

SIGAP — complete code listing

Repository: github.com/Alvnc/loop-tb – prototipe riset UnivaBio 2026.

Berisi: pipeline data (audit dedup), training, studi representasi SSL, evaluasi rigor, API, dan UI.

Daftar file

ml/evaluate.py
ml/export_onnx.py
ml/ssl_probe.py
ml/train.py
ml/data/prepare_data.py
ml/data/prepare_meta.py
ml/kaggle/hello/hello.py
ml/kaggle/make_nlm_dataset.py
ml/kaggle/out_ssl/loop-tb/api/main.py
ml/kaggle/out_ssl/loop-tb/ml/data/prepare_data.py
ml/kaggle/out_ssl/loop-tb/ml/data/prepare_meta.py
ml/kaggle/out_ssl/loop-tb/ml/evaluate.py
ml/kaggle/out_ssl/loop-tb/ml/export_onnx.py
ml/kaggle/out_ssl/loop-tb/ml/kaggle/hello/hello.py
ml/kaggle/out_ssl/loop-tb/ml/kaggle/make_nlm_dataset.py
ml/kaggle/out_ssl/loop-tb/ml/kaggle/ssl/ssl_extract.py
ml/kaggle/out_ssl/loop-tb/ml/kaggle/train/train_kernel.py
ml/kaggle/out_ssl/loop-tb/ml/ssl_probe.py
ml/kaggle/out_ssl/loop-tb/ml/train.py
ml/kaggle/out_ssl/loop-tb/tools/e2e_screen.py
ml/kaggle/out_ssl/loop-tb/tools/screenshot_ui.py
ml/kaggle/out_ssl_diag/loop-tb/api/main.py
ml/kaggle/out_ssl_diag/loop-tb/ml/data/prepare_data.py
ml/kaggle/out_ssl_diag/loop-tb/ml/data/prepare_meta.py
ml/kaggle/out_ssl_diag/loop-tb/ml/evaluate.py
ml/kaggle/out_ssl_diag/loop-tb/ml/export_onnx.py
ml/kaggle/out_ssl_diag/loop-tb/ml/kaggle/hello/hello.py
ml/kaggle/out_ssl_diag/loop-tb/ml/kaggle/make_nlm_dataset.py
ml/kaggle/out_train/loop-tb/api/main.py
ml/kaggle/out_train/loop-tb/ml/data/prepare_data.py
ml/kaggle/out_train/loop-tb/ml/data/prepare_meta.py
ml/kaggle/out_train/loop-tb/ml/evaluate.py
ml/kaggle/out_train/loop-tb/ml/export_onnx.py
ml/kaggle/out_train/loop-tb/ml/kaggle/hello/hello.py
ml/kaggle/out_train/loop-tb/ml/kaggle/train/train_kernel.py
ml/kaggle/out_train/loop-tb/ml/train.py
ml/kaggle/ssl/ssl_extract.py
ml/kaggle/train/train_kernel.py
api/main.py
api/model_config.json
api/model_config_probe.json
api/model_config_space.json
scripts/fetch_models.sh
scripts/redeploy_all.sh
tools/e2e_screen.py
tools/gallery_check.py
tools/klinisi_shot.py
tools/make_case_gallery.py
tools/make_pdfs.py
tools/make_video.py
tools/model_checks.py
tools/print_check.py
tools/screenshot_ui.py
tools/ui_verify.py
web/src/lib/api.ts
web/src/app/klinisi/page.tsx
web/src/app/layout.tsx
web/src/app/page.tsx
web/src/app/pasien/page.tsx
web/src/app/skrining/page.tsx
web/src/app/tentang/page.tsx
web/src/components/BottomNav.tsx
web/src/components/ScreeningFlow.tsx
web/src/components/SiteFooter.tsx
web/src/components/SiteHeader.tsx
web/src/lib/i18n.tsx
web/src/app/globals.css

web/package-lock.json
web/package.json
web/tsconfig.json
web/next-env.d.ts
web/next.config.ts
docs/00-charter.md
docs/01-pohon-masalah.md
docs/02-arsitektur.md
docs/03-eksekusi.md
docs/04-evidensi-data.md
docs/05-model-card.md

ml/evaluate.py

```
#!/usr/bin/env python3
"""Evaluasi rigor: kalibrasi, CI bootstrap, transfer, band triase, subgroup.

Contoh (ensemble + temperature scaling):
python ml/evaluate.py --runs ml/runs/efficientnet_b0 ml/runs/convnext_tiny \
    --manifest ml/data/artifacts/manifest.json --split external --temp-scale --out ml/runs/eval_external
"""
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import brier_score_loss, roc_auc_score, roc_curve
from torch.utils.data import DataLoader
from tqdm import tqdm

import sys
sys.path.insert(0, str(Path(__file__).resolve().parent))
from train import CXRManifest # noqa: E402

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-x))

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 -> fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

def threshold_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    fpr, tpr, thr = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(thr[ok][np.argmax(tpr[ok])]) if ok.any() else 0.5

def ece(y: np.ndarray, p: np.ndarray, bins: int = 15) -> float:
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for i in range(bins):
        m = (p >= edges[i]) & ((p < edges[i + 1]) if i < bins - 1 else (p <= edges[i + 1]))
        if m.sum():
            total += m.mean() * abs(p[m].mean() - y[m].mean())
    return float(total)

def bootstrap_auroc(y: np.ndarray, p: np.ndarray, n: int = 1000, seed: int = 42) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    vals = []
    for _ in range(n):
        idx = rng.integers(0, len(y), len(y))
        if len(np.unique(y[idx])) > 1:
            vals.append(roc_auc_score(y[idx], p[idx]))
    return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))

@torch.no_grad()
def infer(ckpt_path: Path, items: list[dict], img_size: int, device: torch.device, workers: int) -> np.ndarray:
    ckpt = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    arch = ckpt["arch"]
    model = timm.create_model(arch, pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.to(device).eval()
    dl = DataLoader(CXRManifest(items, train=False, img_size=img_size), batch_size=64,
                    shuffle=False, num_workers=workers)

    out = []
    for x, _, _ in tqdm(dl, desc=f"infer {ckpt_path.parent.name}", leave=False):
        x = x.to(device)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            out.append(model(x).squeeze(1).float().cpu().numpy())
    return np.concatenate(out)
```

```

def fit_temperature(val_logits: list[np.ndarray], val_y: np.ndarray) -> float:
    """Cari T>0 yang meminimalkan NLL pada val (grid log-space lebar untuk logit besar)."""
    logits = torch.tensor(np.mean(val_logits, axis=0), dtype=torch.float32)
    y = torch.tensor(val_y, dtype=torch.float32)
    best_t, best_nll = 1.0, float("inf")
    for t in np.logspace(-1, 3.5, 500):
        nll = nn.functional.binary_cross_entropy_with_logits(logits / t, y).item()
        if nll < best_nll:
            best_t, best_nll = float(t), nll
    return best_t

def reliability_plot(y: np.ndarray, p: np.ndarray, path: Path, title: str) -> None:
    edges = np.linspace(0, 1, 11)
    mids, accs = [], []
    for i in range(10):
        m = (p >= edges[i]) & (p < edges[i + 1] if i < 9 else p <= edges[i + 1])
        if m.sum():
            mids.append(p[m].mean())
            accs.append(y[m].mean())
    plt.figure(figsize=(4.2, 4.2))
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.plot(mids, accs, "o-", color="#d62728", lw=2)
    plt.xlabel("Confidence"), plt.ylabel("Empirical TB rate"), plt.title(title)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def roc_plot(curves: dict[str, tuple[np.ndarray, np.ndarray]], path: Path) -> None:
    plt.figure(figsize=(5, 5))
    for name, (y, p) in curves.items():
        fpr, tpr, _ = roc_curve(y, p)
        plt.plot(fpr, tpr, lw=2, label=f"{name} (AUROC {roc_auc_score(y, p):.3f})")
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.xlabel("FPR"), plt.ylabel("TPR (sensitivitas)"), plt.legend(fontsize=8)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--runs", type=Path, nargs="+", required=True)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--split", default="external", choices=["val", "external"])
    ap.add_argument("--temp-scale", action="store_true")
    ap.add_argument("--meta", type=Path, default=None, help="CSV filename,sex,age untuk subgrup (opsional)")
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--workers", type=int, default=8)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    manifest = json.loads(args.manifest.read_text())
    items = manifest[args.split]
    y = np.array([it["label"] for it in items])

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    per_run_target, per_run_val, tscales = {}, {}, {}
    for run in args.runs:
        ckpt = torch.load(run / "best.pt", map_location="cpu", weights_only=False)
        img_size = ckpt["config"]["img_size"]
        per_run_target[run.name] = infer(run / "best.pt", items, img_size, device, args.workers)
        val_items = manifest["val"]
        per_run_val[run.name] = infer(run / "best.pt", val_items, img_size, device, args.workers)
        if args.temp_scale:
            tscales[run.name] = fit_temperature([per_run_val[run.name]], np.array([it["label"] for it in val_items]))

    per_run_target = {k: sigmoid(v) for k, v in per_run_target.items()}
    per_run_val = {k: sigmoid(v) for k, v in per_run_val.items()}
    val_y = np.array([it["label"] for it in manifest["val"]])

    # kalibrasi per-run pada val, lalu terapkan T ke target (bukan val-fit pada target!)
    if args.temp_scale:
        for k in per_run_target:
            per_run_val[k] = sigmoid(np.log(np.clip(per_run_val[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_val[k], 1e-6, 1 - 1e-6)))
            / tscales[k])
            lg = np.log(np.clip(per_run_target[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_target[k], 1e-6, 1 - 1e-6)))
            per_run_target[k] = sigmoid(lg / tscales[k])

    # ensemble = rata-rata; ketidakpastian = std antar anggota
    stack_t = np.stack(list(per_run_target.values()))
    p_ens = stack_t.mean(axis=0)
    u_ens = stack_t.std(axis=0)
    stack_v = np.stack(list(per_run_val.values()))
    p_ens_v = stack_v.mean(axis=0)

    tau_high = threshold_at_spec(val_y, p_ens_v, 0.70) # spec>=0.70 (TPP WHO)
    tau_low = threshold_at_spec(val_y, p_ens_v, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    band = np.where(p_ens >= tau_high, "rujuk_prioritas", np.where(p_ens < tau_low, "negatif_skrining", "ragu"))
    band_report = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b

```

```

        band_report[b] = {"n": int(m.sum()), "frac": float(m.mean()),
                          "tb_rate": float(y[m].mean()) if m.sum() else None}

lo, hi = bootstrap_aucroc(y, p_ens)
metrics = {
    "split": args.split, "n": int(len(y)), "pos": int(y.sum()),
    "aucroc": float(roc_auc_score(y, p_ens)), "aucroc_ci95": [lo, hi],
    "sens_at_spec_90": sens_at_spec(y, p_ens, 0.90),
    "sens_at_spec_70": sens_at_spec(y, p_ens, 0.70),
    "ece": ece(y, p_ens), "brier": float(brier_score_loss(y, p_ens)),
    "temp_scales": tscales, "tau_low": tau_low, "tau_high": tau_high,
    "bands": band_report,
    "deferral": {"rate": float((band == "ragu").mean()),
                 "tb_rate_on_deferred": float(y[band == "ragu"].mean()) if (band == "ragu").any() else None},
    "per_run_aucroc": {k: float(roc_auc_score(y, v)) for k, v in per_run_target.items()},
}

if args.meta and args.meta.exists():
    meta = {r["filename"]: r for r in csv.DictReader(args.meta.open())}
    for key in ("sex", "age"):
        groups: dict[str, list[int]] = {}
        for i, it in enumerate(items):
            row = meta.get(Path(it["path"]).name, {})
            g = row.get(key, "NA") or "NA"
            if key == "age":
                try:
                    a = float(g)
                    g = "0-30" if a < 30 else "30-50" if a < 50 else "50+"
                except ValueError:
                    g = "NA"
            groups.setdefault(g, []).append(i)
        metrics[f"aucroc_by_{key}"] = {
            g: (float(roc_auc_score(y[idx], p_ens[idx])) if len(np.unique(y[idx])) > 1 else None)
              for g, idx in groups.items() if g != "NA" and len(idx) >= 20
        }

(args.out / "metrics.json").write_text(json.dumps(metrics, indent=1))
# dump prediksi per-citra (untuk analisis lanjutan & stacking)
import csv as _csv
with (args.out / "preds.csv").open("w", newline="") as f:
    w = _csv.writer(f)
    w.writerow(["path", "label", "p_ens", "u_ens"])
    for it, p, u in zip(items, p_ens, u_ens):
        w.writerow([it["path"], it["label"], float(p), float(u)])
reliability_plot(y, p_ens, args.out / "reliability.png", f"Kalibrasi - {args.split}")
curves = {f"ens ({args.split})": (y, p_ens)}
if len(per_run_target) > 1:
    curves.update({k: (y, v) for k, v in per_run_target.items()})
roc_plot(curves, args.out / "roc.png")

print("\n=== METRICS ===")
for k, v in metrics.items():
    if isinstance(v, (int, float, str)):
        print(f"{k:18s} {v}")
print(f"bands          {band_report}")

if __name__ == "__main__":
    main()

```

ml/export_onnx.py

```
#!/usr/bin/env python3
"""Export checkpoint ke ONNX (opsi 17) + verifikasi onnxruntime.

Contoh:
python ml/export_onnx.py --ckpt ml/runs/efficientnet_b0/best.pt --out artifacts/tb_efficientnet_b0.onnx
"""
from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import torch

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm")

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--ckpt", type=Path, required=True)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--opset", type=int, default=17)
    args = ap.parse_args()

    ckpt = torch.load(args.ckpt, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()

    img_size = ckpt["config"].get("img_size", 224)
    dummy = torch.randn(1, 3, img_size, img_size)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    torch.onnx.export(
        model, dummy, args.out,
        input_names=["input"], output_names=["logit"],
        dynamic_axes={"input": {0: "batch"}, "logit": {0: "batch"}},
        opset_version=args.opset, dynamo=False, # legacy: satu file mandiri, opset stabil
    )

    import onnxruntime as ort
    sess = ort.InferenceSession(str(args.out), providers=["CPUExecutionProvider"])
    ref = model(dummy).detach().numpy()
    got = sess.run(None, {"input": dummy.numpy()})[0]
    # Verifikasi di ruang probabilitas (deployment-relevant); logit bisa sangat besar
    # pada model overkonfiden sehingga toleransi absolut di ruang logit tidak adil.
    sig = lambda z: 1 / (1 + np.exp(-z)) # noqa: E731
    prob_diff = float(np.abs(sig(ref) - sig(got)).max())
    size_mb = args.out.stat().st_size / 1e6
    print(f"ONNX ok: {args.out} ({size_mb:.1f} MB) max|Δprob|={prob_diff:.2e} opset={args.opset}")
    if prob_diff > 1e-3:
        print(f"WARN: perbedaan probabilitas {prob_diff:.4f} > 1e-3 – periksa sebelum produksi")

if __name__ == "__main__":
    main()
```

ml/ssl_probe.py

```
#!/usr/bin/env python3
"""Latih probe linear di atas embedding SSL + evaluasi eksternal rigor (ADR-007).

Contoh:
.venv/bin/python ml/ssl_probe.py --emb-dir ml/kaggle/out_ssl/ssl_embeddings \
  --out ml/runs/ssl_eval --supervised-preds ml/runs/eval_external_ens/preds.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import sys
from pathlib import Path

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss, roc_auc_score
from sklearn.preprocessing import StandardScaler

sys.path.insert(0, str(Path(__file__).resolve().parent))
from evaluate import ( # noqa: E402
    bootstrap_auroc,
    ece,
    fit_temperature,
    reliability_plot,
    roc_plot,
    sens_at_spec,
    threshold_at_spec,
)

# Statistik preprocessing piksel yang HARUS sama dengan saat ekstraksi embedding.
# (dinov2: default timm/ImageNet; rad_dino: diisi dari log kernel – processor HF)
PIXEL = {
    "dinov2": {
        "type": "timm", "id": "vit_base_patch14_dinov2.lvd142m", "size": 518,
        "mean": (0.485, 0.456, 0.406), "std": (0.229, 0.224, 0.225),
    },
    "rad_dino": {
        "type": "hf", "id": "microsoft/rad-dino", "size": 518,
        "mean": (0.5307, 0.5307, 0.5307), "std": (0.2588, 0.2588, 0.2588), # diverifikasi dari log kernel; koreksi bila beda
    },
}

def sigmoid(z: np.ndarray) -> np.ndarray:
    z = np.clip(z, -60, 60)
    return 1 / (1 + np.exp(-z))

def load_split(emb_dir: Path, enc: str, split: str) -> tuple[np.ndarray, np.ndarray, list[dict]]:
    X = np.load(emb_dir / f"emb_{enc}_{split}.npy")
    meta = json.loads((emb_dir / f"meta_{split}.json").read_text())
    y = np.array([m["label"] for m in meta])
    return X, y, meta

def pixel_of(emb_dir: Path, enc: str) -> dict:
    """Statistik preprocessing piksel: dari encmeta kernel bila ada, else fallback PIXEL."""
    f = emb_dir / f"encmeta_{enc}.json"
    if f.exists():
        d = json.loads(f.read_text())
        return {"type": d["type"], "id": d["id"], "size": d["size"], "mean": tuple(d["mean"]), "std": tuple(d["std"])}
    return PIXEL.get(enc, PIXEL["dinov2"])

def band_metrics(y: np.ndarray, p: np.ndarray, tau_low: float, tau_high: float) -> dict:
    band = np.where(p >= tau_high, "rujuk_prioritas", np.where(p < tau_low, "negatif_skrining", "ragu"))
    rep = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b
        rep[b] = {
            "n": int(m.sum()),
            "frac": float(m.mean()),
            "tb_rate": float(y[m].mean()) if m.sum() else None,
        }
    return rep

def full_metrics(y: np.ndarray, p: np.ndarray) -> dict:
    lo, hi = bootstrap_auroc(y, p)
    return {
        "n": int(len(y)),
        "pos": int(y.sum()),
        "auroc": float(roc_auc_score(y, p)),
        "auroc_ci95": [lo, hi],
        "sens_at_spec_90": sens_at_spec(y, p, 0.90),
        "sens_at_spec_70": sens_at_spec(y, p, 0.70),
    }
```

```

        "ece": ece(y, p),
        "brier": float(brier_score_loss(y, p)),
    }

def run_encoder(enc: str, emb_dir: Path, out_dir: Path, supervised_preds: Path | None) -> dict:
    Xtr, ytr, _ = load_split(emb_dir, enc, "train")
    Xva, yva, meta_va = load_split(emb_dir, enc, "val")
    Xex, yex, meta_ex = load_split(emb_dir, enc, "external")

    scaler = StandardScaler().fit(Xtr)
    Xtr_s, Xva_s, Xex_s = scaler.transform(Xtr), scaler.transform(Xva), scaler.transform(Xex)

    best = {"auroc": -1.0, "C": None}
    for C in (0.001, 0.01, 0.1, 1.0, 10.0):
        clf = LogisticRegression(C=C, class_weight="balanced", max_iter=3000, solver="lbfgs")
        clf.fit(Xtr_s, ytr)
        z_va = clf.decision_function(Xva_s)
        try:
            auroc = roc_auc_score(yva, sigmoid(z_va))
        except ValueError:
            auroc = float("nan")
        if auroc > best["auroc"]:
            best = {"auroc": float(auroc), "C": C, "clf": clf}
        print(f" [{enc}] C={C:<6} val AUROC={auroc:.4f}")

    clf: LogisticRegression = best["clf"]
    z_va, z_ex = clf.decision_function(Xva_s), clf.decision_function(Xex_s)
    T = fit_temperature([z_va], yva)
    p_va, p_ex = sigmoid(z_va / T), sigmoid(z_ex / T)

    tau_high = threshold_at_spec(yva, p_va, 0.70)
    tau_low = threshold_at_spec(yva, p_va, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    res = {
        "encoder": enc,
        "chosen_C": best["C"],
        "val": full_metrics(yva, p_va),
        "external": full_metrics(yex, p_ex),
        "temp": T,
        "tau_low": tau_low,
        "tau_high": tau_high,
        "bands_external": band_metrics(yex, p_ex, tau_low, tau_high),
    }

    out_dir.mkdir(parents=True, exist_ok=True)
    (out_dir / f"metrics_{enc}.json").write_text(json.dumps(res, indent=1))

    # ekspor probe untuk serving API (scaler + LR + T + metadata encoder)
    pix = pixel_of(emb_dir, enc)
    np.savez(
        out_dir / f"probe_{enc}.npz",
        scaler_mean=scaler.mean_,
        scaler_scale=scaler.scale_,
        coef=clf.coef_.ravel(),
        intercept=np.array([float(clf.intercept_[0])]),
        T=np.array([T]),
        size=np.array([pix["size"]]),
        pixel_mean=np.array(pix["mean"], dtype=np.float64),
        pixel_std=np.array(pix["std"], dtype=np.float64),
        encoder_type=np.array([pix["type"]]),
        encoder_id=np.array([pix["id"]]),
    )

    reliability_plot(yex, p_ex, out_dir / f"reliability_{enc}.png", f"SSL probe {enc} - external")
    roc_plot({f"{enc} (external)": (yex, p_ex)}, out_dir / f"roc_{enc}.png")
    with (out_dir / f"preds_{enc}.csv").open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["path", "label", "p", "z"])
        for m, p, z in zip(meta_ex, p_ex, z_ex):
            w.writerow([m["path"], m["label"], float(p), float(z)])

    # stacking dengan ensemble supervised (jika tersedia)
    if supervised_preds and supervised_preds.exists():
        sup = {}
        for row in csv.DictReader(supervised_preds.open()):
            sup[Path(row["path"]).stem] = float(row["p_ens"])
        pairs = [(Path(m["path"]).stem, m["label"], p) for m, p in zip(meta_ex, p_ex)]
        aligned = [(k, lab, p, sup[k]) for k, lab, p in pairs if k in sup]
        if len(aligned) >= 50:
            ya = np.array([a[1] for a in aligned])
            ps = (np.array([a[2] for a in aligned]) + np.array([a[3] for a in aligned])) / 2
            res["stack_with_supervised"] = {**full_metrics(ya, ps), "n_matched": len(aligned)}
            print(f" [{enc}] stack n={len(aligned)} external AUROC={res['stack_with_supervised']['auroc']:.4f}")

    (out_dir / f"metrics_{enc}.json").write_text(json.dumps(res, indent=1))
    return res

def main() -> None:

```

```

ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
ap.add_argument("--emb-dir", type=Path, required=True)
ap.add_argument("--out", type=Path, required=True)
ap.add_argument("--encoder", nargs="*", default=None, help="default: semua yang ada")
ap.add_argument("--supervised-preds", type=Path, default=None)
args = ap.parse_args()

encs = args.encoder or sorted(
    p.name[len("emb_") : -len("_train.npy")] for p in args.emb_dir.glob("emb_*_train.npy")
)
results = []
for enc in encs:
    print(f"=== {enc} ===")
    results.append(run_encoder(enc, args.emb_dir, args.out, args.supervised_preds))

print("\n=== RINGKASAN (eksternal) ===")
for r in results:
    e = r["external"]
    print(f"{r['encoder']:10s} AUROC {e['auroc']:.4f} [{e['auroc_ci95'][0]:.3f},{e['auroc_ci95'][1]:.3f}] "
          f"| sens@spec90 {e['sens_at_spec_90']:.3f} | ECE {e['ece']:.3f}"
          + (f" | stack {r['stack_with_supervised'] ['auroc']:.4f}" if "stack_with_supervised" in r else ""))

# kombinasi rata-rata probabilitas antar encoder SSL
preds_files = sorted(args.out.glob("preds_*.csv"))
if len(preds_files) > 1:
    maps = []
    for f in preds_files:
        rows = list(csv.DictReader(f.open()))
        maps.append({r["path"]: (float(r["p"]), int(r["label"])) for r in rows})
    keys = set(maps[0])
    for m in maps[1:]:
        keys |= set(m)
    ks = sorted(keys)
    y = np.array([maps[0][k][1] for k in ks])
    p = np.mean([np.array([m[k][0] for k in ks]) for m in maps], axis=0)
    comb = full_metrics(y, p)
    print(f"combined{'x'.join([]) or ''} SSL ({len(maps)} encoder): AUROC {comb['auroc']:.4f} "
          f"[{comb['auroc_ci95'][0]:.3f},{comb['auroc_ci95'][1]:.3f}] | sens@spec90 {comb['sens_at_spec_90']:.3f}")
    (args.out / "metrics_combined.json").write_text(json.dumps(comb, indent=1))

if __name__ == "__main__":
    main()

```

ml/train.py

```
#!/usr/bin/env python3
"""Train TB CXR classifier dari manifest provenance-aware.

Contoh:
python ml/train.py --manifest ml/data/artifacts/manifest.json --arch efficientnet_b0 \
    --epochs 20 --out ml/runs/efficientnet_b0

Baseline linear probe (fitur beku):
python ml/train.py ... --freeze --lr 1e-3 --out ml/runs/linear_probe
"""
from __future__ import annotations

import argparse
import csv
import json
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import roc_auc_score, roc_curve
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import transforms as T
from tqdm import tqdm

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

class CXRManifest(Dataset):
    def __init__(self, items: list[dict], train: bool, img_size: int):
        self.items = items
        if train:
            self.tx = T.Compose([
                T.RandomResizedCrop(img_size, scale=(0.75, 1.0), ratio=(0.9, 1.1)),
                T.RandomRotation(7, fill=0),
                T.ColorJitter(brightness=0.15, contrast=0.15),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])
        else:
            self.tx = T.Compose([
                T.Resize(int(img_size * 1.14)),
                T.CenterCrop(img_size),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])

    def __len__(self) -> int:
        return len(self.items)

    def __getitem__(self, i: int):
        from PIL import Image
        it = self.items[i]
        with Image.open(it["path"]) as im:
            x = self.tx(im.convert("RGB"))
        return x, torch.tensor(float(it["label"])), i

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 → fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

@torch.no_grad()
def predict(model: nn.Module, loader: DataLoader, device: torch.device) -> tuple[np.ndarray, np.ndarray]:
    model.eval()
    logits_all, labels_all = [], []
    for x, y, _ in loader:
        x = x.to(device, non_blocking=True)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            logits = model(x).squeeze(1).float()
        logits_all.append(logits.cpu().numpy())
        labels_all.append(y.numpy())
    return np.concatenate(logits_all), np.concatenate(labels_all)

def evaluate(logits: np.ndarray, y: np.ndarray) -> dict:
    p = 1 / (1 + np.exp(-logits))
    out = {"n": int(len(y)), "pos": int(y.sum())}
    if len(np.unique(y)) > 1:
```

```

        out["auroc"] = float(roc_auc_score(y, p))
        out["sens_at_spec_90"] = sens_at_spec(y, p, 0.90)
        out["sens_at_spec_70"] = sens_at_spec(y, p, 0.70)
    return out

```

```

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--arch", default="efficientnet_b0")
    ap.add_argument("--epochs", type=int, default=20)
    ap.add_argument("--batch", type=int, default=32)
    ap.add_argument("--lr", type=float, default=3e-4)
    ap.add_argument("--wd", type=float, default=1e-4)
    ap.add_argument("--img-size", type=int, default=224)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--workers", type=int, default=8)
    ap.add_argument("--freeze", action="store_true", help="linear probe: backbone beku")
    ap.add_argument("--balanced", action="store_true", help="WeightedRandomSampler")
    ap.add_argument("--limit", type=int, default=0, help="smoke test: batasi n sampel (0=semua)")
    args = ap.parse_args()

    torch.manual_seed(args.seed)
    np.random.seed(args.seed)
    args.out.mkdir(parents=True, exist_ok=True)

    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
    manifest = json.loads(args.manifest.read_text())

    def take(items: list[dict], n: int) -> list[dict]:
        """Subset berstratifikasi untuk smoke test."""
        if not n:
            return items
        pos = [it for it in items if it["label"] == 1]
        neg = [it for it in items if it["label"] == 0]
        n = min(n, len(items))
        kp = min(len(pos), max(1, n // 4))
        return pos[:kp] + neg[: max(n - kp, 0)]

    train_items = take(manifest["train"], args.limit)
    val_items = take(manifest["val"], args.limit)

    ds_tr = CXRMManifest(train_items, train=True, img_size=args.img_size)
    ds_va = CXRMManifest(val_items, train=False, img_size=args.img_size)
    sampler = None
    if args.balanced:
        labels = np.array([it["label"] for it in train_items])
        w = np.where(labels == 1, 1.0 / labels.sum(), 1.0 / (len(labels) - labels.sum()))
        sampler = WeightedRandomSampler(torch.from_numpy(w), num_samples=len(labels), replacement=True)
    dl_tr = DataLoader(ds_tr, batch_size=args.batch, shuffle=sampler is None, sampler=sampler,
                      num_workers=args.workers, pin_memory=(device.type == "cuda"), drop_last=True)
    dl_va = DataLoader(ds_va, batch_size=args.batch, shuffle=False, num_workers=args.workers, pin_memory=(device.type == "cuda"))

    model = timm.create_model(args.arch, pretrained=True, num_classes=1).to(device)
    if args.freeze:
        for n, p in model.named_parameters():
            p.requires_grad = "head" in n or "classifier" in n or "fc" in n
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=args.lr, weight_decay=args.wd)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(args.epochs, 1))
    loss_fn = nn.BCEWithLogitsLoss()
    scaler = torch.amp.GradScaler(enabled=(device.type == "cuda"))

    print(f"device={device} arch={args.arch} freeze={args.freeze} "
          f"train={len(ds_tr)} val={len(ds_va)} params={sum(p.numel() for p in params)/1e6:.2f}M")

    hist = []
    best = {"auroc": -1.0, "epoch": -1}
    for epoch in range(1, args.epochs + 1):
        model.train()
        t0, losses = time.time(), []
        for x, y, _ in tqdm(dl_tr, desc=f"epoch {epoch:02d}", leave=False):
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device.type, enabled=(device.type == "cuda")):
                logits = model(x).squeeze(1)
                loss = loss_fn(logits.float(), y)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            losses.append(loss.item())
        sched.step()

        logits, y_va = predict(model, dl_va, device)
        m = evaluate(logits, y_va)
        row = {"epoch": epoch, "loss": float(np.mean(losses)), "lr": opt.param_groups[0]["lr"],
              "sec": round(time.time() - t0, 1), **m}
        hist.append(row)
    print(f"epoch {epoch:02d} | loss {row['loss']:.4f} | AUROC {m.get('auroc', float('nan')):.4f} "
          f"| sens@spec90 {m.get('sens_at_spec_90', float('nan')):.3f} | {row['sec']}s")

```

```

torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
           "epoch": epoch, "val": m}, args.out / "last.pt")
if m.get("auroc", -1) > best["auroc"]:
    best = {"auroc": m["auroc"], "epoch": epoch}
    torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
               "epoch": epoch, "val": m}, args.out / "best.pt")
    with (args.out / "val_preds.csv").open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["path", "label", "logit"])
        for it, lg in zip(val_items, logits):
            w.writerow([it["path"], it["label"], float(lg)])

(args.out / "history.csv").write_text(
    "epoch,loss,lr,sec,n,pos,auroc,sens_at_spec_90,sens_at_spec_70\n" + "\n".join(
        ", ".join(str(r.get(k, "")) for k in ("epoch", "loss", "lr", "sec", "n", "pos", "auroc", "sens_at_spec_90",
        "sens_at_spec_70"))
        for r in hist) + "\n")
(args.out / "metrics.json").write_text(json.dumps({"best": best, "history": hist, "config": vars(args)}, indent=1, default=str))
print(f"\nBEST epoch {best['epoch']} AUROC {best['auroc']:.4f} -> {args.out/'best.pt'}")

if __name__ == "__main__":
    main()

```

ml/data/prepare_data.py

```
#!/usr/bin/env python3
"""Build a provenance-aware manifest for TB chest X-ray training.

Sumber & aturan label (lihat docs/02-arsitektur.md):
- TBX11K (Kaggle: vbookshelf/tbx11k-simplified): folder per kelas.
  positif = {tb}; negatif = {healthy, sick}; latent_tb DIKELUARKAN (ambigu).
- Rahman composite (Kaggle: tawsifurrahman/tuberculosis-tb-chest-xray-dataset):
  folder Normal/ -> 0, Tuberculosis/ -> 1.
- NLM Shenzhen/Montgomery: konvensi nama file resmi `*_0` = normal, `*_1` = TB.

Dedup (WAJIB – mitigasi class-conditional acquisition confounding):
1. pHash 64-bit; Hamming <= --dup-threshold dianggap duplikat (union-find transitif).
2. Integritas external: citra train yang sekelompok dengan citra external DIBUANG dari train.
3. Duplikat internal train dibuang (keep-first, urutan deterministik).
4. Split val dilakukan per-klaster (tidak ada kebocoran near-dup train->val).

Output (ml/data/artifacts/):
- manifest.json, dedup_report.json, train.csv, val.csv, external.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import re
from collections import Counter
from pathlib import Path

import numpy as np
from PIL import Image
from tqdm import tqdm

try:
    import imagehash
except ImportError: # pragma: no cover
    raise SystemExit("Butuh imagehash: pip install imagehash")

IMG_EXTS = {".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff"}
POS_TOKENS = {"tb", "tuberculosis", "tuberculose"}
NEG_TOKENS = {"healthy", "normal", "sick"}
EXCLUDE_TOKENS = {"latent", "latent_tb", "latenttb"}

def _rel(p: Path, root: Path) -> str:
    """Path relatif terhadap root repo (portable lokal <-> Colab); fallback absolut."""
    try:
        return str(p.relative_to(root))
    except ValueError:
        return str(p)

def _tokens(path: Path) -> set[str]:
    parts = [p.lower() for p in path.parts]
    parts += [path.stem.lower()]
    return set(parts)

def load_tbx11k_map(raw: Path) -> dict[str, int | None]:
    """Label otoritatif dari data.csv TBX11K: 1=active_tb, 0=no_tb, None=latent (dikeluarkan)."""
    m: dict[str, int | None] = {}
    for p in raw.rglob("data.csv"):
        if "tbx11k" not in str(p).lower():
            continue
        for r in csv.DictReader(p.open()):
            f = (r.get("fname") or "").strip()
            if not f:
                continue
            t = (r.get("tb_type") or "").lower()
            tgt = (r.get("target") or "").lower()
            if "latent" in t:
                m.setdefault(f, None)
            elif tgt == "tb":
                m[f] = 1
            else:
                m.setdefault(f, 0)
    return m

def classify(path: Path, raw: Path, tbx_map: dict[str, int | None]) -> tuple[int, str] | None:
    """Return (label, source) atau None kalau tidak bisa diklasifikasi."""
    rel = path.relative_to(raw)
    low = str(rel).lower()

    # --- NLM Shenzhen / Montgomery: konvensi suffix nama file ---
    m = re.search(r"(chncxr|mcucxr)_d+_(\d+)", path.stem.lower())
    if m:
        label = int(m.group(2))
        source = "shenzhen" if m.group(1) == "chncxr" else "montgomery"
```

```

        return label, source

toks = _tokens(rel)
if toks & EXCLUDE_TOKENS or any(t.startswith("latent") for t in toks):
    return None

if "tbx11k" in low:
    if tbx_map:
        lab = tbx_map.get(path.name, None)
        return None if lab is None else (lab, "tbx11k")
    # fallback bila data.csv tidak ada: prefix nama file
    stem = path.stem.lower()
    if stem.startswith("t"):
        return 1, "tbx11k"
    if stem.startswith(("h", "s")):
        return 0, "tbx11k"
    return None

if toks & POS_TOKENS:
    return 1, "rahman"
if toks & NEG_TOKENS:
    return 0, "rahman"
return None

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[rb] = ra

_POP = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def hamming_pairs(hashes: np.ndarray, threshold: int, chunk: int = 512) -> list[tuple[int, int]]:
    """Semua pasangan (i<j) dengan Hamming distance <= threshold (chunked, hemat memori)."""
    n = len(hashes)
    h8 = hashes.view(np.uint8).reshape(n, 8)
    pairs: list[tuple[int, int]] = []
    for start in range(0, n, chunk):
        stop = min(start + chunk, n)
        xor = np.bitwise_xor(h8[start:stop, None, :], h8[None, :, :])
        dist = _POP[xor].sum(axis=-1) # (chunk, n)
        rows, cols = np.where(dist <= threshold)
        for r, c in zip(rows, cols):
            i, j = start + int(r), int(c)
            if i < j:
                pairs.append((i, j))
    return pairs

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--raw", type=Path, default=Path("data/raw"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts"))
    ap.add_argument("--dup-threshold", type=int, default=3, help="Hamming pHash <= ini dianggap duplikat")
    ap.add_argument("--val-frac", type=float, default=0.15)
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    raw = args.raw.resolve()
    root = raw.parent.parent # asumsi struktur <root>/data/raw

    # ----- 1) discovery -----
    labeled: list[dict] = []
    unlabeled = Counter()
    tbx_map = load_tbx11k_map(raw)
    print(f"[discovery] label TBX11K dari data.csv: {len(tbx_map)} entri")
    for p in sorted(raw.rglob("*")):
        if p.suffix.lower() not in IMG_EXTS:
            continue
        cls = classify(p, raw, tbx_map)
        if cls is None:
            unlabeled[str(p.relative_to(raw).parent)] += 1
            continue
        label, source = cls
        labeled.append({"path": _rel(p, root), "label": label, "source": source})

    print(f"[discovery] berlabel={len(labeled)} tak-terklasifikasi={sum(unlabeled.values())}")
    for k, v in unlabeled.most_common(10):
        print(f" skip {v:6d} {k}")

```

```

if not labeled:
    raise SystemExit("Tidak ada citra berlabel – cek struktur data/raw")

# ----- 2) pHash + kluster duplikat -----
print("[dedup] menghitung pHash...")
ok: list[dict] = []
for item in tqdm(labeled, unit="img"):
    try:
        with Image.open(item["path"]) as im:
            h = imagehash.phash(im.convert("L"), hash_size=8)
            item["phash"] = int(str(h), 16)
            ok.append(item)
    except Exception as e: # gambar korup
        print(f" warn: gagal baca {item['path']}: {e}")
labeled = ok

hashes = np.array([it["phash"] for it in labeled], dtype=np.uint64)
pairs = hamming_pairs(hashes, args.dup_threshold)
dsu = DSU(len(labeled))
for i, j in pairs:
    dsu.union(i, j)
for idx, it in enumerate(labeled):
    it["cluster"] = dsu.find(idx)

by_cluster: dict[int, list[dict]] = {}
for it in labeled:
    by_cluster.setdefault(it["cluster"], []).append(it)

# ----- 3) lock external, buang kebocoran dari train -----
ext = [it for it in labeled if it["source"] in ("shenzhen", "montgomery")]
ext_clusters = {it["cluster"] for it in ext}
train_pool, dropped_cross = [], []
for it in labeled:
    if it["source"] in ("shenzhen", "montgomery"):
        continue
    (dropped_cross if it["cluster"] in ext_clusters else train_pool).append(it)

# duplikat internal train: keep-first per kluster (deterministik via path)
keep, dropped_internal = [], []
seen_clusters: set[int] = set()
for it in sorted(train_pool, key=lambda x: x["path"]):
    if it["cluster"] in seen_clusters:
        dropped_internal.append(it)
    else:
        seen_clusters.add(it["cluster"])
        keep.append(it)
train_pool = keep

# ----- 4) split val per-kluster -----
rng = np.random.default_rng(args.seed)
sizes = Counter(it["cluster"] for it in train_pool)
pool_clusters = sorted(sizes)
rng.shuffle(pool_clusters)
target_val = int(len(train_pool) * args.val_frac)
val_clusters: set[int] = set()
acc = 0
for c in pool_clusters:
    if acc >= target_val:
        break
    val_clusters.add(c)
    acc += sizes[c]
train = [it for it in train_pool if it["cluster"] not in val_clusters]
val = [it for it in train_pool if it["cluster"] in val_clusters]

def dump(items: list[dict]) -> list[dict]:
    return [
        {k: it[k] for k in ("path", "label", "source", "cluster", "phash")}
        for it in sorted(items, key=lambda x: x["path"])
    ]

manifest = {
    "meta": {
        "root": str(root),
        "raw": _rel(raw, root),
        "dup_threshold": args.dup_threshold,
        "val_frac": args.val_frac,
        "seed": args.seed,
        "counts": {
            "train": len(train), "val": len(val), "external": len(ext),
            "dropped_cross_corpus": len(dropped_cross),
            "dropped_internal_dup": len(dropped_internal),
            "unlabeled": sum(unlabeled.values()),
        },
    },
    "train": dump(train),
    "val": dump(val),
    "external": dump(ext),
}
(args.out / "manifest.json").write_text(json.dumps(manifest, indent=1))
(args.out / "dedup_report.json").write_text(json.dumps({
    "pairs_within_threshold": len(pairs),
    "clusters": len(by_cluster),
}))

```

```

        "dropped_cross_corpus": dump(dropped_cross),
        "dropped_internal_dup": dump(dropped_internal),
        "unlabeled_by_dir": dict(unlabeled),
    }, indent=1))

for split in ("train", "val", "external"):
    with (args.out / f"{split}.csv").open("w") as f:
        f.write("path,label,source,cluster\n")
        for it in manifest[split]:
            f.write(f"{it['path']},{it['label']},{it['source']},{it['cluster']}\n")

def comp(items):
    c = Counter((it["source"], it["label"]) for it in items)
    return " ".join(f"{s}/{ 'TB' if l else 'NEG' }={n}" for (s, l), n in sorted(c.items()))

print("\n=== RINGKASAN ===")
print(f"train      n={len(train):6d}  {comp(train)}")
print(f"val        n={len(val):6d}    {comp(val)}")
print(f"external  n={len(ext):6d}    {comp(ext)}")
print(f"dibuang  cross-corpus={len(dropped_cross)}  internal-dup={len(dropped_internal)}")
print(f"manifest -> {args.out/'manifest.json'}")

if __name__ == "__main__":
    main()

```

ml/data/prepare_meta.py

```
#!/usr/bin/env python3
"""Ekstrak metadata klinis NLM (Montgomery + Shenzhen) → CSV untuk analisis subgroup.

Output: ml/data/artifacts/meta_nlm.csv
Kolom: filename,sex,age,reading,scar_flag
- Montgomery: "Patient's Sex: F / Patient's Age: 027Y / <reading>"
- Shenzhen: "male 45yrs / <reading>" (kadang terkontaminasi antar-file)
- scar_flag: menyebut scar/inactive/fibrosis → untuk pembahasan mode gagal
            "healed scar false positive" (selaras audit Bilal 2026).
"""
from __future__ import annotations

import argparse
import csv
import re
from pathlib import Path

SEX_M = re.compile(r"sex[:\s]+(m|male)\b", re.I)
SEX_F = re.compile(r"sex[:\s]+(f|female)\b", re.I)
SEX_FMT2 = re.compile(r"\b(male|female)\b", re.I)
AGE_MONT = re.compile(r"age[:\s]+(\d{1,3})\s*Y?\b", re.I)
AGE_FMT2 = re.compile(r"\b(\d{1,3})\s*yrs\b", re.I)
SCAR = re.compile(r"scar|inactive|fibro", re.I)

def parse_montgomery(text: str) -> dict | None:
    age = AGE_MONT.search(text)
    sex = "F" if SEX_F.search(text) else ("M" if SEX_M.search(text) else "")
    if not age:
        return None
    # reading = baris setelah Age
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    reading = lines[2] if len(lines) > 2 else ""
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": int(bool(SCAR.search(reading)))}

def parse_shenzhen(text: str) -> dict | None:
    age = AGE_FMT2.search(text)
    sex_m = SEX_FMT2.findall(text)
    sex = sex_m[0][0].upper() if sex_m else ""
    if not age:
        return None
    # reading = token huruf setelah "yrs"
    tail = text[age.end():]
    reading = ""
    m = re.search(r"[A-Za-z]{2,}", tail)
    if m:
        reading = m.group(0)
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": 0}

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--nlm", type=Path, default=Path("data/raw/nlm"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts/meta_nlm.csv"))
    args = ap.parse_args()
    args.out.parent.mkdir(parents=True, exist_ok=True)

    rows, n_fail = [], 0
    for corpus, parser in (("montgomery", parse_montgomery), ("shenzhen", parse_shenzhen)):
        d = args.nlm / corpus / "ClinicalReadings"
        for p in sorted(d.glob("*.txt")):
            r = parser(p.read_text(errors="ignore"))
            if r is None:
                n_fail += 1
                continue
            r["filename"] = p.stem + ".png"
            r["corpus"] = corpus
            rows.append(r)

    with args.out.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["filename", "corpus", "sex", "age", "reading", "scar_flag"])
        w.writeheader()
        w.writerows(rows)

    from collections import Counter
    print(f"meta -> {args.out} n={len(rows)} gagal-parse={n_fail}")
    print("per korpus:", Counter(r["corpus"] for r in rows))
    print("sex:", Counter(r["sex"] for r in rows))
    print("scar_flag:", Counter(r["scar_flag"] for r in rows))

if __name__ == "__main__":
    main()
```

ml/kaggle/hello/hello.py

```
"""Kernel uji: verifikasi GPU, internet, versi lib, dan struktur dataset ter-mount."""
import os
import sys

lines: list[str] = []

def log(*a) -> None:
    print(*a, flush=True)
    lines.append(" ".join(str(x) for x in a))

log("python:", sys.version.split()[0])

try:
    import torch

    log("torch:", torch.__version__, "| cuda:", torch.cuda.is_available())
    if torch.cuda.is_available():
        log("gpu:", torch.cuda.get_device_name(0))
except Exception as e: # noqa: BLE001
    log("torch error:", e)

for mod in ("timm", "sklearn", "onnx", "onnxruntime"):
    try:
        m = __import__(mod)
        log(f"{mod}: {getattr(m, '__version__', '?')}")
    except Exception as e: # noqa: BLE001
        log(f"{mod}: MISSING ({e})")

log("--- input mounts ---")
for root, dirs, _ in os.walk("/kaggle/input"):
    log(root, "dirs:", dirs[:6])
    if root.count("/") > 3:
        dirs[:] = []

log("HELLO DONE")
with open("/kaggle/working/hello_result.txt", "w") as f:
    f.write("\n".join(lines) + "\n")
```

ml/kaggle/make_nlm_dataset.py

```
#!/usr/bin/env python3
"""Siapkan dataset Kaggle privat: NLM CXR di-resize (max dim 512) + ClinicalReadings.

Dipisah dari data asli agar unggahan ringan (~150 MB) dan resolusi tetap cukup
untuk encoder SSL (DINOv2/RAD-DINO resize ke 518).
"""
from __future__ import annotations

import shutil
from pathlib import Path

from PIL import Image

SRC = Path("data/raw/nlm")
DST = Path("ml/kaggle/nlm_dataset/nlm")

def main() -> None:
    n = 0
    for corpus in ("shenzhen", "montgomery"):
        out_dir = DST / corpus / "CXR_png"
        out_dir.mkdir(parents=True, exist_ok=True)
        for p in sorted((SRC / corpus / "CXR_png").glob("*.png")):
            with Image.open(p) as im:
                im = im.convert("L")
                w, h = im.size
                scale = 512 / max(w, h)
                if scale < 1:
                    im = im.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
                im.save(out_dir / (p.stem + ".jpg"), quality=92)
            n += 1
        readings = SRC / corpus / "ClinicalReadings"
        if readings.exists():
            out_r = DST / corpus / "ClinicalReadings"
            out_r.mkdir(parents=True, exist_ok=True)
            for f in readings.glob("*.txt"):
                shutil.copy(f, out_r / f.name)
        print(f"selesai: {n} citra -> {DST}")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl/loop-tb/api/main.py

```
"""SIGAP API – inference ensemble (supervised ckpt &/atau probe SSL) + Grad-CAM + band triase.

Jalankan lokal:
    uvicorn api.main:app --host 0.0.0.0 --port 8000

Konfigurasi (default `api/model_config.json`, override env LOOPTB_CONFIG):
{
    "score_members": [ {"path": "ml/runs/ssl_eval/probe_dinov2.npz"} ],
    "uncertainty_members": [ {"path": "ml/runs/eb0/best.pt", "T": 1.2336},
                             {"path": "ml/runs/convnext/best.pt", "T": 1.3405} ],
    "bands": {"tau_low": ..., "tau_high": ..., "tau_uncertainty": 0.2}
}

Skor p = rata-rata prob anggota `score_members` (model terbaik untuk AUROC eksternal).
Ketidakpastian u = std antar SEMUA anggota unik (disagreement lintas-keluarga SSL-supervised).
Jika hanya ada satu anggota total, u = 0.
"""
from __future__ import annotations

import base64
import io
import json
import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.cm as cm
import numpy as np
import timm
import torch
import torch.nn as nn
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
from torchvision import transforms as T

CONFIG_PATH = Path(os.environ.get("LOOPTB_CONFIG", "api/model_config.json"))
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

app = FastAPI(title="loop-tb (SIGAP)", version="0.4.0")
app.add_middleware(
    CORSMiddleware, allow_origins=["*"], allow_credentials=False,
    allow_methods=["*"], allow_headers=["*"],
)

_state: dict = {}

def sigmoid(z: float) -> float:
    return float(1.0 / (1.0 + np.exp(-np.clip(z, -60, 60))))

class HFViT(nn.Module):
    """Pembungkus model HF (mis. RAD-DINO) → CLS token sebagai fitur."""
    def __init__(self, model_id: str):
        super().__init__()
        from transformers import AutoModel

        self.m = AutoModel.from_pretrained(model_id)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        out = self.m(pixel_values=x)
        return out.last_hidden_state[:, 0] if hasattr(out, "last_hidden_state") else out.pooler_output

def _load_member(spec: dict) -> dict:
    path = spec["path"]
    if path.endswith(".npz"):
        d = np.load(path, allow_pickle=False)
        enc_type, enc_id = str(d["encoder_type"][0]), str(d["encoder_id"][0])
        model: nn.Module = HFViT(enc_id) if enc_type == "hf" else timm.create_model(
            enc_id, pretrained=True, num_classes=0, dynamic_img_size=True
        )
    return {
        "kind": "probe",
        "model": model.eval(),
        "T": float(d["T"][0]),
        "size": int(d["size"][0]),
        "pixel_mean": tuple(float(v) for v in d["pixel_mean"]),
        "pixel_std": tuple(float(v) for v in d["pixel_std"]),
        "scaler_mean": d["scaler_mean"].astype(np.float32),
        "scaler_scale": d["scaler_scale"].astype(np.float32),
        "coef": d["coef"].astype(np.float32),
        "intercept": float(d["intercept"][0]),
        "path": path,
    }
```

```

    }
    ckpt = torch.load(path, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    return {
        "kind": "torch",
        "model": model.eval(),
        "T": float(spec.get("T", 1.0)),
        "size": int(ckpt["config"].get("img_size", 224)),
        "path": path,
    }

def _load() -> None:
    cfg = json.loads(CONFIG_PATH.read_text())
    score_specs = cfg.get("score_members") or cfg.get("members") or []
    unc_specs = cfg.get("uncertainty_members") or score_specs
    cache: dict[str, dict] = {}

    def get(spec: dict) -> dict:
        p = spec["path"]
        if p not in cache:
            cache[p] = _load_member(spec)
        return cache[p]

    _state.update(
        score_members=[get(s) for s in score_specs],
        unc_members=[get(s) for s in unc_specs],
        bands={"tau_high": 0.5, "tau_low": 0.5, **cfg.get("bands", {})},
    )
    print(f"[SIGAP] score={len(_state['score_members'])} unc={len(_state['unc_members'])} bands={_state['bands']}")

def _names(members: list[dict]) -> list[str]:
    return [f"{Path(m['path']).parent.name}/{Path(m['path']).name}" for m in members]

def preprocess(img: Image.Image, size: int, mean=MEAN, std=STD) -> torch.Tensor:
    tx = T.Compose([T.Resize(int(size * 1.14)), T.CenterCrop(size), T.ToTensor(), T.Normalize(mean, std)])
    return tx(img.convert("RGB")).unsqueeze(0)

def band_of(p: float, u: float, bands: dict) -> str:
    if u > float(bands.get("tau_uncertainty", 1.0)):
        return "ragu"
    if p >= bands["tau_high"]:
        return "rujuk_prioritas"
    if p < bands["tau_low"]:
        return "negatif_skrining"
    return "ragu"

def infer_prob(member: dict, img: Image.Image) -> float:
    if member["kind"] == "probe":
        x = preprocess(img, member["size"], member.get("pixel_mean", MEAN), member.get("pixel_std", STD))
        with torch.no_grad():
            feats = member["model"](x).float().numpy()[0]
            z = float(((feats - member["scaler_mean"]) / member["scaler_scale"]) @ member["coef"] + member["intercept"])
    else:
        x = preprocess(img, member["size"])
        with torch.no_grad():
            z = float(member["model"](x).squeeze(1).item())
    return sigmoid(z / member["T"])

def _cam_from_tokens(grads: torch.Tensor, acts: torch.Tensor, out_hw: tuple[int, int]) -> torch.Tensor | None:
    g, a = grads[1:], acts[1:]
    n = g.shape[0]
    side = int(round(n ** 0.5))
    if side * side != n:
        return None
    g = g.reshape(side, side, -1)
    a = a.reshape(side, side, -1)
    cam = (g * a).sum(-1).clamp(min=0)[None, None]
    return nn.functional.interpolate(cam, size=out_hw, mode="bilinear", align_corners=False)[0, 0]

def gradcam(member: dict, img: Image.Image) -> np.ndarray | None:
    model = member["model"]
    if member["kind"] == "torch":
        x = preprocess(img, member["size"])
        model.zero_grad(set_to_none=True)
        feats = model.forward_features(x)
        feats.retain_grad()
        model.forward_head(feats).squeeze(1).backward()
        grads, acts = feats.grad, feats
        if grads.dim() == 4:
            w = grads.mean(dim=(2, 3), keepdim=True)
            cam = (w * acts).sum(1, keepdim=True).clamp(min=0)
            cam = nn.functional.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)[0, 0]
        else:
            cam = _cam_from_tokens(grads[0], acts[0], tuple(x.shape[-2:]))

```

```

else:
    mean, std = member.get("pixel_mean", MEAN), member.get("pixel_std", STD)
    x = preprocess(img, member["size"], mean, std)
    model.zero_grad(set_to_none=True)
    if hasattr(model, "forward_features"):
        feats = model.forward_features(x)
        pooled = model.forward_head(feats)
    else:
        feats = model.m(pixel_values=x).last_hidden_state
        pooled = feats[:, 0]
    feats = feats if feats.dim() == 3 else feats.flatten(2).transpose(1, 2)
    feats.retain_grad()
    mt = torch.tensor(member["scaler_mean"])
    st = torch.tensor(member["scaler_scale"])
    cf = torch.tensor(member["coef"])
    z = ((pooled[0].float() - mt) / st) @ cf + member["intercept"]
    z.backward()
    cam = _cam_from_tokens(feats.grad[0], feats[0], tuple(x.shape[-2:]))
if cam is None:
    return None
cam = cam.detach()
return (cam / (cam.amax() + 1e-8)).numpy()

def overlay_png(img: Image.Image, cam: np.ndarray) -> str:
    img_np = np.asarray(img.convert("RGB").resize((cam.shape[1], cam.shape[0])), dtype=np.float32) / 255.0
    heat = cm.jet(cam)[..., :3].astype(np.float32)
    alpha = cam[..., None] * 0.45
    blend = img_np * (1 - alpha) + heat * alpha
    out = Image.fromarray((np.clip(blend, 0, 1) * 255).astype(np.uint8))
    buf = io.BytesIO()
    out.save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode()

@app.on_event("startup")
def startup() -> None:
    _load()

@app.get("/health")
def health() -> dict:
    return {
        "status": "ok" if _state.get("score_members") else "no-model",
        "score_members": _names(_state.get("score_members", [])),
        "uncertainty_members": _names(_state.get("unc_members", [])),
        "bands": _state.get("bands"),
    }

@app.post("/predict")
async def predict(file: UploadFile = File(...)) -> dict:
    if not _state.get("score_members"):
        raise HTTPException(503, "model belum siap")
    raw = await file.read()
    try:
        img = Image.open(io.BytesIO(raw))
        img.load()
    except Exception as e:
        raise HTTPException(400, f"bukan citra valid: {e}")

    score_probs = [infer_prob(m, img) for m in _state["score_members"]]
    extra = [m for m in _state["unc_members"] if m not in _state["score_members"]]
    all_probs = score_probs + [infer_prob(m, img) for m in extra]

    p = float(np.mean(score_probs))
    u = float(np.std(all_probs)) if len(all_probs) > 1 else 0.0
    band = band_of(p, u, _state["bands"])

    cam = gradcam(_state["score_members"][0], img)
    heatmap_b64 = overlay_png(img, cam) if cam is not None else None

    return {
        "p_tb": round(p, 4),
        "uncertainty": round(u, 4),
        "band": band,
        "bands": _state["bands"],
        "members": _names(_state["score_members"]),
        "uncertainty_members": _names(_state["unc_members"]),
        "disclaimer": "Alat triase skrining – bukan diagnosis. Wajib konfirmasi (GeneXpert) oleh tenaga kesehatan.",
        "heatmap_png_b64": heatmap_b64,
    }

```

ml/kaggle/out_ssl/loop-tb/ml/data/prepare_data.py

```
#!/usr/bin/env python3
"""Build a provenance-aware manifest for TB chest X-ray training.

Sumber & aturan label (lihat docs/02-arsitektur.md):
- TBX11K (Kaggle: vbookshelf/tbx11k-simplified): folder per kelas.
  positif = {tb}; negatif = {healthy, sick}; latent_tb DIKELUARKAN (ambigu).
- Rahman composite (Kaggle: tawsifurrahman/tuberculosis-tb-chest-xray-dataset):
  folder Normal/ -> 0, Tuberculosis/ -> 1.
- NLM Shenzhen/Montgomery: konvensi nama file resmi `*_0` = normal, `*_1` = TB.

Dedup (WAJIB – mitigasi class-conditional acquisition confounding):
1. pHash 64-bit; Hamming <= --dup-threshold dianggap duplikat (union-find transitif).
2. Integritas external: citra train yang sekelompok dengan citra external DIBUANG dari train.
3. Duplikat internal train dibuang (keep-first, urutan deterministik).
4. Split val dilakukan per-klaster (tidak ada kebocoran near-dup train->val).

Output (ml/data/artifacts/):
- manifest.json, dedup_report.json, train.csv, val.csv, external.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import re
from collections import Counter
from pathlib import Path

import numpy as np
from PIL import Image
from tqdm import tqdm

try:
    import imagehash
except ImportError: # pragma: no cover
    raise SystemExit("Butuh imagehash: pip install imagehash")

IMG_EXTS = {".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff"}
POS_TOKENS = {"tb", "tuberculosis", "tuberculose"}
NEG_TOKENS = {"healthy", "normal", "sick"}
EXCLUDE_TOKENS = {"latent", "latent_tb", "latenttb"}

def _rel(p: Path, root: Path) -> str:
    """Path relatif terhadap root repo (portable lokal <-> Colab); fallback absolut."""
    try:
        return str(p.relative_to(root))
    except ValueError:
        return str(p)

def _tokens(path: Path) -> set[str]:
    parts = [p.lower() for p in path.parts]
    parts += [path.stem.lower()]
    return set(parts)

def load_tbx11k_map(raw: Path) -> dict[str, int | None]:
    """Label otoritatif dari data.csv TBX11K: 1=active_tb, 0=no_tb, None=latent (dikeluarkan)."""
    m: dict[str, int | None] = {}
    for p in raw.rglob("data.csv"):
        if "tbx11k" not in str(p).lower():
            continue
        for r in csv.DictReader(p.open()):
            f = (r.get("fname") or "").strip()
            if not f:
                continue
            t = (r.get("tb_type") or "").lower()
            tgt = (r.get("target") or "").lower()
            if "latent" in t:
                m.setdefault(f, None)
            elif tgt == "tb":
                m[f] = 1
            else:
                m.setdefault(f, 0)
    return m

def classify(path: Path, raw: Path, tbx_map: dict[str, int | None]) -> tuple[int, str] | None:
    """Return (label, source) atau None kalau tidak bisa diklasifikasi."""
    rel = path.relative_to(raw)
    low = str(rel).lower()

    # --- NLM Shenzhen / Montgomery: konvensi suffix nama file ---
    m = re.search(r"(chncxr|mcucxr)_d+_(\d+)", path.stem.lower())
    if m:
        label = int(m.group(2))
        source = "shenzhen" if m.group(1) == "chncxr" else "montgomery"
```

```

        return label, source

toks = _tokens(rel)
if toks & EXCLUDE_TOKENS or any(t.startswith("latent") for t in toks):
    return None

if "tbx11k" in low:
    if tbx_map:
        lab = tbx_map.get(path.name, None)
        return None if lab is None else (lab, "tbx11k")
    # fallback bila data.csv tidak ada: prefix nama file
    stem = path.stem.lower()
    if stem.startswith("t"):
        return 1, "tbx11k"
    if stem.startswith(("h", "s")):
        return 0, "tbx11k"
    return None

if toks & POS_TOKENS:
    return 1, "rahman"
if toks & NEG_TOKENS:
    return 0, "rahman"
return None

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[rb] = ra

_POP = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def hamming_pairs(hashes: np.ndarray, threshold: int, chunk: int = 512) -> list[tuple[int, int]]:
    """Semua pasangan (i<j) dengan Hamming distance <= threshold (chunked, hemat memori)."""
    n = len(hashes)
    h8 = hashes.view(np.uint8).reshape(n, 8)
    pairs: list[tuple[int, int]] = []
    for start in range(0, n, chunk):
        stop = min(start + chunk, n)
        xor = np.bitwise_xor(h8[start:stop, None, :], h8[None, :, :])
        dist = _POP[xor].sum(axis=-1) # (chunk, n)
        rows, cols = np.where(dist <= threshold)
        for r, c in zip(rows, cols):
            i, j = start + int(r), int(c)
            if i < j:
                pairs.append((i, j))
    return pairs

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--raw", type=Path, default=Path("data/raw"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts"))
    ap.add_argument("--dup-threshold", type=int, default=3, help="Hamming pHash <= ini dianggap duplikat")
    ap.add_argument("--val-frac", type=float, default=0.15)
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    raw = args.raw.resolve()
    root = raw.parent.parent # asumsi struktur <root>/data/raw

    # ----- 1) discovery -----
    labeled: list[dict] = []
    unlabeled = Counter()
    tbx_map = load_tbx11k_map(raw)
    print(f"[discovery] label TBX11K dari data.csv: {len(tbx_map)} entri")
    for p in sorted(raw.rglob("*")):
        if p.suffix.lower() not in IMG_EXTS:
            continue
        cls = classify(p, raw, tbx_map)
        if cls is None:
            unlabeled[str(p.relative_to(raw).parent)] += 1
            continue
        label, source = cls
        labeled.append({"path": _rel(p, root), "label": label, "source": source})

    print(f"[discovery] berlabel={len(labeled)} tak-terklasifikasi={sum(unlabeled.values())}")
    for k, v in unlabeled.most_common(10):
        print(f" skip {v:6d} {k}")

```

```

if not labeled:
    raise SystemExit("Tidak ada citra berlabel – cek struktur data/raw")

# ----- 2) pHash + kluster duplikat -----
print("[dedup] menghitung pHash...")
ok: list[dict] = []
for item in tqdm(labeled, unit="img"):
    try:
        with Image.open(item["path"]) as im:
            h = imagehash.phash(im.convert("L"), hash_size=8)
            item["phash"] = int(str(h), 16)
            ok.append(item)
    except Exception as e: # gambar korup
        print(f" warn: gagal baca {item['path']}: {e}")
labeled = ok

hashes = np.array([it["phash"] for it in labeled], dtype=np.uint64)
pairs = hamming_pairs(hashes, args.dup_threshold)
dsu = DSU(len(labeled))
for i, j in pairs:
    dsu.union(i, j)
for idx, it in enumerate(labeled):
    it["cluster"] = dsu.find(idx)

by_cluster: dict[int, list[dict]] = {}
for it in labeled:
    by_cluster.setdefault(it["cluster"], []).append(it)

# ----- 3) lock external, buang kebocoran dari train -----
ext = [it for it in labeled if it["source"] in ("shenzhen", "montgomery")]
ext_clusters = {it["cluster"] for it in ext}
train_pool, dropped_cross = [], []
for it in labeled:
    if it["source"] in ("shenzhen", "montgomery"):
        continue
    (dropped_cross if it["cluster"] in ext_clusters else train_pool).append(it)

# duplikat internal train: keep-first per kluster (deterministik via path)
keep, dropped_internal = [], []
seen_clusters: set[int] = set()
for it in sorted(train_pool, key=lambda x: x["path"]):
    if it["cluster"] in seen_clusters:
        dropped_internal.append(it)
    else:
        seen_clusters.add(it["cluster"])
        keep.append(it)
train_pool = keep

# ----- 4) split val per-kluster -----
rng = np.random.default_rng(args.seed)
sizes = Counter(it["cluster"] for it in train_pool)
pool_clusters = sorted(sizes)
rng.shuffle(pool_clusters)
target_val = int(len(train_pool) * args.val_frac)
val_clusters: set[int] = set()
acc = 0
for c in pool_clusters:
    if acc >= target_val:
        break
    val_clusters.add(c)
    acc += sizes[c]
train = [it for it in train_pool if it["cluster"] not in val_clusters]
val = [it for it in train_pool if it["cluster"] in val_clusters]

def dump(items: list[dict]) -> list[dict]:
    return [
        {k: it[k] for k in ("path", "label", "source", "cluster", "phash")}
        for it in sorted(items, key=lambda x: x["path"])
    ]

manifest = {
    "meta": {
        "root": str(root),
        "raw": _rel(raw, root),
        "dup_threshold": args.dup_threshold,
        "val_frac": args.val_frac,
        "seed": args.seed,
        "counts": {
            "train": len(train), "val": len(val), "external": len(ext),
            "dropped_cross_corpus": len(dropped_cross),
            "dropped_internal_dup": len(dropped_internal),
            "unlabeled": sum(unlabeled.values()),
        },
    },
    "train": dump(train),
    "val": dump(val),
    "external": dump(ext),
}
(args.out / "manifest.json").write_text(json.dumps(manifest, indent=1))
(args.out / "dedup_report.json").write_text(json.dumps({
    "pairs_within_threshold": len(pairs),
    "clusters": len(by_cluster),
}))

```

```

        "dropped_cross_corpus": dump(dropped_cross),
        "dropped_internal_dup": dump(dropped_internal),
        "unlabeled_by_dir": dict(unlabeled),
    }, indent=1))

for split in ("train", "val", "external"):
    with (args.out / f"{split}.csv").open("w") as f:
        f.write("path,label,source,cluster\n")
        for it in manifest[split]:
            f.write(f"{it['path']},{it['label']},{it['source']},{it['cluster']}\n")

def comp(items):
    c = Counter((it["source"], it["label"]) for it in items)
    return " ".join(f"{s}/{ 'TB' if l else 'NEG' }={n}" for (s, l), n in sorted(c.items()))

print("\n=== RINGKASAN ===")
print(f"train      n={len(train):6d}  {comp(train)}")
print(f"val        n={len(val):6d}    {comp(val)}")
print(f"external  n={len(ext):6d}    {comp(ext)}")
print(f"dibuang  cross-corpus={len(dropped_cross)}  internal-dup={len(dropped_internal)}")
print(f"manifest -> {args.out/'manifest.json'}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl/loop-tb/ml/data/prepare_meta.py

```
#!/usr/bin/env python3
"""Ekstrak metadata klinis NLM (Montgomery + Shenzhen) → CSV untuk analisis subgroup.

Output: ml/data/artifacts/meta_nlm.csv
Kolom: filename,sex,age,reading,scar_flag
- Montgomery: "Patient's Sex: F / Patient's Age: 027Y / <reading>"
- Shenzhen: "male 45yrs / <reading>" (kadang terkontaminasi antar-file)
- scar_flag: menyebut scar/inactive/fibrosis → untuk pembahasan mode gagal
            "healed scar false positive" (selaras audit Bilal 2026).
"""
from __future__ import annotations

import argparse
import csv
import re
from pathlib import Path

SEX_M = re.compile(r"sex[:\s]+(m|male)\b", re.I)
SEX_F = re.compile(r"sex[:\s]+(f|female)\b", re.I)
SEX_FMT2 = re.compile(r"\b(male|female)\b", re.I)
AGE_MONT = re.compile(r"age[:\s]+(\d{1,3})\s*Y?\b", re.I)
AGE_FMT2 = re.compile(r"\b(\d{1,3})\s*yrs\b", re.I)
SCAR = re.compile(r"scar|inactive|fibro", re.I)

def parse_montgomery(text: str) -> dict | None:
    age = AGE_MONT.search(text)
    sex = "F" if SEX_F.search(text) else ("M" if SEX_M.search(text) else "")
    if not age:
        return None
    # reading = baris setelah Age
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    reading = lines[2] if len(lines) > 2 else ""
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": int(bool(SCAR.search(reading)))}

def parse_shenzhen(text: str) -> dict | None:
    age = AGE_FMT2.search(text)
    sex_m = SEX_FMT2.findall(text)
    sex = sex_m[0][0].upper() if sex_m else ""
    if not age:
        return None
    # reading = token huruf setelah "yrs"
    tail = text[age.end():]
    reading = ""
    m = re.search(r"[A-Za-z]{2,}", tail)
    if m:
        reading = m.group(0)
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": 0}

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--nlm", type=Path, default=Path("data/raw/nlm"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts/meta_nlm.csv"))
    args = ap.parse_args()
    args.out.parent.mkdir(parents=True, exist_ok=True)

    rows, n_fail = [], 0
    for corpus, parser in (("montgomery", parse_montgomery), ("shenzhen", parse_shenzhen)):
        d = args.nlm / corpus / "ClinicalReadings"
        for p in sorted(d.glob("*.txt")):
            r = parser(p.read_text(errors="ignore"))
            if r is None:
                n_fail += 1
                continue
            r["filename"] = p.stem + ".png"
            r["corpus"] = corpus
            rows.append(r)

    with args.out.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["filename", "corpus", "sex", "age", "reading", "scar_flag"])
        w.writeheader()
        w.writerows(rows)

    from collections import Counter
    print(f"meta -> {args.out} n={len(rows)} gagal-parse={n_fail}")
    print("per korpus:", Counter(r["corpus"] for r in rows))
    print("sex:", Counter(r["sex"] for r in rows))
    print("scar_flag:", Counter(r["scar_flag"] for r in rows))

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl/loop-tb/ml/evaluate.py

```
#!/usr/bin/env python3
"""Evaluasi rigor: kalibrasi, CI bootstrap, transfer, band triase, subgroup.

Contoh (ensemble + temperature scaling):
python ml/evaluate.py --runs ml/runs/efficientnet_b0 ml/runs/convnext_tiny \
    --manifest ml/data/artifacts/manifest.json --split external --temp-scale --out ml/runs/eval_external
"""
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import brier_score_loss, roc_auc_score, roc_curve
from torch.utils.data import DataLoader
from tqdm import tqdm

import sys
sys.path.insert(0, str(Path(__file__).resolve().parent))
from train import CXRManifest # noqa: E402

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-x))

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 -> fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

def threshold_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    fpr, tpr, thr = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(thr[ok][np.argmax(tpr[ok])]) if ok.any() else 0.5

def ece(y: np.ndarray, p: np.ndarray, bins: int = 15) -> float:
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for i in range(bins):
        m = (p >= edges[i]) & ((p < edges[i + 1]) if i < bins - 1 else (p <= edges[i + 1]))
        if m.sum():
            total += m.mean() * abs(p[m].mean() - y[m].mean())
    return float(total)

def bootstrap_auroc(y: np.ndarray, p: np.ndarray, n: int = 1000, seed: int = 42) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    vals = []
    for _ in range(n):
        idx = rng.integers(0, len(y), len(y))
        if len(np.unique(y[idx])) > 1:
            vals.append(roc_auc_score(y[idx], p[idx]))
    return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))

@torch.no_grad()
def infer(ckpt_path: Path, items: list[dict], img_size: int, device: torch.device, workers: int) -> np.ndarray:
    ckpt = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    arch = ckpt["arch"]
    model = timm.create_model(arch, pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.to(device).eval()
    dl = DataLoader(CXRManifest(items, train=False, img_size=img_size), batch_size=64,
                    shuffle=False, num_workers=workers)

    out = []
    for x, _, _ in tqdm(dl, desc=f"infer {ckpt_path.parent.name}", leave=False):
        x = x.to(device)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            out.append(model(x).squeeze(1).float().cpu().numpy())
    return np.concatenate(out)
```

```

def fit_temperature(val_logits: list[np.ndarray], val_y: np.ndarray) -> float:
    """Cari T>0 yang meminimalkan NLL pada val (grid log-space lebar untuk logit besar)."""
    logits = torch.tensor(np.mean(val_logits, axis=0), dtype=torch.float32)
    y = torch.tensor(val_y, dtype=torch.float32)
    best_t, best_nll = 1.0, float("inf")
    for t in np.logspace(-1, 3.5, 500):
        nll = nn.functional.binary_cross_entropy_with_logits(logits / t, y).item()
        if nll < best_nll:
            best_t, best_nll = float(t), nll
    return best_t

def reliability_plot(y: np.ndarray, p: np.ndarray, path: Path, title: str) -> None:
    edges = np.linspace(0, 1, 11)
    mids, accs = [], []
    for i in range(10):
        m = (p >= edges[i]) & (p < edges[i + 1] if i < 9 else p <= edges[i + 1])
        if m.sum():
            mids.append(p[m].mean())
            accs.append(y[m].mean())
    plt.figure(figsize=(4.2, 4.2))
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.plot(mids, accs, "o-", color="#d62728", lw=2)
    plt.xlabel("Confidence"), plt.ylabel("Empirical TB rate"), plt.title(title)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def roc_plot(curves: dict[str, tuple[np.ndarray, np.ndarray]], path: Path) -> None:
    plt.figure(figsize=(5, 5))
    for name, (y, p) in curves.items():
        fpr, tpr, _ = roc_curve(y, p)
        plt.plot(fpr, tpr, lw=2, label=f"{name} (AUROC {roc_auc_score(y, p):.3f})")
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.xlabel("FPR"), plt.ylabel("TPR (sensitivitas)"), plt.legend(fontsize=8)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--runs", type=Path, nargs="+", required=True)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--split", default="external", choices=["val", "external"])
    ap.add_argument("--temp-scale", action="store_true")
    ap.add_argument("--meta", type=Path, default=None, help="CSV filename,sex,age untuk subgroup (opsional)")
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--workers", type=int, default=8)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    manifest = json.loads(args.manifest.read_text())
    items = manifest[args.split]
    y = np.array([it["label"] for it in items])

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    per_run_target, per_run_val, tscales = {}, {}, {}
    for run in args.runs:
        ckpt = torch.load(run / "best.pt", map_location="cpu", weights_only=False)
        img_size = ckpt["config"]["img_size"]
        per_run_target[run.name] = infer(run / "best.pt", items, img_size, device, args.workers)
        val_items = manifest["val"]
        per_run_val[run.name] = infer(run / "best.pt", val_items, img_size, device, args.workers)
        if args.temp_scale:
            tscales[run.name] = fit_temperature([per_run_val[run.name]], np.array([it["label"] for it in val_items]))

    per_run_target = {k: sigmoid(v) for k, v in per_run_target.items()}
    per_run_val = {k: sigmoid(v) for k, v in per_run_val.items()}
    val_y = np.array([it["label"] for it in manifest["val"]])

    # kalibrasi per-run pada val, lalu terapkan T ke target (bukan val-fit pada target!)
    if args.temp_scale:
        for k in per_run_target:
            per_run_val[k] = sigmoid(np.log(np.clip(per_run_val[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_val[k], 1e-6, 1 - 1e-6)))
            / tscales[k])
            lg = np.log(np.clip(per_run_target[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_target[k], 1e-6, 1 - 1e-6)))
            per_run_target[k] = sigmoid(lg / tscales[k])

    # ensemble = rata-rata; ketidakpastian = std antar anggota
    stack_t = np.stack(list(per_run_target.values()))
    p_ens = stack_t.mean(axis=0)
    u_ens = stack_t.std(axis=0)
    stack_v = np.stack(list(per_run_val.values()))
    p_ens_v = stack_v.mean(axis=0)

    tau_high = threshold_at_spec(val_y, p_ens_v, 0.70) # spec>=0.70 (TPP WHO)
    tau_low = threshold_at_spec(val_y, p_ens_v, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    band = np.where(p_ens >= tau_high, "rujuk_prioritas", np.where(p_ens < tau_low, "negatif_skrining", "ragu"))
    band_report = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b

```

```

        band_report[b] = {"n": int(m.sum()), "frac": float(m.mean()),
                          "tb_rate": float(y[m].mean()) if m.sum() else None}

lo, hi = bootstrap_aucroc(y, p_ens)
metrics = {
    "split": args.split, "n": int(len(y)), "pos": int(y.sum()),
    "aucroc": float(roc_auc_score(y, p_ens)), "aucroc_ci95": [lo, hi],
    "sens_at_spec_90": sens_at_spec(y, p_ens, 0.90),
    "sens_at_spec_70": sens_at_spec(y, p_ens, 0.70),
    "ece": ece(y, p_ens), "brier": float(brier_score_loss(y, p_ens)),
    "temp_scales": tscales, "tau_low": tau_low, "tau_high": tau_high,
    "bands": band_report,
    "deferral": {"rate": float((band == "ragu").mean()),
                 "tb_rate_on_deferred": float(y[band == "ragu"].mean()) if (band == "ragu").any() else None},
    "per_run_aucroc": {k: float(roc_auc_score(y, v)) for k, v in per_run_target.items()},
}

if args.meta and args.meta.exists():
    meta = {r["filename"]: r for r in csv.DictReader(args.meta.open())}
    for key in ("sex", "age"):
        groups: dict[str, list[int]] = {}
        for i, it in enumerate(items):
            row = meta.get(Path(it["path"]).name, {})
            g = row.get(key, "NA") or "NA"
            if key == "age":
                try:
                    a = float(g)
                    g = "0-30" if a < 30 else "30-50" if a < 50 else "50+"
                except ValueError:
                    g = "NA"
            groups.setdefault(g, []).append(i)
        metrics[f"aucroc_by_{key}"] = {
            g: (float(roc_auc_score(y[idx], p_ens[idx])) if len(np.unique(y[idx])) > 1 else None)
              for g, idx in groups.items() if g != "NA" and len(idx) >= 20
        }

(args.out / "metrics.json").write_text(json.dumps(metrics, indent=1))
# dump prediksi per-citra (untuk analisis lanjutan & stacking)
import csv as _csv
with (args.out / "preds.csv").open("w", newline="") as f:
    w = _csv.writer(f)
    w.writerow(["path", "label", "p_ens", "u_ens"])
    for it, p, u in zip(items, p_ens, u_ens):
        w.writerow([it["path"], it["label"], float(p), float(u)])
reliability_plot(y, p_ens, args.out / "reliability.png", f"Kalibrasi - {args.split}")
curves = {f"ens ({args.split})": (y, p_ens)}
if len(per_run_target) > 1:
    curves.update({k: (y, v) for k, v in per_run_target.items()})
roc_plot(curves, args.out / "roc.png")

print("\n=== METRICS ===")
for k, v in metrics.items():
    if isinstance(v, (int, float, str)):
        print(f"{k:18s} {v}")
print(f"bands          {band_report}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl/loop-tb/ml/export_onnx.py

```
#!/usr/bin/env python3
"""Export checkpoint ke ONNX (opsi 17) + verifikasi onnxruntime.

Contoh:
python ml/export_onnx.py --ckpt ml/runs/efficientnet_b0/best.pt --out artifacts/tb_efficientnet_b0.onnx
"""
from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import torch

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm")

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--ckpt", type=Path, required=True)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--opset", type=int, default=17)
    args = ap.parse_args()

    ckpt = torch.load(args.ckpt, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()

    img_size = ckpt["config"].get("img_size", 224)
    dummy = torch.randn(1, 3, img_size, img_size)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    torch.onnx.export(
        model, dummy, args.out,
        input_names=["input"], output_names=["logit"],
        dynamic_axes={"input": {0: "batch"}, "logit": {0: "batch"}},
        opset_version=args.opset, dynamo=False, # legacy: satu file mandiri, opset stabil
    )

    import onnxruntime as ort
    sess = ort.InferenceSession(str(args.out), providers=["CPUExecutionProvider"])
    ref = model(dummy).detach().numpy()
    got = sess.run(None, {"input": dummy.numpy()})[0]
    # Verifikasi di ruang probabilitas (deployment-relevant); logit bisa sangat besar
    # pada model overkonfiden sehingga toleransi absolut di ruang logit tidak adil.
    sig = lambda z: 1 / (1 + np.exp(-z)) # noqa: E731
    prob_diff = float(np.abs(sig(ref) - sig(got)).max())
    size_mb = args.out.stat().st_size / 1e6
    print(f"ONNX ok: {args.out} ({size_mb:.1f} MB) max|Δprob|={prob_diff:.2e} opset={args.opset}")
    if prob_diff > 1e-3:
        print(f"WARN: perbedaan probabilitas {prob_diff:.4f} > 1e-3 – periksa sebelum produksi")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl/loop-tb/ml/kaggle/hello/hello.py

```
"""Kernel uji: verifikasi GPU, internet, versi lib, dan struktur dataset ter-mount."""
import os
import sys

lines: list[str] = []

def log(*a) -> None:
    print(*a, flush=True)
    lines.append(" ".join(str(x) for x in a))

log("python:", sys.version.split()[0])

try:
    import torch

    log("torch:", torch.__version__, "| cuda:", torch.cuda.is_available())
    if torch.cuda.is_available():
        log("gpu:", torch.cuda.get_device_name(0))
except Exception as e: # noqa: BLE001
    log("torch error:", e)

for mod in ("timm", "sklearn", "onnx", "onnxruntime"):
    try:
        m = __import__(mod)
        log(f"{mod}: {getattr(m, '__version__', '?')}")
    except Exception as e: # noqa: BLE001
        log(f"{mod}: MISSING ({e})")

log("--- input mounts ---")
for root, dirs, _ in os.walk("/kaggle/input"):
    log(root, "dirs:", dirs[:6])
    if root.count("/") > 3:
        dirs[:] = []

log("HELLO DONE")
with open("/kaggle/working/hello_result.txt", "w") as f:
    f.write("\n".join(lines) + "\n")
```

ml/kaggle/out_ssl/loop-tb/ml/kaggle/make_nlm_dataset.py

```
#!/usr/bin/env python3
"""Siapkan dataset Kaggle privat: NLM CXR di-resize (max dim 512) + ClinicalReadings.

Dipisah dari data asli agar unggahan ringan (~150 MB) dan resolusi tetap cukup
untuk encoder SSL (DINOv2/RAD-DINO resize ke 518).
"""
from __future__ import annotations

import shutil
from pathlib import Path

from PIL import Image

SRC = Path("data/raw/nlm")
DST = Path("ml/kaggle/nlm_dataset/nlm")

def main() -> None:
    n = 0
    for corpus in ("shenzhen", "montgomery"):
        out_dir = DST / corpus / "CXR_png"
        out_dir.mkdir(parents=True, exist_ok=True)
        for p in sorted((SRC / corpus / "CXR_png").glob("*.png")):
            with Image.open(p) as im:
                im = im.convert("L")
                w, h = im.size
                scale = 512 / max(w, h)
                if scale < 1:
                    im = im.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
                im.save(out_dir / (p.stem + ".jpg"), quality=92)
            n += 1
        readings = SRC / corpus / "ClinicalReadings"
        if readings.exists():
            out_r = DST / corpus / "ClinicalReadings"
            out_r.mkdir(parents=True, exist_ok=True)
            for f in readings.glob("*.txt"):
                shutil.copy(f, out_r / f.name)
        print(f"selesai: {n} citra -> {DST}")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl/loop-tb/ml/kaggle/ssl/ssl_extract.py

```
"""Ekstraksi embedding SSL untuk studi representasi (ADR-007).

Encoder:
- dinov2 : timm vit_base_patch14_dinov2.lvd142m (SSL umum, 518 px)
- rad_dino: hf-hub:microsoft/rad-dino (DINOv2 pada X-ray dada, 518 px)

Output di /kaggle/working/loop-tb/ssl_embeddings/:
meta_{train,val,external}.json + emb_{encoder}_{split}.npy
Dipakai oleh ml/ssl_probe.py secara lokal (sklearn, tanpa GPU).
"""
from __future__ import annotations

import json
import os
import subprocess
import sys

import numpy as np

WORK = "/kaggle/working"
REPO = f"{WORK}/loop-tb"

def sh(cmd: list[str]) -> None:
    print("+", " ".join(map(str, cmd)), flush=True)
    subprocess.run(cmd, check=True)

if not os.path.exists(REPO):
    sh(["git", "clone", "--depth", "1", "https://github.com/Alvnc/loop-tb.git", REPO])
os.chdir(REPO)
sh([sys.executable, "-m", "pip", "install", "-q", "timm", "huggingface_hub", "transformers"])

# --- mount datasets (self-healing) ---
os.makedirs("data/raw", exist_ok=True)

def pick(dst: str, cands: list[str], markers: tuple[str, ...]) -> None:
    if os.path.exists(dst):
        return
    for c in cands:
        if any(os.path.isdir(os.path.join(c, m)) for m in markers):
            os.symlink(c, dst)
            print("symlink", dst, "->", c, flush=True)
            return
    for c in cands:
        if os.path.exists(c):
            os.symlink(c, dst)
            print("symlink (fallback)", dst, "->", c, flush=True)
            return
    print("WARNING: tidak ada mount untuk", dst, flush=True)

pick("data/raw/tbx11k-simplified",
     ["/kaggle/input/datasets/vbookshelf/tbx11k-simplified/tbx11k-simplified",
      "/kaggle/input/datasets/vbookshelf/tbx11k-simplified", "/kaggle/input/tbx11k-simplified"],
     ("images",))
pick("data/raw/TB_Chest_Radiography_Database",
     ["/kaggle/input/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
      "/kaggle/input/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database"],
     ("Normal", "Tuberculosis"))
pick("data/raw/nlm",
     ["/kaggle/input/datasets/alvinreba/nlm-tb-cxr-resized/nlm",
      "/kaggle/input/datasets/alvinreba/nlm-tb-cxr-resized",
      "/kaggle/input/nlm-tb-cxr-resized/nlm",
      "/kaggle/input/nlm-tb-cxr-resized"],
     ("shenzhen", "montgomery"))

# --- siapkan daftar item (external: .png -> .jpg hasil resize) ---
manifest = json.load(open("ml/data/artifacts/manifest.json"))
train_items = manifest["train"]
val_items = manifest["val"]
ext_items = []
for it in manifest["external"]:
    p = it["path"]
    jp = p[:-4] + ".jpg" if p.lower().endswith(".png") else p
    ext_items.append({**it, "path": jp})

for name, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
    missing = [it["path"] for it in items if not os.path.exists(it["path"])]
    print(f"{name}: n={len(items)} missing={len(missing)}", missing[:2], flush=True)
    assert not missing, f"data {name} tidak lengkap"

out_dir = "ssl_embeddings"
os.makedirs(out_dir, exist_ok=True)
for split, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
    with open(f"{out_dir}/meta_{split}.json", "w") as f:
        json.dump([{"path": it["path"], "label": it["label"], "source": it["source"]} for it in items], f)
```

```

# --- ekstraksi ---
import torch # noqa: E402
from PIL import Image # noqa: E402
from torch.utils.data import DataLoader, Dataset # noqa: E402
from torchvision import transforms as T # noqa: E402

MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

class Items(Dataset):
    def __init__(self, items: list[dict], size: int, mean: tuple = MEAN, std: tuple = STD):
        self.items = items
        self.tx = T.Compose([T.Resize(size), T.CenterCrop(size), T.ToTensor(), T.Normalize(mean, std)])

    def __len__(self) -> int:
        return len(self.items)

    def __getitem__(self, i: int):
        with Image.open(self.items[i]["path"]) as im:
            x = self.tx(im.convert("RGB"))
        return x, i

@torch.no_grad()
def extract(model, items: list[dict], size: int, device: torch.device, tag: str,
            mean: tuple = MEAN, std: tuple = STD) -> np.ndarray:
    dl = DataLoader(Items(items, size, mean, std), batch_size=32, shuffle=False, num_workers=4, pin_memory=True)
    out = np.zeros((len(items), 0), dtype=np.float32)
    embs = []
    for bi, (x, _) in enumerate(dl):
        x = x.to(device, non_blocking=True)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            feats = model(x)
            feats = feats.float().cpu().numpy()
            embs.append(feats)
        if bi % 20 == 0:
            print(f" {tag} batch {bi}/{len(dl)}", flush=True)
    out = np.concatenate(embs, axis=0)
    return out

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device, flush=True)

ENCODERS = [
    ("dinov2", "vit_base_patch14_dinov2.lvd142m", 518),
    ("rad_dino", "hf-hub:microsoft/rad-dino", 518),
]

import timm # noqa: E402

for enc_name, model_id, size in ENCODERS:
    if all(os.path.exists(f"{out_dir}/emb_{enc_name}_{s}.npy") for s in ("train", "val", "external")):
        print(f"skip {enc_name} (sudah ada)", flush=True)
        continue
    try:
        print(f"=== encoder {enc_name} ({model_id}) ===", flush=True)
        mean, std = MEAN, STD
        if enc_name == "rad_dino":
            from transformers import AutoImageProcessor, AutoModel

            hf_id = model_id.replace("hf-hub:", "")

            class HFWrap(torch.nn.Module):
                def __init__(self, m):
                    super().__init__()
                    self.m = m

                def forward(self, x):
                    out = self.m(pixel_values=x)
                    return out.last_hidden_state[:, 0] if hasattr(out, "last_hidden_state") else out.pooler_output

            proc = AutoImageProcessor.from_pretrained(hf_id)
            print("rad_dino processor:", proc.image_mean, proc.image_std, proc.size, flush=True)
            if proc.image_mean:
                mean, std = tuple(proc.image_mean), tuple(proc.image_std)
            model = HFWrap(AutoModel.from_pretrained(hf_id))
        else:
            model = timm.create_model(model_id, pretrained=True, num_classes=0, dynamic_img_size=True)
        model.to(device).eval()
        for split, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
            emb = extract(model, items, size, device, f"{enc_name}/{split}", mean, std)
            np.save(f"{out_dir}/emb_{enc_name}_{split}.npy", emb)
            print(f"{enc_name} {split}: {emb.shape}", flush=True)
        with open(f"{out_dir}/encmeta_{enc_name}.json", "w") as f:
            json.dump({
                "type": "hf" if enc_name == "rad_dino" else "timm",
                "id": model_id.replace("hf-hub:", ""),
                "size": size,
                "mean": list(mean),
                "std": list(std),
            }, f)
    except Exception as e:
        print(f"Error processing {enc_name} {model_id}: {e}")

```

```
    }, f)
    del model
    torch.cuda.empty_cache()
except Exception as e: # noqa: BLE001
    print(f"ERROR encoder {enc_name}: {e}", flush=True)

print("SSL EXTRACT DONE", flush=True)
```

ml/kaggle/out_ssl/loop-tb/ml/kaggle/train/train_kernel.py

```
"""Kernel training loop-tb: clone repo → symlink dataset → train 2 arsitektur + baseline → export ONNX.

Output di /kaggle/working/loop-tb/{runs,artifacts}; diunduh via `kaggle kernels output`.
Manifest yang dipakai = split v1 frozen dari repo (deterministik; NLM external tidak dibutuhkan di sini).
"""
import json
import os
import subprocess
import sys

WORK = "/kaggle/working"
REPO = f"{WORK}/loop-tb"

def sh(cmd: list[str], **kw) -> None:
    print("+", " ".join(map(str, cmd)), flush=True)
    subprocess.run(cmd, check=True, **kw)

# 1) repo + deps
if not os.path.exists(REPO):
    sh(["git", "clone", "--depth", "1", "https://github.com/Alvnc/loop-tb.git", REPO])
os.chdir(REPO)
sh([sys.executable, "-m", "pip", "install", "-q", "timm", "onnxruntime"])

# 2) symlink dataset ke struktur yang diharapkan manifest (tahan variasi struktur mount)
os.makedirs("data/raw", exist_ok=True)

def pick(dst: str, cands: list[str], markers: tuple[str, ...]) -> None:
    """Symlink kandidat mount pertama yang punya 'marker' (images/Normal/Tuberculosis)."""
    if os.path.exists(dst):
        return
    for c in cands:
        if any(os.path.isdir(os.path.join(c, m)) for m in markers):
            os.symlink(c, dst)
            print("symlink", dst, "->", c, flush=True)
            return
    for c in cands:
        if os.path.exists(c):
            os.symlink(c, dst)
            print("symlink (fallback)", dst, "->", c, flush=True)
            return
    print("WARNING: tidak ada kandidat mount untuk", dst, flush=True)

TBX_CANDS = [
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified",
]
RAH_CANDS = [
    "/kaggle/input/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
    "/kaggle/input/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
]
pick("data/raw/tbx11k-simplified", TBX_CANDS, ("images",))
pick("data/raw/TB_Chest_Radiography_Database", RAH_CANDS, ("Normal", "Tuberculosis"))

for root, dirs, files in os.walk("/kaggle/input"):
    print("mount:", root, "dirs:", dirs[:6], "files:", len(files), flush=True)
    if root.count("/") > 4:
        dirs[:] = []

# 3) sanity: semua path train/val harus ada
manifest = json.load(open("ml/data/artifacts/manifest.json"))
missing = [it["path"] for s in ("train", "val") for it in manifest[s] if not os.path.exists(it["path"])]
print(f"train={len(manifest['train'])} val={len(manifest['val'])} missing={len(missing)}", flush=True)
print("missing contoh:", missing[:3], flush=True)
assert not missing, "data train/val tidak lengkap – cek mount dataset"

# 4) training (dua arsitektur + baseline fitur beku)
JOBS = [
    ("efficientnet_b0", "runs/eb0", ["--epochs", "14", "--batch", "64"]),
    ("convnext_tiny", "runs/convnext", ["--epochs", "12", "--batch", "48"]),
]
for arch, out, extra in JOBS:
    sh(
        [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
         "--arch", arch, "--balanced", "--out", out, *extra]
    )

sh(
    [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
     "--arch", "efficientnet_b0", "--freeze", "--lr", "1e-3", "--epochs", "8", "--batch", "64",
     "--out", "runs/linear_probe"]
)
```

```
# 5) export ONNX (satu file mandiri per model)
os.makedirs("artifacts", exist_ok=True)
for run, name in (("runs/eb0", "eb0"), ("runs/convnext", "convnext")):
    sh([sys.executable, "ml/export_onnx.py", "--ckpt", f"{run}/best.pt", "--out", f"artifacts/{name}.onnx"])

print("ALL DONE", flush=True)
```

ml/kaggle/out_ssl/loop-tb/ml/ssl_probe.py

```
#!/usr/bin/env python3
"""Latih probe linear di atas embedding SSL + evaluasi eksternal rigor (ADR-007).

Contoh:
.venv/bin/python ml/ssl_probe.py --emb-dir ml/kaggle/out_ssl/ssl_embeddings \
  --out ml/runs/ssl_eval --supervised-preds ml/runs/eval_external_ens/preds.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import sys
from pathlib import Path

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss, roc_auc_score
from sklearn.preprocessing import StandardScaler

sys.path.insert(0, str(Path(__file__).resolve().parent))
from evaluate import ( # noqa: E402
    bootstrap_auroc,
    ece,
    fit_temperature,
    reliability_plot,
    roc_plot,
    sens_at_spec,
    threshold_at_spec,
)

# Statistik preprocessing piksel yang HARUS sama dengan saat ekstraksi embedding.
# (dinov2: default timm/ImageNet; rad_dino: diisi dari log kernel – processor HF)
PIXEL = {
    "dinov2": {
        "type": "timm", "id": "vit_base_patch14_dinov2.lvd142m", "size": 518,
        "mean": (0.485, 0.456, 0.406), "std": (0.229, 0.224, 0.225),
    },
    "rad_dino": {
        "type": "hf", "id": "microsoft/rad-dino", "size": 518,
        "mean": (0.5307, 0.5307, 0.5307), "std": (0.2588, 0.2588, 0.2588), # diverifikasi dari log kernel; koreksi bila beda
    },
}

def sigmoid(z: np.ndarray) -> np.ndarray:
    z = np.clip(z, -60, 60)
    return 1 / (1 + np.exp(-z))

def load_split(emb_dir: Path, enc: str, split: str) -> tuple[np.ndarray, np.ndarray, list[dict]]:
    X = np.load(emb_dir / f"emb_{enc}_{split}.npy")
    meta = json.loads((emb_dir / f"meta_{split}.json").read_text())
    y = np.array([m["label"] for m in meta])
    return X, y, meta

def pixel_of(emb_dir: Path, enc: str) -> dict:
    """Statistik preprocessing piksel: dari encmeta kernel bila ada, else fallback PIXEL."""
    f = emb_dir / f"encmeta_{enc}.json"
    if f.exists():
        d = json.loads(f.read_text())
        return {"type": d["type"], "id": d["id"], "size": d["size"], "mean": tuple(d["mean"]), "std": tuple(d["std"])}
    return PIXEL.get(enc, PIXEL["dinov2"])

def band_metrics(y: np.ndarray, p: np.ndarray, tau_low: float, tau_high: float) -> dict:
    band = np.where(p >= tau_high, "rujuk_prioritas", np.where(p < tau_low, "negatif_skrining", "ragu"))
    rep = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b
        rep[b] = {
            "n": int(m.sum()),
            "frac": float(m.mean()),
            "tb_rate": float(y[m].mean()) if m.sum() else None,
        }
    return rep

def full_metrics(y: np.ndarray, p: np.ndarray) -> dict:
    lo, hi = bootstrap_auroc(y, p)
    return {
        "n": int(len(y)),
        "pos": int(y.sum()),
        "auroc": float(roc_auc_score(y, p)),
        "auroc_ci95": [lo, hi],
        "sens_at_spec_90": sens_at_spec(y, p, 0.90),
        "sens_at_spec_70": sens_at_spec(y, p, 0.70),
    }
```

```

        "ece": ece(y, p),
        "brier": float(brier_score_loss(y, p)),
    }

def run_encoder(enc: str, emb_dir: Path, out_dir: Path, supervised_preds: Path | None) -> dict:
    Xtr, ytr, _ = load_split(emb_dir, enc, "train")
    Xva, yva, meta_va = load_split(emb_dir, enc, "val")
    Xex, yex, meta_ex = load_split(emb_dir, enc, "external")

    scaler = StandardScaler().fit(Xtr)
    Xtr_s, Xva_s, Xex_s = scaler.transform(Xtr), scaler.transform(Xva), scaler.transform(Xex)

    best = {"auroc": -1.0, "C": None}
    for C in (0.001, 0.01, 0.1, 1.0, 10.0):
        clf = LogisticRegression(C=C, class_weight="balanced", max_iter=3000, solver="lbfgs")
        clf.fit(Xtr_s, ytr)
        z_va = clf.decision_function(Xva_s)
        try:
            auroc = roc_auc_score(yva, sigmoid(z_va))
        except ValueError:
            auroc = float("nan")
        if auroc > best["auroc"]:
            best = {"auroc": float(auroc), "C": C, "clf": clf}
        print(f" [{enc}] C={C:<6} val AUROC={auroc:.4f}")

    clf: LogisticRegression = best["clf"]
    z_va, z_ex = clf.decision_function(Xva_s), clf.decision_function(Xex_s)
    T = fit_temperature([z_va], yva)
    p_va, p_ex = sigmoid(z_va / T), sigmoid(z_ex / T)

    tau_high = threshold_at_spec(yva, p_va, 0.70)
    tau_low = threshold_at_spec(yva, p_va, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    res = {
        "encoder": enc,
        "chosen_C": best["C"],
        "val": full_metrics(yva, p_va),
        "external": full_metrics(yex, p_ex),
        "temp": T,
        "tau_low": tau_low,
        "tau_high": tau_high,
        "bands_external": band_metrics(yex, p_ex, tau_low, tau_high),
    }

    out_dir.mkdir(parents=True, exist_ok=True)
    (out_dir / f"metrics_{enc}.json").write_text(json.dumps(res, indent=1))

    # ekspor probe untuk serving API (scaler + LR + T + metadata encoder)
    pix = pixel_of(emb_dir, enc)
    np.savez(
        out_dir / f"probe_{enc}.npz",
        scaler_mean=scaler.mean_,
        scaler_scale=scaler.scale_,
        coef=clf.coef_.ravel(),
        intercept=np.array([float(clf.intercept_[0])]),
        T=np.array([T]),
        size=np.array([pix["size"]]),
        pixel_mean=np.array(pix["mean"], dtype=np.float64),
        pixel_std=np.array(pix["std"], dtype=np.float64),
        encoder_type=np.array([pix["type"]]),
        encoder_id=np.array([pix["id"]]),
    )

    reliability_plot(yex, p_ex, out_dir / f"reliability_{enc}.png", f"SSL probe {enc} - external")
    roc_plot({f"{enc} (external)": (yex, p_ex)}, out_dir / f"roc_{enc}.png")
    with (out_dir / f"preds_{enc}.csv").open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["path", "label", "p", "z"])
        for m, p, z in zip(meta_ex, p_ex, z_ex):
            w.writerow([m["path"], m["label"], float(p), float(z)])

    # stacking dengan ensemble supervised (jika tersedia)
    if supervised_preds and supervised_preds.exists():
        sup = {}
        for row in csv.DictReader(supervised_preds.open()):
            sup[Path(row["path"]).stem] = float(row["p_ens"])
        pairs = [(Path(m["path"]).stem, m["label"], p) for m, p in zip(meta_ex, p_ex)]
        aligned = [(k, lab, p, sup[k]) for k, lab, p in pairs if k in sup]
        if len(aligned) >= 50:
            ya = np.array([a[1] for a in aligned])
            ps = (np.array([a[2] for a in aligned]) + np.array([a[3] for a in aligned])) / 2
            res["stack_with_supervised"] = {**full_metrics(ya, ps), "n_matched": len(aligned)}
            print(f" [{enc}] stack n={len(aligned)} external AUROC={res['stack_with_supervised']['auroc']:.4f}")

    (out_dir / f"metrics_{enc}.json").write_text(json.dumps(res, indent=1))
    return res

def main() -> None:

```

```

ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
ap.add_argument("--emb-dir", type=Path, required=True)
ap.add_argument("--out", type=Path, required=True)
ap.add_argument("--encoder", nargs="*", default=None, help="default: semua yang ada")
ap.add_argument("--supervised-preds", type=Path, default=None)
args = ap.parse_args()

encs = args.encoder or sorted(
    p.name[len("emb_") : -len("_train.npy")] for p in args.emb_dir.glob("emb_*_train.npy")
)
results = []
for enc in encs:
    print(f"=== {enc} ===")
    results.append(run_encoder(enc, args.emb_dir, args.out, args.supervised_preds))

print("\n=== RINGKASAN (eksternal) ===")
for r in results:
    e = r["external"]
    print(f"{r['encoder']:10s} AUROC {e['auroc']:.4f} [{e['auroc_ci95'][0]:.3f},{e['auroc_ci95'][1]:.3f}] "
          f"| sens@spec90 {e['sens_at_spec_90']:.3f} | ECE {e['ece']:.3f}"
          + (f" | stack {r['stack_with_supervised'] ['auroc']:.4f}" if "stack_with_supervised" in r else ""))

# kombinasi rata-rata probabilitas antar encoder SSL
preds_files = sorted(args.out.glob("preds_*.csv"))
if len(preds_files) > 1:
    maps = []
    for f in preds_files:
        rows = list(csv.DictReader(f.open()))
        maps.append({r["path"]: (float(r["p"]), int(r["label"])) for r in rows})
    keys = set(maps[0])
    for m in maps[1:]:
        keys |= set(m)
    ks = sorted(keys)
    y = np.array([maps[0][k][1] for k in ks])
    p = np.mean([np.array([m[k][0] for k in ks]) for m in maps], axis=0)
    comb = full_metrics(y, p)
    print(f"combined{'x'.join([]) or ''} SSL ({len(maps)} encoder): AUROC {comb['auroc']:.4f} "
          f"[{comb['auroc_ci95'][0]:.3f},{comb['auroc_ci95'][1]:.3f}] | sens@spec90 {comb['sens_at_spec_90']:.3f}")
    (args.out / "metrics_combined.json").write_text(json.dumps(comb, indent=1))

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl/loop-tb/ml/train.py

```
#!/usr/bin/env python3
"""Train TB CXR classifier dari manifest provenance-aware.

Contoh:
python ml/train.py --manifest ml/data/artifacts/manifest.json --arch efficientnet_b0 \
    --epochs 20 --out ml/runs/efficientnet_b0

Baseline linear probe (fitur beku):
python ml/train.py ... --freeze --lr 1e-3 --out ml/runs/linear_probe
"""
from __future__ import annotations

import argparse
import csv
import json
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import roc_auc_score, roc_curve
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import transforms as T
from tqdm import tqdm

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

class CXRManifest(Dataset):
    def __init__(self, items: list[dict], train: bool, img_size: int):
        self.items = items
        if train:
            self.tx = T.Compose([
                T.RandomResizedCrop(img_size, scale=(0.75, 1.0), ratio=(0.9, 1.1)),
                T.RandomRotation(7, fill=0),
                T.ColorJitter(brightness=0.15, contrast=0.15),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])
        else:
            self.tx = T.Compose([
                T.Resize(int(img_size * 1.14)),
                T.CenterCrop(img_size),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])

    def __len__(self) -> int:
        return len(self.items)

    def __getitem__(self, i: int):
        from PIL import Image
        it = self.items[i]
        with Image.open(it["path"]) as im:
            x = self.tx(im.convert("RGB"))
        return x, torch.tensor(float(it["label"])), i

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 → fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

@torch.no_grad()
def predict(model: nn.Module, loader: DataLoader, device: torch.device) -> tuple[np.ndarray, np.ndarray]:
    model.eval()
    logits_all, labels_all = [], []
    for x, y, _ in loader:
        x = x.to(device, non_blocking=True)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            logits = model(x).squeeze(1).float()
        logits_all.append(logits.cpu().numpy())
        labels_all.append(y.numpy())
    return np.concatenate(logits_all), np.concatenate(labels_all)

def evaluate(logits: np.ndarray, y: np.ndarray) -> dict:
    p = 1 / (1 + np.exp(-logits))
    out = {"n": int(len(y)), "pos": int(y.sum())}
    if len(np.unique(y)) > 1:
```

```

        out["auroc"] = float(roc_auc_score(y, p))
        out["sens_at_spec_90"] = sens_at_spec(y, p, 0.90)
        out["sens_at_spec_70"] = sens_at_spec(y, p, 0.70)
    return out

```

```

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--arch", default="efficientnet_b0")
    ap.add_argument("--epochs", type=int, default=20)
    ap.add_argument("--batch", type=int, default=32)
    ap.add_argument("--lr", type=float, default=3e-4)
    ap.add_argument("--wd", type=float, default=1e-4)
    ap.add_argument("--img-size", type=int, default=224)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--workers", type=int, default=8)
    ap.add_argument("--freeze", action="store_true", help="linear probe: backbone beku")
    ap.add_argument("--balanced", action="store_true", help="WeightedRandomSampler")
    ap.add_argument("--limit", type=int, default=0, help="smoke test: batasi n sampel (0=semua)")
    args = ap.parse_args()

    torch.manual_seed(args.seed)
    np.random.seed(args.seed)
    args.out.mkdir(parents=True, exist_ok=True)

    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
    manifest = json.loads(args.manifest.read_text())

    def take(items: list[dict], n: int) -> list[dict]:
        """Subset berstratifikasi untuk smoke test."""
        if not n:
            return items
        pos = [it for it in items if it["label"] == 1]
        neg = [it for it in items if it["label"] == 0]
        n = min(n, len(items))
        kp = min(len(pos), max(1, n // 4))
        return pos[:kp] + neg[: max(n - kp, 0)]

    train_items = take(manifest["train"], args.limit)
    val_items = take(manifest["val"], args.limit)

    ds_tr = CXRMManifest(train_items, train=True, img_size=args.img_size)
    ds_va = CXRMManifest(val_items, train=False, img_size=args.img_size)
    sampler = None
    if args.balanced:
        labels = np.array([it["label"] for it in train_items])
        w = np.where(labels == 1, 1.0 / labels.sum(), 1.0 / (len(labels) - labels.sum()))
        sampler = WeightedRandomSampler(torch.from_numpy(w), num_samples=len(labels), replacement=True)
    dl_tr = DataLoader(ds_tr, batch_size=args.batch, shuffle=sampler is None, sampler=sampler,
                      num_workers=args.workers, pin_memory=(device.type == "cuda"), drop_last=True)
    dl_va = DataLoader(ds_va, batch_size=args.batch, shuffle=False, num_workers=args.workers, pin_memory=(device.type == "cuda"))

    model = timm.create_model(args.arch, pretrained=True, num_classes=1).to(device)
    if args.freeze:
        for n, p in model.named_parameters():
            p.requires_grad = "head" in n or "classifier" in n or "fc" in n
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=args.lr, weight_decay=args.wd)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(args.epochs, 1))
    loss_fn = nn.BCEWithLogitsLoss()
    scaler = torch.amp.GradScaler(enabled=(device.type == "cuda"))

    print(f"device={device} arch={args.arch} freeze={args.freeze} "
          f"train={len(ds_tr)} val={len(ds_va)} params={sum(p.numel() for p in params)/1e6:.2f}M")

    hist = []
    best = {"auroc": -1.0, "epoch": -1}
    for epoch in range(1, args.epochs + 1):
        model.train()
        t0, losses = time.time(), []
        for x, y, _ in tqdm(dl_tr, desc=f"epoch {epoch:02d}", leave=False):
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device.type, enabled=(device.type == "cuda")):
                logits = model(x).squeeze(1)
                loss = loss_fn(logits.float(), y)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            losses.append(loss.item())
        sched.step()

        logits, y_va = predict(model, dl_va, device)
        m = evaluate(logits, y_va)
        row = {"epoch": epoch, "loss": float(np.mean(losses)), "lr": opt.param_groups[0]["lr"],
              "sec": round(time.time() - t0, 1), **m}
        hist.append(row)
    print(f"epoch {epoch:02d} | loss {row['loss']:.4f} | AUROC {m.get('auroc', float('nan')):.4f} "
          f"| sens@spec90 {m.get('sens_at_spec_90', float('nan')):.3f} | {row['sec']}s")

```

```

torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
           "epoch": epoch, "val": m}, args.out / "last.pt")
if m.get("auroc", -1) > best["auroc"]:
    best = {"auroc": m["auroc"], "epoch": epoch}
    torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
               "epoch": epoch, "val": m}, args.out / "best.pt")
    with (args.out / "val_preds.csv").open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["path", "label", "logit"])
        for it, lg in zip(val_items, logits):
            w.writerow([it["path"], it["label"], float(lg)])

(args.out / "history.csv").write_text(
    "epoch,loss,lr,sec,n,pos,auroc,sens_at_spec_90,sens_at_spec_70\n" + "\n".join(
        ", ".join(str(r.get(k, "")) for k in ("epoch", "loss", "lr", "sec", "n", "pos", "auroc", "sens_at_spec_90",
        "sens_at_spec_70"))
        for r in hist) + "\n")
(args.out / "metrics.json").write_text(json.dumps({"best": best, "history": hist, "config": vars(args)}, indent=1, default=str))
print(f"\nBEST epoch {best['epoch']} AUROC {best['auroc']:.4f} -> {args.out/'best.pt'}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl/loop-tb/tools/e2e_screen.py

```
#!/usr/bin/env python3
"""Uji integrasi UI+API: unggah citra X-ray lewat halaman /skrining, ambil screenshot hasil.

Jalankan dengan helper server (web 3210 + api 8000):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm start -- --port 3210" --port 3210 \
--server ".venv/bin/python -m uvicorn api.main:app --port 8000" --port 8000 \
-- .venv/bin/python tools/e2e_screen.py
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

OUT = Path("submission/screenshots")
OUT.mkdir(parents=True, exist_ok=True)

CASES = [
    ("data/raw/nlm/shenzhen/CXR_png/CHNCXR_0327_1.png", "tb", "S-014"),
    ("data/raw/nlm/shenzhen/CXR_png/CHNCXR_0001_0.png", "normal", "S-015"),
]

def run_case(page, img: str, tag: str, pid: str) -> None:
    page.goto("http://localhost:3210/skrining", wait_until="networkidle")
    page.set_input_files('input[type="file"]', img)
    page.fill('input[placeholder="mis. S-014"]', pid)
    page.fill('input[placeholder="34"]', "42")
    page.click("text=Analisis citra")
    page.wait_for_selector("text=Hasil analisis", timeout=120_000)
    page.wait_for_timeout(600)
    page.screenshot(path=str(OUT / f"skrining_hasil_{tag}_mobile.png"), full_page=True)
    verdict = page.locator("section >> text=Hasil analisis").first
    print(f"[ok] {tag}: {img}")

def main() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        ctx = browser.new_context(viewport={"width": 390, "height": 844}, device_scale_factor=2)
        page = ctx.new_page()
        console: list[str] = []
        page.on("console", lambda m: console.append(f"{m.type}: {m.text}") if m.type == "error" else None)
        for img, tag, pid in CASES:
            if not Path(img).exists():
                print("[skip]", img)
                continue
            run_case(page, img, tag, pid)
        if console:
            print("[console errors]", console[:6])
        ctx.close()
        browser.close()
    print("E2E SCREENSHOTS DONE")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl/loop-tb/tools/screenshot_ui.py

```
#!/usr/bin/env python3
"""QA visual + screenshot semua halaman SIGAP (mobile & desktop).

Jalankan lewat helper server (lihat ml/kaggle/README atau riwayat commit):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm run dev -- --port 3210" --port 3210 \
-- .env/bin/python tools/screenshot_ui.py http://localhost:3210
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots")
OUT.mkdir(parents=True, exist_ok=True)

ROUTES = [
    ("", "landing"),
    ("skrining", "skrining"),
    ("pasien", "pasien"),
    ("klinisi", "klinisi"),
    ("tentang", "tentang"),
]
VIEWPORTS = [("mobile", 390, 844), ("desktop", 1280, 900)]

def main() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        for vp_name, w, h in VIEWPORTS:
            ctx = browser.new_context(viewport={"width": w, "height": h}, device_scale_factor=2)
            page = ctx.new_page()
            console_errors: list[str] = []
            page.on("console", lambda m: console_errors.append(m.text) if m.type == "error" else None)
            for route, name in ROUTES:
                page.goto(f"{BASE}/{route}", wait_until="networkidle")
                page.wait_for_timeout(400)
                page.screenshot(path=str(OUT / f"{name}_{vp_name}.png"), full_page=True)
                print(f"[ok] {vp_name:7s} {name}")
            if console_errors:
                print(f"[warn] console errors ({vp_name}): {console_errors[:5]}")
            ctx.close()
        browser.close()
    print(f"SCREENSHOTS -> {OUT}")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl_diag/loop-tb/api/main.py

```
"""Loop-TB API – inference + Grad-CAM + band triase.

Jalankan lokal:
    uvicorn api.main:app --host 0.0.0.0 --port 8000

Env:
    LOOPTB_CKPT path ke checkpoint best.pt (default ml/runs/convnext_tiny/best.pt)
    LOOPTB_BANDS path ke bands.json hasil evaluate.py (opsional; default  $\tau=0.5/0.5$ )
"""
from __future__ import annotations

import base64
import io
import json
import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.cm as cm
import numpy as np
import timm
import torch
import torch.nn as nn
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
from torchvision import transforms as T

CKPT_PATH = Path(os.environ.get("LOOPTB_CKPT", "ml/runs/convnext_tiny/best.pt"))
BANDS_PATH = Path(os.environ.get("LOOPTB_BANDS", "")) if os.environ.get("LOOPTB_BANDS") else None
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

app = FastAPI(title="loop-tb", version="0.1.0")
app.add_middleware(
    CORSMiddleware, allow_origins=["*"], allow_credentials=False,
    allow_methods=["*"], allow_headers=["*"],
)

_state: dict = {}

def _load() -> None:
    if not CKPT_PATH.exists():
        raise RuntimeError(f"Checkpoint tidak ditemukan: {CKPT_PATH}")
    ckpt = torch.load(CKPT_PATH, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()
    bands = {"tau_high": 0.5, "tau_low": 0.5}
    if BANDS_PATH and BANDS_PATH.exists():
        b = json.loads(BANDS_PATH.read_text())
        bands = {"tau_high": float(b.get("tau_high", 0.5)), "tau_low": float(b.get("tau_low", 0.5))}
    size = int(ckpt["config"].get("img_size", 224))
    _state.update(model=model, arch=ckpt["arch"], img_size=size, bands=bands)
    print(f"[loop-tb] loaded {ckpt['arch']} @ {size}px bands={bands}")

_tx = None

def preprocess(img: Image.Image) -> torch.Tensor:
    global _tx
    if _tx is None:
        s = _state["img_size"]
        _tx = T.Compose([T.Resize(int(s * 1.14)), T.CenterCrop(s), T.ToTensor(), T.Normalize(MEAN, STD)])
    return _tx(img.convert("RGB")).unsqueeze(0)

def band_of(p: float, bands: dict) -> str:
    if p >= bands["tau_high"]:
        return "rujuk_prioritas"
    if p < bands["tau_low"]:
        return "negatif_skrining"
    return "ragu"

def gradcam(model: nn.Module, x: torch.Tensor) -> np.ndarray:
    """Grad-CAM generik via forward_features (timm), tanpa asumsi arsitektur."""
    model.zero_grad(set_to_none=True)
    feats = model.forward_features(x)
    feats.retain_grad()
    logit = model.forward_head(feats).squeeze(1)
    logit.backward()
    w = feats.grad.mean(dim=(2, 3), keepdim=True)
    cam = (w * feats).sum(dim=1, keepdim=True).clamp(min=0)
    cam = nn.functional.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
```

```

cam = cam / (cam.amax(dim=(2, 3), keepdim=True) + 1e-8)
return cam[0, 0].detach().numpy()

def overlay_png(img: Image.Image, cam: np.ndarray) -> str:
    img_np = np.asarray(img.convert("RGB").resize((cam.shape[1], cam.shape[0])), dtype=np.float32) / 255.0
    heat = cm.jet(cam)[..., :3]
    alpha = (cam[..., None] * 0.45)
    blend = img_np * (1 - alpha) + heat.astype(np.float32) * alpha
    out = Image.fromarray((np.clip(blend, 0, 1) * 255).astype(np.uint8))
    buf = io.BytesIO()
    out.save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode()

@app.on_event("startup")
def startup() -> None:
    _load()

@app.get("/health")
def health() -> dict:
    return {
        "status": "ok",
        "arch": _state.get("arch"),
        "img_size": _state.get("img_size"),
        "bands": _state.get("bands"),
    }

@app.post("/predict")
async def predict(file: UploadFile = File(...)) -> dict:
    if "model" not in _state:
        raise HTTPException(503, "model belum siap")
    raw = await file.read()
    try:
        img = Image.open(io.BytesIO(raw))
        img.load()
    except Exception as e:
        raise HTTPException(400, f"bukan citra valid: {e}")

    x = preprocess(img)
    with torch.no_grad():
        logit = _state["model"](x).squeeze(1)
    p = float(torch.sigmoid(logit).item())
    band = band_of(p, _state["bands"])

    cam = gradcam(_state["model"], x)
    heatmap_b64 = overlay_png(img, cam)

    return {
        "p_tb": round(p, 4),
        "band": band,
        "bands": _state["bands"],
        "arch": _state.get("arch"),
        "disclaimer": "Alat triase skrining – bukan diagnosis. Wajib konfirmasi (GeneXpert) oleh tenaga kesehatan.",
        "heatmap_png_b64": heatmap_b64,
    }

```

ml/kaggle/out_ssl_diag/loop-tb/ml/data/prepare_data.py

```
#!/usr/bin/env python3
"""Build a provenance-aware manifest for TB chest X-ray training.

Sumber & aturan label (lihat docs/02-arsitektur.md):
- TBX11K (Kaggle: vbookshelf/tbx11k-simplified): folder per kelas.
  positif = {tb}; negatif = {healthy, sick}; latent_tb DIKELUARKAN (ambigu).
- Rahman composite (Kaggle: tawsifurrahman/tuberculosis-tb-chest-xray-dataset):
  folder Normal/ -> 0, Tuberculosis/ -> 1.
- NLM Shenzhen/Montgomery: konvensi nama file resmi `*_0` = normal, `*_1` = TB.

Dedup (WAJIB – mitigasi class-conditional acquisition confounding):
1. pHash 64-bit; Hamming <= --dup-threshold dianggap duplikat (union-find transitif).
2. Integritas external: citra train yang sekelompok dengan citra external DIBUANG dari train.
3. Duplikat internal train dibuang (keep-first, urutan deterministik).
4. Split val dilakukan per-klaster (tidak ada kebocoran near-dup train->val).

Output (ml/data/artifacts/):
- manifest.json, dedup_report.json, train.csv, val.csv, external.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import re
from collections import Counter
from pathlib import Path

import numpy as np
from PIL import Image
from tqdm import tqdm

try:
    import imagehash
except ImportError: # pragma: no cover
    raise SystemExit("Butuh imagehash: pip install imagehash")

IMG_EXTS = {".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff"}
POS_TOKENS = {"tb", "tuberculosis", "tuberculose"}
NEG_TOKENS = {"healthy", "normal", "sick"}
EXCLUDE_TOKENS = {"latent", "latent_tb", "latenttb"}

def _rel(p: Path, root: Path) -> str:
    """Path relatif terhadap root repo (portable lokal <-> Colab); fallback absolut."""
    try:
        return str(p.relative_to(root))
    except ValueError:
        return str(p)

def _tokens(path: Path) -> set[str]:
    parts = [p.lower() for p in path.parts]
    parts += [path.stem.lower()]
    return set(parts)

def load_tbx11k_map(raw: Path) -> dict[str, int | None]:
    """Label otoritatif dari data.csv TBX11K: 1=active_tb, 0=no_tb, None=latent (dikeluarkan)."""
    m: dict[str, int | None] = {}
    for p in raw.rglob("data.csv"):
        if "tbx11k" not in str(p).lower():
            continue
        for r in csv.DictReader(p.open()):
            f = (r.get("fname") or "").strip()
            if not f:
                continue
            t = (r.get("tb_type") or "").lower()
            tgt = (r.get("target") or "").lower()
            if "latent" in t:
                m.setdefault(f, None)
            elif tgt == "tb":
                m[f] = 1
            else:
                m.setdefault(f, 0)
    return m

def classify(path: Path, raw: Path, tbx_map: dict[str, int | None]) -> tuple[int, str] | None:
    """Return (label, source) atau None kalau tidak bisa diklasifikasi."""
    rel = path.relative_to(raw)
    low = str(rel).lower()

    # --- NLM Shenzhen / Montgomery: konvensi suffix nama file ---
    m = re.search(r"(chncxr|mcucxr)_d+_(\d+)", path.stem.lower())
    if m:
        label = int(m.group(2))
        source = "shenzhen" if m.group(1) == "chncxr" else "montgomery"
```

```

        return label, source

toks = _tokens(rel)
if toks & EXCLUDE_TOKENS or any(t.startswith("latent") for t in toks):
    return None

if "tbx11k" in low:
    if tbx_map:
        lab = tbx_map.get(path.name, None)
        return None if lab is None else (lab, "tbx11k")
    # fallback bila data.csv tidak ada: prefix nama file
    stem = path.stem.lower()
    if stem.startswith("t"):
        return 1, "tbx11k"
    if stem.startswith(("h", "s")):
        return 0, "tbx11k"
    return None

if toks & POS_TOKENS:
    return 1, "rahman"
if toks & NEG_TOKENS:
    return 0, "rahman"
return None

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[rb] = ra

_POP = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def hamming_pairs(hashes: np.ndarray, threshold: int, chunk: int = 512) -> list[tuple[int, int]]:
    """Semua pasangan (i<j) dengan Hamming distance <= threshold (chunked, hemat memori)."""
    n = len(hashes)
    h8 = hashes.view(np.uint8).reshape(n, 8)
    pairs: list[tuple[int, int]] = []
    for start in range(0, n, chunk):
        stop = min(start + chunk, n)
        xor = np.bitwise_xor(h8[start:stop, None, :], h8[None, :, :])
        dist = _POP[xor].sum(axis=-1) # (chunk, n)
        rows, cols = np.where(dist <= threshold)
        for r, c in zip(rows, cols):
            i, j = start + int(r), int(c)
            if i < j:
                pairs.append((i, j))
    return pairs

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--raw", type=Path, default=Path("data/raw"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts"))
    ap.add_argument("--dup-threshold", type=int, default=3, help="Hamming pHash <= ini dianggap duplikat")
    ap.add_argument("--val-frac", type=float, default=0.15)
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    raw = args.raw.resolve()
    root = raw.parent.parent # asumsi struktur <root>/data/raw

    # ----- 1) discovery -----
    labeled: list[dict] = []
    unlabeled = Counter()
    tbx_map = load_tbx11k_map(raw)
    print(f"[discovery] label TBX11K dari data.csv: {len(tbx_map)} entri")
    for p in sorted(raw.rglob("*")):
        if p.suffix.lower() not in IMG_EXTS:
            continue
        cls = classify(p, raw, tbx_map)
        if cls is None:
            unlabeled[str(p.relative_to(raw).parent)] += 1
            continue
        label, source = cls
        labeled.append({"path": _rel(p, root), "label": label, "source": source})

    print(f"[discovery] berlabel={len(labeled)} tak-terklasifikasi={sum(unlabeled.values())}")
    for k, v in unlabeled.most_common(10):
        print(f" skip {v:6d} {k}")

```

```

if not labeled:
    raise SystemExit("Tidak ada citra berlabel – cek struktur data/raw")

# ----- 2) pHash + kluster duplikat -----
print("[dedup] menghitung pHash...")
ok: list[dict] = []
for item in tqdm(labeled, unit="img"):
    try:
        with Image.open(item["path"]) as im:
            h = imagehash.phash(im.convert("L"), hash_size=8)
            item["phash"] = int(str(h), 16)
            ok.append(item)
    except Exception as e: # gambar korup
        print(f"warn: gagal baca {item['path']}: {e}")
labeled = ok

hashes = np.array([it["phash"] for it in labeled], dtype=np.uint64)
pairs = hamming_pairs(hashes, args.dup_threshold)
dsu = DSU(len(labeled))
for i, j in pairs:
    dsu.union(i, j)
for idx, it in enumerate(labeled):
    it["cluster"] = dsu.find(idx)

by_cluster: dict[int, list[dict]] = {}
for it in labeled:
    by_cluster.setdefault(it["cluster"], []).append(it)

# ----- 3) lock external, buang kebocoran dari train -----
ext = [it for it in labeled if it["source"] in ("shenzhen", "montgomery")]
ext_clusters = {it["cluster"] for it in ext}
train_pool, dropped_cross = [], []
for it in labeled:
    if it["source"] in ("shenzhen", "montgomery"):
        continue
    (dropped_cross if it["cluster"] in ext_clusters else train_pool).append(it)

# duplikat internal train: keep-first per kluster (deterministik via path)
keep, dropped_internal = [], []
seen_clusters: set[int] = set()
for it in sorted(train_pool, key=lambda x: x["path"]):
    if it["cluster"] in seen_clusters:
        dropped_internal.append(it)
    else:
        seen_clusters.add(it["cluster"])
        keep.append(it)
train_pool = keep

# ----- 4) split val per-kluster -----
rng = np.random.default_rng(args.seed)
sizes = Counter(it["cluster"] for it in train_pool)
pool_clusters = sorted(sizes)
rng.shuffle(pool_clusters)
target_val = int(len(train_pool) * args.val_frac)
val_clusters: set[int] = set()
acc = 0
for c in pool_clusters:
    if acc >= target_val:
        break
    val_clusters.add(c)
    acc += sizes[c]
train = [it for it in train_pool if it["cluster"] not in val_clusters]
val = [it for it in train_pool if it["cluster"] in val_clusters]

def dump(items: list[dict]) -> list[dict]:
    return [
        {k: it[k] for k in ("path", "label", "source", "cluster", "phash")}
        for it in sorted(items, key=lambda x: x["path"])
    ]

manifest = {
    "meta": {
        "root": str(root),
        "raw": _rel(raw, root),
        "dup_threshold": args.dup_threshold,
        "val_frac": args.val_frac,
        "seed": args.seed,
        "counts": {
            "train": len(train), "val": len(val), "external": len(ext),
            "dropped_cross_corpus": len(dropped_cross),
            "dropped_internal_dup": len(dropped_internal),
            "unlabeled": sum(unlabeled.values()),
        },
    },
    "train": dump(train),
    "val": dump(val),
    "external": dump(ext),
}
(args.out / "manifest.json").write_text(json.dumps(manifest, indent=1))
(args.out / "dedup_report.json").write_text(json.dumps({
    "pairs_within_threshold": len(pairs),
    "clusters": len(by_cluster),
}))

```

```

        "dropped_cross_corpus": dump(dropped_cross),
        "dropped_internal_dup": dump(dropped_internal),
        "unlabeled_by_dir": dict(unlabeled),
    }, indent=1))

for split in ("train", "val", "external"):
    with (args.out / f"{split}.csv").open("w") as f:
        f.write("path,label,source,cluster\n")
        for it in manifest[split]:
            f.write(f"{it['path']},{it['label']},{it['source']},{it['cluster']}\n")

def comp(items):
    c = Counter((it["source"], it["label"]) for it in items)
    return " ".join(f"{s}/{ 'TB' if l else 'NEG' }={n}" for (s, l), n in sorted(c.items()))

print("\n=== RINGKASAN ===")
print(f"train      n={len(train):6d}  {comp(train)}")
print(f"val        n={len(val):6d}    {comp(val)}")
print(f"external  n={len(ext):6d}    {comp(ext)}")
print(f"dibuang  cross-corpus={len(dropped_cross)}  internal-dup={len(dropped_internal)}")
print(f"manifest -> {args.out/'manifest.json'}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl_diag/loop-tb/ml/data/prepare_meta.py

```
#!/usr/bin/env python3
"""Ekstrak metadata klinis NLM (Montgomery + Shenzhen) → CSV untuk analisis subgroup.

Output: ml/data/artifacts/meta_nlm.csv
Kolom: filename,sex,age,reading,scar_flag
- Montgomery: "Patient's Sex: F / Patient's Age: 027Y / <reading>"
- Shenzhen: "male 45yrs / <reading>" (kadang terkontaminasi antar-file)
- scar_flag: menyebut scar/inactive/fibrosis → untuk pembahasan mode gagal
            "healed scar false positive" (selaras audit Bilal 2026).
"""
from __future__ import annotations

import argparse
import csv
import re
from pathlib import Path

SEX_M = re.compile(r"sex[:\s]+(m|male)\b", re.I)
SEX_F = re.compile(r"sex[:\s]+(f|female)\b", re.I)
SEX_FMT2 = re.compile(r"\b(male|female)\b", re.I)
AGE_MONT = re.compile(r"age[:\s]+(\d{1,3})\s*Y?\b", re.I)
AGE_FMT2 = re.compile(r"\b(\d{1,3})\s*yrs\b", re.I)
SCAR = re.compile(r"scar|inactive|fibro", re.I)

def parse_montgomery(text: str) -> dict | None:
    age = AGE_MONT.search(text)
    sex = "F" if SEX_F.search(text) else ("M" if SEX_M.search(text) else "")
    if not age:
        return None
    # reading = baris setelah Age
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    reading = lines[2] if len(lines) > 2 else ""
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": int(bool(SCAR.search(reading)))}

def parse_shenzhen(text: str) -> dict | None:
    age = AGE_FMT2.search(text)
    sex_m = SEX_FMT2.findall(text)
    sex = sex_m[0][0].upper() if sex_m else ""
    if not age:
        return None
    # reading = token huruf setelah "yrs"
    tail = text[age.end():]
    reading = ""
    m = re.search(r"[A-Za-z]{2,}", tail)
    if m:
        reading = m.group(0)
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": 0}

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--nlm", type=Path, default=Path("data/raw/nlm"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts/meta_nlm.csv"))
    args = ap.parse_args()
    args.out.parent.mkdir(parents=True, exist_ok=True)

    rows, n_fail = [], 0
    for corpus, parser in (("montgomery", parse_montgomery), ("shenzhen", parse_shenzhen)):
        d = args.nlm / corpus / "ClinicalReadings"
        for p in sorted(d.glob("*.txt")):
            r = parser(p.read_text(errors="ignore"))
            if r is None:
                n_fail += 1
                continue
            r["filename"] = p.stem + ".png"
            r["corpus"] = corpus
            rows.append(r)

    with args.out.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["filename", "corpus", "sex", "age", "reading", "scar_flag"])
        w.writeheader()
        w.writerows(rows)

    from collections import Counter
    print(f"meta -> {args.out} n={len(rows)} gagal-parse={n_fail}")
    print("per korpus:", Counter(r["corpus"] for r in rows))
    print("sex:", Counter(r["sex"] for r in rows))
    print("scar_flag:", Counter(r["scar_flag"] for r in rows))

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl_diag/loop-tb/ml/evaluate.py

```
#!/usr/bin/env python3
"""Evaluasi rigor: kalibrasi, CI bootstrap, transfer, band triase, subgroup.

Contoh (ensemble + temperature scaling):
python ml/evaluate.py --runs ml/runs/efficientnet_b0 ml/runs/convnext_tiny \
    --manifest ml/data/artifacts/manifest.json --split external --temp-scale --out ml/runs/eval_external
"""
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import brier_score_loss, roc_auc_score, roc_curve
from torch.utils.data import DataLoader
from tqdm import tqdm

import sys
sys.path.insert(0, str(Path(__file__).resolve().parent))
from train import CXRManifest # noqa: E402

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-x))

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 -> fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

def threshold_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    fpr, tpr, thr = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(thr[ok][np.argmax(tpr[ok])]) if ok.any() else 0.5

def ece(y: np.ndarray, p: np.ndarray, bins: int = 15) -> float:
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for i in range(bins):
        m = (p >= edges[i]) & ((p < edges[i + 1]) if i < bins - 1 else (p <= edges[i + 1]))
        if m.sum():
            total += m.mean() * abs(p[m].mean() - y[m].mean())
    return float(total)

def bootstrap_auroc(y: np.ndarray, p: np.ndarray, n: int = 1000, seed: int = 42) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    vals = []
    for _ in range(n):
        idx = rng.integers(0, len(y), len(y))
        if len(np.unique(y[idx])) > 1:
            vals.append(roc_auc_score(y[idx], p[idx]))
    return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))

@torch.no_grad()
def infer(ckpt_path: Path, items: list[dict], img_size: int, device: torch.device, workers: int) -> np.ndarray:
    ckpt = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    arch = ckpt["arch"]
    model = timm.create_model(arch, pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.to(device).eval()
    dl = DataLoader(CXRManifest(items, train=False, img_size=img_size), batch_size=64,
                    shuffle=False, num_workers=workers)

    out = []
    for x, _, _ in tqdm(dl, desc=f"infer {ckpt_path.parent.name}", leave=False):
        x = x.to(device)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            out.append(model(x).squeeze(1).float().cpu().numpy())
    return np.concatenate(out)
```

```

def fit_temperature(val_logits: list[np.ndarray], val_y: np.ndarray) -> float:
    """Cari T>0 yang meminimalkan NLL pada val (grid log-space lebar untuk logit besar)."""
    logits = torch.tensor(np.mean(val_logits, axis=0), dtype=torch.float32)
    y = torch.tensor(val_y, dtype=torch.float32)
    best_t, best_nll = 1.0, float("inf")
    for t in np.logspace(-1, 3.5, 500):
        nll = nn.functional.binary_cross_entropy_with_logits(logits / t, y).item()
        if nll < best_nll:
            best_t, best_nll = float(t), nll
    return best_t

def reliability_plot(y: np.ndarray, p: np.ndarray, path: Path, title: str) -> None:
    edges = np.linspace(0, 1, 11)
    mids, accs = [], []
    for i in range(10):
        m = (p >= edges[i]) & (p < edges[i + 1] if i < 9 else p <= edges[i + 1])
        if m.sum():
            mids.append(p[m].mean())
            accs.append(y[m].mean())
    plt.figure(figsize=(4.2, 4.2))
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.plot(mids, accs, "o-", color="#d62728", lw=2)
    plt.xlabel("Confidence"), plt.ylabel("Empirical TB rate"), plt.title(title)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def roc_plot(curves: dict[str, tuple[np.ndarray, np.ndarray]], path: Path) -> None:
    plt.figure(figsize=(5, 5))
    for name, (y, p) in curves.items():
        fpr, tpr, _ = roc_curve(y, p)
        plt.plot(fpr, tpr, lw=2, label=f"{name} (AUROC {roc_auc_score(y, p):.3f})")
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.xlabel("FPR"), plt.ylabel("TPR (sensitivitas)"), plt.legend(fontsize=8)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--runs", type=Path, nargs="+", required=True)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--split", default="external", choices=["val", "external"])
    ap.add_argument("--temp-scale", action="store_true")
    ap.add_argument("--meta", type=Path, default=None, help="CSV filename,sex,age untuk subgroup (opsional)")
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--workers", type=int, default=8)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    manifest = json.loads(args.manifest.read_text())
    items = manifest[args.split]
    y = np.array([it["label"] for it in items])

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    per_run_target, per_run_val, tscales = {}, {}, {}
    for run in args.runs:
        ckpt = torch.load(run / "best.pt", map_location="cpu", weights_only=False)
        img_size = ckpt["config"]["img_size"]
        per_run_target[run.name] = infer(run / "best.pt", items, img_size, device, args.workers)
        val_items = manifest["val"]
        per_run_val[run.name] = infer(run / "best.pt", val_items, img_size, device, args.workers)
        if args.temp_scale:
            tscales[run.name] = fit_temperature([per_run_val[run.name]], np.array([it["label"] for it in val_items]))

    per_run_target = {k: sigmoid(v) for k, v in per_run_target.items()}
    per_run_val = {k: sigmoid(v) for k, v in per_run_val.items()}
    val_y = np.array([it["label"] for it in manifest["val"]])

    # kalibrasi per-run pada val, lalu terapkan T ke target (bukan val-fit pada target!)
    if args.temp_scale:
        for k in per_run_target:
            per_run_val[k] = sigmoid(np.log(np.clip(per_run_val[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_val[k], 1e-6, 1 - 1e-6)))
            / tscales[k])
            lg = np.log(np.clip(per_run_target[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_target[k], 1e-6, 1 - 1e-6)))
            per_run_target[k] = sigmoid(lg / tscales[k])

    # ensemble = rata-rata; ketidakpastian = std antar anggota
    stack_t = np.stack(list(per_run_target.values()))
    p_ens = stack_t.mean(axis=0)
    u_ens = stack_t.std(axis=0)
    stack_v = np.stack(list(per_run_val.values()))
    p_ens_v = stack_v.mean(axis=0)

    tau_high = threshold_at_spec(val_y, p_ens_v, 0.70) # spec>=0.70 (TPP WHO)
    tau_low = threshold_at_spec(val_y, p_ens_v, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    band = np.where(p_ens >= tau_high, "rujuk_prioritas", np.where(p_ens < tau_low, "negatif_skrining", "ragu"))
    band_report = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b

```

```

        band_report[b] = {"n": int(m.sum()), "frac": float(m.mean()),
                          "tb_rate": float(y[m].mean()) if m.sum() else None}

lo, hi = bootstrap_auc(y, p_ens)
metrics = {
    "split": args.split, "n": int(len(y)), "pos": int(y.sum()),
    "aucroc": float(roc_auc_score(y, p_ens)), "aucroc_ci95": [lo, hi],
    "sens_at_spec_90": sens_at_spec(y, p_ens, 0.90),
    "sens_at_spec_70": sens_at_spec(y, p_ens, 0.70),
    "ece": ece(y, p_ens), "brier": float(brier_score_loss(y, p_ens)),
    "temp_scales": tscales, "tau_low": tau_low, "tau_high": tau_high,
    "bands": band_report,
    "deferral": {"rate": float((band == "ragu").mean()),
                 "tb_rate_on_deferred": float(y[band == "ragu"].mean()) if (band == "ragu").any() else None},
    "per_run_aucroc": {k: float(roc_auc_score(y, v)) for k, v in per_run_target.items()},
}

if args.meta and args.meta.exists():
    meta = {r["filename"]: r for r in csv.DictReader(args.meta.open())}
    for key in ("sex", "age"):
        groups: dict[str, list[int]] = {}
        for i, it in enumerate(items):
            row = meta.get(Path(it["path"]).name, {})
            g = row.get(key, "NA") or "NA"
            if key == "age":
                try:
                    a = float(g)
                    g = "0-30" if a < 30 else "30-50" if a < 50 else "50+"
                except ValueError:
                    g = "NA"
            groups.setdefault(g, []).append(i)
        metrics[f"aucroc_by_{key}"] = {
            g: (float(roc_auc_score(y[idx], p_ens[idx])) if len(np.unique(y[idx])) > 1 else None)
              for g, idx in groups.items() if g != "NA" and len(idx) >= 20
        }

(args.out / "metrics.json").write_text(json.dumps(metrics, indent=1))
# dump prediksi per-citra (untuk analisis lanjutan & stacking)
import csv as _csv
with (args.out / "preds.csv").open("w", newline="") as f:
    w = _csv.writer(f)
    w.writerow(["path", "label", "p_ens", "u_ens"])
    for it, p, u in zip(items, p_ens, u_ens):
        w.writerow([it["path"], it["label"], float(p), float(u)])
reliability_plot(y, p_ens, args.out / "reliability.png", f"Kalibrasi - {args.split}")
curves = {f"ens ({args.split})": (y, p_ens)}
if len(per_run_target) > 1:
    curves.update({k: (y, v) for k, v in per_run_target.items()})
roc_plot(curves, args.out / "roc.png")

print("\n=== METRICS ===")
for k, v in metrics.items():
    if isinstance(v, (int, float, str)):
        print(f"{k:18s} {v}")
print(f"bands          {band_report}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_ssl_diag/loop-tb/ml/export_onnx.py

```
#!/usr/bin/env python3
"""Export checkpoint ke ONNX (opsi 17) + verifikasi onnxruntime.

Contoh:
python ml/export_onnx.py --ckpt ml/runs/efficientnet_b0/best.pt --out artifacts/tb_efficientnet_b0.onnx
"""
from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import torch

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm")

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--ckpt", type=Path, required=True)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--opset", type=int, default=17)
    args = ap.parse_args()

    ckpt = torch.load(args.ckpt, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()

    img_size = ckpt["config"].get("img_size", 224)
    dummy = torch.randn(1, 3, img_size, img_size)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    torch.onnx.export(
        model, dummy, args.out,
        input_names=["input"], output_names=["logit"],
        dynamic_axes={"input": {0: "batch"}, "logit": {0: "batch"}},
        opset_version=args.opset, dynamo=False, # legacy: satu file mandiri, opset stabil
    )

    import onnxruntime as ort
    sess = ort.InferenceSession(str(args.out), providers=["CPUExecutionProvider"])
    ref = model(dummy).detach().numpy()
    got = sess.run(None, {"input": dummy.numpy()})[0]
    # Verifikasi di ruang probabilitas (deployment-relevant); logit bisa sangat besar
    # pada model overkonfiden sehingga toleransi absolut di ruang logit tidak adil.
    sig = lambda z: 1 / (1 + np.exp(-z)) # noqa: E731
    prob_diff = float(np.abs(sig(ref) - sig(got)).max())
    size_mb = args.out.stat().st_size / 1e6
    print(f"ONNX ok: {args.out} ({size_mb:.1f} MB) max|Δprob|={prob_diff:.2e} opset={args.opset}")
    if prob_diff > 1e-3:
        print(f"WARN: perbedaan probabilitas {prob_diff:.4f} > 1e-3 – periksa sebelum produksi")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_ssl_diag/loop-tb/ml/kaggle/hello/hello.py

```
"""Kernel uji: verifikasi GPU, internet, versi lib, dan struktur dataset ter-mount."""
import os
import sys

lines: list[str] = []

def log(*a) -> None:
    print(*a, flush=True)
    lines.append(" ".join(str(x) for x in a))

log("python:", sys.version.split()[0])

try:
    import torch

    log("torch:", torch.__version__, "| cuda:", torch.cuda.is_available())
    if torch.cuda.is_available():
        log("gpu:", torch.cuda.get_device_name(0))
except Exception as e: # noqa: BLE001
    log("torch error:", e)

for mod in ("timm", "sklearn", "onnx", "onnxruntime"):
    try:
        m = __import__(mod)
        log(f"{mod}: {getattr(m, '__version__', '?')}")
    except Exception as e: # noqa: BLE001
        log(f"{mod}: MISSING ({e})")

log("--- input mounts ---")
for root, dirs, _ in os.walk("/kaggle/input"):
    log(root, "dirs:", dirs[:6])
    if root.count("/") > 3:
        dirs[:] = []

log("HELLO DONE")
with open("/kaggle/working/hello_result.txt", "w") as f:
    f.write("\n".join(lines) + "\n")
```

ml/kaggle/out_ssl_diag/loop-tb/ml/kaggle/make_nlm_dataset.py

```
#!/usr/bin/env python3
"""Siapkan dataset Kaggle privat: NLM CXR di-resize (max dim 512) + ClinicalReadings.

Dipisah dari data asli agar unggahan ringan (~150 MB) dan resolusi tetap cukup
untuk encoder SSL (DINOv2/RAD-DINO resize ke 518).
"""
from __future__ import annotations

import shutil
from pathlib import Path

from PIL import Image

SRC = Path("data/raw/nlm")
DST = Path("ml/kaggle/nlm_dataset/nlm")

def main() -> None:
    n = 0
    for corpus in ("shenzhen", "montgomery"):
        out_dir = DST / corpus / "CXR_png"
        out_dir.mkdir(parents=True, exist_ok=True)
        for p in sorted((SRC / corpus / "CXR_png").glob("*.png")):
            with Image.open(p) as im:
                im = im.convert("L")
                w, h = im.size
                scale = 512 / max(w, h)
                if scale < 1:
                    im = im.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
                im.save(out_dir / (p.stem + ".jpg"), quality=92)
            n += 1
        readings = SRC / corpus / "ClinicalReadings"
        if readings.exists():
            out_r = DST / corpus / "ClinicalReadings"
            out_r.mkdir(parents=True, exist_ok=True)
            for f in readings.glob("*.txt"):
                shutil.copy(f, out_r / f.name)
        print(f"selesai: {n} citra -> {DST}")

if __name__ == "__main__":
    main()
```

ml/kaggle/out_train/loop-tb/api/main.py

```
"""Loop-TB API – inference + Grad-CAM + band triase.

Jalankan lokal:
    uvicorn api.main:app --host 0.0.0.0 --port 8000

Env:
    LOOPTB_CKPT path ke checkpoint best.pt (default ml/runs/convnext_tiny/best.pt)
    LOOPTB_BANDS path ke bands.json hasil evaluate.py (opsional; default  $\tau=0.5/0.5$ )
"""
from __future__ import annotations

import base64
import io
import json
import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.cm as cm
import numpy as np
import timm
import torch
import torch.nn as nn
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
from torchvision import transforms as T

CKPT_PATH = Path(os.environ.get("LOOPTB_CKPT", "ml/runs/convnext_tiny/best.pt"))
BANDS_PATH = Path(os.environ.get("LOOPTB_BANDS", "")) if os.environ.get("LOOPTB_BANDS") else None
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

app = FastAPI(title="loop-tb", version="0.1.0")
app.add_middleware(
    CORSMiddleware, allow_origins=["*"], allow_credentials=False,
    allow_methods=["*"], allow_headers=["*"],
)

_state: dict = {}

def _load() -> None:
    if not CKPT_PATH.exists():
        raise RuntimeError(f"Checkpoint tidak ditemukan: {CKPT_PATH}")
    ckpt = torch.load(CKPT_PATH, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()
    bands = {"tau_high": 0.5, "tau_low": 0.5}
    if BANDS_PATH and BANDS_PATH.exists():
        b = json.loads(BANDS_PATH.read_text())
        bands = {"tau_high": float(b.get("tau_high", 0.5)), "tau_low": float(b.get("tau_low", 0.5))}
    size = int(ckpt["config"].get("img_size", 224))
    _state.update(model=model, arch=ckpt["arch"], img_size=size, bands=bands)
    print(f"[loop-tb] loaded {ckpt['arch']} @ {size}px bands={bands}")

_tx = None

def preprocess(img: Image.Image) -> torch.Tensor:
    global _tx
    if _tx is None:
        s = _state["img_size"]
        _tx = T.Compose([T.Resize(int(s * 1.14)), T.CenterCrop(s), T.ToTensor(), T.Normalize(MEAN, STD)])
    return _tx(img.convert("RGB")).unsqueeze(0)

def band_of(p: float, bands: dict) -> str:
    if p >= bands["tau_high"]:
        return "rujuk_prioritas"
    if p < bands["tau_low"]:
        return "negatif_skrining"
    return "ragu"

def gradcam(model: nn.Module, x: torch.Tensor) -> np.ndarray:
    """Grad-CAM generik via forward_features (timm), tanpa asumsi arsitektur."""
    model.zero_grad(set_to_none=True)
    feats = model.forward_features(x)
    feats.retain_grad()
    logit = model.forward_head(feats).squeeze(1)
    logit.backward()
    w = feats.grad.mean(dim=(2, 3), keepdim=True)
    cam = (w * feats).sum(dim=1, keepdim=True).clamp(min=0)
    cam = nn.functional.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
```

```

cam = cam / (cam.amax(dim=(2, 3), keepdim=True) + 1e-8)
return cam[0, 0].detach().numpy()

def overlay_png(img: Image.Image, cam: np.ndarray) -> str:
    img_np = np.asarray(img.convert("RGB").resize((cam.shape[1], cam.shape[0])), dtype=np.float32) / 255.0
    heat = cm.jet(cam)[..., :3]
    alpha = (cam[..., None] * 0.45)
    blend = img_np * (1 - alpha) + heat.astype(np.float32) * alpha
    out = Image.fromarray((np.clip(blend, 0, 1) * 255).astype(np.uint8))
    buf = io.BytesIO()
    out.save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode()

@app.on_event("startup")
def startup() -> None:
    _load()

@app.get("/health")
def health() -> dict:
    return {
        "status": "ok",
        "arch": _state.get("arch"),
        "img_size": _state.get("img_size"),
        "bands": _state.get("bands"),
    }

@app.post("/predict")
async def predict(file: UploadFile = File(...)) -> dict:
    if "model" not in _state:
        raise HTTPException(503, "model belum siap")
    raw = await file.read()
    try:
        img = Image.open(io.BytesIO(raw))
        img.load()
    except Exception as e:
        raise HTTPException(400, f"bukan citra valid: {e}")

    x = preprocess(img)
    with torch.no_grad():
        logit = _state["model"](x).squeeze(1)
        p = float(torch.sigmoid(logit).item())
        band = band_of(p, _state["bands"])

    cam = gradcam(_state["model"], x)
    heatmap_b64 = overlay_png(img, cam)

    return {
        "p_tb": round(p, 4),
        "band": band,
        "bands": _state["bands"],
        "arch": _state.get("arch"),
        "disclaimer": "Alat triase skrining – bukan diagnosis. Wajib konfirmasi (GeneXpert) oleh tenaga kesehatan.",
        "heatmap_png_b64": heatmap_b64,
    }

```

ml/kaggle/out_train/loop-tb/ml/data/prepare_data.py

```
#!/usr/bin/env python3
"""Build a provenance-aware manifest for TB chest X-ray training.

Sumber & aturan label (lihat docs/02-arsitektur.md):
- TBX11K (Kaggle: vbookshelf/tbx11k-simplified): folder per kelas.
  positif = {tb}; negatif = {healthy, sick}; latent_tb DIKELUARKAN (ambigu).
- Rahman composite (Kaggle: tawsifurrahman/tuberculosis-tb-chest-xray-dataset):
  folder Normal/ -> 0, Tuberculosis/ -> 1.
- NLM Shenzhen/Montgomery: konvensi nama file resmi `*_0` = normal, `*_1` = TB.

Dedup (WAJIB – mitigasi class-conditional acquisition confounding):
1. pHash 64-bit; Hamming <= --dup-threshold dianggap duplikat (union-find transitif).
2. Integritas external: citra train yang sekelompok dengan citra external DIBUANG dari train.
3. Duplikat internal train dibuang (keep-first, urutan deterministik).
4. Split val dilakukan per-klaster (tidak ada kebocoran near-dup train->val).

Output (ml/data/artifacts/):
- manifest.json, dedup_report.json, train.csv, val.csv, external.csv
"""
from __future__ import annotations

import argparse
import csv
import json
import re
from collections import Counter
from pathlib import Path

import numpy as np
from PIL import Image
from tqdm import tqdm

try:
    import imagehash
except ImportError: # pragma: no cover
    raise SystemExit("Butuh imagehash: pip install imagehash")

IMG_EXTS = {".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff"}
POS_TOKENS = {"tb", "tuberculosis", "tuberculose"}
NEG_TOKENS = {"healthy", "normal", "sick"}
EXCLUDE_TOKENS = {"latent", "latent_tb", "latenttb"}

def _rel(p: Path, root: Path) -> str:
    """Path relatif terhadap root repo (portable lokal <-> Colab); fallback absolut."""
    try:
        return str(p.relative_to(root))
    except ValueError:
        return str(p)

def _tokens(path: Path) -> set[str]:
    parts = [p.lower() for p in path.parts]
    parts += [path.stem.lower()]
    return set(parts)

def load_tbx11k_map(raw: Path) -> dict[str, int | None]:
    """Label otoritatif dari data.csv TBX11K: 1=active_tb, 0=no_tb, None=latent (dikeluarkan)."""
    m: dict[str, int | None] = {}
    for p in raw.rglob("data.csv"):
        if "tbx11k" not in str(p).lower():
            continue
        for r in csv.DictReader(p.open()):
            f = (r.get("fname") or "").strip()
            if not f:
                continue
            t = (r.get("tb_type") or "").lower()
            tgt = (r.get("target") or "").lower()
            if "latent" in t:
                m.setdefault(f, None)
            elif tgt == "tb":
                m[f] = 1
            else:
                m.setdefault(f, 0)
    return m

def classify(path: Path, raw: Path, tbx_map: dict[str, int | None]) -> tuple[int, str] | None:
    """Return (label, source) atau None kalau tidak bisa diklasifikasi."""
    rel = path.relative_to(raw)
    low = str(rel).lower()

    # --- NLM Shenzhen / Montgomery: konvensi suffix nama file ---
    m = re.search(r"(chncxr|mcucxr)_d+_(\d+)", path.stem.lower())
    if m:
        label = int(m.group(2))
        source = "shenzhen" if m.group(1) == "chncxr" else "montgomery"
```

```

        return label, source

toks = _tokens(rel)
if toks & EXCLUDE_TOKENS or any(t.startswith("latent") for t in toks):
    return None

if "tbx11k" in low:
    if tbx_map:
        lab = tbx_map.get(path.name, None)
        return None if lab is None else (lab, "tbx11k")
    # fallback bila data.csv tidak ada: prefix nama file
    stem = path.stem.lower()
    if stem.startswith("t"):
        return 1, "tbx11k"
    if stem.startswith(("h", "s")):
        return 0, "tbx11k"
    return None

if toks & POS_TOKENS:
    return 1, "rahman"
if toks & NEG_TOKENS:
    return 0, "rahman"
return None

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[rb] = ra

_POP = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)

def hamming_pairs(hashes: np.ndarray, threshold: int, chunk: int = 512) -> list[tuple[int, int]]:
    """Semua pasangan (i<j) dengan Hamming distance <= threshold (chunked, hemat memori)."""
    n = len(hashes)
    h8 = hashes.view(np.uint8).reshape(n, 8)
    pairs: list[tuple[int, int]] = []
    for start in range(0, n, chunk):
        stop = min(start + chunk, n)
        xor = np.bitwise_xor(h8[start:stop, None, :], h8[None, :, :])
        dist = _POP[xor].sum(axis=-1) # (chunk, n)
        rows, cols = np.where(dist <= threshold)
        for r, c in zip(rows, cols):
            i, j = start + int(r), int(c)
            if i < j:
                pairs.append((i, j))
    return pairs

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--raw", type=Path, default=Path("data/raw"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts"))
    ap.add_argument("--dup-threshold", type=int, default=3, help="Hamming pHash <= ini dianggap duplikat")
    ap.add_argument("--val-frac", type=float, default=0.15)
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    raw = args.raw.resolve()
    root = raw.parent.parent # asumsi struktur <root>/data/raw

    # ----- 1) discovery -----
    labeled: list[dict] = []
    unlabeled = Counter()
    tbx_map = load_tbx11k_map(raw)
    print(f"[discovery] label TBX11K dari data.csv: {len(tbx_map)} entri")
    for p in sorted(raw.rglob("*")):
        if p.suffix.lower() not in IMG_EXTS:
            continue
        cls = classify(p, raw, tbx_map)
        if cls is None:
            unlabeled[str(p.relative_to(raw).parent)] += 1
            continue
        label, source = cls
        labeled.append({"path": _rel(p, root), "label": label, "source": source})

    print(f"[discovery] berlabel={len(labeled)} tak-terklasifikasi={sum(unlabeled.values())}")
    for k, v in unlabeled.most_common(10):
        print(f" skip {v:6d} {k}")

```

```

if not labeled:
    raise SystemExit("Tidak ada citra berlabel – cek struktur data/raw")

# ----- 2) pHash + kluster duplikat -----
print("[dedup] menghitung pHash...")
ok: list[dict] = []
for item in tqdm(labeled, unit="img"):
    try:
        with Image.open(item["path"]) as im:
            h = imagehash.phash(im.convert("L"), hash_size=8)
            item["phash"] = int(str(h), 16)
            ok.append(item)
    except Exception as e: # gambar korup
        print(f" warn: gagal baca {item['path']}: {e}")
labeled = ok

hashes = np.array([it["phash"] for it in labeled], dtype=np.uint64)
pairs = hamming_pairs(hashes, args.dup_threshold)
dsu = DSU(len(labeled))
for i, j in pairs:
    dsu.union(i, j)
for idx, it in enumerate(labeled):
    it["cluster"] = dsu.find(idx)

by_cluster: dict[int, list[dict]] = {}
for it in labeled:
    by_cluster.setdefault(it["cluster"], []).append(it)

# ----- 3) lock external, buang kebocoran dari train -----
ext = [it for it in labeled if it["source"] in ("shenzhen", "montgomery")]
ext_clusters = {it["cluster"] for it in ext}
train_pool, dropped_cross = [], []
for it in labeled:
    if it["source"] in ("shenzhen", "montgomery"):
        continue
    (dropped_cross if it["cluster"] in ext_clusters else train_pool).append(it)

# duplikat internal train: keep-first per kluster (deterministik via path)
keep, dropped_internal = [], []
seen_clusters: set[int] = set()
for it in sorted(train_pool, key=lambda x: x["path"]):
    if it["cluster"] in seen_clusters:
        dropped_internal.append(it)
    else:
        seen_clusters.add(it["cluster"])
        keep.append(it)
train_pool = keep

# ----- 4) split val per-kluster -----
rng = np.random.default_rng(args.seed)
sizes = Counter(it["cluster"] for it in train_pool)
pool_clusters = sorted(sizes)
rng.shuffle(pool_clusters)
target_val = int(len(train_pool) * args.val_frac)
val_clusters: set[int] = set()
acc = 0
for c in pool_clusters:
    if acc >= target_val:
        break
    val_clusters.add(c)
    acc += sizes[c]
train = [it for it in train_pool if it["cluster"] not in val_clusters]
val = [it for it in train_pool if it["cluster"] in val_clusters]

def dump(items: list[dict]) -> list[dict]:
    return [
        {k: it[k] for k in ("path", "label", "source", "cluster", "phash")}
        for it in sorted(items, key=lambda x: x["path"])
    ]

manifest = {
    "meta": {
        "root": str(root),
        "raw": _rel(raw, root),
        "dup_threshold": args.dup_threshold,
        "val_frac": args.val_frac,
        "seed": args.seed,
        "counts": {
            "train": len(train), "val": len(val), "external": len(ext),
            "dropped_cross_corpus": len(dropped_cross),
            "dropped_internal_dup": len(dropped_internal),
            "unlabeled": sum(unlabeled.values()),
        },
    },
    "train": dump(train),
    "val": dump(val),
    "external": dump(ext),
}
(args.out / "manifest.json").write_text(json.dumps(manifest, indent=1))
(args.out / "dedup_report.json").write_text(json.dumps({
    "pairs_within_threshold": len(pairs),
    "clusters": len(by_cluster),
}))

```

```

        "dropped_cross_corpus": dump(dropped_cross),
        "dropped_internal_dup": dump(dropped_internal),
        "unlabeled_by_dir": dict(unlabeled),
    }, indent=1))

for split in ("train", "val", "external"):
    with (args.out / f"{split}.csv").open("w") as f:
        f.write("path,label,source,cluster\n")
        for it in manifest[split]:
            f.write(f"{it['path']},{it['label']},{it['source']},{it['cluster']}\n")

def comp(items):
    c = Counter((it["source"], it["label"]) for it in items)
    return " ".join(f"{s}/{ 'TB' if l else 'NEG' }={n}" for (s, l), n in sorted(c.items()))

print("\n=== RINGKASAN ===")
print(f"train      n={len(train):6d}  {comp(train)}")
print(f"val        n={len(val):6d}    {comp(val)}")
print(f"external  n={len(ext):6d}    {comp(ext)}")
print(f"dibuang  cross-corpus={len(dropped_cross)}  internal-dup={len(dropped_internal)}")
print(f"manifest -> {args.out/'manifest.json'}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_train/loop-tb/ml/data/prepare_meta.py

```
#!/usr/bin/env python3
"""Ekstrak metadata klinis NLM (Montgomery + Shenzhen) → CSV untuk analisis subgroup.

Output: ml/data/artifacts/meta_nlm.csv
Kolom: filename,sex,age,reading,scar_flag
- Montgomery: "Patient's Sex: F / Patient's Age: 027Y / <reading>"
- Shenzhen: "male 45yrs / <reading>" (kadang terkontaminasi antar-file)
- scar_flag: menyebut scar/inactive/fibrosis → untuk pembahasan mode gagal
            "healed scar false positive" (selaras audit Bilal 2026).
"""
from __future__ import annotations

import argparse
import csv
import re
from pathlib import Path

SEX_M = re.compile(r"sex[:\s]+(m|male)\b", re.I)
SEX_F = re.compile(r"sex[:\s]+(f|female)\b", re.I)
SEX_FMT2 = re.compile(r"\b(male|female)\b", re.I)
AGE_MONT = re.compile(r"age[:\s]+(\d{1,3})\s*Y?\b", re.I)
AGE_FMT2 = re.compile(r"\b(\d{1,3})\s*yrs\b", re.I)
SCAR = re.compile(r"scar|inactive|fibro", re.I)

def parse_montgomery(text: str) -> dict | None:
    age = AGE_MONT.search(text)
    sex = "F" if SEX_F.search(text) else ("M" if SEX_M.search(text) else "")
    if not age:
        return None
    # reading = baris setelah Age
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    reading = lines[2] if len(lines) > 2 else ""
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": int(bool(SCAR.search(reading)))}

def parse_shenzhen(text: str) -> dict | None:
    age = AGE_FMT2.search(text)
    sex_m = SEX_FMT2.findall(text)
    sex = sex_m[0][0].upper() if sex_m else ""
    if not age:
        return None
    # reading = token huruf setelah "yrs"
    tail = text[age.end():]
    reading = ""
    m = re.search(r"[A-Za-z]{2,}", tail)
    if m:
        reading = m.group(0)
    return {"sex": sex, "age": int(age.group(1)), "reading": reading, "scar_flag": 0}

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--nlm", type=Path, default=Path("data/raw/nlm"))
    ap.add_argument("--out", type=Path, default=Path("ml/data/artifacts/meta_nlm.csv"))
    args = ap.parse_args()
    args.out.parent.mkdir(parents=True, exist_ok=True)

    rows, n_fail = [], 0
    for corpus, parser in (("montgomery", parse_montgomery), ("shenzhen", parse_shenzhen)):
        d = args.nlm / corpus / "ClinicalReadings"
        for p in sorted(d.glob("*.txt")):
            r = parser(p.read_text(errors="ignore"))
            if r is None:
                n_fail += 1
                continue
            r["filename"] = p.stem + ".png"
            r["corpus"] = corpus
            rows.append(r)

    with args.out.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["filename", "corpus", "sex", "age", "reading", "scar_flag"])
        w.writeheader()
        w.writerows(rows)

    from collections import Counter
    print(f"meta -> {args.out} n={len(rows)} gagal-parse={n_fail}")
    print("per korpus:", Counter(r["corpus"] for r in rows))
    print("sex:", Counter(r["sex"] for r in rows))
    print("scar_flag:", Counter(r["scar_flag"] for r in rows))

if __name__ == "__main__":
    main()
```

ml/kaggle/out_train/loop-tb/ml/evaluate.py

```
#!/usr/bin/env python3
"""Evaluasi rigor: kalibrasi, CI bootstrap, transfer, band triase, subgroup.

Contoh (ensemble + temperature scaling):
python ml/evaluate.py --runs ml/runs/efficientnet_b0 ml/runs/convnext_tiny \
    --manifest ml/data/artifacts/manifest.json --split external --temp-scale --out ml/runs/eval_external
"""
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import brier_score_loss, roc_auc_score, roc_curve
from torch.utils.data import DataLoader
from tqdm import tqdm

import sys
sys.path.insert(0, str(Path(__file__).resolve().parent))
from train import CXRManifest # noqa: E402

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-x))

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 -> fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

def threshold_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    fpr, tpr, thr = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(thr[ok][np.argmax(tpr[ok])]) if ok.any() else 0.5

def ece(y: np.ndarray, p: np.ndarray, bins: int = 15) -> float:
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for i in range(bins):
        m = (p >= edges[i]) & ((p < edges[i + 1]) if i < bins - 1 else (p <= edges[i + 1]))
        if m.sum():
            total += m.mean() * abs(p[m].mean() - y[m].mean())
    return float(total)

def bootstrap_auroc(y: np.ndarray, p: np.ndarray, n: int = 1000, seed: int = 42) -> tuple[float, float]:
    rng = np.random.default_rng(seed)
    vals = []
    for _ in range(n):
        idx = rng.integers(0, len(y), len(y))
        if len(np.unique(y[idx])) > 1:
            vals.append(roc_auc_score(y[idx], p[idx]))
    return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))

@torch.no_grad()
def infer(ckpt_path: Path, items: list[dict], img_size: int, device: torch.device, workers: int) -> np.ndarray:
    ckpt = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    arch = ckpt["arch"]
    model = timm.create_model(arch, pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.to(device).eval()
    dl = DataLoader(CXRManifest(items, train=False, img_size=img_size), batch_size=64,
                    shuffle=False, num_workers=workers)

    out = []
    for x, _, _ in tqdm(dl, desc=f"infer {ckpt_path.parent.name}", leave=False):
        x = x.to(device)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            out.append(model(x).squeeze(1).float().cpu().numpy())
    return np.concatenate(out)
```

```

def fit_temperature(val_logits: list[np.ndarray], val_y: np.ndarray) -> float:
    """Cari T>0 yang meminimalkan NLL pada val (grid search halus)."""
    logits = torch.tensor(np.mean(val_logits, axis=0), dtype=torch.float32)
    y = torch.tensor(val_y, dtype=torch.float32)
    best_t, best_nll = 1.0, float("inf")
    for t in np.linspace(0.25, 10.0, 200):
        nll = nn.functional.binary_cross_entropy_with_logits(logits / t, y).item()
        if nll < best_nll:
            best_t, best_nll = float(t), nll
    return best_t

def reliability_plot(y: np.ndarray, p: np.ndarray, path: Path, title: str) -> None:
    edges = np.linspace(0, 1, 11)
    mids, accs = [], []
    for i in range(10):
        m = (p >= edges[i]) & (p < edges[i + 1] if i < 9 else p <= edges[i + 1])
        if m.sum():
            mids.append(p[m].mean())
            accs.append(y[m].mean())
    plt.figure(figsize=(4.2, 4.2))
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.plot(mids, accs, "o-", color="#d62728", lw=2)
    plt.xlabel("Confidence"), plt.ylabel("Empirical TB rate"), plt.title(title)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def roc_plot(curves: dict[str, tuple[np.ndarray, np.ndarray]], path: Path) -> None:
    plt.figure(figsize=(5, 5))
    for name, (y, p) in curves.items():
        fpr, tpr, _ = roc_curve(y, p)
        plt.plot(fpr, tpr, lw=2, label=f"{name} (AUROC {roc_auc_score(y, p):.3f})")
    plt.plot([0, 1], [0, 1], "--", color="#999", lw=1)
    plt.xlabel("FPR"), plt.ylabel("TPR (sensitivitas)"), plt.legend(fontsize=8)
    plt.tight_layout(), plt.savefig(path, dpi=160), plt.close()

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--runs", type=Path, nargs="+", required=True)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--split", default="external", choices=["val", "external"])
    ap.add_argument("--temp-scale", action="store_true")
    ap.add_argument("--meta", type=Path, default=None, help="CSV filename,sex,age untuk subgroup (opsional)")
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--workers", type=int, default=8)
    args = ap.parse_args()

    args.out.mkdir(parents=True, exist_ok=True)
    manifest = json.loads(args.manifest.read_text())
    items = manifest[args.split]
    y = np.array([it["label"] for it in items])

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    per_run_target, per_run_val, tscales = {}, {}, {}
    for run in args.runs:
        ckpt = torch.load(run / "best.pt", map_location="cpu", weights_only=False)
        img_size = ckpt["config"]["img_size"]
        per_run_target[run.name] = infer(run / "best.pt", items, img_size, device, args.workers)
        val_items = manifest["val"]
        per_run_val[run.name] = infer(run / "best.pt", val_items, img_size, device, args.workers)
        if args.temp_scale:
            tscales[run.name] = fit_temperature([per_run_val[run.name]], np.array([it["label"] for it in val_items]))

    per_run_target = {k: sigmoid(v) for k, v in per_run_target.items()}
    per_run_val = {k: sigmoid(v) for k, v in per_run_val.items()}
    val_y = np.array([it["label"] for it in manifest["val"]])

    # kalibrasi per-run pada val, lalu terapkan T ke target (bukan val-fit pada target!)
    if args.temp_scale:
        for k in per_run_target:
            per_run_val[k] = sigmoid(np.log(np.clip(per_run_val[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_val[k], 1e-6, 1 - 1e-6)))
            / tscales[k])
            lg = np.log(np.clip(per_run_target[k], 1e-6, 1 - 1e-6) / (1 - np.clip(per_run_target[k], 1e-6, 1 - 1e-6)))
            per_run_target[k] = sigmoid(lg / tscales[k])

    # ensemble = rata-rata; ketidakpastian = std antar anggota
    stack_t = np.stack(list(per_run_target.values()))
    p_ens = stack_t.mean(axis=0)
    u_ens = stack_t.std(axis=0)
    stack_v = np.stack(list(per_run_val.values()))
    p_ens_v = stack_v.mean(axis=0)

    tau_high = threshold_at_spec(val_y, p_ens_v, 0.70) # spec>=0.70 (TPP WHO)
    tau_low = threshold_at_spec(val_y, p_ens_v, 0.95)
    if tau_low > tau_high:
        tau_low, tau_high = tau_high, tau_low

    band = np.where(p_ens >= tau_high, "rujuk_prioritas", np.where(p_ens < tau_low, "negatif_skrining", "ragu"))
    band_report = {}
    for b in ("rujuk_prioritas", "ragu", "negatif_skrining"):
        m = band == b

```

```

        band_report[b] = {"n": int(m.sum()), "frac": float(m.mean()),
                          "tb_rate": float(y[m].mean()) if m.sum() else None}

lo, hi = bootstrap_aucroc(y, p_ens)
metrics = {
    "split": args.split, "n": int(len(y)), "pos": int(y.sum()),
    "aucroc": float(roc_auc_score(y, p_ens)), "aucroc_ci95": [lo, hi],
    "sens_at_spec_90": sens_at_spec(y, p_ens, 0.90),
    "sens_at_spec_70": sens_at_spec(y, p_ens, 0.70),
    "ece": ece(y, p_ens), "brier": float(brier_score_loss(y, p_ens)),
    "temp_scales": tscales, "tau_low": tau_low, "tau_high": tau_high,
    "bands": band_report,
    "deferral": {"rate": float((band == "ragu").mean()),
                  "error_on_deferred": float(1 - (band[band == "ragu"] == y[band == "ragu"]).mean()) if (band == "ragu").any()
                  else None},
    "per_run_aucroc": {k: float(roc_auc_score(y, v)) for k, v in per_run_target.items()},
}

if args.meta and args.meta.exists():
    meta = {r["filename"]: r for r in csv.DictReader(args.meta.open())}
    for key in ("sex", "age"):
        groups: dict[str, list[int]] = {}
        for i, it in enumerate(items):
            row = meta.get(Path(it["path"]).name, {})
            g = row.get(key, "NA") or "NA"
            if key == "age":
                try:
                    a = float(g)
                    g = "0-30" if a < 30 else "30-50" if a < 50 else "50+"
                except ValueError:
                    g = "NA"
            groups.setdefault(g, []).append(i)
        metrics[f"auroc_by_{key}"] = {
            g: (float(roc_auc_score(y[idx], p_ens[idx])) if len(np.unique(y[idx])) > 1 else None)
              for g, idx in groups.items() if g != "NA" and len(idx) >= 20
        }

(args.out / "metrics.json").write_text(json.dumps(metrics, indent=1))
reliability_plot(y, p_ens, args.out / "reliability.png", f"Kalibrasi - {args.split}")
curves = {f"ens ({args.split})": (y, p_ens)}
if len(per_run_target) > 1:
    curves.update({k: (y, v) for k, v in per_run_target.items()})
roc_plot(curves, args.out / "roc.png")

print("\n=== METRICS ===")
for k, v in metrics.items():
    if isinstance(v, (int, float, str)):
        print(f"{k:18s} {v}")
print(f"bands          {band_report}")

if __name__ == "__main__":
    main()

```

ml/kaggle/out_train/loop-tb/ml/export_onnx.py

```
#!/usr/bin/env python3
"""Export checkpoint ke ONNX (opsi 17) + verifikasi onnxruntime.

Contoh:
python ml/export_onnx.py --ckpt ml/runs/efficientnet_b0/best.pt --out artifacts/tb_efficientnet_b0.onnx
"""
from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import torch

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm")

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--ckpt", type=Path, required=True)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--opset", type=int, default=17)
    args = ap.parse_args()

    ckpt = torch.load(args.ckpt, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    model.eval()

    img_size = ckpt["config"].get("img_size", 224)
    dummy = torch.randn(1, 3, img_size, img_size)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    torch.onnx.export(
        model, dummy, args.out,
        input_names=["input"], output_names=["logit"],
        dynamic_axes={"input": {0: "batch"}, "logit": {0: "batch"}},
        opset_version=args.opset, dynamo=False, # legacy: satu file mandiri, opset stabil
    )

    import onnxruntime as ort
    sess = ort.InferenceSession(str(args.out), providers=["CPUExecutionProvider"])
    ref = model(dummy).detach().numpy()
    got = sess.run(None, {"input": dummy.numpy()})[0]
    max_diff = float(np.abs(ref - got).max())
    ok = bool(np.allclose(ref, got, rtol=1e-3, atol=1e-3))
    size_mb = args.out.stat().st_size / 1e6
    print(f"ONNX ok: {args.out} ({size_mb:.1f} MB) max|diff|={max_diff:.2e} opset={args.opset}")
    assert ok, "Verifikasi ONNX gagal (allclose rtol/atol 1e-3)"

if __name__ == "__main__":
    main()
```

ml/kaggle/out_train/loop-tb/ml/kaggle/hello/hello.py

```
"""Kernel uji: verifikasi GPU, internet, versi lib, dan struktur dataset ter-mount."""
import os
import sys

lines: list[str] = []

def log(*a) -> None:
    print(*a, flush=True)
    lines.append(" ".join(str(x) for x in a))

log("python:", sys.version.split()[0])

try:
    import torch

    log("torch:", torch.__version__, "| cuda:", torch.cuda.is_available())
    if torch.cuda.is_available():
        log("gpu:", torch.cuda.get_device_name(0))
except Exception as e: # noqa: BLE001
    log("torch error:", e)

for mod in ("timm", "sklearn", "onnx", "onnxruntime"):
    try:
        m = __import__(mod)
        log(f"{mod}: {getattr(m, '__version__', '?')}")
    except Exception as e: # noqa: BLE001
        log(f"{mod}: MISSING ({e})")

log("--- input mounts ---")
for root, dirs, _ in os.walk("/kaggle/input"):
    log(root, "dirs:", dirs[:6])
    if root.count("/") > 3:
        dirs[:] = []

log("HELLO DONE")
with open("/kaggle/working/hello_result.txt", "w") as f:
    f.write("\n".join(lines) + "\n")
```

ml/kaggle/out_train/loop-tb/ml/kaggle/train/train_kernel.py

```
"""Kernel training loop-tb: clone repo → symlink dataset → train 2 arsitektur + baseline → export ONNX.

Output di /kaggle/working/loop-tb/{runs,artifacts}; diunduh via `kaggle kernels output`.
Manifest yang dipakai = split v1 frozen dari repo (deterministik; NLM external tidak dibutuhkan di sini).
"""
import json
import os
import subprocess
import sys

WORK = "/kaggle/working"
REPO = f"{WORK}/loop-tb"

def sh(cmd: list[str], **kw) -> None:
    print("+", " ".join(map(str, cmd)), flush=True)
    subprocess.run(cmd, check=True, **kw)

# 1) repo + deps
if not os.path.exists(REPO):
    sh(["git", "clone", "--depth", "1", "https://github.com/Alvnc/loop-tb.git", REPO])
os.chdir(REPO)
sh([sys.executable, "-m", "pip", "install", "-q", "timm", "onnxruntime"])

# 2) symlink dataset ke struktur yang diharapkan manifest (tahan variasi struktur mount)
os.makedirs("data/raw", exist_ok=True)

def pick(dst: str, cands: list[str], markers: tuple[str, ...]) -> None:
    """Symlink kandidat mount pertama yang punya 'marker' (images/Normal/Tuberculosis)."""
    if os.path.exists(dst):
        return
    for c in cands:
        if any(os.path.isdir(os.path.join(c, m)) for m in markers):
            os.symlink(c, dst)
            print("symlink", dst, "->", c, flush=True)
            return
    for c in cands:
        if os.path.exists(c):
            os.symlink(c, dst)
            print("symlink (fallback)", dst, "->", c, flush=True)
            return
    print("WARNING: tidak ada kandidat mount untuk", dst, flush=True)

TBX_CANDS = [
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified",
]
RAH_CANDS = [
    "/kaggle/input/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
    "/kaggle/input/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
]
pick("data/raw/tbx11k-simplified", TBX_CANDS, ("images",))
pick("data/raw/TB_Chest_Radiography_Database", RAH_CANDS, ("Normal", "Tuberculosis"))

for root, dirs, files in os.walk("/kaggle/input"):
    print("mount:", root, "dirs:", dirs[:6], "files:", len(files), flush=True)
    if root.count("/") > 4:
        dirs[:] = []

# 3) sanity: semua path train/val harus ada
manifest = json.load(open("ml/data/artifacts/manifest.json"))
missing = [it["path"] for s in ("train", "val") for it in manifest[s] if not os.path.exists(it["path"])]
print(f"train={len(manifest['train'])} val={len(manifest['val'])} missing={len(missing)}", flush=True)
print("missing contoh:", missing[:3], flush=True)
assert not missing, "data train/val tidak lengkap – cek mount dataset"

# 4) training (dua arsitektur + baseline fitur beku)
JOBS = [
    ("efficientnet_b0", "runs/eb0", ["--epochs", "14", "--batch", "64"]),
    ("convnext_tiny", "runs/convnext", ["--epochs", "12", "--batch", "48"]),
]
for arch, out, extra in JOBS:
    sh(
        [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
         "--arch", arch, "--balanced", "--out", out, *extra]
    )

sh(
    [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
     "--arch", "efficientnet_b0", "--freeze", "--lr", "1e-3", "--epochs", "8", "--batch", "64",
     "--out", "runs/linear_probe"]
)
```

```
# 5) export ONNX (satu file mandiri per model)
os.makedirs("artifacts", exist_ok=True)
for run, name in (("runs/eb0", "eb0"), ("runs/convnext", "convnext")):
    sh([sys.executable, "ml/export_onnx.py", "--ckpt", f"{run}/best.pt", "--out", f"artifacts/{name}.onnx"])

print("ALL DONE", flush=True)
```

ml/kaggle/out_train/loop-tb/ml/train.py

```
#!/usr/bin/env python3
"""Train TB CXR classifier dari manifest provenance-aware.

Contoh:
python ml/train.py --manifest ml/data/artifacts/manifest.json --arch efficientnet_b0 \
    --epochs 20 --out ml/runs/efficientnet_b0

Baseline linear probe (fitur beku):
python ml/train.py ... --freeze --lr 1e-3 --out ml/runs/linear_probe
"""
from __future__ import annotations

import argparse
import csv
import json
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import roc_auc_score, roc_curve
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import transforms as T
from tqdm import tqdm

try:
    import timm
except ImportError: # pragma: no cover
    raise SystemExit("Butuh timm: pip install timm")

MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

class CXRManifest(Dataset):
    def __init__(self, items: list[dict], train: bool, img_size: int):
        self.items = items
        if train:
            self.tx = T.Compose([
                T.RandomResizedCrop(img_size, scale=(0.75, 1.0), ratio=(0.9, 1.1)),
                T.RandomRotation(7, fill=0),
                T.ColorJitter(brightness=0.15, contrast=0.15),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])
        else:
            self.tx = T.Compose([
                T.Resize(int(img_size * 1.14)),
                T.CenterCrop(img_size),
                T.ToTensor(),
                T.Normalize(MEAN, STD),
            ])

    def __len__(self) -> int:
        return len(self.items)

    def __getitem__(self, i: int):
        from PIL import Image
        it = self.items[i]
        with Image.open(it["path"]) as im:
            x = self.tx(im.convert("RGB"))
        return x, torch.tensor(float(it["label"])), i

def sens_at_spec(y: np.ndarray, p: np.ndarray, spec: float) -> float:
    """Sensitivitas pada spesifisitas >= spec (mis. spec=0.90 → fpr <= 0.10)."""
    fpr, tpr, _ = roc_curve(y, p)
    ok = fpr <= 1 - spec + 1e-9
    return float(tpr[ok].max()) if ok.any() else float("nan")

@torch.no_grad()
def predict(model: nn.Module, loader: DataLoader, device: torch.device) -> tuple[np.ndarray, np.ndarray]:
    model.eval()
    logits_all, labels_all = [], []
    for x, y, _ in loader:
        x = x.to(device, non_blocking=True)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            logits = model(x).squeeze(1).float()
        logits_all.append(logits.cpu().numpy())
        labels_all.append(y.numpy())
    return np.concatenate(logits_all), np.concatenate(labels_all)

def evaluate(logits: np.ndarray, y: np.ndarray) -> dict:
    p = 1 / (1 + np.exp(-logits))
    out = {"n": int(len(y)), "pos": int(y.sum())}
    if len(np.unique(y)) > 1:
```

```

        out["auroc"] = float(roc_auc_score(y, p))
        out["sens_at_spec_90"] = sens_at_spec(y, p, 0.90)
        out["sens_at_spec_70"] = sens_at_spec(y, p, 0.70)
    return out

```

```

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--manifest", type=Path, required=True)
    ap.add_argument("--arch", default="efficientnet_b0")
    ap.add_argument("--epochs", type=int, default=20)
    ap.add_argument("--batch", type=int, default=32)
    ap.add_argument("--lr", type=float, default=3e-4)
    ap.add_argument("--wd", type=float, default=1e-4)
    ap.add_argument("--img-size", type=int, default=224)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--workers", type=int, default=8)
    ap.add_argument("--freeze", action="store_true", help="linear probe: backbone beku")
    ap.add_argument("--balanced", action="store_true", help="WeightedRandomSampler")
    ap.add_argument("--limit", type=int, default=0, help="smoke test: batasi n sampel (0=semua)")
    args = ap.parse_args()

    torch.manual_seed(args.seed)
    np.random.seed(args.seed)
    args.out.mkdir(parents=True, exist_ok=True)

    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
    manifest = json.loads(args.manifest.read_text())

    def take(items: list[dict], n: int) -> list[dict]:
        """Subset berstratifikasi untuk smoke test."""
        if not n:
            return items
        pos = [it for it in items if it["label"] == 1]
        neg = [it for it in items if it["label"] == 0]
        n = min(n, len(items))
        kp = min(len(pos), max(1, n // 4))
        return pos[:kp] + neg[: max(n - kp, 0)]

    train_items = take(manifest["train"], args.limit)
    val_items = take(manifest["val"], args.limit)

    ds_tr = CXRMManifest(train_items, train=True, img_size=args.img_size)
    ds_va = CXRMManifest(val_items, train=False, img_size=args.img_size)
    sampler = None
    if args.balanced:
        labels = np.array([it["label"] for it in train_items])
        w = np.where(labels == 1, 1.0 / labels.sum(), 1.0 / (len(labels) - labels.sum()))
        sampler = WeightedRandomSampler(torch.from_numpy(w), num_samples=len(labels), replacement=True)
    dl_tr = DataLoader(ds_tr, batch_size=args.batch, shuffle=sampler is None, sampler=sampler,
                      num_workers=args.workers, pin_memory=(device.type == "cuda"), drop_last=True)
    dl_va = DataLoader(ds_va, batch_size=args.batch, shuffle=False, num_workers=args.workers, pin_memory=(device.type == "cuda"))

    model = timm.create_model(args.arch, pretrained=True, num_classes=1).to(device)
    if args.freeze:
        for n, p in model.named_parameters():
            p.requires_grad = "head" in n or "classifier" in n or "fc" in n
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=args.lr, weight_decay=args.wd)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(args.epochs, 1))
    loss_fn = nn.BCEWithLogitsLoss()
    scaler = torch.amp.GradScaler(enabled=(device.type == "cuda"))

    print(f"device={device} arch={args.arch} freeze={args.freeze} "
          f"train={len(ds_tr)} val={len(ds_va)} params={sum(p.numel() for p in params)/1e6:.2f}M")

    hist = []
    best = {"auroc": -1.0, "epoch": -1}
    for epoch in range(1, args.epochs + 1):
        model.train()
        t0, losses = time.time(), []
        for x, y, _ in tqdm(dl_tr, desc=f"epoch {epoch:02d}", leave=False):
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device.type, enabled=(device.type == "cuda")):
                logits = model(x).squeeze(1)
                loss = loss_fn(logits.float(), y)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            losses.append(loss.item())
        sched.step()

        logits, y_va = predict(model, dl_va, device)
        m = evaluate(logits, y_va)
        row = {"epoch": epoch, "loss": float(np.mean(losses)), "lr": opt.param_groups[0]["lr"],
              "sec": round(time.time() - t0, 1), **m}
        hist.append(row)
    print(f"epoch {epoch:02d} | loss {row['loss']:.4f} | AUROC {m.get('auroc', float('nan')):.4f} "
          f"| sens@spec90 {m.get('sens_at_spec_90', float('nan')):.3f} | {row['sec']}s")

```

```

torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
           "epoch": epoch, "val": m}, args.out / "last.pt")
if m.get("auroc", -1) > best["auroc"]:
    best = {"auroc": m["auroc"], "epoch": epoch}
    torch.save({"arch": args.arch, "state_dict": model.state_dict(), "config": vars(args),
               "epoch": epoch, "val": m}, args.out / "best.pt")
    with (args.out / "val_preds.csv").open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["path", "label", "logit"])
        for it, lg in zip(val_items, logits):
            w.writerow([it["path"], it["label"], float(lg)])

(args.out / "history.csv").write_text(
    "epoch,loss,lr,sec,n,pos,auroc,sens_at_spec_90,sens_at_spec_70\n" + "\n".join(
        ", ".join(str(r.get(k, "")) for k in ("epoch", "loss", "lr", "sec", "n", "pos", "auroc", "sens_at_spec_90",
        "sens_at_spec_70"))
        for r in hist) + "\n")
(args.out / "metrics.json").write_text(json.dumps({"best": best, "history": hist, "config": vars(args)}, indent=1, default=str))
print(f"\nBEST epoch {best['epoch']} AUROC {best['auroc']:.4f} -> {args.out/'best.pt'}")

if __name__ == "__main__":
    main()

```

ml/kaggle/ssl/ssl_extract.py

```
"""Ekstraksi embedding SSL untuk studi representasi (ADR-007).

Encoder:
- dinov2 : timm vit_base_patch14_dinov2.lvd142m (SSL umum, 518 px)
- rad_dino: hf-hub:microsoft/rad-dino (DINOv2 pada X-ray data, 518 px)

Output di /kaggle/working/loop-tb/ssl_embeddings/:
meta_{train,val,external}.json + emb_{encoder}_{split}.npy
Dipakai oleh ml/ssl_probe.py secara lokal (sklearn, tanpa GPU).
"""
from __future__ import annotations

import json
import os
import subprocess
import sys

import numpy as np

WORK = "/kaggle/working"
REPO = f"{WORK}/loop-tb"

def sh(cmd: list[str]) -> None:
    print("+", " ".join(map(str, cmd)), flush=True)
    subprocess.run(cmd, check=True)

if not os.path.exists(REPO):
    sh(["git", "clone", "--depth", "1", "https://github.com/Alvnc/loop-tb.git", REPO])
os.chdir(REPO)
sh([sys.executable, "-m", "pip", "install", "-q", "timm", "huggingface_hub", "transformers"])

# --- mount datasets (self-healing) ---
os.makedirs("data/raw", exist_ok=True)

def pick(dst: str, cands: list[str], markers: tuple[str, ...]) -> None:
    if os.path.exists(dst):
        return
    for c in cands:
        if any(os.path.isdir(os.path.join(c, m)) for m in markers):
            os.symlink(c, dst)
            print("symlink", dst, "->", c, flush=True)
            return
    for c in cands:
        if os.path.exists(c):
            os.symlink(c, dst)
            print("symlink (fallback)", dst, "->", c, flush=True)
            return
    print("WARNING: tidak ada mount untuk", dst, flush=True)

pick("data/raw/tbx11k-simplified",
     ["/kaggle/input/datasets/vbookshelf/tbx11k-simplified/tbx11k-simplified",
      "/kaggle/input/datasets/vbookshelf/tbx11k-simplified", "/kaggle/input/tbx11k-simplified"],
     ("images",))
pick("data/raw/TB_Chest_Radiography_Database",
     ["/kaggle/input/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
      "/kaggle/input/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database"],
     ("Normal", "Tuberculosis"))
pick("data/raw/nlm",
     ["/kaggle/input/datasets/alvinreba/nlm-tb-cxr-resized/nlm",
      "/kaggle/input/datasets/alvinreba/nlm-tb-cxr-resized",
      "/kaggle/input/nlm-tb-cxr-resized/nlm",
      "/kaggle/input/nlm-tb-cxr-resized"],
     ("shenzhen", "montgomery"))

# --- siapkan daftar item (external: .png -> .jpg hasil resize) ---
manifest = json.load(open("ml/data/artifacts/manifest.json"))
train_items = manifest["train"]
val_items = manifest["val"]
ext_items = []
for it in manifest["external"]:
    p = it["path"]
    jp = p[:-4] + ".jpg" if p.lower().endswith(".png") else p
    ext_items.append({**it, "path": jp})

for name, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
    missing = [it["path"] for it in items if not os.path.exists(it["path"])]
    print(f"{name}: n={len(items)} missing={len(missing)}", missing[:2], flush=True)
    assert not missing, f"data {name} tidak lengkap"

out_dir = "ssl_embeddings"
os.makedirs(out_dir, exist_ok=True)
for split, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
    with open(f"{out_dir}/meta_{split}.json", "w") as f:
        json.dump([{"path": it["path"], "label": it["label"], "source": it["source"]} for it in items], f)
```

```

# --- ekstraksi ---
import torch # noqa: E402
from PIL import Image # noqa: E402
from torch.utils.data import DataLoader, Dataset # noqa: E402
from torchvision import transforms as T # noqa: E402

MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

class Items(Dataset):
    def __init__(self, items: list[dict], size: int, mean: tuple = MEAN, std: tuple = STD):
        self.items = items
        self.tx = T.Compose([T.Resize(size), T.CenterCrop(size), T.ToTensor(), T.Normalize(mean, std)])

    def __len__(self) -> int:
        return len(self.items)

    def __getitem__(self, i: int):
        with Image.open(self.items[i]["path"]) as im:
            x = self.tx(im.convert("RGB"))
        return x, i

@torch.no_grad()
def extract(model, items: list[dict], size: int, device: torch.device, tag: str,
            mean: tuple = MEAN, std: tuple = STD) -> np.ndarray:
    dl = DataLoader(Items(items, size, mean, std), batch_size=32, shuffle=False, num_workers=4, pin_memory=True)
    out = np.zeros((len(items), 0), dtype=np.float32)
    embs = []
    for bi, (x, _) in enumerate(dl):
        x = x.to(device, non_blocking=True)
        with torch.autocast(device.type, enabled=(device.type == "cuda")):
            feats = model(x)
        feats = feats.float().cpu().numpy()
        embs.append(feats)
        if bi % 20 == 0:
            print(f" {tag} batch {bi}/{len(dl)}", flush=True)
    out = np.concatenate(embs, axis=0)
    return out

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device, flush=True)

ENCODERS = [
    ("dinov2", "vit_base_patch14_dinov2.lvd142m", 518),
    ("rad_dino", "hf-hub:microsoft/rad-dino", 518),
]

import timm # noqa: E402

for enc_name, model_id, size in ENCODERS:
    if all(os.path.exists(f"{out_dir}/emb_{enc_name}_{s}.npy") for s in ("train", "val", "external")):
        print(f"skip {enc_name} (sudah ada)", flush=True)
        continue
    try:
        print(f"=== encoder {enc_name} ({model_id}) ===", flush=True)
        mean, std = MEAN, STD
        if enc_name == "rad_dino":
            from transformers import AutoImageProcessor, AutoModel

            hf_id = model_id.replace("hf-hub:", "")

            class HFWrap(torch.nn.Module):
                def __init__(self, m):
                    super().__init__()
                    self.m = m

                def forward(self, x):
                    out = self.m(pixel_values=x)
                    return out.last_hidden_state[:, 0] if hasattr(out, "last_hidden_state") else out.pooler_output

            proc = AutoImageProcessor.from_pretrained(hf_id)
            print("rad_dino processor:", proc.image_mean, proc.image_std, proc.size, flush=True)
            if proc.image_mean:
                mean, std = tuple(proc.image_mean), tuple(proc.image_std)
            model = HFWrap(AutoModel.from_pretrained(hf_id))
        else:
            model = timm.create_model(model_id, pretrained=True, num_classes=0, dynamic_img_size=True)
        model.to(device).eval()
        for split, items in (("train", train_items), ("val", val_items), ("external", ext_items)):
            emb = extract(model, items, size, device, f"{enc_name}/{split}", mean, std)
            np.save(f"{out_dir}/emb_{enc_name}_{split}.npy", emb)
            print(f"{enc_name} {split}: {emb.shape}", flush=True)
        with open(f"{out_dir}/encmeta_{enc_name}.json", "w") as f:
            json.dump({
                "type": "hf" if enc_name == "rad_dino" else "timm",
                "id": model_id.replace("hf-hub:", ""),
                "size": size,
                "mean": list(mean),
                "std": list(std),
            }, f)
    except Exception as e:
        print(f"Error processing {enc_name} {model_id}: {e}")

```

```
    }, f)
    del model
    torch.cuda.empty_cache()
except Exception as e: # noqa: BLE001
    print(f"ERROR encoder {enc_name}: {e}", flush=True)

print("SSL EXTRACT DONE", flush=True)
```

ml/kaggle/train/train_kernel.py

```
"""Kernel training loop-tb: clone repo → symlink dataset → train 2 arsitektur + baseline → export ONNX.

Output di /kaggle/working/loop-tb/{runs,artifacts}; diunduh via `kaggle kernels output`.
Manifest yang dipakai = split v1 frozen dari repo (deterministik; NLM external tidak dibutuhkan di sini).
"""
import json
import os
import subprocess
import sys

WORK = "/kaggle/working"
REPO = f"{WORK}/loop-tb"

def sh(cmd: list[str], **kw) -> None:
    print("+", " ".join(map(str, cmd)), flush=True)
    subprocess.run(cmd, check=True, **kw)

# 1) repo + deps
if not os.path.exists(REPO):
    sh(["git", "clone", "--depth", "1", "https://github.com/Alvnc/loop-tb.git", REPO])
os.chdir(REPO)
sh([sys.executable, "-m", "pip", "install", "-q", "timm", "onnxruntime"])

# 2) symlink dataset ke struktur yang diharapkan manifest (tahan variasi struktur mount)
os.makedirs("data/raw", exist_ok=True)

def pick(dst: str, cands: list[str], markers: tuple[str, ...]) -> None:
    """Symlink kandidat mount pertama yang punya 'marker' (images/Normal/Tuberculosis)."""
    if os.path.exists(dst):
        return
    for c in cands:
        if any(os.path.isdir(os.path.join(c, m)) for m in markers):
            os.symlink(c, dst)
            print("symlink", dst, "->", c, flush=True)
            return
    for c in cands:
        if os.path.exists(c):
            os.symlink(c, dst)
            print("symlink (fallback)", dst, "->", c, flush=True)
            return
    print("WARNING: tidak ada kandidat mount untuk", dst, flush=True)

TBX_CANDS = [
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/datasets/vbookshelf/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified/tbx11k-simplified",
    "/kaggle/input/tbx11k-simplified",
]
RAH_CANDS = [
    "/kaggle/input/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
    "/kaggle/input/tuberculosis-tb-chest-xray-dataset/TB_Chest_Radiography_Database",
]
pick("data/raw/tbx11k-simplified", TBX_CANDS, ("images",))
pick("data/raw/TB_Chest_Radiography_Database", RAH_CANDS, ("Normal", "Tuberculosis"))

for root, dirs, files in os.walk("/kaggle/input"):
    print("mount:", root, "dirs:", dirs[:6], "files:", len(files), flush=True)
    if root.count("/") > 4:
        dirs[:] = []

# 3) sanity: semua path train/val harus ada
manifest = json.load(open("ml/data/artifacts/manifest.json"))
missing = [it["path"] for s in ("train", "val") for it in manifest[s] if not os.path.exists(it["path"])]
print(f"train={len(manifest['train'])} val={len(manifest['val'])} missing={len(missing)}", flush=True)
print("missing contoh:", missing[:3], flush=True)
assert not missing, "data train/val tidak lengkap – cek mount dataset"

# 4) training (dua arsitektur + baseline fitur beku)
JOBS = [
    ("efficientnet_b0", "runs/eb0", ["--epochs", "14", "--batch", "64"]),
    ("convnext_tiny", "runs/convnext", ["--epochs", "12", "--batch", "48"]),
]
for arch, out, extra in JOBS:
    sh(
        [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
         "--arch", arch, "--balanced", "--out", out, *extra]
    )

sh(
    [sys.executable, "ml/train.py", "--manifest", "ml/data/artifacts/manifest.json",
     "--arch", "efficientnet_b0", "--freeze", "--lr", "1e-3", "--epochs", "8", "--batch", "64",
     "--out", "runs/linear_probe"]
)
```

```
# 5) export ONNX (satu file mandiri per model)
os.makedirs("artifacts", exist_ok=True)
for run, name in (("runs/eb0", "eb0"), ("runs/convnext", "convnext")):
    sh([sys.executable, "ml/export_onnx.py", "--ckpt", f"{run}/best.pt", "--out", f"artifacts/{name}.onnx"])

print("ALL DONE", flush=True)
```

api/main.py

```
"""SIGAP API – inference ensemble (supervised ckpt &/atau probe SSL) + Grad-CAM + band triase.

Jalankan lokal:
    uvicorn api.main:app --host 0.0.0.0 --port 8000

Konfigurasi (default `api/model_config.json`, override env LOOPTB_CONFIG):
{
    "score_members": [ {"path": "ml/runs/ssl_eval/probe_dinov2.npz"} ],
    "uncertainty_members": [ {"path": "ml/runs/eb0/best.pt", "T": 1.2336},
                             {"path": "ml/runs/convnext/best.pt", "T": 1.3405} ],
    "bands": {"tau_low": ..., "tau_high": ..., "tau_uncertainty": 0.2}
}

Skor p = rata-rata prob anggota `score_members` (model terbaik untuk AUROC eksternal).
Ketidakpastian u = std antar SEMUA anggota unik (disagreement lintas-keluarga SSL-supervised).
Jika hanya ada satu anggota total, u = 0.
"""
from __future__ import annotations

import base64
import io
import json
import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.cm as cm
import numpy as np
import timm
import torch
import torch.nn as nn
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
from torchvision import transforms as T

CONFIG_PATH = Path(os.environ.get("LOOPTB_CONFIG", "api/model_config.json"))
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)

app = FastAPI(title="loop-tb (SIGAP)", version="0.4.0")
app.add_middleware(
    CORSMiddleware, allow_origins=["*"], allow_credentials=False,
    allow_methods=["*"], allow_headers=["*"],
)

_state: dict = {}

def sigmoid(z: float) -> float:
    return float(1.0 / (1.0 + np.exp(-np.clip(z, -60, 60))))

class HFViT(nn.Module):
    """Pembungkus model HF (mis. RAD-DINO) → CLS token sebagai fitur."""
    def __init__(self, model_id: str):
        super().__init__()
        from transformers import AutoModel

        self.m = AutoModel.from_pretrained(model_id)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        out = self.m(pixel_values=x)
        return out.last_hidden_state[:, 0] if hasattr(out, "last_hidden_state") else out.pooler_output

def _load_member(spec: dict) -> dict:
    path = spec["path"]
    if path.endswith(".npz"):
        d = np.load(path, allow_pickle=False)
        enc_type, enc_id = str(d["encoder_type"][0]), str(d["encoder_id"][0])
        model: nn.Module = HFViT(enc_id) if enc_type == "hf" else timm.create_model(
            enc_id, pretrained=True, num_classes=0, dynamic_img_size=True
        )
    return {
        "kind": "probe",
        "model": model.eval(),
        "T": float(d["T"][0]),
        "size": int(d["size"][0]),
        "pixel_mean": tuple(float(v) for v in d["pixel_mean"]),
        "pixel_std": tuple(float(v) for v in d["pixel_std"]),
        "scaler_mean": d["scaler_mean"].astype(np.float32),
        "scaler_scale": d["scaler_scale"].astype(np.float32),
        "coef": d["coef"].astype(np.float32),
        "intercept": float(d["intercept"][0]),
        "path": path,
    }
```

```

    }
    ckpt = torch.load(path, map_location="cpu", weights_only=False)
    model = timm.create_model(ckpt["arch"], pretrained=False, num_classes=1)
    model.load_state_dict(ckpt["state_dict"])
    return {
        "kind": "torch",
        "model": model.eval(),
        "T": float(spec.get("T", 1.0)),
        "size": int(ckpt["config"].get("img_size", 224)),
        "path": path,
    }

def _load() -> None:
    cfg = json.loads(CONFIG_PATH.read_text())
    score_specs = cfg.get("score_members") or cfg.get("members") or []
    unc_specs = cfg.get("uncertainty_members") or score_specs
    cache: dict[str, dict] = {}

    def get(spec: dict) -> dict:
        p = spec["path"]
        if p not in cache:
            cache[p] = _load_member(spec)
        return cache[p]

    _state.update(
        score_members=[get(s) for s in score_specs],
        unc_members=[get(s) for s in unc_specs],
        bands={"tau_high": 0.5, "tau_low": 0.5, **cfg.get("bands", {})},
    )
    print(f"[SIGAP] score={len(_state['score_members'])} unc={len(_state['unc_members'])} bands={_state['bands']}")

def _names(members: list[dict]) -> list[str]:
    return [f"{Path(m['path']).parent.name}/{Path(m['path']).name}" for m in members]

def preprocess(img: Image.Image, size: int, mean=MEAN, std=STD) -> torch.Tensor:
    tx = T.Compose([T.Resize(int(size * 1.14)), T.CenterCrop(size), T.ToTensor(), T.Normalize(mean, std)])
    return tx(img.convert("RGB")).unsqueeze(0)

def band_of(p: float, u: float, bands: dict) -> str:
    if u > float(bands.get("tau_uncertainty", 1.0)):
        return "ragu"
    if p >= bands["tau_high"]:
        return "rujuk_prioritas"
    if p < bands["tau_low"]:
        return "negatif_skrining"
    return "ragu"

def _pre_resize(img: Image.Image, max_dim: int) -> Image.Image:
    """Samakan preprocessing dengan saat ekstraksi embedding SSL (dataset NLM di-resize 512px)."""
    if max_dim <= 0:
        return img
    w, h = img.size
    scale = max_dim / max(w, h)
    if scale < 1:
        return img.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
    return img

def infer_prob(member: dict, img: Image.Image) -> float:
    if member["kind"] == "probe":
        img = _pre_resize(img, member.get("pre_resize", 512))
        x = preprocess(img, member["size"], member.get("pixel_mean", MEAN), member.get("pixel_std", STD))
        with torch.no_grad():
            feats = member["model"](x).float().numpy()[0]
            z = float((feats - member["scaler_mean"]) / member["scaler_scale"]) @ member["coef"] + member["intercept"])
    else:
        x = preprocess(img, member["size"])
        with torch.no_grad():
            z = float(member["model"](x).squeeze(1).item())
    return sigmoid(z / member["T"])

def _cam_from_tokens(grads: torch.Tensor, acts: torch.Tensor, out_hw: tuple[int, int]) -> torch.Tensor | None:
    g, a = grads[1:], acts[1:]
    n = g.shape[0]
    side = int(round(n ** 0.5))
    if side * side != n:
        return None
    g = g.reshape(side, side, -1)
    a = a.reshape(side, side, -1)
    cam = (g * a).sum(-1).clamp(min=0)[None, None]
    return nn.functional.interpolate(cam, size=out_hw, mode="bilinear", align_corners=False)[0, 0]

def gradcam(member: dict, img: Image.Image) -> np.ndarray | None:
    model = member["model"]
    if member["kind"] == "torch":

```

```

x = preprocess(img, member["size"])
model.zero_grad(set_to_none=True)
feats = model.forward_features(x)
feats.retain_grad()
model.forward_head(feats).squeeze(1).backward()
grads, acts = feats.grad, feats
if grads.dim() == 4:
    w = grads.mean(dim=(2, 3), keepdim=True)
    cam = (w * acts).sum(1, keepdim=True).clamp(min=0)
    cam = nn.functional.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)[0, 0]
else:
    cam = _cam_from_tokens(grads[0], acts[0], tuple(x.shape[-2:]))
else:
    mean, std = member.get("pixel_mean", MEAN), member.get("pixel_std", STD)
    img_in = _pre_resize(img, member.get("pre_resize", 512))
    x = preprocess(img_in, member["size"], mean, std)
    model.zero_grad(set_to_none=True)
    if hasattr(model, "forward_features"):
        feats = model.forward_features(x)
        pooled = model.forward_head(feats)
    else:
        feats = model.m(pixel_values=x).last_hidden_state
        pooled = feats[:, 0]
    feats = feats if feats.dim() == 3 else feats.flatten(2).transpose(1, 2)
    feats.retain_grad()
    mt = torch.tensor(member["scaler_mean"])
    st = torch.tensor(member["scaler_scale"])
    cf = torch.tensor(member["coef"])
    z = ((pooled[0].float() - mt) / st) @ cf + member["intercept"]
    z.backward()
    cam = _cam_from_tokens(feats.grad[0], feats[0], tuple(x.shape[-2:]))
if cam is None:
    return None
cam = cam.detach()
return (cam / (cam.amax() + 1e-8)).numpy()

def overlay_png(img: Image.Image, cam: np.ndarray) -> str:
    img_np = np.asarray(img.convert("RGB").resize((cam.shape[1], cam.shape[0])), dtype=np.float32) / 255.0
    heat = cm.jet(cam)[..., :3].astype(np.float32)
    alpha = cam[..., None] * 0.45
    blend = img_np * (1 - alpha) + heat * alpha
    out = Image.fromarray((np.clip(blend, 0, 1) * 255).astype(np.uint8))
    buf = io.BytesIO()
    out.save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode()

@app.on_event("startup")
def startup() -> None:
    _load()

@app.get("/health")
def health() -> dict:
    return {
        "status": "ok" if _state.get("score_members") else "no-model",
        "score_members": _names(_state.get("score_members", [])),
        "uncertainty_members": _names(_state.get("unc_members", [])),
        "bands": _state.get("bands"),
    }

@app.post("/predict")
async def predict(file: UploadFile = File(...)) -> dict:
    if not _state.get("score_members"):
        raise HTTPException(503, "model belum siap")
    raw = await file.read()
    try:
        img = Image.open(io.BytesIO(raw))
        img.load()
    except Exception as e:
        raise HTTPException(400, f"bukan citra valid: {e}")

    score_probs = [infer_prob(m, img) for m in _state["score_members"]]
    extra = [m for m in _state["unc_members"] if m not in _state["score_members"]]
    all_probs = score_probs + [infer_prob(m, img) for m in extra]

    p = float(np.mean(score_probs))
    u = float(np.std(all_probs)) if len(all_probs) > 1 else 0.0
    band = band_of(p, u, _state["bands"])

    cam = gradcam(_state["score_members"][0], img)
    heatmap_b64 = overlay_png(img, cam) if cam is not None else None

    return {
        "p_tb": round(p, 4),
        "uncertainty": round(u, 4),
        "band": band,
        "bands": _state["bands"],
        "members": _names(_state["score_members"]),
        "uncertainty_members": _names(_state["unc_members"]),
    }

```

```
        "disclaimer": "Alat triase skrining – bukan diagnosis. Wajib konfirmasi (GeneXpert) oleh tenaga kesehatan.",
        "heatmap_png_b64": heatmap_b64,
    }

# --- UI statis (mode deployment unified: satu Space melayani UI + API) ---
_STATIC = Path(os.environ.get("LOOP_TB_STATIC", "static"))
if _STATIC.is_dir():
    from fastapi.staticfiles import StaticFiles

    app.mount("/", StaticFiles(directory=str(_STATIC), html=True), name="ui")
    print(f"[SIGAP] UI statis dilayani dari {_STATIC}")
```

api/model_config.json

```
{
  "score_members": [{ "path": "ml/runs/ssl_eval/probe_rad_dino.npz" }],
  "uncertainty_members": [
    { "path": "ml/runs/ssl_eval/probe_rad_dino.npz" },
    { "path": "ml/runs/ssl_eval/probe_dinov2.npz" },
    { "path": "ml/runs/eb0/best.pt", "T": 1.2336 },
    { "path": "ml/runs/convnext/best.pt", "T": 1.3405 }
  ],
  "bands": { "tau_low": 0.0233, "tau_high": 0.0977, "tau_uncertainty": 0.25 }
}
```

api/model_config_probe.json

```
{  
  "members": [{ "path": "ml/runs/ssl_eval/probe_dinov2.npz" }],  
  "bands": { "tau_low": 0.0346, "tau_high": 0.0863, "tau_uncertainty": 1.0 }  
}
```

api/model_config_space.json

```
{
  "score_members": [{ "path": "models/probe_rad_dino.npz" }],
  "uncertainty_members": [
    { "path": "models/probe_rad_dino.npz" },
    { "path": "models/probe_dinov2.npz" },
    { "path": "models/eb0.pt", "T": 1.2336 },
    { "path": "models/convnext.pt", "T": 1.3405 }
  ],
  "bands": { "tau_low": 0.0233, "tau_high": 0.0977, "tau_uncertainty": 0.25 }
}
```

scripts/fetch_models.sh

```
#!/usr/bin/env bash
# Unduh artefak model dari GitHub Release `models-v1` ke lokasi yang diharapkan
# api/model_config.json dan ml/evaluate.py (ml/runs/...).
set -euo pipefail
REPO="{REPO:-Alvnc/loop-tb}"
TAG="{TAG:-models-v1}"
BASE="https://github.com/$REPO/releases/download/$TAG"

mkdir -p ml/runs/eb0 ml/runs/convnext ml/runs/linear_probe ml/runs/ssl_eval

fetch() { # $1 = nama file, $2 = tujuan
  if [ -f "$2" ]; then echo "skip (ada): $2"; return; fi
  echo "unduh $1 -> $2"
  curl -fL --retry 3 -o "$2" "$BASE/$1"
}

fetch eb0.pt ml/runs/eb0/best.pt
fetch convnext.pt ml/runs/convnext/best.pt
fetch linear_probe.pt ml/runs/linear_probe/best.pt
fetch probe_rad_dino.npz ml/runs/ssl_eval/probe_rad_dino.npz
fetch probe_dinov2.npz ml/runs/ssl_eval/probe_dinov2.npz

echo "Model siap. Jalankan API:"
echo " .venv/bin/python -m uvicorn api.main:app --port 8000"
echo " (atau LOOPTB_CONFIG=api/model_config_probe.json untuk probe DINOv2 saja)"
```

scripts/redeploy_all.sh

```
#!/usr/bin/env bash
# Redeploy SIGAP ke dua target sekaligus:
# 1) GitHub Pages (static export, basePath /loop-tb, API menunjuk ke VPS)
# 2) VPS (container Docker unified UI+API, dilayani via Tailscale Funnel)
# Lalu verifikasi via funnel + Pages.
set -euo pipefail
cd "$(dirname "$0")/.."

FUNNEL_URL="${FUNNEL_URL:-https://ip-172-26-13-244.tail40f715.ts.net}"
VPS_HOST="${VPS_HOST:-Lightsail-Coretax}"

echo "== 1/4) Build static export untuk Pages =="
( cd web && NEXT_EXPORT=1 NEXT_BASE_PATH=/loop-tb NEXT_PUBLIC_BASE_PATH=/loop-tb NEXT_PUBLIC_API_URL="$FUNNEL_URL" npm run build 2>&1
| tail -2 )

echo "== 2/4) Push gh-pages =="
D="$HOME/.cache/sigap/pages-deploy"
rm -rf "$D" && mkdir -p "$D"
cp -r web/out/. "$D/"
touch "$D/.nojekyll"
(
  cd "$D"
  git init -q -b gh-pages
  git add -A
  git -c user.name="Alvnnvc" -c user.email="alvnnvc@users.noreply.github.com" commit -q -m "deploy: SIGAP static (v2 UI)"
  git remote add origin git@github.com:Alvnnvc/loop-tb.git
  git push -f -q origin gh-pages
)
echo "gh-pages pushed"

echo "== 3/4) Rebuild VPS container =="
STAGE_ONLY=1 bash api/deploy_hf.sh > /dev/null
ssh -o BatchMode=yes "$VPS_HOST" 'mkdir -p ~/sigap && rm -rf ~/sigap/*'
rsync -az --delete "$HOME/.cache/sigap/hf_space/" "$VPS_HOST:~/sigap/" > /dev/null
ssh -o BatchMode=yes "$VPS_HOST" 'cd ~/sigap && sudo docker build -q -t sigap-api . > /dev/null && { sudo docker rm -f sigap > /dev/null 2>&1 || true; } && sudo docker run -d --name sigap --restart unless-stopped -p 8099:7860 sigap-api > /dev/null && echo container-up'

echo "== 4/4) Verifikasi =="
H=""
for i in $(seq 1 45); do
  sleep 10
  H=$(curl -s -m 10 "$FUNNEL_URL/health" || true)
  [ -n "$H" ] && break
  echo "[${i}] menunggu health..."
done
echo "health: ${H:0:160}"
curl -s -m 15 -o /dev/null -w "VPS UI -> %{http_code}\n" "$FUNNEL_URL/"
sleep 60
curl -s -m 15 -o /dev/null -w "Pages UI -> %{http_code}\n" https://alvnnvc.github.io/loop-tb/
echo "REDEPLOY DONE"
```

tools/e2e_screen.py

```
#!/usr/bin/env python3
"""Uji integrasi UI+API: unggah citra X-ray lewat halaman /skrining, ambil screenshot hasil.

Menjalankan dua pass: mobile (390x844) untuk kedua kasus dan desktop (1440x900)
untuk kasus TB – bukti workspace web-first.

Jalankan dengan helper server (web 3210 + api 8000):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm start -- --port 3210" --port 3210 \
--server ".venv/bin/python -m uvicorn api.main:app --port 8000" --port 8000 \
-- .venv/bin/python tools/e2e_screen.py
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots")
OUT.mkdir(parents=True, exist_ok=True)

CASES = [
    ("data/raw/nlm/shenzhen/CXR_png/CHNCXR_0327_1.png", "tb", "S-014"),
    ("data/raw/nlm/shenzhen/CXR_png/CHNCXR_0001_0.png", "normal", "S-015"),
]

def run_case(page, img: str, tag: str, pid: str, suffix: str) -> None:
    page.goto(f"{BASE}/skrining", wait_until="networkidle")
    page.set_input_files('input[type="file"]', img)
    page.fill('input[placeholder="e.g. S-014"]', pid)
    page.fill('input[placeholder="34"]', "42")
    page.click("text=Analyze image")
    page.wait_for_selector("text=Analysis result", timeout=120_000)
    page.wait_for_timeout(600)
    page.screenshot(path=str(OUT / f"skrining_hasil_{tag}_{suffix}.png"), full_page=True)
    print(f"[ok] {tag} ({suffix}): {img}")

def main() -> None:
    passes = [
        ("mobile", {"width": 390, "height": 844}, 2, CASES),
        ("desktop", {"width": 1440, "height": 900}, 1, CASES[:1]),
    ]
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        for suffix, vp, ds, cases in passes:
            ctx = browser.new_context(viewport=vp, device_scale_factor=ds)
            page = ctx.new_page()
            console: list[str] = []
            page.on(
                "console",
                lambda m: console.append(f"{m.type}: {m.text}") if m.type == "error" else None,
            )
            for img, tag, pid in cases:
                if not Path(img).exists():
                    print("[skip]", img)
                    continue
                run_case(page, img, tag, pid, suffix)
            if console:
                print(f"[{suffix} console errors]", console[:6])
            ctx.close()
            browser.close()
    print("E2E SCREENSHOTS DONE")

if __name__ == "__main__":
    main()
```

tools/gallery_check.py

```
#!/usr/bin/env python3
"""Uji galeri contoh arsip: klik satu kasus → hasil tampil tanpa upload.

Jalankan dengan helper server (web 3210; API opsional):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm start -- --port 3210" --port 3210 \
-- .env/bin/python tools/gallery_check.py http://localhost:3210
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots/gallery_case_b.png")

def main() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_context(viewport={"width": 390, "height": 844}, device_scale_factor=2).new_page()
        page.goto(f"{BASE}/skrining", wait_until="networkidle")
        page.wait_for_selector("text=Archive examples", timeout=30_000)
        page.click("text=Case B")
        page.wait_for_selector("text=Analysis result", timeout=30_000)
        page.wait_for_selector("text=Needs re-read", timeout=30_000)
        page.wait_for_timeout(600)
        page.screenshot(path=str(OUT), full_page=True)
        print("GALLERY CHECK OK -", OUT)
        browser.close()

if __name__ == "__main__":
    main()
```

tools/klinisi_shot.py

```
#!/usr/bin/env python3
"""Screenshot halaman klinisi dalam keadaan TERISI (seed localStorage) – desktop & mobile.

.venv/bin/python tools/klinisi_shot.py <BASE_URL>
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots")
CASE = [
    {
        "id": "seed-1",
        "date": "2026-10-03T08:00:00.000Z",
        "name": "S-014",
        "age": "42",
        "sex": "L",
        "p": 0.9937,
        "band": "rujuk_prioritas",
        "history": [
            {"date": "2026-10-04T08:00:00.000Z", "med": True, "symptoms": 1, "danger": []},
            {"date": "2026-10-03T08:00:00.000Z", "med": False, "symptoms": 2, "danger": ["Shortness of breath"]},
        ],
    }
]

def main() -> None:
    seed = f"localStorage.setItem('sigap.cases', {json.dumps(json.dumps(CASE))});"
    with sync_playwright() as p:
        b = p.chromium.launch(headless=True)
        for vp, w, h, ds in (("desktop", 1440, 900, 1), ("mobile", 390, 844, 2)):
            ctx = b.new_context(viewport={"width": w, "height": h}, device_scale_factor=ds)
            ctx.add_init_script(seed)
            page = ctx.new_page()
            page.goto(f"{BASE}/klinisi", wait_until="networkidle")
            page.wait_for_selector("text=Clinician summary", timeout=30_000)
            page.wait_for_selector("text=S-014", timeout=15_000)
            page.wait_for_timeout(400)
            page.screenshot(path=str(OUT / f"klinisi_filled_{vp}.png"), full_page=True)
            print("ok", vp)
            ctx.close()
        b.close()
    print("KLINISI SHOT DONE")

if __name__ == "__main__":
    main()
```

tools/make_case_gallery.py

```
#!/usr/bin/env python3
"""Bangun galeri kasus arsip untuk deployment statis (tanpa server).

Memilih 4 kasus showcase dari prediksi offline (satu per perilaku band:
TB-rujuk, normal-ragu, TB-ragu, normal-negatif bila ada), lalu memanggil API
lokal untuk merekam keluaran ensemble asli + heatmap, dan menyimpan gambar
tampilan. Output -> web/public/cases/ (ikut ke static export / GitHub Pages).

Jalankan (API lokal harus hidup di :8000):
.venv/bin/python tools/make_case_gallery.py
"""
from __future__ import annotations

import csv
import io
import json
import urllib.request
import uuid
from pathlib import Path

from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "web/public/cases"
API = "http://localhost:8000"

TAU_LOW, TAU_HIGH, TAU_U = 0.0233, 0.0977, 0.25

def load(f: str, col: str) -> dict[str, float]:
    return {Path(r["path"]).stem: float(r[col]) for r in csv.DictReader(open(ROOT / f))}

def band_of(p: float, u: float) -> str:
    if u > TAU_U:
        return "ragu"
    if p >= TAU_HIGH:
        return "rujuk_prioritas"
    if p < TAU_LOW:
        return "negatif_skrining"
    return "ragu"

def infer(path: Path) -> dict:
    boundary = uuid.uuid4().hex
    body = (
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="file"; filename="{path.name}"\r\n'
        f'Content-Type: image/png\r\n\r\n'
    ).encode() + path.read_bytes() + f"\r\n--{boundary}--\r\n".encode()
    req = urllib.request.Request(
        f"{API}/predict", data=body, method="POST",
        headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
    )
    with urllib.request.urlopen(req, timeout=300) as r:
        return json.loads(r.read())

def main() -> None:
    OUT.mkdir(parents=True, exist_ok=True)
    pr = load("ml/runs/ssl_eval/preds_rad_dino.csv", "p")
    pd = load("ml/runs/ssl_eval/preds_dinov2.csv", "p")
    ps = load("ml/runs/eval_external_ens/preds.csv", "p_ens")

    man = json.loads((ROOT / "ml/data/artifacts/manifest.json").read_text())
    cans = []
    for it in man["external"]:
        stem = Path(it["path"]).stem
        if stem not in pr or stem not in pd or stem not in ps:
            continue
        import statistics
        u = statistics.pstdev([pr[stem], pd[stem], ps[stem]])
        cans.append({
            "stem": stem, "path": it["path"], "label": it["label"],
            "p": pr[stem], "u": u, "band": band_of(pr[stem], u),
        })

    def best(pred, key, rev=True):
        xs = [c for c in cans if pred(c)]
        xs.sort(key=key, reverse=rev)
        return xs[0] if xs else None

    picks = [
        best(lambda c: c["label"] == 1 and c["band"] == "rujuk_prioritas", lambda c: c["p"]),
        best(lambda c: c["label"] == 0 and c["band"] == "ragu", lambda c: c["u"]),
        best(lambda c: c["label"] == 1 and c["band"] == "ragu", lambda c: c["u"]),
        best(lambda c: c["label"] == 0 and c["band"] == "ragu", lambda c: c["u"], rev=False),
    ]
```

```

picks = [p for p in picks if p]
# dedupe bila ada pemilihan sama
seen, uniq = set(), []
for c in picks:
    if c["stem"] not in seen:
        seen.add(c["stem"])
        uniq.append(c)
picks = uniq

TITLES = [
    ("Kasus A – TB aktif", "Ensemble sepakat: prioritas rujukan hari ini."),
    ("Kasus B – normal dari RS asing", "Keluarga model tidak sepakat → sistem menahan keputusan."),
    ("Kasus C – TB, bukti tidak konklusif", "Bahkan untuk TB, ketidakpastian tinggi → pembacaan ulang oleh manusia."),
    ("Kasus D – normal, mendekati negatif", "Model cenderung rendah, tetapi ketidakpastian menjaga band ragu."),
]

index = []
for i, c in enumerate(picks):
    src = ROOT / c["path"]
    res = infer(src)
    cid = c["stem"].lower()
    # gambar tampilan (maks 900px, jpg)
    with Image.open(src) as im:
        im = im.convert("L")
        w, h = im.size
        sc = 900 / max(w, h)
        if sc < 1:
            im = im.resize((round(w * sc), round(h * sc)), Image.LANCZOS)
        buf = io.BytesIO()
        im.save(buf, format="JPEG", quality=88)
    (OUT / f"{cid}.jpg").write_bytes(buf.getvalue())
    payload = {**res, "case_id": cid, "source_image": src.name, "dataset": c["path"].split("/")[2]}
    (OUT / f"{cid}.json").write_text(json.dumps(payload))
    title, desc = TITLES[i] if i < len(TITLES) else (f"Kasus {cid}", "")
    index.append({
        "id": cid, "title": title, "desc": desc,
        "label": "TB" if c["label"] == 1 else "normal",
        "image": f"{cid}.jpg", "result": f"{cid}.json",
        "band": res["band"], "p": res["p_tb"], "u": res["uncertainty"],
    })
    print(f"{cid}: band={res['band']} p={res['p_tb']} u={res['uncertainty']} ({title})")

(OUT / "index.json").write_text(json.dumps(index, indent=1))
print("index ->", OUT / "index.json")

if __name__ == "__main__":
    main()

```

tools/make_pdfs.py

```
#!/usr/bin/env python3
"""Hasilkan PDF submission: one-pager (dari HTML) + code PDF (dari repo).

Jalankan:
.venv/bin/python tools/make_pdfs.py
"""
from __future__ import annotations

import html
import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "submission"

CODE_GLOBS = [
    "ml/*.py",
    "ml/data/*.py",
    "ml/kaggle/**/*.py",
    "api/*.py",
    "api/*.json",
    "scripts/*.sh",
    "tools/*.py",
    "web/src/**/*.ts",
    "web/src/**/*.tsx",
    "web/src/**/*.css",
    "web/*.json",
    "web/*.ts",
    "docs/*.md",
]

EXCLUDE_PARTS = {"node_modules", ".next", "__pycache__", ".venv"}

def collect_files() -> list[Path]:
    files: list[Path] = []
    seen: set[Path] = set()
    for g in CODE_GLOBS:
        for p in sorted(ROOT.glob(g)):
            if not p.is_file() or any(part in EXCLUDE_PARTS for part in p.parts):
                continue
            if p in seen or p.suffix == ".png":
                continue
            seen.add(p)
            files.append(p.relative_to(ROOT))
    return files

def build_code_html(files: list[Path]) -> str:
    parts = [
        """<!DOCTYPE html><html><head><meta charset="utf-8"><title>SIGAP – Code</title>

<style>
@page { size: A4; margin: 14mm 12mm; }
body { font-family: "DejaVu Sans Mono", monospace; font-size: 8.4px; color: #14181d; }
h1 { font-family: sans-serif; font-size: 20px; margin-bottom: 6px; }
h2 { font-family: sans-serif; font-size: 12px; margin: 14px 0 6px; border-bottom: 1px solid #ccc; padding-bottom: 3px; }
.filetree { font-size: 9px; line-height: 1.5; margin-bottom: 10px; }
pre { white-space: pre-wrap; word-break: break-word; border: 1px solid #ddd; border-radius: 4px; padding: 6px 8px; margin-bottom: 10px; }
.filehead { font-family: sans-serif; font-weight: bold; font-size: 10px; margin: 14px 0 4px; page-break-before: always; }
</style></head><body>""",
        "<h1>SIGAP – complete code listing</h1>",
        "<p class='filetree'><b>Repository:</b> github.com/Alvncnc/loop-tb – prototipe riset UnivaBio 2026.<br>"
        "<Berisi: pipeline data (audit dedup), training, studi representasi SSL, evaluasi rigor, API, dan UI.</p>",
        "<h2>Daftar file</h2><div class='filetree'>" + "<br>".join(html.escape(str(f)) for f in files) + "</div>",
    ]

    for f in files:
        text = (ROOT / f).read_text(errors="ignore")
        parts.append(f"<div class='filehead'>{html.escape(str(f))}</div>")
        parts.append(f"<pre>{html.escape(text)}</pre>")
    parts.append("</body></html>")
    return "".join(parts)

def main() -> None:
    OUT.mkdir(exist_ok=True)
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()

        # 1) One-pager
        page.goto((OUT / "onepager.html").as_uri(), wait_until="networkidle")
        page.pdf(path=str(OUT / "onepager.pdf"), format="A4", print_background=True,
            margin={"top": "0", "right": "0", "bottom": "0", "left": "0"})
        print("onepager.pdf OK")

        # 2) Code PDF
```

```
files = collect_files()
code_html = build_code_html(files)
tmp = Path("/tmp/opencode/code_listing.html")
tmp.write_text(code_html)
page.goto(tmp.as_uri(), wait_until="load")
page.pdf(path=str(OUT / "code_listing.pdf"), format="A4", print_background=True)
print(f"code_listing.pdf OK - {len(files)} file")

browser.close()
```

```
if __name__ == "__main__":
    main()
```

tools/make_video.py

```
#!/usr/bin/env python3
"""Produksi video demo SIGAP secara otomatis.

Tahapan: render slide → rekam screencast (Playwright) → sintesis narasi (Piper TTS)
→ rakit dengan ffmpeg → submission/sigap_demo.mp4.

Jalankan lewat helper (web 3210 + api 8000):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
  --server "cd web && npm start -- --port 3210" --port 3210 \
  --server ".venv/bin/python -m uvicorn api.main:app --port 8000" --port 8000 \
  -- .env/bin/python tools/make_video.py
"""
from __future__ import annotations

import os
import subprocess
import wave
from pathlib import Path

from playwright.sync_api import sync_playwright

ROOT = Path(__file__).resolve().parent.parent
ASSETS = ROOT / "submission" / "video_assets"
ASSETS.mkdir(parents=True, exist_ok=True)
FINAL = ROOT / "submission" / "sigap_demo.mp4"
VOICE_MODEL = str(Path.home() / ".cache/piper/en/en_US/lessac/medium/en_US-lessac-medium.onnx")
BASE = "http://localhost:3210"
W, H = 1920, 1080

SLIDE_CSS = """
<style>
  @import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;600;800&display=swap');
  * { margin:0; padding:0; box-sizing:border-box; }
  body { width:1920px; height:1080px; background:#0b0f13; color:#e9e7e0;
    font-family:'Plus Jakarta Sans',sans-serif; display:flex; align-items:center; }
  .pad { padding: 0 140px; width:100%; }
  h1 { font-size:96px; font-weight:800; letter-spacing:-0.03em; line-height:1.04; }
  p { font-size:38px; line-height:1.45; color:rgba(233,231,224,.85); margin-top:36px; max-width:1500px; }
  .small { font-size:26px; color:rgba(233,231,224,.55); margin-top:44px; }
  .accent { color:#ee8b7e; } .defer { color:#e5b95c; } .clear { color:#83c7a4; }
  table { border-collapse:collapse; margin-top:40px; font-size:34px; }
  td, th { text-align:left; padding:14px 28px 14px 0; border-bottom:1px solid rgba(233,231,224,.18); }
  th { font-size:26px; color:rgba(233,231,224,.55); font-weight:600; }
  .num { font-variant-numeric: tabular-nums; font-weight:800; }
</style>"""

SLIDES = {
  "slide_problem": f"""{SLIDE_CSS}<div class="pad">
    <h1>≈200,000 people a year<br>transmit TB <span class="accent">before ever being detected</span>.</h1>
    <p>Indonesia carries 10% of the world's TB burden. The care loop – detect, refer, treat, monitor –
      moves slower than the disease transmits.</p>
    <div class="small">Source: WHO Global TB Report 2025 · Kemenkes 2024</div></div>""",
  "slide_solution": f"""{SLIDE_CSS}<div class="pad">
    <h1>One calibrated risk state.<br>Three interfaces.</h1>
    <p><b>Screen</b> – CXR triage with a <span class="defer">deferral band</span>. &nbsp;&nbsp;&nbsp;
      <b>Refer</b> – action band + printable referral letter. &nbsp;&nbsp;&nbsp;
      <b>Support</b> – adherence loop with escalation.</p>
    <p>When model families disagree, SIGAP says <i>"uncertain – human re-read"</i> instead of forcing a verdict.</p></div>""",
  "slide_evidence": f"""{SLIDE_CSS}<div class="pad">
    <h1>We report the number most projects hide.</h1>
    <table>
      <tr><th>Evaluation</th><th>AUROC</th><th>sens@spec90</th></tr>
      <tr><td>Internal random split <i>(the illusion)</i></td><td class="num">0.98 – 0.9999</td><td class="num">–</td></tr>
      <tr><td>External hospitals · supervised fine-tune</td><td class="num">0.675</td><td class="num">0.27</td></tr>
      <tr><td><b>External · RAD-DINO self-supervised probe (final)</b></td><td class="num clear">0.887</td><td class="num
clear">0.71</td></tr>
    </table>
    <div class="small">Data audit removed 338 duplicate images and 19 training images that leaked into the external test set.</div>
</div>""",
  "slide_close": f"""{SLIDE_CSS}<div class="pad">
    <h1>SIGAP – screen faster<br>than the disease spreads.</h1>
    <p>Research triage prototype – not a diagnostic device. WHO sensitivity target not yet met; reported as-is.
      Next: prospective validation, per-site calibration, cough audio.</p>
    <div class="small">Code, frozen splits &amp; evaluation: github.com/Alvnvnc/loop-tb</div></div>""",
}

# (jenis, id, teks narasi)
SEGMENTS = [
  ("slide", "slide_problem",
    "Indonesia carries ten percent of the world's tuberculosis burden. Around two hundred thousand people "
    "each year transmit TB before ever being detected. The root cause is latency. The care loop moves "
    "slower than the disease."),
  ("slide", "slide_solution",
    "SIGAP compresses that latency with one calibrated risk state. It drives screening, referral, and "
    "treatment support. And when model families disagree, it says so, instead of forcing a decision."),
  ("clip", "clip_tb",
    "A health worker uploads a chest X-ray. For a real TB case, the model families agree: refer today, "
    "with a referral letter for GeneXpert confirmation. The attention map shows where the model looked."),
```

```

("clip", "clip_normal",
 "For a normal X-ray from an unseen hospital, the families disagree – so SIGAP answers: uncertain, "
 "human re-read. That is the deferral band doing its job on a case the model should not decide."),
("clip", "clip_pasien",
 "The same risk state follows the patient home. Missed doses and symptoms raise the score and trigger "
 "escalation, so the loop keeps moving until the patient is cured."),
("slide", "slide_evidence",
 "Here is the evidence, honestly. On an internal split the model looks perfect. On external hospitals, "
 "supervised fine-tuning falls to zero point six eight AUROC. Our final model, a self-supervised chest "
 "X-ray representation called RAD-DINO, holds zero point eight nine. We also removed nineteen training "
 "images that had leaked into the test set."),
("slide", "slide_close",
 "This is a research triage prototype, not a diagnostic device. The WHO sensitivity target is not met "
 "yet, and we report that. Next steps: prospective validation, per-site threshold calibration, and "
 "cough audio. SIGAP – screen faster than the disease spreads."),
]

def sh(cmd: list[str], **kw) -> None:
    subprocess.run(cmd, check=True, **kw)

def wav_duration(p: Path) -> float:
    with wave.open(str(p)) as w:
        return w.getnframes() / w.getframerate()

def render_slides() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page(viewport={"width": W, "height": H, "device_scale_factor": 1})
        for name, html in SLIDES.items():
            tmp = ASSETS / f"{name}.html"
            tmp.write_text(f"<!DOCTYPE html><html><head><meta charset='utf-8'></head><body>{html}</body></html>")
            page.goto(tmp.resolve().as_uri(), wait_until="networkidle")
            page.wait_for_timeout(400)
            page.screenshot(path=str(ASSETS / f"{name}.png"))
            print("slide:", name)
        browser.close()

def record_clips() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)

        def new_ctx(tag: str):
            return browser.new_context(
                viewport={"width": W, "height": H},
                record_video_dir=str(ASSETS / "raw"),
                record_video_size={"width": W, "height": H},
            )

        # --- clip TB ---
        ctx = new_ctx("tb")
        page = ctx.new_page()
        page.goto(f"{BASE}/skringing", wait_until="networkidle")
        page.wait_for_timeout(1200)
        page.set_input_files('input[type="file"]', str(ROOT / "data/raw/nlm/shenzhen/CXR_png/CHNCXR_0327_1.png"))
        page.fill('input[placeholder="e.g. S-014"]', "S-014")
        page.wait_for_timeout(800)
        page.click("text=Analyze image")
        page.wait_for_selector("text=Analysis result", timeout=120_000)
        page.wait_for_timeout(2500)
        page.click("text=Original")
        page.wait_for_timeout(1400)
        page.click("text=Attention map")
        page.wait_for_timeout(1200)
        page.mouse.wheel(0, 420)
        page.wait_for_timeout(1400)
        video = page.video
        ctx.close()
        video.save_as(ASSETS / "clip_tb.webm")
        print("clip_tb ok")

        # --- clip normal ---
        ctx = new_ctx("normal")
        page = ctx.new_page()
        page.goto(f"{BASE}/skringing", wait_until="networkidle")
        page.wait_for_timeout(900)
        page.set_input_files('input[type="file"]', str(ROOT / "data/raw/nlm/shenzhen/CXR_png/CHNCXR_0001_0.png"))
        page.fill('input[placeholder="e.g. S-014"]', "S-015")
        page.click("text=Analyze image")
        page.wait_for_selector("text=Analysis result", timeout=120_000)
        page.wait_for_timeout(3200)
        page.mouse.wheel(0, 380)
        page.wait_for_timeout(1600)
        video = page.video
        ctx.close()
        video.save_as(ASSETS / "clip_normal.webm")
        print("clip_normal ok")

        # --- clip pasien ---

```

```

    ctx = new_ctx("pasien")
    page = ctx.new_page()
    page.goto(f"{BASE}/pasien", wait_until="networkidle")
    page.wait_for_timeout(1200)
    page.locator('input[type="checkbox"]').first.click() # batalkan "minum obat"
    page.select_option("select", "2")
    page.wait_for_timeout(700)
    page.click("text=Save check-in")
    page.wait_for_timeout(2600)
    page.mouse.wheel(0, 300)
    page.wait_for_timeout(1400)
    video = page.video
    ctx.close()
    video.save_as(ASSETS / "clip_pasien.webm")
    print("clip_pasien ok")

    browser.close()

def narrate() -> list[Path]:
    from piper import PiperVoice

    voice = PiperVoice.load(VOICE_MODEL)
    wavs = []
    for i, (_, _, text) in enumerate(SEGMENTS):
        w = ASSETS / f"seg{i}.wav"
        with wave.open(str(w), "wb") as f:
            voice.synthesize_wav(text, f)
        wavs.append(w)
        print(f"narasi seg{i}: {wav_duration(w):.1f}s")
    return wavs

def clip_duration(p: Path) -> float:
    out = subprocess.run(
        ["ffprobe", "-v", "error", "-show_entries", "format=duration", "-of", "csv=p=0", str(p)],
        capture_output=True, text=True, check=True,
    )
    return float(out.stdout.strip())

def assemble(wavs: list[Path]) -> None:
    parts = []
    for i, ((kind, ident, _), wav) in enumerate(zip(SEGMENTS, wavs)):
        dur = wav_duration(wav) + 0.6
        out = ASSETS / f"part{i}.mp4"
        if kind == "slide":
            sh(["ffmpeg", "-y", "-loglevel", "error",
                "-loop", "1", "-i", str(ASSETS / f"{ident}.png"),
                "-i", str(wav),
                "-af", "apad=pad_dur=0.6",
                "-t", f"{dur:.2f}",
                "-r", "30", "-c:v", "libx264", "-pix_fmt", "yuv420p", "-preset", "medium", "-crf", "20",
                "-c:a", "aac", "-ar", "44100", "-ac", "2", str(out)])
        else:
            clip = ASSETS / f"{ident}.webm"
            cd = clip_duration(clip)
            pad = max(0.0, dur - cd)
            vf = f"tpad=stop_mode=clone:stop_duration={pad:.2f},scale={W}:{H},fps=30,format=yuv420p"
            sh(["ffmpeg", "-y", "-loglevel", "error",
                "-i", str(clip), "-i", str(wav),
                "-filter_complex", f"[0:v]{vf}[v]",
                "-map", "[v]", "-map", "1:a",
                "-af", "apad=pad_dur=0.6",
                "-t", f"{dur:.2f}",
                "-c:v", "libx264", "-preset", "medium", "-crf", "20",
                "-c:a", "aac", "-ar", "44100", "-ac", "2", str(out)])
            parts.append(out)
            print(f"part{i} ok ({kind}:{ident}, {dur:.1f}s)")

    lst = ASSETS / "concat.txt"
    lst.write_text("".join(f"file '{p.name}'\n" for p in parts))
    sh(["ffmpeg", "-y", "-loglevel", "error", "-f", "concat", "-safe", "0",
        "-i", str(lst), "-c", "copy", str(FINAL)])
    print("FINAL:", FINAL, f"({clip_duration(FINAL):.1f}s)")

def main() -> None:
    force = os.environ.get("FORCE", "0") == "1"
    need_slides = force or not all((ASSETS / f"{k}.png").exists() for k in SLIDES)
    need_clips = force or not all((ASSETS / f"{n}.webm").exists() for n in ("clip_tb", "clip_normal", "clip_pasien"))
    if need_slides:
        render_slides()
    else:
        print("slides: skip (ada)")
    if need_clips:
        record_clips()
    else:
        print("clips: skip (ada)")
    wavs = narrate()
    assemble(wavs)

```

```
if __name__ == "__main__":  
    main()
```

tools/model_checks.py

```
#!/usr/bin/env python3
"""Uji jalur deployment: API vs evaluasi offline + robustness kompresi JPEG.

- 60 citra eksternal (30 TB + 30 normal) dikirim ke POST /predict,
  dibandingkan dengan probabilitas offline (ml/runs/ssl_eval/preds_rad_dino.csv).
- 15 citra diuji sebagai PNG vs JPEG q85 (sensitivitas kompresi).

Jalankan dengan helper (API saja):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
  --server ".venv/bin/python -m uvicorn api.main:app --port 8000" --port 8000 \
  -- ".venv/bin/python tools/model_checks.py
"""
from __future__ import annotations

import csv
import io
import json
import random
import statistics
import urllib.request
import uuid
from pathlib import Path

from PIL import Image

API = "http://localhost:8000"
ROOT = Path(__file__).resolve().parent.parent
MANIFEST = json.loads((ROOT / "ml/data/artifacts/manifest.json").read_text())
OFFLINE = {Path(r["path"]).stem: float(r["p"]) for r in csv.DictReader((ROOT / "ml/runs/ssl_eval/preds_rad_dino.csv").open())}

def post_image(data: bytes, name: str) -> float:
    boundary = uuid.uuid4().hex
    body = (
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="file"; filename="{name}"\r\n'
        f'Content-Type: image/png\r\n\r\n'
    ).encode() + data + f"\r\n--{boundary}--\r\n".encode()
    req = urllib.request.Request(
        f"{API}/predict", data=body, method="POST",
        headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
    )
    with urllib.request.urlopen(req, timeout=180) as r:
        return float(json.loads(r.read())["p_tb"])

def pearson(a: list[float], b: list[float]) -> float:
    ma, mb = statistics.fmean(a), statistics.fmean(b)
    num = sum((x - ma) * (y - mb) for x, y in zip(a, b))
    den = (sum((x - ma) ** 2 for x in a) * sum((y - mb) ** 2 for y in b)) ** 0.5
    return num / den if den else float("nan")

def auROC(y: list[int], p: list[float]) -> float:
    pairs = sorted(zip(p, y))
    # rank-based AUROC (Mann-Whitney)
    ranks = [0.0] * len(pairs)
    i = 0
    while i < len(pairs):
        j = i
        while j + 1 < len(pairs) and pairs[j + 1][0] == pairs[i][0]:
            j += 1
        avg = (i + j) / 2 + 1
        for k in range(i, j + 1):
            ranks[k] = avg
        i = j + 1
    pos = sum(r for r, (_, lab) in zip(ranks, pairs) if lab == 1)
    n_pos = sum(1 for _, lab in pairs if lab == 1)
    n_neg = len(pairs) - n_pos
    return (pos - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)

def main() -> None:
    ext = MANIFEST["external"]
    rng = random.Random(42)
    tb = [it for it in ext if it["label"] == 1]
    neg = [it for it in ext if it["label"] == 0]
    sample = rng.sample(tb, 30) + rng.sample(neg, 30)
    rng.shuffle(sample)

    api_p, off_p, labels = [], [], []
    for it in sample:
        data = Path(it["path"]).read_bytes()
        p = post_image(data, Path(it["path"]).name)
        api_p.append(p)
        off_p.append(OFFLINE[Path(it["path"]).stem])
        labels.append(it["label"])
```

```

diffs = [abs(a - b) for a, b in zip(api_p, off_p)]
report = {
    "n": len(sample),
    "pearson_r_api_vs_offline": round(pearson(api_p, off_p), 4),
    "mean_abs_delta_p": round(statistics.fmean(diffs), 4),
    "max_abs_delta_p": round(max(diffs), 4),
    "auroc_api_sample": round(auroc(labels, api_p), 4),
    "auroc_offline_sample": round(auroc(labels, off_p), 4),
}
print("== konsistensi API vs offline ==")
print(json.dumps(report, indent=1))

# robustness kompresi
jdiffs = []
for it in sample[:15]:
    img = Image.open(ROOT / it["path"]).convert("L")
    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=85)
    p_jpg = post_image(buf.getvalue(), "x.jpg")
    p_png = post_image((ROOT / it["path"]).read_bytes(), Path(it["path"]).name)
    jdiffs.append(abs(p_jpg - p_png))
report["jpeg_q85_mean_abs_delta_p"] = round(statistics.fmean(jdiffs), 4)
report["jpeg_q85_max_abs_delta_p"] = round(max(jdiffs), 4)
print("== robustness JPEG q85 ==")
print(f"mean|Δp|={report['jpeg_q85_mean_abs_delta_p']}   max|Δp|={report['jpeg_q85_max_abs_delta_p']}")

out = ROOT / "ml/runs/deploy_checks.json"
out.write_text(json.dumps(report, indent=1))
print("saved:", out)

if __name__ == "__main__":
    main()

```

tools/print_check.py

```
#!/usr/bin/env python3
"""Uji tampilan cetak surat rujukan (print media emulation) – screenshot bukti.

Jalankan dengan helper server (web 3210 + api 8000):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm start -- --port 3210" --port 3210 \
--server ".venv/bin/python -m uvicorn api.main:app --port 8000" --port 8000 \
-- .venv/bin/python tools/print_check.py
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots/referral_letter_print.png")
IMG = "data/raw/nlm/shenzhen/CXR_png/CHNCXR_0327_1.png"

def main() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page(viewport={"width": 794, "height": 1123}, device_scale_factor=2)
        page.goto(f"{BASE}/skrining", wait_until="networkidle")
        page.set_input_files('input[type="file"]', IMG)
        page.fill('input[placeholder="e.g. S-014"]', "S-014")
        page.fill('input[placeholder="34"]', "42")
        page.click("text=Analyze image")
        page.wait_for_selector("text=Analysis result", timeout=120_000)
        page.emulate_media(media="print")
        page.wait_for_timeout(500)
        page.screenshot(path=str(OUT), full_page=True)
        print("print check ok ->", OUT)
        browser.close()

if __name__ == "__main__":
    main()
```

tools/screenshot_ui.py

```
#!/usr/bin/env python3
"""QA visual + screenshot semua halaman SIGAP (mobile & desktop).

Jalankan lewat helper server (lihat ml/kaggle/README atau riwayat commit):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm run dev -- --port 3210" --port 3210 \
-- .env/bin/python tools/screenshot_ui.py http://localhost:3210
"""
from __future__ import annotations

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path("submission/screenshots")
OUT.mkdir(parents=True, exist_ok=True)

ROUTES = [
    ("", "landing"),
    ("skrining", "skrining"),
    ("pasien", "pasien"),
    ("klinisi", "klinisi"),
    ("tentang", "tentang"),
]
VIEWPORTS = [("mobile", 390, 844), ("desktop", 1280, 900)]

def main() -> None:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        for vp_name, w, h in VIEWPORTS:
            ctx = browser.new_context(viewport={"width": w, "height": h}, device_scale_factor=2)
            page = ctx.new_page()
            console_errors: list[str] = []
            page.on("console", lambda m: console_errors.append(m.text) if m.type == "error" else None)
            for route, name in ROUTES:
                page.goto(f"{BASE}/{route}", wait_until="networkidle")
                page.wait_for_timeout(400)
                page.screenshot(path=str(OUT / f"{name}_{vp_name}.png"), full_page=True)
                print(f"[ok] {vp_name:7s} {name}")
            if console_errors:
                print(f"[warn] console errors ({vp_name}): {console_errors[:5]}")
            ctx.close()
        browser.close()
    print(f"SCREENSHOTS -> {OUT}")

if __name__ == "__main__":
    main()
```

tools/ui_verify.py

```
#!/usr/bin/env python3
"""Verifikasi UI v2: 3 viewport, console errors, overflow horizontal, tugas utama, toggle bahasa.

Jalankan dengan helper (web saja):
python /home/alvn/.agents/skills/webapp-testing/scripts/with_server.py \
--server "cd web && npm start -- --port 3210" --port 3210 \
-- .env/bin/python tools/ui_verify.py http://localhost:3210
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

from playwright.sync_api import sync_playwright

BASE = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:3210"
OUT = Path(".design/mobbin-stdir/run-01/proof")
OUT.mkdir(parents=True, exist_ok=True)

ROUTES = [("", "landing"), ("skrining", "skrining"), ("pasien", "pasien"), ("klinisi", "klinisi"), ("tentang", "tentang")]
VIEWPORTS = [("desktop", 1440, 900), ("tablet", 768, 1024), ("mobile", 390, 844)]

def main() -> None:
    report: dict = {"viewports": {}, "checks": {}, "consoleErrors": []}
    with sync_playwright() as p:
        b = p.chromium.launch(headless=True)
        for vp, w, h in VIEWPORTS:
            ctx = b.new_context(viewport={"width": w, "height": h}, device_scale_factor=1)
            page = ctx.new_page()
            errors: list[str] = []
            page.on("console", lambda m: errors.append(m.text) if m.type == "error" else None)
            for route, name in ROUTES:
                page.goto(f"{BASE}/{route}", wait_until="networkidle")
                page.wait_for_timeout(300)
                overflow = page.evaluate("document.documentElement.scrollHeight - window.innerWidth")
                report["viewports"].setdefault(vp, {})[name] = {"overflowPx": overflow}
                if vp in ("desktop", "mobile"):
                    page.screenshot(path=str(OUT / f"{name}_{vp}.png"), full_page=(vp == "mobile"))
            report["consoleErrors"] += errors
            ctx.close()

    # tugas utama + toggle bahasa (mobile)
    ctx = b.new_context(viewport={"width": 390, "height": 844}, device_scale_factor=2)
    page = ctx.new_page()
    page.goto(f"{BASE}/skrining", wait_until="networkidle")
    page.wait_for_selector("text=Archive examples", timeout=30_000)
    page.click("text=Case B")
    page.wait_for_selector("text=Needs re-read", timeout=30_000)
    page.wait_for_timeout(400)
    page.screenshot(path=str(OUT / "main_task_archive_case_mobile.png"), full_page=True)
    report["checks"]["archiveCase"] = "passed"

    page.click('button:has-text("ID")')
    page.wait_for_selector("text=Contoh arsip", timeout=15_000)
    report["checks"]["switchToID"] = "passed"
    page.reload(wait_until="networkidle")
    page.wait_for_selector("text=Contoh arsip", timeout=15_000)
    report["checks"]["persistsAfterReload"] = "passed"
    page.screenshot(path=str(OUT / "after_lang_id_persist_mobile.png"), full_page=True)
    page.click('button:has-text("EN")')

    # fokus keyboard: Tab pertama harus masuk elemen interaktif
    page.keyboard.press("Tab")
    focused = page.evaluate("document.activeElement?.tagName")
    report["checks"]["keyboardFocus"] = focused
    ctx.close()
    b.close()
    print(json.dumps(report, indent=1))

if __name__ == "__main__":
    main()
```

web/src/lib/api.ts

```
export type Band = "rujuk_prioritas" | "ragu" | "negatif_skrining";

export interface PredictResponse {
  p_tb: number;
  band: Band;
  bands: { tau_high: number; tau_low: number; tau_uncertainty?: number };
  arch?: string;
  uncertainty?: number;
  disclaimer?: string;
  heatmap_png_b64?: string;
}

const API_URL = (process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000").replace(/\/$/, "");

export async function predict(file: File): Promise<PredictResponse> {
  const fd = new FormData();
  fd.append("file", file);
  const res = await fetch(`${API_URL}/predict`, { method: "POST", body: fd });
  if (!res.ok) throw new Error(`API ${res.status}`);
  return (await res.json()) as PredictResponse;
}

export const BAND_META: Record<
  Band,
  { labelKey: string; titleKey: string; actionKey: string; text: string; dot: string }
> = {
  rujuk_prioritas: {
    labelKey: "scr.band.refer.label",
    titleKey: "scr.band.refer.title",
    actionKey: "scr.band.refer.action",
    text: "text-[#ee8b7e]",
    dot: "bg-refer",
  },
  ragu: {
    labelKey: "scr.band.defer.label",
    titleKey: "scr.band.defer.title",
    actionKey: "scr.band.defer.action",
    text: "text-[#e5b95c]",
    dot: "bg-defer",
  },
  negatif_skrining: {
    labelKey: "scr.band.neg.label",
    titleKey: "scr.band.neg.title",
    actionKey: "scr.band.neg.action",
    text: "text-[#83c7a4]",
    dot: "bg-clear",
  },
};

export function fmtPct(x: number, digits = 0): string {
  return `${(x * 100).toFixed(digits)}%`;
}
```

```

"use client";

import { useEffect, useMemo, useState } from "react";
import Link from "next/link";
import { BAND_META, fmtPct, type Band } from "@lib/api";
import { useLang } from "@lib/i18n";

type CheckIn = { date: string; med: boolean; symptoms: number; danger: string[] };
type Case = {
  id: string;
  date: string;
  name: string;
  age?: string;
  sex?: string;
  p: number;
  band: Band;
  history: CheckIn[];
};

export default function KlinisiPage() {
  const { t, lang } = useLang();
  const [cases, setCases] = useState<Case[]>([]);
  const [selectedId, setSelectedId] = useState("");

  useEffect(() => {
    try {
      const stored = JSON.parse(localStorage.getItem("sigap.cases") ?? "[]") as Case[];
      setCases(stored);
      if (stored[0]) setSelectedId(stored[0].id);
    } catch {
      /* ignore */
    }
  }, []);

  const selected = useMemo(() => cases.find((c) => c.id === selectedId), [cases, selectedId]);
  const adherence = useMemo(() => {
    if (!selected || selected.history.length === 0) return null;
    return selected.history.filter((h) => h.med).length / selected.history.length;
  }, [selected]);
  const dangerList = selected?.history.flatMap((h) => h.danger).slice(0, 3) ?? [];

  return (
    <main className="mx-auto max-w-6xl px-4 sm:px-6">
      <section className="pt-10 print:hidden">
        <h1 className="font-display text-[30px] font-semibold tracking-tight">{t("cli.title")}</h1>
        <p className="mt-2 max-w-xl text-[15px] leading-relaxed text-ink/70">{t("cli.sub")}</p>
      </section>

      {cases.length === 0 ? (
        <p className="mt-8 text-[15px] text-ink/60">
          {t("cli.empty")} <Link className="underline" href="/skrining">{t("nav.screening")}</Link>
        </p>
      ) : (
        <>
          <div className="lg:grid lg:grid-cols-[250px_1fr] lg:items-start lg:gap-8">
            <div className="mt-8 flex flex-wrap gap-2 print:hidden lg:flex-col lg:flex-nowrap">
              {cases.map((c) => (
                <button
                  key={c.id}
                  type="button"
                  onClick={() => setSelectedId(c.id)}
                  aria-pressed={selectedId === c.id}
                  className={`rounded-control border px-3.5 py-2 text-[13px] font-semibold transition-colors lg:w-full lg:text-left ${
                    selectedId === c.id ? "border-ink bg-ink text-paper" : "border-ink/20 bg-white/60 hover:border-ink/45"
                  }`}
                >
                  {c.name}
                </button>
              ))}
            </div>

            {selected && (
              <section className="reveal mt-6 rounded-panel border border-ink/10 bg-white/70 p-5 shadow-tile sm:p-6 print:border-0 print:shadow-none">
                <header className="flex flex-wrap items-start justify-between gap-4">
                  <div>
                    <h2 className="font-display text-[24px] font-semibold">{selected.name}</h2>
                    <p className="mt-0.5 text-[13px] text-ink/60">
                      {selected.age || "-"} · {selected.sex || "-"} · {t("cli.screeningDate")} {t("cli.date")}
                      {new Intl.DateTimeFormat(lang === "id" ? "id-ID" : "en-GB", { dateStyle: "long" }).format(new Date(selected.date))}
                    </p>
                  </div>
                  <div>
                    <p className="flex items-center gap-2 text-[13px] font-semibold">
                      <span className={`inline-block h-2 w-2 rounded-full ${BAND_META[selected.band].dot}` } />
                      {t(BAND_META[selected.band].titleKey)}
                    </p>
                  </div>
                </header>

```

```

<div className="mt-6 grid grid-cols-2 gap-3 sm:grid-cols-4">
  {[
    [String(selected.history.length), t("cli.tile.checkins"), ""],
    [adherence === null ? "-" : fmtPct(adherence), t("cli.tile.adherence"), ""],
    [selected.history[0] ? `${selected.history[0].symptoms}/3` : "-", t("cli.tile.symptoms"), ""],
    [
      dangerList.length ? t("pat.yes") : t("pat.no"),
      t("cli.tile.danger"),
      dangerList.length ? "text-refer" : "text-clear",
    ],
  ]}.map([v, label, tone]) => (
    <div key={label as string} className="rounded-tile border border-ink/10 bg-paper/70 p-3">
      <p className={`font-display text-[24px] font-semibold leading-none tabular-nums ${tone}`}>{v}</p>
      <p className="mt-1.5 text-[11.5px] leading-snug text-ink/55">{label}</p>
    </div>
  ))
</div>

<p className="mt-4 text-[13px] text-ink/60">
  {t("scr.scale.ptb")}: <span className="font-semibold tabular-nums text-ink">{fmtPct(selected.p)}</span>
</p>

{dangerList.length > 0 && (
  <p className="mt-5 flex items-start gap-2.5 rounded-tile border border-refer/35 bg-refer/5 px-4 py-3 text-[13.5px] font-semibold text-refer">
    <svg viewBox="0 0 24 24" className="mt-0.5 h-4 w-4 shrink-0" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
      <path d="M12 4.8 3.2 19.2h17.6L12 4.8Z" />
      <path d="M12 10v4M12 16.6v.2" />
    </svg>
    {t("cli.alert", { list: dangerList.join(", ") })}
  </p>
)}

<div className="mt-6 flex flex-wrap gap-3 print:hidden">
  <button
    type="button"
    onClick={() => window.print()}
    className="rounded-control bg-ink px-5 py-3 text-[14px] font-semibold text-paper transition-colors hover:bg-ink/90"
  >
    {t("cli.print")}
  </button>
</div>

<p className="mt-6 text-[11.5px] leading-relaxed text-ink/45">{t("cli.note")}</p>
</section>
  )}
</div>
</>
)}
</main>
);
}

```

web/src/app/layout.tsx

```
import type { Metadata } from "next";
import { Newsreader, Plus_Jakarta_Sans } from "next/font/google";
import "./globals.css";
import SiteHeader from "@/components/SiteHeader";
import SiteFooter from "@/components/SiteFooter";
import BottomNav from "@/components/BottomNav";
import { LangProvider } from "@/lib/i18n";

const jakarta = Plus_Jakarta_Sans({
  subsets: ["latin"],
  variable: "--font-jakarta",
  display: "swap",
});

const newsreader = Newsreader({
  subsets: ["latin"],
  variable: "--font-newsreader",
  display: "swap",
  style: ["normal", "italic"],
});

export const metadata: Metadata = {
  title: "SIGAP – calibrated TB screening triage",
  description:
    "Research triage tool: calibrated chest X-ray screening → referral → treatment follow-up. UnivaBio 2026.",
};

export default function RootLayout({ children }: LayoutProps<"/>) {
  return (
    <html lang="en" className={` ${jakarta.variable} ${newsreader.variable} h-full antialiased`} >
      <body className="flex min-h-full flex-col">
        <LangProvider>
          <SiteHeader />
          <div className="flex-1 pb-20 md:pb-0">{children}</div>
          <SiteFooter />
          <BottomNav />
        </LangProvider>
      </body>
    </html>
  );
}
```

```

"use client";

import Link from "next/link";
import { useState } from "react";
import { useLang } from "@lib/i18n";

const LIVE_URL = "https://ip-172-26-13-244.tail40f715.ts.net";
const PAGES_URL = "https://alvncvnc.github.io/loop-tb";

export default function Home() {
  const { t } = useLang();
  const [active, setActive] = useState(0);
  const steps = [0, 1, 2].map((i) => ({
    tag: t(`landing.rail.${i}.tag`),
    title: t(`landing.rail.${i}.title`),
    body: t(`landing.rail.${i}.body`),
  }));
  const openList = ["1", "2", "3"].map((n) => t(`landing.evidence.open.${n}`));
  const todoList = ["1", "2", "3"].map((n) => t(`landing.evidence.todo.${n}`));

  return (
    <main className="mx-auto max-w-6xl px-4 sm:px-6">
      <!-- Hero -->
      <section className="grid gap-10 pt-12 pb-14 lg:grid-cols-12 lg:gap-8 lg:pt-16">
        <div className="lg:col-span-5">
          <p className="text-[12.5px] font-semibold tracking-wide text-ink/45">{t("landing.kicker")}</p>
          <h1 className="font-display mt-3 text-[38px] leading-[1.05] font-semibold tracking-tight sm:text-[46px]">
            {t("landing.title")}
          </h1>
          <p className="mt-5 max-w-xl text-[15.5px] leading-relaxed text-ink/70">{t("landing.sub")}</p>

          <div className="mt-7 flex flex-wrap gap-3">
            <a
              href={LIVE_URL}
              target="_blank"
              rel="noreferrer"
              className="rounded-control bg-ink px-5 py-3 text-[14.5px] font-semibold text-paper shadow-tile transition-all hover:-translate-y-0.5 hover:bg-ink/90"
            >
              {t("landing.cta.primary")}
            </a>
            <a
              href="https://github.com/Alvncvnc/loop-tb"
              target="_blank"
              rel="noreferrer"
              className="rounded-control border border-ink/20 px-5 py-3 text-[14.5px] font-semibold transition-colors hover:border-ink/45"
            >
              {t("landing.cta.secondary")}
            </a>
          </div>

          <!-- Rail bernomor -->
          <ol className="mt-10 space-y-1 border-l border-ink/15 pl-0">
            {steps.map((s, i) => {
              const on = i === active;
              return (
                <li key={s.tag}>
                  <button
                    type="button"
                    onClick={() => setActive(i)}
                    aria-expanded={on}
                    className="group w-full border-l-2 border-transparent py-3 pl-4 text-left transition-colors"
                    style={{ borderLeftColor: on ? "var(--color-ink)" : "transparent" }}
                  >
                    <span className={`flex items-baseline gap-3 ${on ? "" : "opacity-45 group-hover:opacity-70"}`>
                      <span className="font-display text-[15px] font-semibold tabular-nums">0{i + 1}</span>
                      <span className="text-[14px] font-bold uppercase tracking-wider">{s.tag}</span>
                    </span>
                    <span>
                      <on && (
                        <span className="reveal mt-2 block">
                          <span className="font-display block text-[21px] font-semibold leading-snug">{s.title}</span>
                          <span className="mt-1.5 block max-w-md text-[14px] leading-relaxed text-ink/65">{s.body}</span>
                        </span>
                      </on>
                    </span>
                  </button>
                </li>
              );
            })}
          </ol>
        </div>

        <!-- Mockup perangkat -->
        <div className="lg:col-span-7">
          <div className="relative mx-auto w-fit">
            <div className="absolute -left-12 top-24 z-10 hidden -rotate-2 rounded-tile border border-ink/10 bg-white px-3 py-2 text-[11.5px] font-semibold text-ink/70 shadow-float sm:block">
              RAD-DINO · external AUROC <span className="tabular-nums">0.887</span>
            </div>
          </div>
        </div>
      </section>
    </main>
  );
}

```

```

</div>
<div className="rounded-[40px] border border-ink/15 bg-film p-3 shadow-float">
  <div className="w-[292px] overflow-hidden rounded-[30px] bg-film text-bone">
    <div className="relative aspect-[4/3]">
      { /* eslint-disable-next-line @next/next/no-img-element */ }
      
      <div className="absolute inset-x-0 bottom-0 bg-gradient-to-t from-film to-transparent p-3">
        <p className="flex items-center gap-1.5 text-[11px] font-bold text-[#ee8b7e]">
          <span className="inline-block h-1.5 w-1.5 rounded-full bg-refer" />
          {t("scr.band.refer.label")}
        </p>
      </div>
    </div>
    <div className="p-4">
      <div className="flex items-end justify-between">
        <p className="text-[12px] text-bone/60">{t("scr.scale.ptb")}</p>
        <p className="font-display text-[30px] font-semibold leading-none tabular-nums">99%</p>
      </div>
      <div className="relative mt-3 h-2 rounded-full bg-[linear-gradient(90deg,#2e7d5b_0%,#2e7d5b_2%,#b7791f_10%,#c0392b_30%,#c0392b_100%)] opacity-90">
        <span className="absolute -top-[3px] h-[14px] w-[3px] rounded-full bg-bone" style={{ left: "calc(99% - 2px)" }} />
      </div>
      <div className="mt-1.5 flex justify-between text-[10px] text-bone/50">
        <span>{t("scr.scale.low")}</span>
        <span>{t("scr.scale.high")}</span>
      </div>
      <p className="mt-3 border-t border-bone/10 pt-2.5 text-[10.5px] leading-relaxed text-bone/55">
        {t("scr.band.refer.action")}
      </p>
    </div>
  </div>
  <div>
    <div>
      <p className="mt-4 text-center text-[11.5px] text-ink/45">{t("landing.mockup.caption")}</p>
    </div>
  </div>
</section>

{ /* Stat strip */ }
<section className="grid grid-cols-1 gap-px overflow-hidden rounded-panel border border-ink/10 bg-ink/10 sm:grid-cols-3">
  {
    ["0.887", t("landing.stats.0")],
    ["19", t("landing.stats.1")],
    ["0.71", t("landing.stats.2")],
  ].map(([, label]) => (
    <div key={label} className="bg-paper px-5 py-6">
      <p className="font-display text-[34px] font-semibold leading-none tabular-nums">{n}</p>
      <p className="mt-2 text-[13px] leading-relaxed text-ink/60">{label}</p>
    </div>
  )))
</section>

{ /* Evidence */ }
<section className="mt-16">
  <h2 className="font-display text-[26px] font-semibold">{t("landing.evidence.title")}</h2>
  <div className="mt-6 grid gap-4 md:grid-cols-2">
    <div className="rounded-panel border border-ink/10 bg-white/70 p-5 shadow-tile">
      <h3 className="flex items-center gap-2 text-[14px] font-bold text-clear">
        <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
          <circle cx="12" cy="12" r="9" />
          <path d="m8.5 12.2 2.4 2.4 4.6-5" />
        </svg>
        {t("landing.evidence.open.title")}
      </h3>
      <ul className="mt-3 space-y-2.5">
        {openList.map((x) => (
          <li key={x} className="text-[13.5px] leading-relaxed text-ink/70">
            {x}
          </li>
        ))}
      </ul>
    </div>
    <div className="rounded-panel border border-defer/30 bg-defer/5 p-5">
      <h3 className="flex items-center gap-2 text-[14px] font-bold text-defer">
        <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
          <path d="M12 4.8 3.2 19.2h17.6l12 4.8Z" />
          <path d="M12 10v4M12 16.6v2" />
        </svg>
        {t("landing.evidence.todo.title")}
      </h3>
      <ul className="mt-3 space-y-2.5">
        {todoList.map((x) => (
          <li key={x} className="text-[13.5px] leading-relaxed text-ink/70">
            {x}
          </li>
        ))}
      </ul>
    </div>
  </div>
</section>

{ /* Stack */ }

```

```

<section className="mt-16 rounded-panel border border-ink/10 bg-white/70 p-6 shadow-tile">
  <h2 className="font-display text-[24px] font-semibold">{t("landing.stack.title")}</h2>
  <p className="mt-3 max-w-3xl text-[14px] leading-relaxed text-ink/70">{t("landing.stack.body")}</p>
  <div className="mt-4 flex flex-wrap gap-2">
    {[ "RAD-DINO", "DINOv2", "PyTorch", "FastAPI", "Next.js", "Kaggle T4", "ONNX"].map((c) => (
      <span key={c} className="rounded-full border border-ink/15 bg-paper px-3 py-1 text-[12px] font-semibold text-ink/70">
        {c}
      </span>
    ))}
  </div>
</section>

{/* Try */}
<section className="mt-16 rounded-panel bg-film p-7 text-bone shadow-float sm:p-9">
  <h2 className="font-display text-[26px] font-semibold">{t("landing.try.title")}</h2>
  <p className="mt-3 max-w-2xl text-[14px] leading-relaxed text-bone/70">{t("landing.try.note")}</p>
  <div className="mt-6 flex flex-wrap gap-3">
    <a
      href={LIVE_URL}
      target="_blank"
      rel="noreferrer"
      className="rounded-control bg-bone px-5 py-3 text-[14px] font-bold text-film transition-all hover:-translate-y-0.5"
    >
      {t("landing.try.full")} ↗
    </a>
    <a
      href={PAGES_URL}
      target="_blank"
      rel="noreferrer"
      className="rounded-control border border-bone/30 px-5 py-3 text-[14px] font-semibold text-bone transition-colors
hover:border-bone/60"
    >
      {t("landing.try.static")} ↗
    </a>
    <Link
      href="/skrining"
      className="rounded-control px-5 py-3 text-[14px] font-semibold text-bone/70 transition-colors hover:text-bone"
    >
      {t("nav.screening")} →
    </Link>
  </div>
</section>
</main>
);
}

```

```

"use client";

import { useEffect, useMemo, useState } from "react";
import { BAND_META, fmtPct, type Band } from "@lib/api";
import { useLang } from "@lib/i18n";

type CheckIn = { date: string; med: boolean; symptoms: number; danger: string[]; note?: string };
type Case = {
  id: string;
  date: string;
  name: string;
  age?: string;
  sex?: string;
  p: number;
  band: Band;
  history: CheckIn[];
};

function step(prev: number, h: CheckIn): number {
  const delta = (h.med ? -0.08 : 0.18) + 0.09 * h.symptoms + (h.danger.length ? 0.3 : 0);
  return Math.min(0.99, Math.max(0.01, prev + delta));
}

function riskOf(c: Case): number {
  let r = c.p;
  for (const h of [...c.history].reverse()) r = step(r, h);
  return r;
}

function StatusNode({ status }: { status: "done" | "active" | "next" }) {
  if (status === "done")
    return (
      <span className="grid h-6 w-6 place-items-center rounded-full bg-clear text-white">
        <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="2.6" strokeLinecap="round">
          <path d="m5.5 12.5 4 4l18.5 8" />
        </svg>
      </span>
    );
  if (status === "active")
    return <span className="grid h-6 w-6 place-items-center rounded-full border-2 border-ink bg-paper"><span className="h-2 w-2 rounded-full bg-ink" /></span>;
  return <span className="grid h-6 w-6 place-items-center rounded-full border border-dashed border-ink/30" />;
}

export default function PasienPage() {
  const { t, lang } = useLang();
  const [cases, setCases] = useState<Case[]>([]);
  const [selectedId, setSelectedId] = useState<string>("demo");
  const [loaded, setLoaded] = useState(false);
  const [med, setMed] = useState(true);
  const [symptoms, setSymptoms] = useState(1);
  const [danger, setDanger] = useState<string[]>([]);
  const [note, setNote] = useState("");

  const demoName = t("pat.demoName");
  const DEMO_CASE: Case = useMemo(
    () => ({
      id: "demo",
      date: new Date(Date.now() - 9 * 864e5).toISOString(),
      name: demoName,
      age: "45",
      sex: "F",
      p: 0.42,
      band: "ragu",
      history: [
        { date: new Date(Date.now() - 1 * 864e5).toISOString(), med: true, symptoms: 1, danger: [] },
        { date: new Date(Date.now() - 3 * 864e5).toISOString(), med: true, symptoms: 2, danger: [] },
        { date: new Date(Date.now() - 5 * 864e5).toISOString(), med: false, symptoms: 2, danger: [] },
      ],
    }),
    [demoName],
  );

  useEffect(() => {
    try {
      const stored = JSON.parse(localStorage.getItem("sigap.cases") ?? "[]") as Case[];
      setCases(stored);
      if (stored[0]) setSelectedId(stored[0].id);
    } catch {
      /* storage blocked */
    }
    setLoaded(true);
  }, []);

  const all = useMemo<Case[]>(() => [DEMO_CASE, ...cases], [DEMO_CASE, cases]);
  const selected = all.find((c) => c.id === selectedId) ?? DEMO_CASE;
  const risk = riskOf(selected);
  const escalate = risk >= 0.61;

```

```

const meta = BAND_META[selected.band];

const taken = selected.history.filter((h) => h.med).length;
const missed = selected.history.length - taken;
const latestSym = selected.history[0]?.symptoms ?? 0;
const hasDanger = selected.history.some((h) => h.danger.length > 0);
const insight =
  selected.history.length < 2
    ? t("pat.insight.low")
    : risk >= 0.61 || missed >= 2 || latestSym >= 3
      ? t("pat.insight.risk", { missed, symptoms: latestSym })
      : t("pat.insight.ok", { taken, total: selected.history.length });

const bars = [...selected.history].reverse().slice(-14);
const stages: { key: string; status: "done" | "active" | "next" }[] = [
  { key: "pat.stage.screening", status: "done" },
  { key: "pat.stage.referral", status: "done" },
  { key: "pat.stage.treatment", status: "active" },
  { key: "pat.stage.followup", status: "next" },
];

function persist(next: Case) {
  if (next.id === "demo") return;
  const updated = cases.map((c) => (c.id === next.id ? next : c));
  setCases(updated);
  localStorage.setItem("sigap.cases", JSON.stringify(updated));
}

function addCheckIn() {
  const h: CheckIn = { date: new Date().toISOString(), med, symptoms, danger, note: note.trim() || undefined };
  const base =
    selected.id === "demo" ? { ...DEMO_CASE, id: crypto.randomUUID(), name: `${DEMO_CASE.name} ${t("pat.demoCopySuffix")}` } :
  selected;
  const next: Case = { ...base, history: [h, ...base.history] };
  if (selected.id === "demo") {
    const updated = [next, ...cases];
    setCases(updated);
    try {
      localStorage.setItem("sigap.cases", JSON.stringify(updated));
    } catch {
      /* ignore */
    }
    setSelectedId(next.id);
  } else {
    persist(next);
  }
  setNote("");
  setDanger([]);
}

return (
  <main className="mx-auto max-w-6xl px-4 sm:px-6">
    <section className="pt-10">
      <h1 className="font-display text-[30px] font-semibold tracking-tight">{t("pat.title")}</h1>
      <p className="mt-2 max-w-xl text-[15px] leading-relaxed text-ink/70">{t("pat.sub")}</p>
    </section>

    /* Pilih kasus */
    <div className="mt-7 flex flex-wrap gap-2">
      {all.map((c) => {
        const dot = c.band === "rujuk_prioritas" ? "bg-refer" : c.band === "negatif_skrining" ? "bg-clear" : "bg-defer";
        return (
          <button
            key={c.id}
            type="button"
            onClick={() => setSelectedId(c.id)}
            aria-pressed={selected.id === c.id}
            className={`flex items-center gap-2 rounded-control border px-3.5 py-2 text-[13px] font-semibold transition-colors ${
              selected.id === c.id ? "border-ink bg-ink text-paper" : "border-ink/20 bg-white/60 hover:border-ink/45"
            }`}
          >
            <span className={`inline-block h-2 w-2 rounded-full ${dot}`} />
            {c.name}
          </button>
        );
      })}
      {loaded && cases.length === 0 && <span className="py-2 text-[13px] text-ink/50">{t("pat.empty")}</span>}
    </div>

    /* Ringkasan */
    <section className="reveal mt-6 rounded-panel border border-ink/10 bg-white/70 p-5 shadow-tile sm:p-6">
      <h2 className="font-display text-[22px] font-semibold leading-snug">{insight}</h2>
      <p className="mt-1.5 text-[13px] text-ink/55">
        {selected.age ? `${selected.age}` : "-"} · {selected.sex || "-"} · {t("cli.screeningDate")}{" "}
        {new Intl.DateTimeFormat(lang === "id" ? "id-ID" : "en-GB", { dateStyle: "medium" }).format(new Date(selected.date))}
      </p>

      {escalate && (
        <p className="mt-4 flex items-start gap-2.5 rounded-tile border border-refer/35 bg-refer/5 px-4 py-3 text-[13.5px] font-semibold text-refer">
          <svg viewBox="0 0 24 24" className="mt-0.5 h-4 w-4 shrink-0 fill="none" stroke="currentColor" strokeWidth="2"
            strokeLinecap="round">

```

```

        <path d="M12 4.8 3.2 19.2h17.6L12 4.8Z" />
        <path d="M12 10v4M12 16.6v.2" />
    </svg>
    {t("pat.escalation"))}
</p>
})

<div className="lg:mt-5 lg:grid lg:grid-cols-12 lg:gap-6">
    <div className="lg:col-span-5">
        {/* Stat tiles */}
        <div className="mt-5 grid grid-cols-2 gap-3 sm:grid-cols-4 lg:mt-0 lg:grid-cols-2">
            {[
                [fmtPct(risk), t("pat.stat.risk"), risk >= 0.61 ? "text-refer" : risk >= 0.3 ? "text-defer" : "text-clear"],
                [String(selected.history.length), t("pat.stat.checkins"), ""],
                [selected.history.length ? fmtPct(taken / selected.history.length) : "-", t("pat.stat.ontime"), ""],
                [hasDanger ? t("pat.yes") : t("pat.no"), t("pat.stat.danger"), hasDanger ? "text-refer" : "text-clear"],
            ].map(([, label, tone]) => (
                <div key={label as string} className="rounded-tile border border-ink/10 bg-paper/70 p-3">
                    <p className={fontDisplayText[24px] fontSemibold leadingNone tabularNums ${tone}}>{v}</p>
                    <p className="mt-1.5 text-[11.5px] leading-snug text-ink/55">{label}</p>
                </div>
            ))}
        </div>

        {/* Bar 14 check-in */}
        {bars.length > 0 && (
            <div className="mt-5 rounded-tile border border-ink/10 bg-paper/70 p-4">
                <p className="text-[12px] font-bold text-ink/60">{t("pat.window")}</p>
                <div className="mt-3 flex items-end gap-1.5">
                    {bars.map((h, i) => (
                        <div key={i} className="flex flex-1 flex-col items-center gap-1">
                            <span className={h-1.5 w-1.5 rounded-full ${h.danger.length ? "bg-refer" : "bg-transparent"}} />
                            <span
                                className={wFull maxW-[14px] rounded-[3px] ${h.med ? "bg-clear/85" : "bg-refer/85"}}
                                style={{ height: `${10 + h.symptoms * 7}px` }}
                            />
                        </div>
                    ))}
                </div>
                <div className="mt-2.5 flex gap-4 text-[11px] text-ink/55">
                    <span className="flex items-center gap-1.5">
                        <span className="h-2 w-2 rounded-full bg-clear" /> {t("pat.legend.taken")}
                    </span>
                    <span className="flex items-center gap-1.5">
                        <span className="h-2 w-2 rounded-full bg-refer" /> {t("pat.legend.missed")}
                    </span>
                </div>
            </div>
        )}
    </div>

    <div className="lg:col-span-7">
        {/* Timeline perawatan */}
        <div className="mt-5 grid gap-5 sm:grid-cols-[1fr_1.2fr] lg:grid-cols-2">
            <div>
                <p className="text-[12px] font-bold text-ink/60">{t("pat.timeline.title")}</p>
                <ol className="mt-3 space-y-3">
                    {stages.map((s) => (
                        <li key={s.key} className="flex items-start gap-3">
                            <StatusNode status={s.status} />
                            <div className={s.status === "next" ? "opacity-45" : ""}>
                                <p className="text-[13.5px] font-semibold leading-snug">{t(s.key)}</p>
                                <p className="text-[11.5px] text-ink/50">
                                    {s.status === "done" ? t("pat.status.done") : s.status === "active" ? t("pat.status.active") :
t("pat.status.next")}
                                </p>
                            </div>
                        </li>
                    ))}
                </ol>
            </div>

            <div>
                <p className="text-[12px] font-bold text-ink/60">{t("pat.checkin.title")}</p>
                <div className="mt-3 space-y-3">
                    <label className="flex items-center gap-2.5 text-[13.5px]">
                        <input type="checkbox" checked={med} onChange={(e) => setMed(e.target.checked)} className="h-4 w-4 accent-ink" />
                        {t("pat.checkin.med")}
                    </label>
                    <label className="flex items-center gap-2.5 text-[13.5px]">
                        {t("pat.checkin.symptoms")}
                        <select
                            value={symptoms}
                            onChange={(e) => setSymptoms(Number(e.target.value))}
                            className="rounded-[7px] border border-ink/20 bg-white px-2 py-1.5 text-[13.5px]"
                        >
                            {[0, 1, 2, 3].map((n) => (
                                <option key={n} value={n}>
                                    {n}
                                </option>
                            ))}
                        </select>
                    </label>
                </div>
            </div>
        </div>
    </div>

```

```

</label>
<div className="flex flex-wrap gap-2">
  [{"d1", "d2", "d3"].map((d) => {
    const label = t(`pat.checkin.${d}`);
    const on = danger.includes(label);
    return (
      <button
        key={d}
        type="button"
        onClick={() => setDanger(on ? danger.filter((x) => x !== label) : [...danger, label])}
        aria-pressed={on}
        className={`rounded-full border px-3 py-1.5 text-[12.5px] font-semibold transition-colors ${
          on ? "border-refer bg-refer/10 text-refer" : "border-ink/20 text-ink/60 hover:border-ink/45"
        }`}
      >
        {label}
      </button>
    );
  });
</div>
<div className="flex flex-col gap-2 sm:flex-row">
  <input
    value={note}
    onChange={(e) => setNote(e.target.value)}
    placeholder={t("pat.checkin.note")}
    className="min-w-0 flex-1 rounded-control border border-ink/20 bg-white px-3 py-2.5 text-[14px] outline-none
focus:border-ink/50"
  />
  <button
    type="button"
    onClick={addCheckIn}
    className="rounded-control bg-ink px-5 py-2.5 text-[14px] font-semibold text-paper transition-colors hover:bg-
ink/90"
  >
    {t("pat.checkin.save")}
  </button>
</div>
</div>
</div>
</div>

{/* Riwayat */}
{selected.history.length > 0 && (
  <div className="mt-6 border-t border-ink/10 pt-5">
    <p className="text-[12px] font-bold text-ink/60">{t("pat.history.title")}</p>
    <ul className="mt-3 space-y-2">
      {selected.history.map((h, i) => (
        <li key={i} className="flex flex-wrap items-baseline gap-x-3 gap-y-1 text-[13.5px] text-ink/75">
          <span className="tabular-nums text-ink/50">
            {new Intl.DateTimeFormat(lang === "id" ? "id-ID" : "en-GB", { dateStyle: "medium" }).format(new Date(h.date))}
          </span>
          <span className={h.med ? "text-clear" : "font-semibold text-refer"}>
            {h.med ? t("pat.history.med") : t("pat.history.missed")}
          </span>
          <span>{t("pat.history.symptoms", { n: h.symptoms })}</span>
          {h.danger.map((d) => (
            <span key={d} className="font-semibold text-refer">
              {d}
            </span>
          ))}
          {h.note && <span className="text-ink/55">{h.note}</span>}
        </li>
      ))}
    </ul>
  </div>
)}
</div>
</div>

<p className="mt-6 text-[11.5px] leading-relaxed text-ink/45">{t("pat.note")}</p>
</section>

<p className="mt-4 flex items-center gap-2 text-[12.5px] text-ink/60">
  <span className={`inline-block h-2 w-2 rounded-full ${meta.dot}`} />
  {t(meta.titleKey)}
</p>
</main>
);
}

```

web/src/app/skrining/page.tsx

```
import type { Metadata } from "next";
import ScreeningFlow from "@/components/ScreeningFlow";

export const metadata: Metadata = {
  title: "Screening – SIGAP",
};

export default function SkriningPage() {
  return (
    <main className="mx-auto max-w-6xl px-4 sm:px-6">
      <ScreeningFlow />
    </main>
  );
}
```

```

"use client";

import { useLang } from "@lib/i18n";

const LIVE_URL = "https://ip-172-26-13-244.tail40f715.ts.net";
const PAGES_URL = "https://alvncvnc.github.io/loop-tb";

export default function TentangPage() {
  const { t } = useLang();

  return (
    <main className="mx-auto max-w-4xl px-4 sm:px-6">
      <section className="pt-12">
        <p className="text-[12.5px] font-semibold tracking-wide text-ink/45">{t("landing.kicker")}</p>
        <h1 className="font-display mt-3 text-[34px] font-semibold leading-tight tracking-tight">{t("ab.title")}</h1>
        <p className="font-display mt-5 max-w-2xl text-[19px] leading-relaxed text-ink/80">{t("ab.lede")}</p>
      </section>

      <section className="mt-12">
        <h2 className="font-display text-[24px] font-semibold">{t("ab.how.title")}</h2>
        <ol className="mt-5 space-y-5">
          {[1, 2, 3].map((n) => (
            <li key={n} className="grid grid-cols-[44px_1fr] gap-4">
              <span className="font-display pt-0.5 text-[17px] font-semibold tabular-nums text-ink/30">0{n}</span>
              <p className="max-w-2xl text-[14.5px] leading-relaxed text-ink/75">{t(`ab.how.${n}`)}</p>
            </li>
          ))}
        </ol>
      </section>

      <section className="mt-12">
        <h2 className="font-display text-[24px] font-semibold">{t("ab.rigor.title")}</h2>
        <div className="mt-5 grid gap-3 lg:grid-cols-3">
          {[1, 2, 3].map((n) => (
            <div key={n} className="flex items-start gap-3 rounded-tile border border-ink/10 bg-white/70 p-4 shadow-tile">
              <svg viewBox="0 0 24 24" className="mt-0.5 h-4 w-4 shrink-0 text-clear" fill="none" stroke="currentColor"
                strokeWidth="2.2" strokeLinecap="round">
                <circle cx="12" cy="12" r="9" />
                <path d="m8.5 12.2 2.4 2.4 4.6-5" />
              </svg>
              <p className="text-[14px] leading-relaxed text-ink/75">{t(`ab.rigor.${n}`)}</p>
            </div>
          ))}
        </div>
      </section>

      <section className="mt-12 rounded-panel bg-film p-6 text-bone shadow-float sm:p-8">
        <h2 className="font-display text-[24px] font-semibold">{t("ab.links.title")}</h2>
        <p className="mt-3 max-w-2xl text-[14px] leading-relaxed text-bone/70">{t("ab.links.body")}</p>
        <div className="mt-5 flex flex-wrap gap-3">
          <a href={LIVE_URL} target="_blank" rel="noreferrer" className="rounded-control bg-bone px-4 py-2.5 text-[13.5px] font-bold text-film transition-all hover:-translate-y-0.5">
            {t("landing.try.full")} ↗
          </a>
          <a href={PAGES_URL} target="_blank" rel="noreferrer" className="rounded-control border border-bone/30 px-4 py-2.5 text-[13.5px] font-semibold text-bone transition-colors hover:border-bone/60">
            {t("landing.try.static")} ↗
          </a>
          <a
            href="https://github.com/Alvncvnc/loop-tb"
            target="_blank"
            rel="noreferrer"
            className="rounded-control border border-bone/30 px-4 py-2.5 text-[13.5px] font-semibold text-bone transition-colors hover:border-bone/60"
          >
            GitHub ↗
          </a>
          <a
            href="https://github.com/Alvncvnc/loop-tb/releases/tag/models-v1"
            target="_blank"
            rel="noreferrer"
            className="rounded-control border border-bone/30 px-4 py-2.5 text-[13.5px] font-semibold text-bone transition-colors hover:border-bone/60"
          >
            models-v1 ↗
          </a>
        </div>
      </section>

      <section className="mt-12 pb-4">
        <p className="rounded-panel border border-ink/15 bg-white/70 p-5 text-[14px] leading-relaxed text-ink/70">
          {t("ab.disclaimer")}
        </p>
      </section>
    </main>
  );
}

```

web/src/components/BottomNav.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useLang } from "@lib/i18n";

const ITEMS = [
  {
    href: "/",
    key: "nav.home",
    icon: (
      <path d="M4 11.5 12 418 7.5M6.5 10v9.5h11V10" />
    ),
  },
  {
    href: "/skrining",
    key: "nav.screening",
    icon: (
      <>
        <rect x="4" y="4" width="16" height="16" rx="3" />
        <path d="M12 8.5v7M8.5 12h7" />
      </>
    ),
  },
  {
    href: "/pasien",
    key: "nav.patients",
    icon: (
      <>
        <circle cx="12" cy="8.2" r="3.4" />
        <path d="M5.4 19.5c1.3-3 3.8-4.5 6.6-4.5s5.3 1.5 6.6 4.5" />
      </>
    ),
  },
  {
    href: "/klinisi",
    key: "nav.clinicians",
    icon: (
      <>
        <rect x="5" y="4" width="14" height="16" rx="2.5" />
        <path d="M9 4.8h6M9 10h6M9 13.6h6M9 17.2h3.5" />
      </>
    ),
  },
  {
    href: "/tentang",
    key: "nav.about",
    icon: (
      <>
        <circle cx="12" cy="12" r="8.2" />
        <path d="M12 11v5.2M12 7.8v.2" />
      </>
    ),
  },
];

export default function BottomNav() {
  const path = usePathname();
  const { t } = useLang();

  return (
    <nav
      aria-label="Primary"
      className="no-print fixed inset-x-0 bottom-0 z-40 border-t border-ink/10 bg-paper/95 pb-[env(safe-area-inset-bottom)] backdrop-blur-sm md:hidden"
    >
      <ul className="mx-auto flex max-w-md items-stretch justify-between px-3">
        {ITEMS.map((it) => {
          const active = it.href === "/" ? path === "/" : path?.startsWith(it.href);
          return (
            <li key={it.href} className="flex-1">
              <Link
                href={it.href}
                aria-current={active ? "page" : undefined}
                className={`flex flex-col items-center gap-0.5 rounded-control px-1 py-2 text-[10.5px] font-semibold transition-colors ${
                  active ? "text-ink" : "text-ink/45 hover:text-ink/75"
                }`}
              >
                <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round">
                  {it.icon}
                </svg>
                {t(it.key)}
              </Link>
            </li>
          );
        })}
      </ul>
    </nav>
  );
}
```

```
        </ul>
      </nav>
    );
  }
```

web/src/components/ScreeningFlow.tsx

```
"use client";

import { useEffect, useRef, useState, type ReactNode } from "react";
import { BAND_META, fmtPct, predict, type Band, type PredictResponse } from "@lib/api";
import { useLang } from "@lib/i18n";

const BASE_PATH = process.env.NEXT_PUBLIC_BASE_PATH ?? "";

type Patient = { name: string; age: string; sex: string; complaint: string };

const EMPTY_PATIENT: Patient = { name: "", age: "", sex: "", complaint: "" };

const DEMO_RESULT: PredictResponse = {
  p_tb: 0.42,
  band: "ragu",
  bands: { tau_low: 0.21, tau_high: 0.61 },
  arch: "demo archive (not from your image)",
};

type GalleryCase = {
  id: string;
  title: string;
  desc: string;
  label: string;
  image: string;
  result: string;
  band: Band;
  p: number;
  u: number;
};

const CASE_KEY: Record<string, string> = {
  chncxr_0338_1: "a",
  chncxr_0178_0: "b",
  chncxr_0495_1: "c",
  mcucxr_0044_0: "d",
};

function StepTitle({ n, children }: { n: number; children: ReactNode }) {
  return (
    <h2 className="flex items-center gap-3 text-[15px] font-bold">
      <span className="grid h-7 w-7 place-items-center rounded-control border border-ink/20 text-[13px] tabular-nums">
        {n}
      </span>
      {children}
    </h2>
  );
}

function CheckIcon() {
  return (
    <svg viewBox="0 0 24 24" className="h-3.5 w-3.5 shrink-0 text-clear" fill="none" stroke="currentColor" strokeWidth="2.4"
      strokeLinecap="round">
      <path d="m5.5 12.5 4 4l18.5 8" />
    </svg>
  );
}

function XRayGlyph() {
  return (
    <svg viewBox="0 0 48 48" className="h-12 w-12 text-ink/35" fill="none" stroke="currentColor" strokeWidth="1.6"
      strokeLinecap="round">
      <rect x="7" y="5" width="34" height="38" rx="3" />
      <path d="M24 10v28" />
      <path d="M24 13c-5 0-8 3-8 0 4 2 7 5 9 1.5 1 2.2 3 2.2 6" />
      <path d="M24 13c5 0 8 3 8 0 4-2 7-5 9-1.5 1-2.2 3-2.2 6" />
      <path d="M20 13.5 24 15l4-1.5" />
    </svg>
  );
}

function ProbScale({
  p,
  u,
  tauLow,
  tauHigh,
}: {
  p: number;
  u: number;
  tauLow: number;
  tauHigh: number;
}) {
  const { t } = useLang();
  const left = (x: number) => `${Math.min(100, Math.max(0, x * 100))}%`;
  const lo = Math.max(0, p - u);
  const hi = Math.min(1, p + u);
  return (
    <div className="rounded-tile border border-bone/12 bg-bone/[0.04] p-4">

```

```

<div className="flex items-baseline justify-between">
  <span className="text-[12px] font-semibold text-bone/55">{t("scr.scale.ptb")}</span>
  <span className="font-display text-[38px] font-semibold leading-none tabular-nums">{fmtPct(p)}</span>
</div>
<div className="relative mt-4">
  <div className="relative h-2.5 overflow-hidden rounded-full bg-bone/10">
    <span className="absolute inset-y-0 left-0 bg-clear/80" style={{ width: left(tauLow) }} />
    <span className="absolute inset-y-0 bg-defer/80" style={{ left: left(tauLow), width: left(Math.max(0, tauHigh - tauLow)) }} />
    <span className="absolute inset-y-0 right-0 bg-refer/80" style={{ left: left(tauHigh) }} />
  </div>
  {u > 0 && (
    <div
      className="absolute -bottom-[8px] h-[4px] rounded-full bg-bone/70"
      style={{ left: left(lo), width: `${Math.max(0.8, (hi - lo) * 100)}%` }}
      title={t("scr.scale.unc")}
    />
  )}
  <div className="absolute -top-[6px] h-[22px]" style={{ left: `calc(${left(p)} - 2px)` }}>
    <div className="grow-x h-full w-[4px] rounded-full bg-bone" />
  </div>
</div>
<div className="mt-4 flex flex-wrap gap-x-4 gap-y-1 text-[10.5px] text-bone/55">
  <span className="flex items-center gap-1.5">
    <span className="h-2 w-2 rounded-full bg-clear" />
    {t("scr.scale.low")} &lt; {fmtPct(tauLow)}
  </span>
  <span className="flex items-center gap-1.5">
    <span className="h-2 w-2 rounded-full bg-defer" />
    {t("scr.scale.tauLow")} {fmtPct(tauLow)} - {fmtPct(tauHigh)}
  </span>
  <span className="flex items-center gap-1.5">
    <span className="h-2 w-2 rounded-full bg-refer" />
    {t("scr.scale.tauHigh")} ≥ {fmtPct(tauHigh)}
  </span>
  {u > 0 && <span className="ml-auto tabular-nums">{t("scr.scale.unc")}: ±{fmtPct(u, 1)}</span>}
</div>
);
}

function FindingCard({ title, body, tone = "bone" }: { title: string; body: string; tone?: "bone" | "defer" }) {
  return (
    <div className={`rounded-tile border p-4 ${tone === "defer" ? "border-defer/40 bg-defer/10" : "border-bone/12 bg-bone/[0.04]"}`>
      <h3 className={`text-[13px] font-bold ${tone === "defer" ? "text-[#e5b95c]" : "text-bone"}`>{title}</h3>
      <p className="mt-1 text-[12.5px] leading-relaxed text-bone/70">{body}</p>
    </div>
  );
}

export default function ScreeningFlow() {
  const { t, lang } = useLang();
  const [file, setFile] = useState<File | null>(null);
  const [gallery, setGallery] = useState<GalleryCase[]>([]);
  const [caseId, setCaseId] = useState("");
  const [preview, setPreview] = useState("");
  const [patient, setPatient] = useState<Patient>(EMPTY_PATIENT);
  const [demo, setDemo] = useState(false);
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState<PredictResponse | null>(null);
  const [error, setError] = useState("");
  const [overlay, setOverlay] = useState<"heatmap" | "original">("heatmap");
  const [saved, setSaved] = useState(false);
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    fetch(`${BASE_PATH}/cases/index.json`)
      .then((r) => (r.ok ? r.json() : []))
      .then((d) => setGallery(Array.isArray(d) ? d : []))
      .catch(() => {});
  }, []);

  async function openCase(c: GalleryCase) {
    try {
      const r = await fetch(`${BASE_PATH}/cases/${c.result}`);
      if (!r.ok) throw new Error("load");
      const data = (await r.json()) as PredictResponse;
      setPreview(`${BASE_PATH}/cases/${c.image}`);
      setResult(data);
      setOverlay("heatmap");
      setCaseId(c.id);
      setSaved(false);
      setError("");
    } catch {
      setError(t("scr.error"));
    }
  }

  function onPick(f: File | null | undefined) {
    if (!f) return;
    if (!f.type.startsWith("image/")) {
      setError(t("scr.error"));
    }
  }
}

```

```

    return;
  }
  setError("");
  setResult(null);
  setSaved(false);
  setCaseId("");
  setFile(f);
  setPreview(URL.createObjectURL(f));
}

function clearFile() {
  setFile(null);
  setPreview("");
  setResult(null);
  setSaved(false);
  setCaseId("");
  if (inputRef.current) inputRef.current.value = "";
}

async function analyze() {
  if (!file || loading) return;
  setLoading(true);
  setError("");
  setResult(null);
  try {
    const r = demo ? DEMO_RESULT : await predict(file);
    setCaseId("");
    setResult(r);
    setOverlay(r.heatmap_png_b64 ? "heatmap" : "original");
  } catch {
    setError(t("scr.error"));
  } finally {
    setLoading(false);
  }
}

function saveToCard() {
  if (!result) return;
  try {
    const cases = JSON.parse(localStorage.getItem("sigap.cases") ?? "[]");
    cases.unshift({
      id: crypto.randomUUID(),
      date: new Date().toISOString(),
      name: patient.name.trim() || "-",
      age: patient.age,
      sex: patient.sex,
      p: result.p_tb,
      band: result.band,
      history: [],
    });
    localStorage.setItem("sigap.cases", JSON.stringify(cases));
    setSaved(true);
  } catch {
    setError(t("scr.error"));
  }
}

const heat = result?.heatmap_png_b64 ? `data:image/png;base64,${result.heatmap_png_b64}` : "";
const meta = result ? BAND_META[result.band] : null;
const today = new Intl.DateTimeFormat(lang === "id" ? "id-ID" : "en-GB", { dateStyle: "long" }).format(new Date());
const unDeferred =
  result && result.bands.tau_uncertainty !== undefined && result.uncertainty !== undefined
    ? result.uncertainty > result.bands.tau_uncertainty
    : false;

return (
  <>
    <div className="print:hidden">
      <section className="pt-10">
        <h1 className="font-display text-[30px] font-semibold tracking-tight">{t("scr.title")}</h1>
        <p className="mt-2 max-w-xl text-[15px] leading-relaxed text-ink/70">{t("scr.sub")}</p>
      </section>

      <div className="lg:grid lg:grid-cols-12 lg:items-start lg:gap-8">
        <div className="lg:col-span-5">
          /* Galeri contoh arsip (mobile: di atas; desktop: rail kanan) */
          {gallery.length > 0 && (
            <section className="mt-8 rounded-panel border border-ink/10 bg-white/60 p-4 shadow-tile sm:p-5 lg:hidden">
              <h2 className="text-[15px] font-bold">{t("scr.gallery.title")}</h2>
              <p className="mt-1 text-[13px] leading-relaxed text-ink/60">{t("scr.gallery.desc")}</p>
              <div className="mt-3 grid gap-2 sm:grid-cols-2">
                {gallery.map((c) => {
                  const key = CASE_KEY[c.id];
                  const title = key ? t(`scr.case.${key}.title`) : c.title;
                  const desc = key ? t(`scr.case.${key}.desc`) : c.desc;
                  const dot = c.band === "rujuk_prioritas" ? "bg-refer" : c.band === "negatif_skrining" ? "bg-clear" : "bg-defer";
                  return (
                    <button
                      key={c.id}
                      type="button"
                      onClick={() => openCase(c)}
                      aria-pressed={caseId === c.id}

```

```

        className={`rounded-tile border p-3 text-left transition-all hover:-translate-y-0.5 hover:shadow-tile ${
          caseId === c.id ? "border-ink bg-ink/5" : "border-ink/15 bg-paper/60 hover:border-ink/40"
        }}`
      >
      <span className="flex items-center gap-2 text-[13px] font-semibold">
        <span className={`inline-block h-2 w-2 rounded-full ${dot}`} />
        {title}
      </span>
      <span className="mt-1 block text-[12px] leading-relaxed text-ink/60">{desc}</span>
    </button>
  );
}
</div>
</section>
)}

{/* 1 – Citra */}
<section className="mt-9">
  <StepTitle n={1}>{t("scr.step1")}</StepTitle>
  <input
    ref={inputRef}
    type="file"
    accept="image/*"
    className="sr-only"
    aria-label={t("scr.drop.title")}
    onChange={(e) => onPick(e.target.files?.[0])}
  />
  {!file ? (
    <button
      type="button"
      onClick={() => inputRef.current?.click()}
      onDragOver={(e) => e.preventDefault()}
      onDrop={(e) => {
        e.preventDefault();
        onPick(e.dataTransfer.files?.[0])};
      }}
      className="mt-4 flex w-full flex-col items-center gap-3 rounded-panel border border-dashed border-ink/20 bg-white/60
px-6 py-10 text-center transition-colors hover:border-ink/40"
    >
      <XRayGlyph />
      <span className="text-[15px] font-semibold">{t("scr.drop.title")}</span>
      <span className="max-w-sm text-[13px] text-ink/55">{t("scr.drop.hint")}</span>
    </button>
  ) : (
    <div className="mt-4 rounded-panel border border-ink/12 bg-white/70 p-3 shadow-tile">
      <div className="flex items-start gap-4">
        <div className="relative shrink-0">
          {/* eslint-disable-next-line @next/next/no-img-element */}
          <img src={preview} alt="" className="h-24 w-24 rounded-tile bg-film object-contain" />
          <button
            type="button"
            onClick={clearFile}
            aria-label={t("scr.remove")}
            className="absolute -right-2 -top-2 grid h-6 w-6 place-items-center rounded-full border border-ink/15 bg-paper
text-ink/60 transition-colors hover:text-ink"
          >
            <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="2.2"
strokeLinecap="round">
              <path d="m7 7 10 10 17 7 7 17" />
            </svg>
          </button>
        </div>
        <div className="min-w-0 flex-1 pt-1">
          <p className="truncate text-[14px] font-semibold">{file.name}</p>
          <p className="mt-0.5 text-[13px] text-ink/55">{(file.size / 1e6).toFixed(1)} MB</p>
          <button
            type="button"
            onClick={() => inputRef.current?.click()}
            className="mt-2 text-[13px] font-semibold underline decoration-ink/30 underline-offset-2 hover:text-ink"
          >
            {t("scr.change")}
          </button>
        </div>
      </div>
    </div>
    <div className="mt-3 rounded-tile bg-mist px-4 py-3">
      <p className="text-[12px] font-bold text-ink/70">{t("scr.guide.title")}</p>
      <ul className="mt-1.5 grid gap-1 sm:grid-cols-2">
        {[1, 2, 3, 4].map((n) => (
          <li key={n} className="flex items-center gap-2 text-[12.5px] text-ink/65">
            <CheckIcon />
            {t(`scr.guide.${n}`)}
          </li>
        ))}
      </ul>
    </div>
  )}
</div>
)}
</section>

{/* 2 – Data pasien */}
<section className="mt-9">
  <StepTitle n={2}>{t("scr.step2")}</StepTitle>

```

```

<div className="mt-4 grid gap-3 sm:grid-cols-2">
  <label className="block text-[13px] font-medium text-ink/70">
    {t("scr.field.name")}
    <input
      value={patient.name}
      onChange={(e) => setPatient({ ...patient, name: e.target.value })}
      className="mt-1 w-full rounded-control border border-ink/20 bg-white px-3 py-2.5 text-[15px] outline-none
focus:border-ink/50"
      placeholder={t("scr.field.name.ph")}
    />
  </label>
  <div className="grid grid-cols-2 gap-3">
    <label className="block text-[13px] font-medium text-ink/70">
      {t("scr.field.age")}
      <input
        value={patient.age}
        onChange={(e) => setPatient({ ...patient, age: e.target.value })}
        inputMode="numeric"
        className="mt-1 w-full rounded-control border border-ink/20 bg-white px-3 py-2.5 text-[15px] outline-none
focus:border-ink/50"
        placeholder="34"
      />
    </label>
    <label className="block text-[13px] font-medium text-ink/70">
      {t("scr.field.sex")}
      <select
        value={patient.sex}
        onChange={(e) => setPatient({ ...patient, sex: e.target.value })}
        className="mt-1 w-full rounded-control border border-ink/20 bg-white px-3 py-2.5 text-[15px] outline-none
focus:border-ink/50"
      >
        <option value=""></option>
        <option value="L">{t("scr.sex.m")}</option>
        <option value="P">{t("scr.sex.f")}</option>
      </select>
    </label>
  </div>
  <label className="block text-[13px] font-medium text-ink/70 sm:col-span-2">
    {t("scr.field.complaint")}
    <input
      value={patient.complaint}
      onChange={(e) => setPatient({ ...patient, complaint: e.target.value })}
      className="mt-1 w-full rounded-control border border-ink/20 bg-white px-3 py-2.5 text-[15px] outline-none
focus:border-ink/50"
      placeholder={t("scr.field.complaint.ph")}
    />
  </label>
</div>
</section>

/* Aksi */
<div className="mt-9 flex flex-col gap-4 sm:flex-row sm:items-center">
  <button
    type="button"
    onClick={analyze}
    disabled={!file || loading}
    className="w-full whitespace-nowrap rounded-control bg-ink px-6 py-3.5 text-[15px] font-semibold text-paper shadow-tile
transition-all hover:-translate-y-0.5 hover:bg-ink/90 disabled:cursor-not-allowed disabled:opacity-40 disabled:hover:translate-y-0
sm:w-auto"
  >
    {loading ? t("scr.analyzing") : t("scr.analyze")}
  </button>
  <label className="flex items-center gap-2 text-[13px] text-ink/65">
    <input type="checkbox" checked={demo} onChange={(e) => setDemo(e.target.checked)} className="h-4 w-4 accent-ink" />
    {t("scr.demo")}
  </label>
</div>
<p className="mt-4 text-[12px] text-ink/45">{t("scr.trust")}</p>

{loading && (
  <div className="mt-5 h-1 w-full overflow-hidden rounded-full bg-ink/10" role="status" aria-label={t("scr.analyzing")}>
    <div className="h-full w-1/3 animate-pulse rounded-full bg-ink/60" />
  </div>
)}

{error && (
  <p role="alert" className="mt-5 rounded-control border border-refer/30 bg-refer/5 px-4 py-3 text-[14px] text-refer">
    {error}
  </p>
)}
</div>

/* Rail kanan (desktop): galeri saat kosong, hasil setelah analisis */
<div className="lg:col-span-7">
{(!result || !meta) && gallery.length > 0 && (
  <section className="mt-8 hidden rounded-panel border border-ink/10 bg-white/60 p-4 shadow-tile sm:p-5 lg:mt-9 lg:block">
    <h2 className="text-[15px] font-bold">{t("scr.gallery.title")}</h2>
    <p className="mt-1 text-[13px] leading-relaxed text-ink/60">{t("scr.gallery.desc")}</p>
    <div className="mt-3 grid gap-2 sm:grid-cols-2">
      {gallery.map((c) => {
        const key = CASE_KEY[c.id];
        const title = key ? t(`scr.case.${key}.title`) : c.title;

```

```

const desc = key ? t(`scr.case.${key}.desc`) : c.desc;
const dot = c.band === "rujuk_prioritas" ? "bg-refer" : c.band === "negatif_skrining" ? "bg-clear" : "bg-defer";
return (
  <button
    key={c.id}
    type="button"
    onClick={() => openCase(c)}
    aria-pressed={caseId === c.id}
    className={`rounded-tile border p-3 text-left transition-all hover:-translate-y-0.5 hover:shadow-tile ${
      caseId === c.id ? "border-ink bg-ink/5" : "border-ink/15 bg-paper/60 hover:border-ink/40"
    }}
  >
    <span className="flex items-center gap-2 text-[13px] font-semibold">
      <span className={`inline-block h-2 w-2 rounded-full ${dot}`} />
      {title}
    </span>
    <span className="mt-1 block text-[12px] leading-relaxed text-ink/60">{desc}</span>
  </button>
);
  })}
</div>
</section>
))
{(!result || !meta) && (
  <div className="mt-6 hidden rounded-panel border border-dashed border-ink/20 bg-film/[0.03] px-6 py-12 text-center
lg:block">
    <div className="mx-auto h-2.5 w-52 overflow-hidden rounded-full bg-[linear-
gradient(90deg,#2e7d5b_0%,#2e7d5b_2%,#b7791f_10%,#c0392b_30%,#c0392b_100%)] opacity-70" />
    <p className="mt-4 text-[14px] font-semibold text-ink/60">{t("scr.resultPlaceholder")}</p>
  </div>
))
{result && meta && (
  <section className="reveal mt-10 lg:mt-9" aria-live="polite">
    <h2 className="text-[15px] font-bold">{t("scr.result")}</h2>
    <div className="mt-4 overflow-hidden rounded-panel bg-film text-bone shadow-float">
      <div className="relative aspect-[4/3]">
        {/* eslint-disable-next-line @next/next/no-img-element */}
        <img
          src={overlay === "heatmap" && heat ? heat : preview}
          alt={t("scr.overlay.heat")}
          className="absolute inset-0 h-full w-full object-contain"
        />
        {heat && (
          <div className="absolute top-3 right-3 flex gap-1 rounded-control bg-film/80 p-1">
            {
              [
                ["heatmap", t("scr.overlay.heat")],
                ["original", t("scr.overlay.orig")],
              ] as const
            }.map(([k, label]) => (
              <button
                key={k}
                type="button"
                onClick={() => setOverlay(k)}
                aria-pressed={overlay === k}
                className={`rounded-[7px] px-2.5 py-1.5 text-[12px] font-semibold transition-colors ${
                  overlay === k ? "bg-bone text-film" : "text-bone/70 hover:text-bone"
                }}
              >
                {label}
              </button>
            ))
          </div>
        )}
      </div>
    </div>
  <div className="border-t border-bone/10 p-5 sm:p-6">
    <p className={`flex items-center gap-2 text-[12.5px] font-bold ${meta.text}`}>
      <span className={`inline-block h-2 w-2 rounded-full ${meta.dot}`} />
      {t(meta.labelKey)}
    </p>
    <h3 className="font-display mt-1 text-[28px] font-semibold leading-tight sm:text-[34px]">{t(meta.titleKey)}</h3>
    <p className="mt-2.5 max-w-2xl text-[14.5px] leading-relaxed text-bone/85">{t(meta.actionKey)}</p>

    <div className="mt-5">
      <ProbScale p={result.p_tb} u={result.uncertainty ?? 0} tauLow={result.bands.tau_low} tauHigh=
{result.bands.tau_high} />
    </div>

    <div className="mt-3 grid gap-3">
      <FindingCard title={t("scr.card.observed.title")} body={t("scr.card.observed.body")} />
      <FindingCard title={t("scr.card.notshown.title")} body={t("scr.card.notshown.body")} />
      <FindingCard
        title={t("scr.card.unc.title")}
        body={
          <uncDeferred
            ? t("scr.card.unc.body", { u: fmtPct(result.uncertainty ?? 0, 1) })
            : t("scr.card.unc.low", { u: fmtPct(result.uncertainty ?? 0, 1) })
          >
          </uncDeferred>
          < ? "defer" : "bone">
        </div>
    </div>

```

```

        {caseId && (
          <p className="mt-4 rounded-tile border border-bone/12 bg-bone/[0.04] px-4 py-3 text-[12.5px] leading-relaxed text-
bone/60">
            {t("scr.archiveNote")}
          </p>
        )}

        <div className="mt-6 flex flex-wrap gap-3 border-t border-bone/10 pt-5">
          <button
            type="button"
            onClick={() => window.print()}
            className="rounded-control bg-bone px-5 py-3 text-[14px] font-bold text-film transition-all hover:-translate-y-
0.5"
          >
            {t("scr.action.print")}
          </button>
          <button
            type="button"
            onClick={saveToCard}
            disabled={saved}
            className="rounded-control border border-bone/30 px-5 py-3 text-[14px] font-semibold text-bone transition-colors
hover:border-bone/60 disabled:opacity-50"
          >
            {saved ? t("scr.action.saved") : t("scr.action.save")}
          </button>
          <button
            type="button"
            onClick={() => {
              setResult(null);
              setSaved(false);
              setCaseId("");
              clearFile();
            }}
            className="rounded-control px-5 py-3 text-[14px] font-semibold text-bone/60 transition-colors hover:text-bone"
          >
            {t("scr.action.new")}
          </button>
        </div>

        <p className="mt-4 text-[11.5px] leading-relaxed text-bone/45">
          {result.arch ? `${result.arch}. ` : ""}
          {t("scr.meta")}
        </p>
        <p className="mt-1.5 text-[11.5px] leading-relaxed text-bone/45">{t("scr.disclaimer")}</p>
      </div>
    </div>
  </section>
)}
</div>
</div>
</div>

{/* Surat rujukan (hanya saat cetak) */}
<div className="hidden print:block">
  <h1 className="font-display text-[24px] font-semibold">{t("letter.title")}</h1>
  <p className="mt-1 text-[12px] text-ink/60">{t("letter.prototype")}</p>
  <table className="mt-6 w-full border-collapse text-[14px]">
    <tbody>
      <tr>
        <td>
          [t("letter.date"), today],
          [t("letter.facility"), t("letter.facility.ph")],
          [t("letter.patient"), patient.name || "_____"],
          [t("letter.agesex"), `${patient.age} || t("letter.dash") / ${patient.sex} || t("letter.dash")`],
          [t("letter.complaint"), patient.complaint || t("letter.dash")],
          [t("letter.risk"), result ? fmtPct(result.p_tb) : t("letter.dash")],
          [t("letter.band"), result ? t(BAND_META[result.band].titleKey) : t("letter.dash")],
        </td>
        <td>
          [
            t("letter.taus"),
            result ? `${fmtPct(result.bands.tau_low)} / ${fmtPct(result.bands.tau_high)}` : t("letter.dash"),
          ],
      </td>
    </tr>
    <tr>
      <td colspan="2">
        .map(([k, v]) => (
          <tr key={k} className="border-b border-ink/15">
            <td className="w-56 py-2 pr-4 align-top font-semibold">{k}</td>
            <td className="py-2">{v}</td>
          </tr>
        ))
      </td>
    </tr>
  </tbody>
</table>
<p className="mt-6 text-[14px] leading-relaxed">{t("letter.request")}</p>
<div className="mt-10 flex justify-between text-[14px]">
  <div>
    {t("letter.sender")}
    <div className="mt-14 w-56 border-t border-ink/40" />
  </div>
  <div>
    {t("letter.receiver")}
    <div className="mt-14 w-56 border-t border-ink/40" />
  </div>
</div>
</div>
</div>
</div>

```


web/src/components/SiteFooter.tsx

```
"use client";

import { useLang } from "@lib/i18n";

export default function SiteFooter() {
  const { t } = useLang();
  return (
    <footer className="no-print mt-16 border-t border-ink/10">
      <div className="mx-auto flex max-w-6xl flex-col gap-2 px-4 py-8 text-[13px] leading-relaxed text-ink/55 sm:px-6">
        <p className="max-w-3xl">{t("footer.text")}</p>
        <p>
          <a
            className="underline decoration-ink/30 underline-offset-2 hover:text-ink"
            href="https://github.com/Alvnc/loop-tb"
            target="_blank"
            rel="noreferrer"
          >
            {t("footer.repo")}
          </a>
        </p>
      </div>
    </footer>
  );
}
```

web/src/components/SiteHeader.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useLang } from "@lib/i18n";

const NAV = [
  { href: "/", key: "nav.home" },
  { href: "/skrining", key: "nav.screening" },
  { href: "/pasien", key: "nav.patients" },
  { href: "/klinisi", key: "nav.clinicians" },
  { href: "/tentang", key: "nav.about" },
];

export default function SiteHeader() {
  const path = usePathname();
  const { lang, setLang, t } = useLang();

  return (
    <header className="no-print sticky top-0 z-40 border-b border-ink/10 bg-paper/95 backdrop-blur-sm">
      <div className="mx-auto flex max-w-6xl items-center gap-3 px-4 py-2.5 sm:px-6 sm:py-3">
        <Link href="/" className="flex shrink-0 items-baseline gap-2">
          <span className="text-[19px] font-extrabold tracking-tight sm:text-[20px]">SIGAP</span>
          <span className="hidden text-[12px] text-ink/50 sm:block">{t("sfx.tagline")}</span>
        </Link>

        <nav aria-label="Primary" className="ml-auto hidden items-center gap-0.5 md:flex">
          {NAV.map((n) => {
            const active = n.href === "/" ? path === "/" : path?.startsWith(n.href);
            return (
              <Link
                key={n.href}
                href={n.href}
                aria-current={active ? "page" : undefined}
                className={`rounded-control px-3 py-2 text-[14px] font-medium transition-colors ${
                  active ? "bg-ink text-paper" : "text-ink/65 hover:bg-ink/5 hover:text-ink"
                }`}
              >
                {t(n.key)}
              </Link>
            );
          })}
        </nav>

        <div className="ml-auto flex items-center md:ml-3" role="group" aria-label={t("lang.switch")}>
          {[["en", "id"] as const].map((l) => (
            <button
              key={l}
              type="button"
              onClick={() => setLang(l)}
              aria-pressed={lang === l}
              className={`rounded-control px-2.5 py-1.5 text-[12px] font-bold uppercase tracking-wide transition-colors ${
                lang === l ? "bg-ink text-paper" : "text-ink/50 hover:text-ink"
              }`}
            >
              {l}
            </button>
          ))}
        </div>
      </div>
    </header>
  );
}
```

```

"use client";

import { createContext, useContext, useEffect, useState, type ReactNode } from "react";

export type Lang = "en" | "id";

const DICT: Record<Lang, Record<string, string>> = {
  en: {
    // common / nav
    "nav.home": "Home",
    "nav.screening": "Screening",
    "nav.patients": "Patients",
    "nav.clinicians": "Clinicians",
    "nav.about": "About",
    "sfx.tagline": "TB screening",
    "lang.switch": "Language",
    // landing
    "landing.kicker": "UnivaBio 2026 · AI for Human Health",
    "landing.title": "Screen faster than the disease spreads.",
    "landing.sub":
      "Indonesia carries 10% of the world's TB burden, and about 200,000 people each year transmit TB before ever being detected. SIGAP compresses the care loop with one calibrated risk state – and says “uncertain” when it should.",
    "landing.cta.primary": "Open live demo",
    "landing.cta.secondary": "Browse the code",
    "landing.rail.0.tag": "Screen",
    "landing.rail.0.title": "Detect earlier",
    "landing.rail.0.body":
      "A chest X-ray becomes a calibrated risk with a deferral band: when model families disagree, the system asks for a human re-read instead of forcing a verdict.",
    "landing.rail.1.tag": "Refer",
    "landing.rail.1.title": "Act without waiting",
    "landing.rail.1.body":
      "The score maps to an action band and a printable referral letter for GeneXpert confirmation, aligned with Indonesia's TOSS-TB workflow.",
    "landing.rail.2.tag": "Support",
    "landing.rail.2.title": "Follow through",
    "landing.rail.2.body":
      "The same risk state follows the patient home: adherence check-ins update the score and trigger escalation when it worsens.",
    "landing.mockup.caption": "Illustrative result – real model output on a public dataset case.",
    "landing.stats.0": "AUROC on unseen hospitals",
    "landing.stats.1": "training images removed after our leakage audit",
    "landing.stats.2": "sensitivity at 90% specificity (external)",
    "landing.evidence.title": "Evidence, reported honestly",
    "landing.evidence.open.title": "Measured & open",
    "landing.evidence.open.1":
      "External validation on two unseen hospitals (800 images), with bootstrap intervals, calibration (ECE) and age/sex subgroups.",
    "landing.evidence.open.2":
      "Data audit before training: 338 internal duplicates and 19 training images that leaked into the external test set were removed. The frozen split manifest is published.",
    "landing.evidence.open.3":
      "Representation study across three model families; the winner (RAD-DINO self-supervised probe) is open in the repo and as downloadable artifacts.",
    "landing.evidence.todo.title": "Not done yet – honestly",
    "landing.evidence.todo.1":
      "No prospective validation in Indonesia yet; this is a research prototype, not a medical device.",
    "landing.evidence.todo.2":
      "Triage thresholds come from public data, not from puskesmas populations; site calibration is planned.",
    "landing.evidence.todo.3": "Cough audio analysis and SITB integration are not implemented.",
    "landing.stack.title": "Under the hood",
    "landing.stack.body":
      "RAD-DINO (self-supervised chest X-ray) + linear probe, temperature scaling, cross-family disagreement deferral. Served with FastAPI and a bilingual Next.js UI; trained on Kaggle GPUs via API.",
    "landing.try.title": "Try it now",
    "landing.try.full": "Full app (live model API)",
    "landing.try.static": "Static mirror (same live API)",
    "landing.try.note":
      "Both links run the real ensemble: upload a chest X-ray and get a triage band, an attention map and a referral letter.",
    // screening
    "scr.title": "Chest X-ray screening",
    "scr.sub": "Upload a chest X-ray, run the analysis, then follow the action band. Under three minutes per patient.",
    "scr.gallery.title": "Archive examples – no server",
    "scr.gallery.desc":
      "Four real cases from the external NLM corpus with outputs from the same final ensemble, computed offline – try the full flow without an API connection.",
    "scr.resultPlaceholder": "Your triage result will appear here after analysis.",
    "scr.case.a.title": "Case A – Active TB",
    "scr.case.a.desc": "Ensemble agrees: refer today.",
    "scr.case.b.title": "Case B – Normal, unseen hospital",
    "scr.case.b.desc": "Model families disagree → the system withholds a decision.",
    "scr.case.c.title": "Case C – TB, inconclusive evidence",
    "scr.case.c.desc": "Even for TB, high uncertainty → human re-read.",
    "scr.case.d.title": "Case D – Normal, near negative",
    "scr.case.d.desc": "Low signal, but uncertainty keeps it in the middle band.",
    "scr.step1": "X-ray image",
    "scr.drop.title": "Choose X-ray image",
    "scr.drop.hint": "PNG or JPG, frontal view (PA/AP). Drag and drop, or click to choose.",
    "scr.change": "Change image",
    "scr.guide.title": "Before you analyze",
  },
  id: {}
};

```

```

"scr.guide.1": "Frontal chest view (PA/AP)",
"scr.guide.2": "Whole lung field visible, not cropped",
"scr.guide.3": "Use the original file – not a photo of a screen",
"scr.guide.4": "Avoid patient identifiers when possible",
"scr.step2": "Patient details (optional)",
"scr.field.name": "Name or patient code",
"scr.field.name.ph": "e.g. S-014",
"scr.field.age": "Age",
"scr.field.sex": "Sex",
"scr.sex.m": "Male",
"scr.sex.f": "Female",
"scr.field.complaint": "Main complaint",
"scr.field.complaint.ph": "e.g. cough for 3 weeks, night sweats",
"scr.demo": "Demo mode (sample output – not from your image)",
"scr.analyze": "Analyze image",
"scr.analyzing": "Analyzing...",
"scr.error":
  "Model server unavailable. Try the archive examples above (final ensemble outputs – no server needed), or switch on demo
mode.",
"scr.result": "Analysis result",
"scr.band.refer.label": "Priority referral",
"scr.band.refer.title": "Refer – priority today",
"scr.band.refer.action":
  "Send the patient for GeneXpert testing at the nearest facility today. Record them as a presumptive TB case.",
"scr.band.defer.label": "Needs re-read",
"scr.band.defer.title": "Uncertain – needs re-read",
"scr.band.defer.action":
  "Do not decide alone. Ask a clinician to read the image, or repeat the screening with a better X-ray.",
"scr.band.neg.label": "Negative screen",
"scr.band.neg.title": "Negative screen",
"scr.band.neg.action":
  "Educate on TB symptoms. Ask the patient to return if symptoms last more than two weeks or worsen.",
"scr.scale.ptb": "p(TB) estimate",
"scr.scale.unc": "Uncertainty (disagreement)",
"scr.scale.low": "Low risk",
"scr.scale.high": "High risk",
"scr.scale.tauLow": "uncertain",
"scr.scale.tauHigh": "refer",
"scr.card.observed.title": "What the model found",
"scr.card.observed.body":
  "The areas that most influenced the score are highlighted on the attention map – for a human to verify, not to quote as proof
of a lesion.",
"scr.card.notshown.title": "What it cannot tell",
"scr.card.notshown.body":
  "The attention map is not evidence of a lesion. The model cannot separate healed TB scars from active TB – that is why the
uncertain band exists and human reading remains mandatory.",
"scr.card.unc.title": "About the uncertainty",
"scr.card.unc.body":
  "Model families disagree on this image (u = {u}). The score and the band may change with a higher-quality X-ray, so treat it as
a screening signal only.",
"scr.card.unc.low":
  "Model families broadly agree on this image (u = {u}), which increases confidence in this band – subject to the limits below.",
"scr.overlay.heat": "Attention map",
"scr.overlay.orig": "Original",
"scr.trust": "Processing runs in this browser session or your clinic's server. Research prototype – not a diagnosis.",
"scr.remove": "Remove image",
"scr.action.print": "Print referral letter",
"scr.action.save": "Save to patient record",
"scr.action.saved": "Saved to record",
"scr.action.new": "New screening",
"scr.disclaimer":
  "Research triage tool – not a diagnosis. Every result must be confirmed by a health professional (GeneXpert).",
"scr.archiveNote":
  "Archive output – computed offline with the final model on an NLM external-corpus case. Live server mode is available when the
API is connected.",
"scr.meta":
  "Thresholds are taken from the sensitivity-target operating point on validation data (WHO TPP reference: sensitivity > 90%,
specificity > 70%).",
"letter.title": "Tuberculosis Screening Referral Letter",
"letter.prototype": "SIGAP research prototype – not an official medical document.",
"letter.date": "Date",
"letter.facility": "Referring facility",
"letter.facility.ph": "Puskesmas / clinic: _____",
"letter.patient": "Patient",
"letter.agesex": "Age / sex",
"letter.complaint": "Complaint",
"letter.risk": "Estimated TB risk",
"letter.band": "Triage band",
"letter.taus": "Thresholds (uncertain / refer)",
"letter.request":
  "Please perform GeneXpert testing and a clinical evaluation according to the national TB program algorithm (TOSS-TB). This
screening result is a triage aid, not a diagnosis.",
"letter.sender": "Referring officer",
"letter.receiver": "Receiving facility",
"letter.dash": "-",
// patients
"pat.title": "Patient follow-up",
"pat.sub": "Patient records connect screening to treatment: daily check-ins update the risk and trigger escalation when it
worsens.",
"pat.demoName": "Demo case – Mrs. S",
"pat.demoCopySuffix": "(copy)",
"pat.empty": "No saved records yet – saving from the Screening page will show them here.",

```

```

"pat.insight.ok": "Adherence is steady – {taken} of {total} check-ins on time.",
"pat.insight.risk": "Risk is rising – missed doses: {missed}, symptom score {symptoms}/3.",
"pat.insight.low": "Not enough check-ins yet – add today's to see the trend.",
"pat.timeline.title": "Care timeline",
"pat.stage.screening": "Screening completed",
"pat.stage.referral": "Referral issued",
"pat.stage.treatment": "Treatment & adherence",
"pat.stage.followup": "Follow-up & cure check",
"pat.status.done": "Done",
"pat.status.active": "In progress",
"pat.status.next": "Upcoming",
"pat.stat.risk": "Current risk",
"pat.stat.checkins": "Check-ins",
"pat.stat.ontime": "On-time doses",
"pat.stat.danger": "Danger signs",
"pat.yes": "Yes",
"pat.no": "No",
"pat.checkin.title": "Today's check-in",
"pat.checkin.med": "Took medication as scheduled",
"pat.checkin.symptoms": "Symptoms (0-3)",
"pat.checkin.d1": "Blood in cough",
"pat.checkin.d2": "Shortness of breath",
"pat.checkin.d3": "High fever",
"pat.checkin.note": "Note (optional)",
"pat.checkin.save": "Save check-in",
"pat.history.title": "Check-in history",
"pat.history.med": "medication ✓",
"pat.history.missed": "missed dose",
"pat.history.symptoms": "symptoms {n}/3",
"pat.escalation":
  "Escalation: risk crossed the threshold. Visit or call the patient today; consider re-referral.",
"pat.window": "Last 14 check-ins",
"pat.legend.taken": "dose taken",
"pat.legend.missed": "missed",
"pat.note":
  "Methodological note: risk updates here use a simple demo rule (adherence -8%, symptoms +9%/point, danger signs +30%) to
demonstrate the loop – not a trained model. The next version reuses the same risk state as the screening engine.",
// clinicians
"cli.title": "Clinician summary",
"cli.sub": "One page for re-reading and follow-up: screening result, adherence and escalation flags.",
"cli.empty": "No saved cases yet. Save a result from the Screening page first.",
"cli.tile.checkins": "Check-ins logged",
"cli.tile.adherence": "Medication adherence",
"cli.tile.symptoms": "Latest symptoms",
"cli.tile.danger": "Danger signs",
"cli.alert":
  "Attention: the patient reported danger signs ({list}). Consider an immediate clinical review.",
"cli.print": "Print summary",
"cli.note": "Research prototype summary – not a medical record.",
"cli.screeningDate": "screening",
// about
"ab.title": "About SIGAP",
"ab.lede":
  "Indonesia has the world's second-highest tuberculosis burden: about 10% of global cases and an estimated 200,000 people each
year who transmit TB without ever being detected. The root cause is latency – the care loop moves slower than the disease.",
"ab.how.title": "How it works",
"ab.how.1":
  "Chest X-rays are scored by a self-supervised chest X-ray representation (RAD-DINO) with a linear probe, trained on audited
public data and evaluated on unseen hospitals.",
"ab.how.2":
  "Scores are calibrated (temperature scaling) and mapped to three bands: priority referral, uncertain (human re-read) and
negative – with thresholds from a sensitivity-target operating point.",
"ab.how.3":
  "The screening result becomes the starting risk state on a patient record; daily check-ins update it and escalate when the
patient worsens.",
"ab.rigor.title": "Rigor you can check",
"ab.rigor.1":
  "Leakage audit: perceptual hashing removed 338 internal duplicates and 19 training images that leaked into the external test
set before any training ran.",
"ab.rigor.2":
  "External evaluation across two unseen hospitals with bootstrap AUROC intervals, sensitivity at fixed specificity, calibration
(ECE) and age/sex subgroups.",
"ab.rigor.3":
  "Limitations are reported, not hidden: public radiographic labels, aggressive re-processing may evade de-duplication, and no
Indonesian data yet.",
"ab.links.title": "Code, data and live demo",
"ab.links.body":
  "Frozen split manifest, evaluation scripts and model artifacts are public. The live demo runs the real ensemble.",
"ab.disclaimer":
  "Honest statement: this is a research prototype built for a competition – not a medical device. Do not use it for clinical
decisions without validation and oversight by health professionals.",
// footer
"footer.text":
  "SIGAP is a research triage prototype – not a diagnostic device. Every result must be confirmed by a health professional
(GeneXpert). Disease-burden figures: WHO Global TB Report 2025 and Kemenkes 2024.",
"footer.repo": "Code, frozen splits and evaluation are open",
},
id: {
  "nav.home": "Beranda",
  "nav.screening": "Skrining",
  "nav.patients": "Pasien",
  "nav.clinicians": "Klinisi",

```

```

"nav.about": "Tentang",
"sfx.tagline": "Skrining TB",
"lang.switch": "Bahasa",
"landing.kicker": "UnivaBio 2026 · AI for Human Health",
"landing.title": "Skrining lebih cepat daripada penyebaran penyakitnya.",
"landing.sub":
  "Indonesia memikul 10% beban TB dunia, dan sekitar 200 ribu orang setiap tahun menularkan TB tanpa pernah terdeteksi. SIGAP
  memampatkan loop perawatan dengan satu state risiko terkalibrasi – dan berkata "ragu" saat memang harus ragu.",
"landing.cta.primary": "Buka live demo",
"landing.cta.secondary": "Lihat kode",
"landing.rail.0.tag": "Skrining",
"landing.rail.0.title": "Deteksi lebih dini",
"landing.rail.0.body":
  "Citra X-ray dada menjadi risiko terkalibrasi dengan pita ragu: saat keluarga model tidak sepakat, sistem meminta pembacaan
  manusia alih-alih memaksakan keputusan.",
"landing.rail.1.tag": "Rujuk",
"landing.rail.1.title": "Bertindak tanpa menunggu",
"landing.rail.1.body":
  "Skor dipetakan ke band tindakan dan surat rujukan siap cetak untuk konfirmasi GeneXpert, selaras alur TOSS-TB.",
"landing.rail.2.tag": "Dampingi",
"landing.rail.2.title": "Tuntaskan sampai sembuh",
"landing.rail.2.body":
  "State risiko yang sama mengikuti pasien sampai rumah: check-in kepatuhan memperbarui skor dan memicu eskalasi bila memburuk.",
"landing.mockup.caption": "Hasil ilustratif – keluaran model nyata pada kasus korpus publik.",
"landing.stats.0": "AUROC di rumah sakit asing",
"landing.stats.1": "citra training dibuang setelah audit kebocoran",
"landing.stats.2": "sensitivitas pada spesifisitas 90% (eksternal)",
"landing.evidence.title": "Bukti, dilaporkan jujur",
"landing.evidence.open.title": "Terukur & terbuka",
"landing.evidence.open.1":
  "Validasi eksternal di dua rumah sakit asing (800 citra), dengan interval bootstrap, kalibrasi (ECE), dan subgroup usia/jenis
  kelamin.",
"landing.evidence.open.2":
  "Audit data sebelum training: 338 duplikat internal dan 19 citra training yang bocor ke test set eksternal dibuang. Manifest
  split beku dipublikasikan.",
"landing.evidence.open.3":
  "Studi representasi tiga keluarga model; pemenangnya (probe self-supervised RAD-DINO) terbuka di repo dan sebagai artefak
  unduhan.",
"landing.evidence.todo.title": "Belum beres – jujur",
"landing.evidence.todo.1":
  "Belum ada validasi prospektif di Indonesia; ini prototipe riset, bukan alat medis.",
"landing.evidence.todo.2":
  "Ambang triase berasal dari data publik, bukan populasi puskesmas; kalibrasi per-site direncanakan.",
"landing.evidence.todo.3": "Analisis suara batuk dan integrasi SITB belum dikerjakan.",
"landing.stack.title": "Di balik layar",
"landing.stack.body":
  "RAD-DINO (self-supervised X-ray dada) + probe linear, temperature scaling, deferral dari disagreement lintas-keluarga.
  Dilayani FastAPI dan UI Next.js dwibahasa; dilatih di GPU Kaggle via API.",
"landing.try.title": "Coba sekarang",
"landing.try.full": "Aplikasi penuh (API model live)",
"landing.try.static": "Cermin statis (API live yang sama)",
"landing.try.note":
  "Kedua tautan menjalankan ensemble asli: unggah X-ray dada dan dapatkan band triase, peta perhatian, dan surat rujukan.",
"scr.title": "Skrining X-ray dada",
"scr.sub": "Unggah citra X-ray dada, jalankan analisis, lalu ikuti band tindakan. Kurang dari tiga menit per pasien.",
"scr.gallery.title": "Contoh arsip – tanpa server",
"scr.gallery.desc":
  "Empat kasus nyata dari korpus eksternal NLM dengan keluaran ensemble final yang sama, dihitung offline – coba alur lengkap
  tanpa koneksi API.",
"scr.resultPlaceholder": "Hasil triase akan muncul di sini setelah analisis.",
"scr.case.a.title": "Kasus A – TB aktif",
"scr.case.a.desc": "Ensemble sepakat: prioritas rujukan hari ini.",
"scr.case.b.title": "Kasus B – normal dari RS asing",
"scr.case.b.desc": "Keluarga model tidak sepakat → sistem menahan keputusan.",
"scr.case.c.title": "Kasus C – TB, bukti tidak konklusif",
"scr.case.c.desc": "Bahkan untuk TB, ketidakpastian tinggi → pembacaan ulang.",
"scr.case.d.title": "Kasus D – normal, mendekati negatif",
"scr.case.d.desc": "Sinyal rendah, tetapi ketidakpastian menjaga band ragu.",
"scr.step1": "Citra X-ray",
"scr.drop.title": "Pilih foto X-ray",
"scr.drop.hint": "PNG atau JPG, tampak depan (PA/AP). Tarik file ke sini atau ketuk untuk memilih.",
"scr.change": "Ganti citra",
"scr.guide.title": "Sebelum menganalisis",
"scr.guide.1": "Tampak depan dada (PA/AP)",
"scr.guide.2": "Seluruh lapang paru terlihat, tidak terpotong",
"scr.guide.3": "Gunakan file asli – bukan foto layar",
"scr.guide.4": "Hindari identitas pasien bila memungkinkan",
"scr.step2": "Data pasien (opsional)",
"scr.field.name": "Nama atau kode pasien",
"scr.field.name.ph": "mis. S-014",
"scr.field.age": "Usia",
"scr.field.sex": "Jenis kelamin",
"scr.sex.m": "Laki-laki",
"scr.sex.f": "Perempuan",
"scr.field.complaint": "Keluhan utama",
"scr.field.complaint.ph": "mis. batuk 3 minggu, keringat malam",
"scr.demo": "Mode peragaan (hasil contoh – bukan dari citra Anda)",
"scr.analyze": "Analisis citra",
"scr.analyzing": "Menganalisis...",
"scr.error":
  "Server model tidak tersedia. Coba contoh arsip di atas (keluaran ensemble final – tanpa server), atau nyalakan mode
  peragaan.",

```

```

"scr.result": "Hasil analisis",
"scr.band.refer.label": "Prioritas rujukan",
"scr.band.refer.title": "Rujuk – prioritas hari ini",
"scr.band.refer.action":
  "Kirim pasien untuk pemeriksaan GeneXpert di fasilitas terdekat hari ini. Catat sebagai suspek TB pada buku register.",
"scr.band.defer.label": "Perlu pembacaan ulang",
"scr.band.defer.title": "Ragu – perlu pembacaan ulang",
"scr.band.defer.action":
  "Jangan diputuskan sendiri. Minta pembacaan oleh petugas/radiolog, atau ulangi skrining dengan citra yang lebih baik.",
"scr.band.neg.label": "Negatif skrining",
"scr.band.neg.title": "Negatif skrining",
"scr.band.neg.action":
  "Edukasi gejala TB. Minta pasien kembali jika gejala berlanjut lebih dari 2 minggu atau memburuk.",
"scr.scale.ptb": "Estimasi p(TB)",
"scr.scale.unc": "Ketidakpastian (disagreement)",
"scr.scale.low": "Risiko rendah",
"scr.scale.high": "Risiko tinggi",
"scr.scale.tauLow": "ragu",
"scr.scale.tauHigh": "rujuk",
"scr.card.observed.title": "Yang dilihat model",
"scr.card.observed.body":
  "Area yang paling memengaruhi skor disorot pada peta perhatian – untuk diverifikasi manusia, bukan dikutip sebagai bukti
lesi.",
"scr.card.notshown.title": "Yang tidak ditunjukkan",
"scr.card.notshown.body":
  "Peta perhatian bukan bukti lesi. Model belum bisa membedakan bekas TB (scar) dari TB aktif – karena itu pita ragu ada dan
pembacaan manusia tetap wajib.",
"scr.card.unc.title": "Tentang ketidakpastian",
"scr.card.unc.body":
  "Keluarga model tidak sepakat pada citra ini (u = {u}). Skor dan band dapat berubah dengan X-ray yang lebih baik, jadi gunakan
hanya sebagai sinyal skrining.",
"scr.card.unc.low":
  "Keluarga model umumnya sepakat pada citra ini (u = {u}), yang menambah keyakinan pada band ini – dengan batasan di bawah.",
"scr.overlay.heat": "Peta perhatian",
"scr.overlay.orig": "Citra asli",
"scr.trust": "Pemrosesan berjalan di sesi browser ini atau server klinikmu. Prototipe riset – bukan diagnosis.",
"scr.remove": "Hapus citra",
"scr.action.print": "Cetak surat rujukan",
"scr.action.save": "Simpan ke kartu pasien",
"scr.action.saved": "Tersimpan di kartu pasien",
"scr.action.new": "Skrining baru",
"scr.disclaimer":
  "Alat triase riset – bukan diagnosis. Setiap hasil wajib dikonfirmasi tenaga kesehatan (GeneXpert).",
"scr.archiveNote":
  "Keluaran arsip – dihitung offline dengan model final pada citra contoh korpus eksternal (NLM). Mode server langsung tersedia
saat API terhubung.",
"scr.meta":
  "Ambang diambil dari titik sensitivitas-target pada data validasi (acuan TPP WHO: sensitivitas > 90%, spesifisitas > 70%).",
"letter.title": "Surat Rujukan Skrining Tuberkulosis",
"letter.prototype": "Prototipe riset SIGAP – bukan dokumen medis resmi.",
"letter.date": "Tanggal",
"letter.facility": "Fasilitas pengirim",
"letter.facility.ph": "Puskesmas / posyandu: _____",
"letter.patient": "Pasien",
"letter.agesex": "Usia / jenis kelamin",
"letter.complaint": "Keluhan",
"letter.risk": "Estimasi risiko TB",
"letter.band": "Band triase",
"letter.taus": "Ambang (ragu / rujuk)",
"letter.request":
  "Mohon dilakukan pemeriksaan GeneXpert dan evaluasi klinis sesuai algoritma program TB nasional (TOSS-TB). Hasil skrining ini
adalah alat bantu triase, bukan diagnosis.",
"letter.sender": "Petugas pengirim",
"letter.receiver": "Penerima rujukan",
"letter.dash": "-",
"pat.title": "Pendampingan pasien",
"pat.sub": "Kartu pasien menyambung skrining ke pengobatan: check-in harian memperbarui risiko dan memicu eskalasi bila
memburuk.",
"pat.demoName": "Contoh peragaan – Ibu S",
"pat.demoCopySuffix": "(salinan)",
"pat.empty": "Belum ada kartu tersimpan – menyimpan dari halaman Skrining akan muncul di sini.",
"pat.insight.ok": "Kepatuhan stabil – {taken} dari {total} check-in tepat waktu.",
"pat.insight.risk": "Risiko menanjak – dosis terlewat: {missed}, skor gejala {symptoms}/3.",
"pat.insight.low": "Check-in masih sedikit – tambahkan hari ini untuk melihat tren.",
"pat.timeline.title": "Perjalanan perawatan",
"pat.stage.screening": "Skrining selesai",
"pat.stage.referral": "Rujukan diterbitkan",
"pat.stage.treatment": "Pengobatan & kepatuhan",
"pat.stage.followup": "Tindak lanjut & evaluasi sembuh",
"pat.status.done": "Selesai",
"pat.status.active": "Berjalan",
"pat.status.next": "Menyusul",
"pat.stat.risk": "Risiko saat ini",
"pat.stat.checkins": "Check-in",
"pat.stat.ontime": "Dosis tepat waktu",
"pat.stat.danger": "Tanda bahaya",
"pat.yes": "Ada",
"pat.no": "Tidak",
"pat.checkin.title": "Check-in hari ini",
"pat.checkin.med": "Minum obat sesuai jadwal",
"pat.checkin.symptoms": "Gejala (0-3)",
"pat.checkin.d1": "Batuk darah",

```

```

    "pat.checkin.d2": "Sesak napas",
    "pat.checkin.d3": "Demam tinggi",
    "pat.checkin.note": "Catatan (opsional)",
    "pat.checkin.save": "Simpan check-in",
    "pat.history.title": "Riwayat check-in",
    "pat.history.med": "obat ✓",
    "pat.history.missed": "obat terlewat",
    "pat.history.symptoms": "gejala {n}/3",
    "pat.escalation":
      "Eskalasi: risiko menembus ambang. Kunjungi atau telepon pasien hari ini; pertimbangkan rujukan ulang.",
    "pat.window": "14 check-in terakhir",
    "pat.legend.taken": "dosis diminum",
    "pat.legend.missed": "terlewat",
    "pat.note":
      "Catatan metodologis: pembaruan risiko di halaman ini memakai aturan peragaan sederhana (kepatuhan -8%, gejala +9%/poin, tanda bahaya +30%) untuk mendemonstrasikan loop – bukan model terlatih. Versi berikutnya memakai state risiko yang sama dengan mesin skrining.",
    "cli.title": "Ringkasan klinisi",
    "cli.sub": "Satu halaman untuk pembacaan ulang dan tindak lanjut: hasil skrining, kepatuhan, dan tanda eskalasi.",
    "cli.empty": "Belum ada kasus tersimpan. Simpan hasil dari halaman Skrining terlebih dahulu.",
    "cli.tile.checkins": "Check-in tercatat",
    "cli.tile.adherence": "Kepatuhan obat",
    "cli.tile.symptoms": "Gejala terakhir",
    "cli.tile.danger": "Tanda bahaya",
    "cli.alert":
      "Perhatian: pasien melaporkan tanda bahaya ({list}). Pertimbangkan evaluasi klinis segera.",
    "cli.print": "Cetak ringkasan",
    "cli.note": "Ringkasan prototipe riset – bukan rekam medis.",
    "cli.screeningDate": "skrining",
    "ab.title": "Tentang SIGAP",
    "ab.lede":
      "Indonesia adalah negara dengan beban tuberkulosis terbesar kedua di dunia: sekitar 10% kasus global dan estimasi 200 ribu orang per tahun yang menularkan TB tanpa pernah terdeteksi. Akarnya adalah latensi – loop perawatan bergerak lebih lambat daripada penyakitnya.",
    "ab.how.title": "Cara kerja",
    "ab.how.1":
      "X-ray dada dinilai oleh representasi self-supervised X-ray dada (RAD-DINO) dengan probe linear, dilatih pada data publik yang diaudit dan dievaluasi di rumah sakit yang belum pernah dilihat.",
    "ab.how.2":
      "Skor dikalibrasi (temperature scaling) dan dipetakan ke tiga band: prioritas rujukan, ragu (pembacaan manusia), dan negatif – dengan ambang dari titik sensitivitas-target.",
    "ab.how.3":
      "Hasil skrining menjadi state risiko awal pada kartu pasien; check-in harian memperbaruinya dan memicu eskalasi bila pasien memburuk.",
    "ab.rigor.title": "Rigor yang bisa diperiksa",
    "ab.rigor.1":
      "Audit kebocoran: perceptual hashing membuang 338 duplikat internal dan 19 citra training yang bocor ke test set eksternal sebelum training dijalankan.",
    "ab.rigor.2":
      "Evaluasi eksternal di dua rumah sakit asing dengan interval bootstrap AUROC, sensitivitas pada spesifisitas tetap, kalibrasi (ECE), dan subgrup usia/jenis kelamin.",
    "ab.rigor.3":
      "Keterbatasan dilaporkan, bukan disembunyikan: label radiografis publik, reprocessing agresif dapat lolos dedup, dan belum ada data Indonesia.",
    "ab.links.title": "Kode, data, dan demo live",
    "ab.links.body":
      "Manifest split beku, skrip evaluasi, dan artefak model terbuka untuk publik. Demo live menjalankan ensemble asli.",
    "ab.disclaimer":
      "Pernyataan jujur: ini prototipe riset untuk kompetisi – bukan alat medis. Jangan dipakai untuk keputusan klinis tanpa validasi dan pengawasan tenaga kesehatan.",
    "footer.text":
      "SIGAP adalah prototipe riset triase skrining – bukan alat diagnosis. Setiap hasil wajib dikonfirmasi tenaga kesehatan (GeneXpert). Angka beban penyakit: WHO Global TB Report 2025 dan Kemenkes 2024.",
    "footer.repo": "Kode, split data, dan evaluasi terbuka",
  },
};

const LangContext = createContext<{
  lang: Lang;
  setLang: (l: Lang) => void;
  t: (key: string, vars?: Record<string, string | number>) => string;
}>({ lang: "en", setLang: () => {}, t: (k) => k });

export function LangProvider({ children }: { children: ReactNode }) {
  const [lang, setLangState] = useState<Lang>("en");

  useEffect(() => {
    const stored = (typeof window !== "undefined" && window.localStorage.getItem("sigap.lang")) as Lang | null;
    if (stored === "id" || stored === "en") setLangState(stored);
  }, []);

  useEffect(() => {
    document.documentElement.lang = lang;
    try {
      window.localStorage.setItem("sigap.lang", lang);
    } catch {
      /* private mode */
    }
  }, [lang]);

  const t = (key: string, vars?: Record<string, string | number>) => {
    let s = DICT[lang][key] ?? DICT.en[key] ?? key;

```

```
    if (vars) for (const [k, v] of Object.entries(vars)) s = s.replaceAll(`${k}`, String(v));
    return s;
  };

  return <LangContext.Provider value={{ lang, setLang: setLangState, t }}>{children}</LangContext.Provider>;
}

export function useLang() {
  return useContext(LangContext);
}
```

```

@import "tailwindcss";

@theme {
  --color-paper: #f3f5f6;
  --color-mist: #e8ecee;
  --color-ink: #12161b;
  --color-film: #0a0e13;
  --color-bone: #e9e7e0;
  --color-refer: #c0392b;
  --color-defer: #b7791f;
  --color-clear: #2e7d5b;
  --font-sans: var(--font-jakarta), system-ui, -apple-system, "Segoe UI", sans-serif;
  --font-display: var(--font-newsreader), Georgia, "Times New Roman", serif;
  --radius-control: 10px;
  --radius-tile: 12px;
  --radius-panel: 16px;
  --shadow-tile: 0 1px 2px rgba(16, 20, 26, 0.05);
  --shadow-float: 0 1px 2px rgba(16, 20, 26, 0.06), 0 16px 40px -20px rgba(16, 20, 26, 0.35);
}

html {
  -webkit-text-size-adjust: 100%;
}

body {
  background: var(--color-paper);
  color: var(--color-ink);
  font-family: var(--font-sans);
}

.font-display {
  font-family: var(--font-display);
  letter-spacing: -0.015em;
}

::selection {
  background: var(--color-ink);
  color: var(--color-paper);
}

:focus-visible {
  outline: 2px solid var(--color-defer);
  outline-offset: 2px;
  border-radius: 2px;
}

@keyframes rise {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: none;
  }
}

@keyframes grow-x {
  from {
    transform: scaleX(0);
  }
  to {
    transform: scaleX(1);
  }
}

.reveal {
  animation: rise 0.4s cubic-bezier(0.2, 0.7, 0.2, 1) both;
}

.reveal-slow {
  animation: rise 0.6s cubic-bezier(0.2, 0.7, 0.2, 1) both;
}

.grow-x {
  transform-origin: left center;
  animation: grow-x 0.7s cubic-bezier(0.2, 0.7, 0.2, 1) both;
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}

```

```
@media print {  
  .no-print {  
    display: none !important;  
  }  
  body {  
    background: #fff;  
  }  
}
```

web/package-lock.json

```
{
  "name": "web",
  "version": "0.1.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "web",
      "version": "0.1.0",
      "dependencies": {
        "next": "16.3.8",
        "react": "19.2.8",
        "react-dom": "19.2.8"
      },
      "devDependencies": {
        "@tailwindcss/postcss": "^4",
        "@types/node": "^20",
        "@types/react": "^19",
        "@types/react-dom": "^19",
        "tailwindcss": "^4",
        "typescript": "^5"
      }
    },
    "node_modules/@alloc/quick-lru": {
      "version": "5.3.0",
      "resolved": "https://registry.npmjs.org/@alloc/quick-lru/-/quick-lru-5.3.0.tgz",
      "integrity": "sha512-ur40pW/OiF4mF41D8Y1F3Wf9l6hyOSFNUHfqz7gXeV64G+zvF78oQvz7oqEnuTx6v1H9q+8lTqupddw67pQ==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/@emnapi/runtime": {
      "version": "1.11.3",
      "resolved": "https://registry.npmjs.org/@emnapi/runtime/-/runtime-1.11.3.tgz",
      "integrity": "sha512-4h+IybnSaJJdUMqJGJZotq407vU4h4oXUg8T4XUwQ8uFgX08e0e8q9E8K8UwXUwXUwXUwXUwXUwXUwXUw==",
      "license": "MIT",
      "optional": true,
      "dependencies": {
        "tslib": "^2.4.0"
      }
    },
    "node_modules/@img/colour": {
      "version": "1.1.0",
      "resolved": "https://registry.npmjs.org/@img/colour/-/colour-1.1.0.tgz",
      "integrity": "sha512-Td7q7T00WwQ0Z1Wwv1z0UW5e303ZUWwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUwXUw==",
      "license": "MIT",
      "optional": true,
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@img/sharp-darwin-arm64": {
      "version": "0.35.5",
      "resolved": "https://registry.npmjs.org/@img/sharp-darwin-arm64/-/sharp-darwin-arm64-0.35.5.tgz",
      "integrity": "sha512-QRUIFQ0WxvdWYqqG/WtI3iupfD5rBzmCHXSDPsY91sAtVtTo7Q4cb6z0ccZ3gqEqkr0f1As1ehLqmEpDsRf+lg==",
      "cpu": [
        "arm64"
      ],
      "license": "Apache-2.0",
      "optional": true,
      "os": [
        "darwin"
      ],
      "engines": {
        "node": ">=20.9.0"
      },
      "funding": {
        "url": "https://opencollective.com/libvips"
      },
      "optionalDependencies": {
        "@img/sharp-libvips-darwin-arm64": "1.3.4"
      }
    },
    "node_modules/@img/sharp-darwin-x64": {
      "version": "0.35.5",
      "resolved": "https://registry.npmjs.org/@img/sharp-darwin-x64/-/sharp-darwin-x64-0.35.5.tgz",
      "integrity": "sha512-+BR255RhDlpygUp0c/Jdt1nT6DQ3XG/ERo5wbcd0f5Q320dKtPCKPLR1LJs9VGXRaMa811uUa0tkCN0XiAxZUw==",
      "cpu": [
        "x64"
      ],
      "license": "Apache-2.0",
      "optional": true,
      "os": [

```

```

    "darwin"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  },
  "optionalDependencies": {
    "@img/sharp-libvips-darwin-x64": "1.3.4"
  }
},
"node_modules/@img/sharp-freebsd-wasm32": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-freebsd-wasm32/-/sharp-freebsd-wasm32-0.35.5.tgz",
  "integrity": "sha512-Y/z91nEZ4uIBX5X3nfTovjU9lHNKFYbL2lpHCLVnmXQK03VIZvXBBt0KxbPGp2SdGSF+2mQU4e+hQaW0t86iAw=",
  "license": "Apache-2.0",
  "optional": true,
  "os": [
    "freebsd"
  ],
  "dependencies": {
    "@img/sharp-wasm32": "0.35.5"
  },
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-libvips-darwin-arm64": {
  "version": "1.3.4",
  "resolved": "https://registry.npmjs.org/@img/sharp-libvips-darwin-arm64/-/sharp-libvips-darwin-arm64-1.3.4.tgz",
  "integrity": "sha512-5R89nBYiRdUlsWJxPh0+GVtaXzXSxKnRu/xqMn3KTA3L9EB90y/P+Nn2f2v1hPuUdy/Zusb2DarbyTpGCfEduw=",
  "cpu": [
    "arm64"
  ],
  "license": "LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "darwin"
  ],
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-libvips-darwin-x64": {
  "version": "1.3.4",
  "resolved": "https://registry.npmjs.org/@img/sharp-libvips-darwin-x64/-/sharp-libvips-darwin-x64-1.3.4.tgz",
  "integrity": "sha512-iR20KH80yi0U+dUplyh3/xdpFvps6YkCwsXenIJxqR1v9o+xtKTGbS9H7cps+2Vxjc8B1j96p75NmTGjIhtpQ=",
  "cpu": [
    "x64"
  ],
  "license": "LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "darwin"
  ],
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-libvips-linux-arm": {
  "version": "1.3.4",
  "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-arm/-/sharp-libvips-linux-arm-1.3.4.tgz",
  "integrity": "sha512-LmRtT0HuvM2+w102Db37d5MiZhB0FvSunciw48Yjd0kZz9KAiRbm8ujeMOA1INqmei5NapFxEK1D1ZSidmw=",
  "cpu": [
    "arm"
  ],
  "license": "LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "linux"
  ],
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-libvips-linux-arm64": {
  "version": "1.3.4",
  "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-arm64/-/sharp-libvips-linux-arm64-1.3.4.tgz",
  "integrity": "sha512-Y3dgX/61E2QhQb+Gxy0WZxfG9MEu/JBjamZpS2Ik1P7xIQoKN4hzAm7KcMVGtaVdt3neE90KBC7vAfonA/Lr1A=",
  "cpu": [
    "arm64"
  ],
  "license": "LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "linux"
  ],
  "funding": {

```

```

    "url": "https://opencollective.com/libvips"
  },
  "node_modules/@img/sharp-libvips-linux-ppc64": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-ppc64/-/sharp-libvips-linux-ppc64-1.3.4.tgz",
    "integrity": "sha512-Le6boB8Tai0Nis+gIxIpKx68UDVVIQdR8Tin5Yf1z2LJJQLDJvCDRqRu+jC2qCoD+eIomonm0wB4smBRxfVpYQ==",
    "cpu": [
      "ppc64"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,
    "os": [
      "linux"
    ],
    "funding": {
      "url": "https://opencollective.com/libvips"
    }
  },
  "node_modules/@img/sharp-libvips-linux-riscv64": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-riscv64/-/sharp-libvips-linux-riscv64-1.3.4.tgz",
    "integrity": "sha512-aHkkIEHPRdQEgJN20MLmGtxYD9R2wQr3Cwpddnu5+YKmt6Uzax7S9h5gpZTo8wyrGuZS1fQ630evL5mTy0C7Q==",
    "cpu": [
      "riscv64"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,
    "os": [
      "linux"
    ],
    "funding": {
      "url": "https://opencollective.com/libvips"
    }
  },
  "node_modules/@img/sharp-libvips-linux-s390x": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-s390x/-/sharp-libvips-linux-s390x-1.3.4.tgz",
    "integrity": "sha512-1aB6MikESDU07Yg+Mi95bFBb9Gs0bURuhnOv30qknjGe9sZrG8tCe9q0xSIGrtLvgw0gKnFWcK4b1SgQ0uQ==",
    "cpu": [
      "s390x"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,
    "os": [
      "linux"
    ],
    "funding": {
      "url": "https://opencollective.com/libvips"
    }
  },
  "node_modules/@img/sharp-libvips-linux-x64": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linux-x64/-/sharp-libvips-linux-x64-1.3.4.tgz",
    "integrity": "sha512-GJ//SSXbnwSDes02umB3nDJLFCQzw8a18V8fyhqr6tV515tOEMdImjxj1AoafMRz56F3PHgftnj1QEKsU1zkw==",
    "cpu": [
      "x64"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,
    "os": [
      "linux"
    ],
    "funding": {
      "url": "https://opencollective.com/libvips"
    }
  },
  "node_modules/@img/sharp-libvips-linuxmusl-arm64": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linuxmusl-arm64/-/sharp-libvips-linuxmusl-arm64-1.3.4.tgz",
    "integrity": "sha512-hvulFwtjUcagsis6BBxHwGFwW0NZjgYmULGVrZcyfNbjA8hKILbRxGg15/7w5HDyXHXUos/j6baAWqnCyQ2DWA==",
    "cpu": [
      "arm64"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,
    "os": [
      "linux"
    ],
    "funding": {
      "url": "https://opencollective.com/libvips"
    }
  },
  "node_modules/@img/sharp-libvips-linuxmusl-x64": {
    "version": "1.3.4",
    "resolved": "https://registry.npmjs.org/@img/sharp-libvips-linuxmusl-x64/-/sharp-libvips-linuxmusl-x64-1.3.4.tgz",
    "integrity": "sha512-6zXKeE/p39I1AmA3cJG35eyBGNqNddLnUXjhwBnsGjFPWqf5VKkDBEqaEkPD0TEtkxwi2vv8Tcr2mDyP4So7Fg==",
    "cpu": [
      "x64"
    ],
    "license": "LGPL-3.0-or-later",
    "optional": true,

```

```

"os": [
  "linux"
],
"funding": {
  "url": "https://opencollective.com/libvips"
}
},
"node_modules/@img/sharp-linux-arm": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-linux-arm/-/sharp-linux-arm-0.35.5.tgz",
  "integrity": "sha512-LEaXK2WdXVK5ykcw0buWyPMsMLL2vpHLD6yzNSW+JGEL3BZPA4tpKN6iaMc4AxTTAoaX/sU1rOL511cIz48ZQ==",
  "cpu": [
    "arm"
  ],
  "license": "Apache-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  },
  "optionalDependencies": {
    "@img/sharp-libvips-linux-arm": "1.3.4"
  }
},
"node_modules/@img/sharp-linux-arm64": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-linux-arm64/-/sharp-linux-arm64-0.35.5.tgz",
  "integrity": "sha512-LYVx5JT50M2CBzmXreh+n164/3H6Xb09iSLknqH47z2T2DFFxDeFLP5y4dJwe6H7uGQlHPyEEtIqyo3DYSRwdQ==",
  "cpu": [
    "arm64"
  ],
  "license": "Apache-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  },
  "optionalDependencies": {
    "@img/sharp-libvips-linux-arm64": "1.3.4"
  }
},
"node_modules/@img/sharp-linux-ppc64": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-linux-ppc64/-/sharp-linux-ppc64-0.35.5.tgz",
  "integrity": "sha512-QVxAAq8evVRI9ia2vqgwzmWucn5Dfv+JdWzj75pD8omHLPSP7f8p2008jxzjCcuCEQE0tY0ZUmX1hkiZ0kdevA==",
  "cpu": [
    "ppc64"
  ],
  "license": "Apache-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  },
  "optionalDependencies": {
    "@img/sharp-libvips-linux-ppc64": "1.3.4"
  }
},
"node_modules/@img/sharp-linux-riscv64": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-linux-riscv64/-/sharp-linux-riscv64-0.35.5.tgz",
  "integrity": "sha512-LtdreXguaavK0DPIfzJ4kffx7UNT1omwtK0rch4EBbbSTXPnxWmYSayXDLJw0fJzQ97kHt1gL/yh4tvU+nCyRQ==",
  "cpu": [
    "riscv64"
  ],
  "license": "Apache-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  },
  "optionalDependencies": {

```

```

    "@img/sharp-libvips-linux-riscv64": "1.3.4"
  },
  "node_modules/@img/sharp-linux-s390x": {
    "version": "0.35.5",
    "resolved": "https://registry.npmjs.org/@img/sharp-linux-s390x/-/sharp-linux-s390x-0.35.5.tgz",
    "integrity": "sha512-UZasT0FiYzotTsG0Cu42BfUzP6Tu6Do/947iRm1RsLKvlllxwGcn4RN27LibGWceix4Y+Pmw3jsnTcCQIgWjqA==",
    "cpu": [
      "s390x"
    ],
    "license": "Apache-2.0",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=20.9.0"
    },
    "funding": {
      "url": "https://opencollective.com/libvips"
    },
    "optionalDependencies": {
      "@img/sharp-libvips-linux-s390x": "1.3.4"
    }
  },
  "node_modules/@img/sharp-linux-x64": {
    "version": "0.35.5",
    "resolved": "https://registry.npmjs.org/@img/sharp-linux-x64/-/sharp-linux-x64-0.35.5.tgz",
    "integrity": "sha512-SxFtLTJInHAA9Q836kux2vZNe0BQEx658qvbb0ZScr0wIARym3IcGmW7KpVD5sbVg00jy+udFQdayYIZyoNog==",
    "cpu": [
      "x64"
    ],
    "license": "Apache-2.0",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=20.9.0"
    },
    "funding": {
      "url": "https://opencollective.com/libvips"
    },
    "optionalDependencies": {
      "@img/sharp-libvips-linux-x64": "1.3.4"
    }
  },
  "node_modules/@img/sharp-linuxmusl-arm64": {
    "version": "0.35.5",
    "resolved": "https://registry.npmjs.org/@img/sharp-linuxmusl-arm64/-/sharp-linuxmusl-arm64-0.35.5.tgz",
    "integrity": "sha512-9HbMcImI1zlnkFRs3z9/eBtDjF0sGlrX1z6b1qwmfiFY5E1DLh4BC0LPBdVp7z1DXFiK1Icznf+ZlsuZzLxQqg==",
    "cpu": [
      "arm64"
    ],
    "license": "Apache-2.0",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=20.9.0"
    },
    "funding": {
      "url": "https://opencollective.com/libvips"
    },
    "optionalDependencies": {
      "@img/sharp-libvips-linuxmusl-arm64": "1.3.4"
    }
  },
  "node_modules/@img/sharp-linuxmusl-x64": {
    "version": "0.35.5",
    "resolved": "https://registry.npmjs.org/@img/sharp-linuxmusl-x64/-/sharp-linuxmusl-x64-0.35.5.tgz",
    "integrity": "sha512-4K0phqB035HrVdqLZfCgMzzERrQkkzOwRh140AkR01YCldbaFjySXMaK534Mo0V+Lndn1Jk+sbUyLeU0ULyD1A==",
    "cpu": [
      "x64"
    ],
    "license": "Apache-2.0",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=20.9.0"
    },
    "funding": {
      "url": "https://opencollective.com/libvips"
    },
    "optionalDependencies": {
      "@img/sharp-libvips-linuxmusl-x64": "1.3.4"
    }
  },
  "node_modules/@img/sharp-wasm32": {

```

```

"version": "0.35.5",
"resolved": "https://registry.npmjs.org/@img/sharp-wasm32/-/sharp-wasm32-0.35.5.tgz",
"integrity": "sha512-Ptsga1su4tQx+LLF1ECS9U6nz5kmrXKo6XVbtR48Ke3ZRxxgaWBu7IDtEe1quo8hiupwm6WFqXV1XaSf7IINGQ==",
"license": "Apache-2.0 AND LGPL-3.0-or-later AND MIT",
"optional": true,
"dependencies": {
  "@emnapi/runtime": "^1.11.3"
},
"engines": {
  "node": ">=20.9.0"
},
"funding": {
  "url": "https://opencollective.com/libvips"
}
},
"node_modules/@img/sharp-webcontainers-wasm32": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-webcontainers-wasm32/-/sharp-webcontainers-wasm32-0.35.5.tgz",
  "integrity": "sha512-hfhF/FmoQyTUKA0bIKF0tw536BQSeBMe6BF6QyWlRpxT754+TFLaZ7sKKTfvvM0yJgKgAYTwnFCIZ/GuDw5SUA==",
  "cpu": [
    "wasm32"
  ],
  "license": "Apache-2.0",
  "optional": true,
  "dependencies": {
    "@img/sharp-wasm32": "0.35.5"
  },
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-win32-arm64": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-win32-arm64/-/sharp-win32-arm64-0.35.5.tgz",
  "integrity": "sha512-X4t7g+7ZA5DKb1CBEXGjUqqemj4vczING/5viFwAL8h4N3qYeyjwdCvRLHi4EdOUI+2Z7UFlp1VM+p/AuEtm6Q==",
  "cpu": [
    "arm64"
  ],
  "license": "Apache-2.0 AND LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-win32-ia32": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-win32-ia32/-/sharp-win32-ia32-0.35.5.tgz",
  "integrity": "sha512-5Zm82LoBc43nhwNybZ1G7Y1K0//Zhsn306fQ129Z0uStHLGT03BWL83q3cnX0poxSAMuYL10n/BHBxkBeKr6A==",
  "cpu": [
    "ia32"
  ],
  "license": "Apache-2.0 AND LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@img/sharp-win32-x64": {
  "version": "0.35.5",
  "resolved": "https://registry.npmjs.org/@img/sharp-win32-x64/-/sharp-win32-x64-0.35.5.tgz",
  "integrity": "sha512-x76eH0vEiHlcmQu8Y8IenntaACtdpT6W0wmXtWnKcnKI7ME5DdgqhAD6SEW0E11v2zDvkZDhFA9KnURwpfqg==",
  "cpu": [
    "x64"
  ],
  "license": "Apache-2.0 AND LGPL-3.0-or-later",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">=20.9.0"
  },
  "funding": {
    "url": "https://opencollective.com/libvips"
  }
},
"node_modules/@jridgewell/gen-mapping": {

```

```

"version": "0.3.13",
"resolved": "https://registry.npmjs.org/@jridgewell/gen-mapping/-/gen-mapping-0.3.13.tgz",
"integrity": "sha512-2kkt77niJ6MgEPxF0bYdQ6etZaA+fQvDcLkckhyIyIQ0zaokjBBjSj63/aLvjYE3qhRt5dvM+uUyfCg6UKCBbA==",
"dev": true,
"license": "MIT",
"dependencies": {
  "@jridgewell/sourcemap-codec": "^1.5.0",
  "@jridgewell/trace-mapping": "^0.3.24"
}
},
"node_modules/@jridgewell/remapping": {
"version": "2.3.5",
"resolved": "https://registry.npmjs.org/@jridgewell/remapping/-/remapping-2.3.5.tgz",
"integrity": "sha512-LI9u/1aYG4Ds1TDKSJW2YPrI1cVY0wi2fUC6xB43lueCjgxV41ff0CZCtYFiH6TN0X+ttQKXx97T4IKHbhyHEQ==",
"dev": true,
"license": "MIT",
"dependencies": {
  "@jridgewell/gen-mapping": "^0.3.5",
  "@jridgewell/trace-mapping": "^0.3.24"
}
},
"node_modules/@jridgewell/resolve-uri": {
"version": "3.1.2",
"resolved": "https://registry.npmjs.org/@jridgewell/resolve-uri/-/resolve-uri-3.1.2.tgz",
"integrity": "sha512-bRISgCIjP20/tbWSPwMEi54QVPRExkuD91JL+UIxUKtwVJA8ww1Trb1jMs1RFxo1CBTNZ/5hpC9QvmKwdopKw==",
"dev": true,
"license": "MIT",
"engines": {
  "node": ">=6.0.0"
}
},
"node_modules/@jridgewell/sourcemap-codec": {
"version": "1.6.0",
"resolved": "https://registry.npmjs.org/@jridgewell/sourcemap-codec/-/sourcemap-codec-1.6.0.tgz",
"integrity": "sha512-T7jtf+5zsZHWNJ4lvq7/aezbyk0nNX+zJVWpmHA7VYsEx7a7qr5Rg5IbtJFqkge5Y2sruq1RUy8Q8370d7iFw==",
"dev": true,
"license": "MIT"
},
"node_modules/@jridgewell/trace-mapping": {
"version": "0.3.31",
"resolved": "https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.31.tgz",
"integrity": "sha512-ZvWE98R3WqZE6VMB1sIAXo0LlKX4D5AEBjYo4W8WUw8A4B19C583I4YUJ8H80qPZK78u94F8o3Z3+5W1g==",
"dev": true,
"license": "MIT",
"dependencies": {
  "@jridgewell/resolve-uri": "^3.1.0",
  "@jridgewell/sourcemap-codec": "^1.4.14"
}
},
"node_modules/@next/env": {
"version": "16.3.8",
"resolved": "https://registry.npmjs.org/@next/env/-/env-16.3.8.tgz",
"integrity": "sha512-A19zqHVV7TJv0eFu0U4U7Lv74PTih4Ch63sk2xCIP5TkE3udFna0cnzP2lQVymiL7yS9Cj2iC1UX1R3EQ5sEw==",
"license": "MIT"
},
"node_modules/@next/swc-darwin-arm64": {
"version": "16.3.8",
"resolved": "https://registry.npmjs.org/@next/swc-darwin-arm64/-/swc-darwin-arm64-16.3.8.tgz",
"integrity": "sha512-2JPRMh2nmQG5Cil7cXGL9AGwnPWJQ//cTtAUCT+w511QHK79SYz3LGv/pc5X643B/WE00rvu3Yww0hqwt3kgeA==",
"cpu": [
  "arm64"
],
"license": "MIT",
"optional": true,
"os": [
  "darwin"
],
"engines": {
  "node": ">= 10"
}
},
"node_modules/@next/swc-darwin-x64": {
"version": "16.3.8",
"resolved": "https://registry.npmjs.org/@next/swc-darwin-x64/-/swc-darwin-x64-16.3.8.tgz",
"integrity": "sha512-GZtCCOBKJ41eVIT/Th011WKHD1ca921zbQiS5R50N9QkoiFn1FsebDae1JU2a3HWoKMEEmEZwGs1AGLavVM72Q==",
"cpu": [
  "x64"
],
"license": "MIT",
"optional": true,
"os": [
  "darwin"
],
"engines": {
  "node": ">= 10"
}
},
"node_modules/@next/swc-linux-arm64-gnu": {
"version": "16.3.8",
"resolved": "https://registry.npmjs.org/@next/swc-linux-arm64-gnu/-/swc-linux-arm64-gnu-16.3.8.tgz",
"integrity": "sha512-0659ygeQYqneJ1fBKMPFxIFqYkYswu8IAS10CKK/4f3ZgJm1dRz4fVJZRi/kLLWjnBKneboePA4Wnv+sv1pVA==",
"cpu": [

```

```

    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@next/swc-linux-arm64-musl": {
  "version": "16.3.8",
  "resolved": "https://registry.npmjs.org/@next/swc-linux-arm64-musl/-/swc-linux-arm64-musl-16.3.8.tgz",
  "integrity": "sha512-dSjKSyWpzo01d3DIZZcP4XJcNaKeLmxQMFOiY15vuBRMmweIqnAhty8tAmRsvTss779cK1FtYnDMj40e4TQ1g==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@next/swc-linux-x64-gnu": {
  "version": "16.3.8",
  "resolved": "https://registry.npmjs.org/@next/swc-linux-x64-gnu/-/swc-linux-x64-gnu-16.3.8.tgz",
  "integrity": "sha512-lbq0uz3RPRcv+o9msNsJw5x4+Y1ZwPTs6vmL6DCf7i0fZfvng/F59wyeDwqHIvV0mK//RBy/jJkZ+nCKsSMXjQ==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@next/swc-linux-x64-musl": {
  "version": "16.3.8",
  "resolved": "https://registry.npmjs.org/@next/swc-linux-x64-musl/-/swc-linux-x64-musl-16.3.8.tgz",
  "integrity": "sha512-+316WswI8ScVgZeUd+1KGaXkHhaYQzCjvH/05TZSpJ8zBizb1a4G7Dt07F12jcBIqM0tsz9ji1t48fmKtzqsGA==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@next/swc-win32-arm64-msvc": {
  "version": "16.3.8",
  "resolved": "https://registry.npmjs.org/@next/swc-win32-arm64-msvc/-/swc-win32-arm64-msvc-16.3.8.tgz",
  "integrity": "sha512-ji0gd4kMYUx0+1fJBibiBVRcjzG/lloiyCccnlebv1ZJ5qXCPZqYg4Jl1DrrixnWNMkylzgpmMWx0yNDYX1zw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@next/swc-win32-x64-msvc": {
  "version": "16.3.8",
  "resolved": "https://registry.npmjs.org/@next/swc-win32-x64-msvc/-/swc-win32-x64-msvc-16.3.8.tgz",
  "integrity": "sha512-WcTlaKt/TWkh5kUjdJcUmB1Xgz+1c6fz4Y9fDHL73YNSdGaUWjceeWrrlwF0nv19iABYWC4iAq1oX1w4Bn0vfg==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">= 10"
  }
},
"node_modules/@swc/helpers": {

```

```

"version": "0.5.23",
"resolved": "https://registry.npmjs.org/@swc/helpers/-/helpers-0.5.23.tgz",
"integrity": "sha512-51SsM0TXURepG1DfVuAQUqGek9Hg2Eks0Yay2m0+XR++b2NWYL/4sWyuVBIIs8oKnJaxkdi9whaL/sqN13afw==",
"license": "Apache-2.0",
"dependencies": {
  "tslib": "^2.8.0"
}
},
"node_modules/@tailwindcss/node": {
"version": "4.3.3",
"resolved": "https://registry.npmjs.org/@tailwindcss/node/-/node-4.3.3.tgz",
"integrity": "sha512-/T8IKesf9VTU6tLjgC7+sv2m0PtQxzE2jMw7u4Tt40Tx+QSZxpzh95/H6cMKoja9XuW7iMdLJYBB0o9G1CaAgg==",
"dev": true,
"license": "MIT",
"dependencies": {
  "@jridgewell/remapping": "^2.3.5",
  "enhanced-resolve": "^5.24.1",
  "jiti": "^2.7.0",
  "lightningcss": "1.32.0",
  "magic-string": "^0.30.21",
  "source-map-js": "^1.2.1",
  "tailwindcss": "4.3.3"
}
},
"node_modules/@tailwindcss/oxide": {
"version": "4.3.3",
"resolved": "https://registry.npmjs.org/@tailwindcss/oxide/-/oxide-4.3.3.tgz",
"integrity": "sha512-kzXjAikiaFSPaK/FkAQT5UTx3VormQaiZ5hBF1JZ9UFGb/rwg1MZIhHAG9smMQRTdyJxP6Qt5MwMtdyU5FWrA==",
"dev": true,
"license": "MIT",
"engines": {
  "node": ">= 20"
}
},
"optionalDependencies": {
  "@tailwindcss/oxide-android-arm64": "4.3.3",
  "@tailwindcss/oxide-darwin-arm64": "4.3.3",
  "@tailwindcss/oxide-darwin-x64": "4.3.3",
  "@tailwindcss/oxide-freebsd-x64": "4.3.3",
  "@tailwindcss/oxide-linux-arm-gnueabi": "4.3.3",
  "@tailwindcss/oxide-linux-arm64-gnu": "4.3.3",
  "@tailwindcss/oxide-linux-arm64-musl": "4.3.3",
  "@tailwindcss/oxide-linux-x64-gnu": "4.3.3",
  "@tailwindcss/oxide-linux-x64-musl": "4.3.3",
  "@tailwindcss/oxide-wasm32-wasi": "4.3.3",
  "@tailwindcss/oxide-win32-arm64-msvc": "4.3.3",
  "@tailwindcss/oxide-win32-x64-msvc": "4.3.3"
}
},
"node_modules/@tailwindcss/oxide-android-arm64": {
"version": "4.3.3",
"resolved": "https://registry.npmjs.org/@tailwindcss/oxide-android-arm64/-/oxide-android-arm64-4.3.3.tgz",
"integrity": "sha512-Y85A2gmPSk15Ve5qR86GL4HT509cfQqh1aes9p3sSkyTPwt0Pppf3GkwGe4JPACcRYjgJIEhQgM6dBC1nr0NYw==",
"cpu": [
  "arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
  "android"
],
"engines": {
  "node": ">= 20"
}
},
"node_modules/@tailwindcss/oxide-darwin-arm64": {
"version": "4.3.3",
"resolved": "https://registry.npmjs.org/@tailwindcss/oxide-darwin-arm64/-/oxide-darwin-arm64-4.3.3.tgz",
"integrity": "sha512-BiaWatpBcERQFD10jRDpIVXuFK5PJez5SA4JMg6VYZdBYU+qKfV/vqjcIs+IYmtitf1xYQZTwXvU/8y41fZUGw==",
"cpu": [
  "arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
  "darwin"
],
"engines": {
  "node": ">= 20"
}
},
"node_modules/@tailwindcss/oxide-darwin-x64": {
"version": "4.3.3",
"resolved": "https://registry.npmjs.org/@tailwindcss/oxide-darwin-x64/-/oxide-darwin-x64-4.3.3.tgz",
"integrity": "sha512-fAeUqfV5ndhXRwai8cXGzdLvU9utW0meTk69unv4ZXixjn61Z+p9lCWdw0wA3TYboG3BwdVuN/RDjhBR10mw==",
"cpu": [
  "x64"
],
"dev": true,
"license": "MIT",
"optional": true,

```

```

    "os": [
      "darwin"
    ],
    "engines": {
      "node": ">= 20"
    }
  },
  "node_modules/@tailwindcss/oxide-freebsd-x64": {
    "version": "4.3.3",
    "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-freebsd-x64/-/oxide-freebsd-x64-4.3.3.tgz",
    "integrity": "sha512-iyf5bV6+wnAlf1VeEy7R25dupxTNECZN5QMI0qNT6eT+EgaGdZcKhGkr5SdoaWiLJ3spLqIY9VCeSGrwtg4kw=",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "freebsd"
    ],
    "engines": {
      "node": ">= 20"
    }
  },
  "node_modules/@tailwindcss/oxide-linux-arm-gnueabihf": {
    "version": "4.3.3",
    "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-linux-arm-gnueabihf/-/oxide-linux-arm-gnueabihf-4.3.3.tgz",
    "integrity": "sha512-aAYUpRJAjQWWbRrPvtjdroZ56Md+JM8pMiopS6xGEwDfLhqj+2ver2p4nU4Mb3CRqcMmNBjo8KkUgcxhkzVQGQ=",
    "cpu": [
      "arm"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">= 20"
    }
  },
  "node_modules/@tailwindcss/oxide-linux-arm64-gnu": {
    "version": "4.3.3",
    "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-linux-arm64-gnu/-/oxide-linux-arm64-gnu-4.3.3.tgz",
    "integrity": "sha512-nDxldcEEN0xZRzC2uu9jrtZDAAQtb+8WWDCSnWL1zvBk1+FN+x6MtDViPB5AJMfttVCUhehGWus3XBPgatM/w=",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">= 20"
    }
  },
  "node_modules/@tailwindcss/oxide-linux-arm64-musl": {
    "version": "4.3.3",
    "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-linux-arm64-musl/-/oxide-linux-arm64-musl-4.3.3.tgz",
    "integrity": "sha512-Md44bD6veX/PC5iyF8cDVnw4HBiANZepRZZ7a8DQ0vkfo5WUBwcp6iAuCUz23u+4SukhJlD3eL7hNdW8ezd/kA=",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">= 20"
    }
  },
  "node_modules/@tailwindcss/oxide-linux-x64-gnu": {
    "version": "4.3.3",
    "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-linux-x64-gnu/-/oxide-linux-x64-gnu-4.3.3.tgz",
    "integrity": "sha512-tx7us1muwOKAKWao2v/GaafFeQboE6aj88vC6ziN2NCGcRm8gWUhwjzg+YdVB1e4boAtdtma4L43onunI6NS4w=",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">= 20"
    }
  },

```

```

"node_modules/@tailwindcss/oxide-linux-x64-musl": {
  "version": "4.3.3",
  "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-linux-x64-musl/-/oxide-linux-x64-musl-4.3.3.tgz",
  "integrity": "sha512-SJxX60smvHgasZoBy1ldX6YRjXJFovwWBoedhbQPOBzgFWBHGB+TVPWB9BxzR7TTxU8FQZAI2AyiNCMzFm8Img=",
  "cpu": [
    "x64"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 20"
  }
},
"node_modules/@tailwindcss/oxide-wasm32-wasi": {
  "version": "4.3.3",
  "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-wasm32-wasi/-/oxide-wasm32-wasi-4.3.3.tgz",
  "integrity": "sha512-jx1+rPhY/5YmpkktD656HBWEbLxP7dH06losBLjff5vgCODXvi9KhtftWcMIwTFIDqBr7cRnQkdLnAG+I0lGvQ==",
  "bundleDependencies": [
    "@napi-rs/wasm-runtime",
    "@emnapi/core",
    "@emnapi/runtime",
    "@tybys/wasm-util",
    "@emnapi/wasi-threads",
    "tslib"
  ],
  "cpu": [
    "wasm32"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "dependencies": {
    "@emnapi/core": "^1.11.1",
    "@emnapi/runtime": "^1.11.1",
    "@emnapi/wasi-threads": "^1.2.2",
    "@napi-rs/wasm-runtime": "^1.1.4",
    "@tybys/wasm-util": "^0.10.2",
    "tslib": "^2.8.1"
  },
  "engines": {
    "node": ">=14.0.0"
  }
},
"node_modules/@tailwindcss/oxide-win32-arm64-msvc": {
  "version": "4.3.3",
  "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-win32-arm64-msvc/-/oxide-win32-arm64-msvc-4.3.3.tgz",
  "integrity": "sha512-3rc292Ca2ceK6Ulc/bAVnTs/3nDtpPhyEKlGpV+yQJQi/JS/AMJlqzxv1DacL1nekbrcf6bTqp/jv4qgnPxNQ=",
  "cpu": [
    "arm64"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">= 20"
  }
},
"node_modules/@tailwindcss/oxide-win32-x64-msvc": {
  "version": "4.3.3",
  "resolved": "https://registry.npmjs.org/@tailwindcss/oxide-win32-x64-msvc/-/oxide-win32-x64-msvc-4.3.3.tgz",
  "integrity": "sha512-yJ0pwIVc/nYeGoV02WtsN8KYyLQv7kyI2wDnkezyJlGGjkd4QLwDGAw147YpPJeuI0M00baXGSPjvWDPeTPggw=",
  "cpu": [
    "x64"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "win32"
  ],
  "engines": {
    "node": ">= 20"
  }
},
"node_modules/@tailwindcss/postcss": {
  "version": "4.3.3",
  "resolved": "https://registry.npmjs.org/@tailwindcss/postcss/-/postcss-4.3.3.tgz",
  "integrity": "sha512-JTSZZGQi1AyKirbLN3azmjVzef92tcX7hiSqPdaeStyFpGpDlKvvpXe0E8njhbUanbRwr3z8DyZhICWnMtQeg=",
  "dev": true,
  "license": "MIT",
  "dependencies": {
    "@alloc/quick-lru": "^5.2.0",
    "@tailwindcss/node": "4.3.3",
    "@tailwindcss/oxide": "4.3.3",
    "postcss": "^8.5.16",
  }
}

```

```

    "tailwindcss": "4.3.3"
  },
  "node_modules/@types/node": {
    "version": "20.19.43",
    "resolved": "https://registry.npmjs.org/@types/node/-/node-20.19.43.tgz",
    "integrity": "sha512-6oYBAi5ikg4P1+kGsoYtawUMBT2zZMCvPNF7pVLnHZfd1zf38DRiWn/gT01RYCdUqkv7Fhr+C9ot4/tb+2sVvA==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "undici-types": "~6.21.0"
    }
  },
  "node_modules/@types/react": {
    "version": "19.3.0",
    "resolved": "https://registry.npmjs.org/@types/react/-/react-19.3.0.tgz",
    "integrity": "sha512-N0rFcuH9YoxG9/m6119MfpJKfmLOVU0em7ipIz6TRgSSkvReLB9vL85GB+yr8Bs5leqpvg96JSwF4ZS1s4viQg==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "csstype": "^3.2.2"
    }
  },
  "node_modules/@types/react-dom": {
    "version": "19.3.0",
    "resolved": "https://registry.npmjs.org/@types/react-dom/-/react-dom-19.3.0.tgz",
    "integrity": "sha512-Z177384ZG90tBpsZUPC1dZL5YpUEV7YVp0jY1Q9v5Uh0njz2xD5QwKz3ZUy0S9yFwBqLW1Pqj8KQ==",
    "dev": true,
    "license": "MIT",
    "peerDependencies": {
      "@types/react": "^19.3.0"
    }
  },
  "node_modules/baseline-browser-mapping": {
    "version": "2.11.27",
    "resolved": "https://registry.npmjs.org/baseline-browser-mapping/-/baseline-browser-mapping-2.11.27.tgz",
    "integrity": "sha512-ELy12DaR0Guan+1MmZ8Cvo/ZUbXPe7Enc/9VU/b1T3Kp4dwytRcNdR8DoSJN5SNJT/CuvCA0DHDVmMOCePdRQ==",
    "license": "Apache-2.0",
    "bin": {
      "baseline-browser-mapping": "dist/cli.cjs"
    },
    "engines": {
      "node": ">=6.0.0"
    }
  },
  "node_modules/caniuse-lite": {
    "version": "1.0.30001814",
    "resolved": "https://registry.npmjs.org/caniuse-lite/-/caniuse-lite-1.0.30001814.tgz",
    "integrity": "sha512-/Uaf1lAzr59XcMpW0o96WoEfr+VXK20X4U9AgFoiShsVJ4HppnIFUjtYzsyDH2+tgANaQb2/oxYGwCPapN1FpA==",
    "funding": [
      {
        "type": "opencollective",
        "url": "https://opencollective.com/browserslist"
      },
      {
        "type": "tidelift",
        "url": "https://tidelift.com/funding/github/npm/caniuse-lite"
      },
      {
        "type": "github",
        "url": "https://github.com/sponsors/ai"
      }
    ],
    "license": "CC-BY-4.0"
  },
  "node_modules/client-only": {
    "version": "0.0.1",
    "resolved": "https://registry.npmjs.org/client-only/-/client-only-0.0.1.tgz",
    "integrity": "sha512-IV30u0jSMzZrd3pZ48nLkT9DA7Ag1pnPzaiQhpW7c3RbcqqzvzVu+L8gfgMp/8IM2MQTSiqaCxrccfu8I8rMA==",
    "license": "MIT"
  },
  "node_modules/csstype": {
    "version": "3.2.3",
    "resolved": "https://registry.npmjs.org/csstype/-/csstype-3.2.3.tgz",
    "integrity": "sha512-E/eT/10U77HfIhM4b/3tZ2U7tYtZrKdknTQJnQZv1S7I2NtIUo/OEMuHq5a6eB6p2hZB7V6LQ==",
    "dev": true,
    "license": "MIT"
  },
  "node_modules/detect-libc": {
    "version": "2.1.2",
    "resolved": "https://registry.npmjs.org/detect-libc/-/detect-libc-2.1.2.tgz",
    "integrity": "sha512-6dyCvFqvY1qF4qQyF8PdUdU+1qaoZiZ1e7pfW4VFDVJ+dzIrJlqLwOjIdUjdcUaPZlSXv61I6A==",
    "devOptional": true,
    "license": "Apache-2.0",
    "engines": {
      "node": ">=8"
    }
  },
  "node_modules/enhanced-resolve": {
    "version": "5.26.0",
    "resolved": "https://registry.npmjs.org/enhanced-resolve/-/enhanced-resolve-5.26.0.tgz",
    "integrity": "sha512-9W1GKd2yH0YG7P20w0Y7b/0tWY1Pb3F3GJ9G8a5sjsMptw/USmEzGnm+9o6SF0O899nUy0Y4A==",

```

```

    "dev": true,
    "license": "MIT",
    "dependencies": {
      "graceful-fs": "^4.2.4",
      "tapable": "^2.3.3"
    },
    "engines": {
      "node": ">=10.13.0"
    }
  },
  "node_modules/graceful-fs": {
    "version": "4.2.11",
    "resolved": "https://registry.npmjs.org/graceful-fs/-/graceful-fs-4.2.11.tgz",
    "integrity": "sha512-RbJ5/jmFcNNCcdV5o9eTnBLJ/HszWV0P73bc+Ff4nS/rJj+YaS6IGyiOL0VoBYX+11Wrl3k63h/KrH+nhJ0XvQ==",
    "dev": true,
    "license": "ISC"
  },
  "node_modules/jiti": {
    "version": "2.7.0",
    "resolved": "https://registry.npmjs.org/jiti/-/jiti-2.7.0.tgz",
    "integrity": "sha512-AC/7JofJvZGrnEWNaEnJeOLUx+JlGt7Na0wZiRPT4MY1wmfKjt2+602p2uz2+sk1l80ZZMjMNqe7kKbNgQ==",
    "dev": true,
    "license": "MIT",
    "bin": {
      "jiti": "lib/jiti-cli.mjs"
    }
  },
  "node_modules/lightningcss": {
    "version": "1.32.0",
    "resolved": "https://registry.npmjs.org/lightningcss/-/lightningcss-1.32.0.tgz",
    "integrity": "sha512-NXYBzinNrb1fraPGyrbPoD19C1h91fI/1mzgWYvXUte414Gz/X1FD2XBZSZM7rRTiMA8JL30tAaGifrIKhQ5yQ==",
    "dev": true,
    "license": "MPL-2.0",
    "dependencies": {
      "detect-libc": "^2.0.3"
    },
    "engines": {
      "node": ">= 12.0.0"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/parcel"
    },
    "optionalDependencies": {
      "lightningcss-android-arm64": "1.32.0",
      "lightningcss-darwin-arm64": "1.32.0",
      "lightningcss-darwin-x64": "1.32.0",
      "lightningcss-freebsd-x64": "1.32.0",
      "lightningcss-linux-arm-gnueabihf": "1.32.0",
      "lightningcss-linux-arm64-gnu": "1.32.0",
      "lightningcss-linux-arm64-musl": "1.32.0",
      "lightningcss-linux-x64-gnu": "1.32.0",
      "lightningcss-linux-x64-musl": "1.32.0",
      "lightningcss-win32-arm64-msvc": "1.32.0",
      "lightningcss-win32-x64-msvc": "1.32.0"
    }
  },
  "node_modules/lightningcss-android-arm64": {
    "version": "1.32.0",
    "resolved": "https://registry.npmjs.org/lightningcss-android-arm64/-/lightningcss-android-arm64-1.32.0.tgz",
    "integrity": "sha512-YK7/ClTt4kAK0vo6w3X+Pnm0D2cf2vPHbh0XdoNti1Ga0al1P4TBzhwjATvjNwLEBCnKvjJc2jQgHXH0NEw1Ag==",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MPL-2.0",
    "optional": true,
    "os": [
      "android"
    ]
  },
  "node_modules/lightningcss-darwin-arm64": {
    "version": "1.32.0",
    "resolved": "https://registry.npmjs.org/lightningcss-darwin-arm64/-/lightningcss-darwin-arm64-1.32.0.tgz",
    "integrity": "sha512-RzeG9Ju5bag2Bv1/1w1VJvBE3q6TtXskdZLLCyfg5pt+HLz9Bq1IC07LZM7VHNTTn/5PRhHFB5jk51c4cmscPQ==",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MPL-2.0",
    "optional": true,
    "os": [
      "darwin"
    ]
  },

```

```

    "node": ">= 12.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/parcel"
  }
},
"node_modules/lightningcss-darwin-x64": {
  "version": "1.32.0",
  "resolved": "https://registry.npmjs.org/lightningcss-darwin-x64/-/lightningcss-darwin-x64-1.32.0.tgz",
  "integrity": "sha512-U+QsBp2m/s2wqpUYT/6wnlagdZbtZdndSmut/NJqLCcMLTwp5muCrID+K5UJ6jqD2BFshejCYXniPDbNh73V8w==",
  "cpu": [
    "x64"
  ],
  "dev": true,
  "license": "MPL-2.0",
  "optional": true,
  "os": [
    "darwin"
  ],
  "engines": {
    "node": ">= 12.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/parcel"
  }
},
"node_modules/lightningcss-freebsd-x64": {
  "version": "1.32.0",
  "resolved": "https://registry.npmjs.org/lightningcss-freebsd-x64/-/lightningcss-freebsd-x64-1.32.0.tgz",
  "integrity": "sha512-JCTigedEksZk3tHTthnMdVfGf61Fky8Ji2E4YjUTEQX14xiy/1TzXnu1vwiZe3bYe0q+SpsSH/CTeDXK6WHig==",
  "cpu": [
    "x64"
  ],
  "dev": true,
  "license": "MPL-2.0",
  "optional": true,
  "os": [
    "freebsd"
  ],
  "engines": {
    "node": ">= 12.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/parcel"
  }
},
"node_modules/lightningcss-linux-arm-gnueabihf": {
  "version": "1.32.0",
  "resolved": "https://registry.npmjs.org/lightningcss-linux-arm-gnueabihf/-/lightningcss-linux-arm-gnueabihf-1.32.0.tgz",
  "integrity": "sha512-x6rnnpRa2GL0zQ0kt6rts3YDPzduLpWvWAF6EMhXFVZXD4tPrBkEFqzGowzCsIWsPjqSK+tyNEODUBXeeVHSkw==",
  "cpu": [
    "arm"
  ],
  "dev": true,
  "license": "MPL-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 12.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/parcel"
  }
},
"node_modules/lightningcss-linux-arm64-gnu": {
  "version": "1.32.0",
  "resolved": "https://registry.npmjs.org/lightningcss-linux-arm64-gnu/-/lightningcss-linux-arm64-gnu-1.32.0.tgz",
  "integrity": "sha512-0nnMyoyOLRXfbM0ilaSRcLH3Jw5z9HDNGfT/gwCPgaDjnx0i8w7vBzFLFR1f6CMLKF8gVbebmKUN3fa/kJpQ==",
  "cpu": [
    "arm64"
  ],
  "dev": true,
  "license": "MPL-2.0",
  "optional": true,
  "os": [
    "linux"
  ],
  "engines": {
    "node": ">= 12.0.0"
  },
  "funding": {
    "type": "opencollective",
    "url": "https://opencollective.com/parcel"
  }
},
"node_modules/lightningcss-linux-arm64-musl": {

```

```

"version": "1.32.0",
"resolved": "https://registry.npmjs.org/lightningcss-linux-arm64-musl/-/lightningcss-linux-arm64-musl-1.32.0.tgz",
"integrity": "sha512-UpQkoenr4UJEzgVIYpI80lDFvRmPVg6oqboNHfoH4CQIfNA+H0rZ7Mo7KZP02dC6LjghPQJeBsvXhJod/wnIBg==",
"cpu": [
  "arm64"
],
"dev": true,
"license": "MPL-2.0",
"optional": true,
"os": [
  "linux"
],
"engines": {
  "node": ">= 12.0.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/parcel"
}
},
"node_modules/lightningcss-linux-x64-gnu": {
"version": "1.32.0",
"resolved": "https://registry.npmjs.org/lightningcss-linux-x64-gnu/-/lightningcss-linux-x64-gnu-1.32.0.tgz",
"integrity": "sha512-V7Qr52IhZmdKPVr+Vtw8o+WLSQJYCTd8l0IfpDaMRWGUZfBOYEJeyJIkqGIDMZPwPx24pUMfwSxxI8phr/Mb0A==",
"cpu": [
  "x64"
],
"dev": true,
"license": "MPL-2.0",
"optional": true,
"os": [
  "linux"
],
"engines": {
  "node": ">= 12.0.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/parcel"
}
},
"node_modules/lightningcss-linux-x64-musl": {
"version": "1.32.0",
"resolved": "https://registry.npmjs.org/lightningcss-linux-x64-musl/-/lightningcss-linux-x64-musl-1.32.0.tgz",
"integrity": "sha512-bYcLp+Vb0awsixG/80uCREzCYHNg1/l3mt0gzHnWV9XP1W5sKa5/TCdGwaR/zBM2PeF/HbsQv/j2URN0iVuxWg==",
"cpu": [
  "x64"
],
"dev": true,
"license": "MPL-2.0",
"optional": true,
"os": [
  "linux"
],
"engines": {
  "node": ">= 12.0.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/parcel"
}
},
"node_modules/lightningcss-win32-arm64-msvc": {
"version": "1.32.0",
"resolved": "https://registry.npmjs.org/lightningcss-win32-arm64-msvc/-/lightningcss-win32-arm64-msvc-1.32.0.tgz",
"integrity": "sha512-8SbC8BR40pS6baCM8sbtYDSwEVQd4JlFT0laD3gWGhfThTcABnNDBda6eTZeQbofalIJhFx0qKzghJmcPtNgdw==",
"cpu": [
  "arm64"
],
"dev": true,
"license": "MPL-2.0",
"optional": true,
"os": [
  "win32"
],
"engines": {
  "node": ">= 12.0.0"
},
"funding": {
  "type": "opencollective",
  "url": "https://opencollective.com/parcel"
}
},
"node_modules/lightningcss-win32-x64-msvc": {
"version": "1.32.0",
"resolved": "https://registry.npmjs.org/lightningcss-win32-x64-msvc/-/lightningcss-win32-x64-msvc-1.32.0.tgz",
"integrity": "sha512-Amq9B/SoZYDi1kFrojnoqPLxYhQ4Wo5XiL8EVJrVsB8ARoC1PWW6VGtT0WKCemjy8aC+louJnjS7U18x3b06Q==",
"cpu": [
  "x64"
],
"dev": true,
"license": "MPL-2.0",

```

```

    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">= 12.0.0"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/parcel"
    }
  },
  "node_modules/magic-string": {
    "version": "0.30.21",
    "resolved": "https://registry.npmjs.org/magic-string/-/magic-string-0.30.21.tgz",
    "integrity": "sha512-ud20fy69LgR0R7oB3w68z6gQcS6uYz38R3oT/GP4oDgB689Y0zPdZmpZhHd9mrslU2TKpYgnwE0QKNjWZQ==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@jridgewell/sourcemap-codec": "^1.5.5"
    }
  },
  "node_modules/nanoid": {
    "version": "3.3.19",
    "resolved": "https://registry.npmjs.org/nanoid/-/nanoid-3.3.19.tgz",
    "integrity": "sha512-Y22tU9Y40uW6tq5oDSKeQYG0yhkUBhN0cGV/02KC+6kd9eDGqdZd++mMiIDilrBYvjEnCYvVtsuHCuP+okSfug==",
    "funding": [
      {
        "type": "github",
        "url": "https://github.com/sponsors/ai"
      }
    ],
    "license": "MIT",
    "bin": {
      "nanoid": "bin/nanoid.cjs"
    },
    "engines": {
      "node": ">=10 || ^12 || ^13.7 || ^14 || >=15.0.1"
    }
  },
  "node_modules/next": {
    "version": "16.3.8",
    "resolved": "https://registry.npmjs.org/next/-/next-16.3.8.tgz",
    "integrity": "sha512-U7QEZA7Tini6wKrb8A8hqlLQYQyCetegKjCpJ0yxk642wWoMoU1x5PyZCJFvgYgiptA8xc5j/9xY1ZF07w9Sjmw==",
    "license": "MIT",
    "dependencies": {
      "@next/env": "16.3.8",
      "@swc/helpers": "0.5.23",
      "baseline-browser-mapping": "^2.9.19",
      "caniuse-lite": "^1.0.30001579",
      "postcss": "8.5.23",
      "styled-jsx": "5.1.6"
    },
    "bin": {
      "next": "dist/bin/next"
    },
    "engines": {
      "node": ">=20.9.0"
    },
    "optionalDependencies": {
      "@next/swc-darwin-arm64": "16.3.8",
      "@next/swc-darwin-x64": "16.3.8",
      "@next/swc-linux-arm64-gnu": "16.3.8",
      "@next/swc-linux-arm64-musl": "16.3.8",
      "@next/swc-linux-x64-gnu": "16.3.8",
      "@next/swc-linux-x64-musl": "16.3.8",
      "@next/swc-win32-arm64-msvc": "16.3.8",
      "@next/swc-win32-x64-msvc": "16.3.8",
      "sharp": "^0.35.4"
    },
    "peerDependencies": {
      "@opentelemetry/api": "^1.1.0",
      "@playwright/test": "^1.51.1",
      "babel-plugin-react-compiler": "*",
      "react": "^18.2.0 || 19.0.0-rc-de68d2f4-20241204 || ^19.0.0",
      "react-dom": "^18.2.0 || 19.0.0-rc-de68d2f4-20241204 || ^19.0.0",
      "sass": "^1.3.0"
    },
    "peerDependenciesMeta": {
      "@opentelemetry/api": {
        "optional": true
      },
      "@playwright/test": {
        "optional": true
      },
      "babel-plugin-react-compiler": {
        "optional": true
      },
      "sass": {
        "optional": true
      }
    }
  }
}

```

```

},
"node_modules/next/node_modules/postcss": {
  "version": "8.5.23",
  "resolved": "https://registry.npmjs.org/postcss/-/postcss-8.5.23.tgz",
  "integrity": "sha512-g5058Zr4bZmwFiTlflMu8E0bDTb5I5gertgwAKmsdULTQIhZtunzUld1WSzwcVWPoAVpsrA6v1fCD7oXvRwgg==",
  "funding": [
    {
      "type": "opencollective",
      "url": "https://opencollective.com/postcss/"
    },
    {
      "type": "tidelift",
      "url": "https://tidelift.com/funding/github/npm/postcss"
    },
    {
      "type": "github",
      "url": "https://github.com/sponsors/ai"
    }
  ],
  "license": "MIT",
  "dependencies": {
    "nanoid": "^3.3.16",
    "picocolors": "^1.1.1",
    "source-map-js": "^1.2.1"
  },
  "engines": {
    "node": ">=10 || ^12 || >=14"
  }
},
"node_modules/picocolors": {
  "version": "1.1.1",
  "resolved": "https://registry.npmjs.org/picocolors/-/picocolors-1.1.1.tgz",
  "integrity": "sha512-xUe5H7uXj3G8M1622X0I0N0wv6zjX811zK0Uj08z0X3J8WvUoKjZiBYZ7OkIeKwQfC5PVatK+E7U0/35kg==",
  "license": "ISC"
},
"node_modules/postcss": {
  "version": "8.5.28",
  "resolved": "https://registry.npmjs.org/postcss/-/postcss-8.5.28.tgz",
  "integrity": "sha512-UzI1VtZ4Zq2uV9H7DYUmP06aF5E9eMPYj0545YDnWwYF91/9H24i+5W04gYD9Jnq6wP2E9F83x1R1qQ==",
  "dev": true,
  "funding": [
    {
      "type": "opencollective",
      "url": "https://opencollective.com/postcss/"
    },
    {
      "type": "tidelift",
      "url": "https://tidelift.com/funding/github/npm/postcss"
    },
    {
      "type": "github",
      "url": "https://github.com/sponsors/ai"
    }
  ],
  "license": "MIT",
  "dependencies": {
    "nanoid": "^3.3.18",
    "picocolors": "^1.1.1",
    "source-map-js": "^1.2.1"
  },
  "engines": {
    "node": ">=10 || ^12 || >=14"
  }
},
"node_modules/react": {
  "version": "19.2.8",
  "resolved": "https://registry.npmjs.org/react/-/react-19.2.8.tgz",
  "integrity": "sha512-yyGgR197iWYUqZrD81BpL1xR6oB3CrqchVgKvMFhZCN1ZBNHsZTgCzSlXjBmUfP0MGW1oLzYgBZg1yuiLQ==",
  "license": "MIT",
  "engines": {
    "node": ">=0.10.0"
  }
},
"node_modules/react-dom": {
  "version": "19.2.8",
  "resolved": "https://registry.npmjs.org/react-dom/-/react-dom-19.2.8.tgz",
  "integrity": "sha512-3i8L73f6QJH5F0W7/2siui0A5L1hKx5HqCzYw75y44u/sJx2pbg7S+5A8WsJ/vNzQwSJbcx4eS9aKJpw==",
  "license": "MIT",
  "dependencies": {
    "scheduler": "^0.27.0"
  },
  "peerDependencies": {
    "react": "^19.2.8"
  }
},
"node_modules/scheduler": {
  "version": "0.27.0",
  "resolved": "https://registry.npmjs.org/scheduler/-/scheduler-0.27.0.tgz",
  "integrity": "sha512-ElgJZS6PI3v14h0ZCZgPRF0VZp1uG0zEzT9XU6B0Q8P9ZuT9mXvU7DscqYG5D9Ys9L89u7g5Y5V6Dg==",
  "license": "MIT"
},
"node_modules/semver": {

```

```

"version": "7.8.5",
"resolved": "https://registry.npmjs.org/semver/-/semver-7.8.5.tgz",
"integrity": "sha512-Y7/KDsb8LjooZpwaqGyu106DQlksGcncchHGk+sZiY4SBvUocMBEFH5Ur1fI4dV+Jv10w6cjuvaIi40puRioA==",
"license": "ISC",
"optional": true,
"bin": {
  "semver": "bin/semver.js"
},
"engines": {
  "node": ">=10"
}
},
"node_modules/sharp": {
"version": "0.35.5",
"resolved": "https://registry.npmjs.org/sharp/-/sharp-0.35.5.tgz",
"integrity": "sha512-Ywn40nzGukp7CDMrp08RQ50YKmuwG47brZgIVPTvBaaAfQlRlygrRqSrxDCil9M+LlZLBIJ68IR1QqvzHyjC7g==",
"license": "Apache-2.0",
"optional": true,
"dependencies": {
  "@img/colour": "^1.1.0",
  "detect-libc": "^2.1.2",
  "semver": "^7.8.5"
},
"engines": {
  "node": ">=20.9.0"
},
"funding": {
  "url": "https://opencollective.com/libvips"
},
"optionalDependencies": {
  "@img/sharp-darwin-arm64": "0.35.5",
  "@img/sharp-darwin-x64": "0.35.5",
  "@img/sharp-freebsd-wasm32": "0.35.5",
  "@img/sharp-libvips-darwin-arm64": "1.3.4",
  "@img/sharp-libvips-darwin-x64": "1.3.4",
  "@img/sharp-libvips-linux-arm": "1.3.4",
  "@img/sharp-libvips-linux-arm64": "1.3.4",
  "@img/sharp-libvips-linux-ppc64": "1.3.4",
  "@img/sharp-libvips-linux-riscv64": "1.3.4",
  "@img/sharp-libvips-linux-s390x": "1.3.4",
  "@img/sharp-libvips-linux-x64": "1.3.4",
  "@img/sharp-libvips-linuxmusl-arm64": "1.3.4",
  "@img/sharp-libvips-linuxmusl-x64": "1.3.4",
  "@img/sharp-linux-arm": "0.35.5",
  "@img/sharp-linux-arm64": "0.35.5",
  "@img/sharp-linux-ppc64": "0.35.5",
  "@img/sharp-linux-riscv64": "0.35.5",
  "@img/sharp-linux-s390x": "0.35.5",
  "@img/sharp-linux-x64": "0.35.5",
  "@img/sharp-linuxmusl-arm64": "0.35.5",
  "@img/sharp-linuxmusl-x64": "0.35.5",
  "@img/sharp-webcontainers-wasm32": "0.35.5",
  "@img/sharp-win32-arm64": "0.35.5",
  "@img/sharp-win32-ia32": "0.35.5",
  "@img/sharp-win32-x64": "0.35.5"
},
"peerDependenciesMeta": {
  "@types/node": {
    "optional": true
  }
}
},
"node_modules/source-map-js": {
"version": "1.2.2",
"resolved": "https://registry.npmjs.org/source-map-js/-/source-map-js-1.2.2.tgz",
"integrity": "sha512-8Y433333aZVDtt+J4mK1hoLGHULMYfSkODJNQjNDC3oW1PqPoxMwo0pLUswM/UEGzON/NxeHywEfNXPN3Vw==",
"license": "BSD-3-Clause",
"engines": {
  "node": ">=0.10.0"
}
},
"node_modules/styled-jsx": {
"version": "5.1.6",
"resolved": "https://registry.npmjs.org/styled-jsx/-/styled-jsx-5.1.6.tgz",
"integrity": "sha512-qSVyDTeMotdvQYoHwLNGwRFJHC+i+ZvdBRYosOFgC+Wg1vx4frN2/RG/NA7SYqqvKNLf39P2LSRA2pu6n0XYZA==",
"license": "MIT",
"dependencies": {
  "client-only": "0.0.1"
},
"engines": {
  "node": ">= 12.0.0"
},
"peerDependencies": {
  "react": ">= 16.8.0 || 17.x.x || ^18.0.0-0 || ^19.0.0-0"
},
"peerDependenciesMeta": {
  "@babel/core": {
    "optional": true
  },
  "babel-plugin-macros": {
    "optional": true
  }
}
}

```

```

    },
    "node_modules/tailwindcss": {
      "version": "4.3.3",
      "resolved": "https://registry.npmjs.org/tailwindcss/-/tailwindcss-4.3.3.tgz",
      "integrity": "sha512-g0hV3P7ufE62QDGg1zVaTgCR+EtPv92k2nIhVcVKcLmxT1sUBsQGhnZj175j+MqRt4zLF7ic+sCYjfhxMxj7YQ==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/tapable": {
      "version": "2.3.3",
      "resolved": "https://registry.npmjs.org/tapable/-/tapable-2.3.3.tgz",
      "integrity": "sha512-tapable",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6"
      },
      "funding": {
        "type": "opencollective",
        "url": "https://opencollective.com/webpack"
      }
    },
    "node_modules/tslib": {
      "version": "2.8.1",
      "resolved": "https://registry.npmjs.org/tslib/-/tslib-2.8.1.tgz",
      "integrity": "sha512-oJFu94HQB+KVduSUQL7wnpmqnfmls0A/nAh6b6EH0wCEoK0/mPeXU6c3wKDV83Mk0uHPRHtSXXKU99IBazS/2w==",
      "license": "0BSD"
    },
    "node_modules/typescript": {
      "version": "5.9.3",
      "resolved": "https://registry.npmjs.org/typescript/-/typescript-5.9.3.tgz",
      "integrity": "sha512-jl1vZzPDinLr9eUt3J/t7V6FgNEw9QjvBPdysz9KfQDD41fQrC2Y4vKQdiaUpFT4bX1b1RHHLpp8wtm6M5TgSw==",
      "dev": true,
      "license": "Apache-2.0",
      "bin": {
        "tsc": "bin/tsc",
        "tsserver": "bin/tsserver"
      },
      "engines": {
        "node": ">=14.17"
      }
    },
    "node_modules/undici-types": {
      "version": "6.21.0",
      "resolved": "https://registry.npmjs.org/undici-types/-/undici-types-6.21.0.tgz",
      "integrity": "sha512-iwDZqg0QAgr9Rav5H4n0M64c3mkR59cJ6wQp+7C4nI0gsmExaedaYLN044eT4AtBBWjbTiGPM1t2Md0T9H9JQ==",
      "dev": true,
      "license": "MIT"
    }
  }
}

```

web/package.json

```
{
  "name": "web",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "16.3.8",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

web/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": [".src/*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

web/next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "../next/types/routes.d.ts";
import "../next/types/root-params.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

web/next.config.ts

```
import type { NextConfig } from "next";

// NEXT_EXPORT=1 → static export (deployment statis / unified Space).
// NEXT_BASE_PATH → prefix path (mis. "/loop-tb" untuk GitHub project pages).
const base = process.env.NEXT_BASE_PATH || undefined;

const nextConfig: NextConfig = {
  ...(base ? { basePath: base, assetPrefix: base } : {}),
  ...(process.env.NEXT_EXPORT === "1"
    ? { output: "export" as const, trailingSlash: true, images: { unoptimized: true } }
    : {}),
};

export default nextConfig;
```

```
# Charter – Proyek `loop-tb` (UnivaBio 2026)

> **Status: v1 FROZEN – disetujui & dikunci 2 Okt 2026.**
> Perubahan hanya via ADR baru (lihat §5).

## 1. Problem statement

Indonesia adalah negara dengan beban TB terbesar ke-2 dunia (10% dari 10,7 juta kasus global 2024 – WHO Global TB Report 2025). Diperkirakan ~1,07-1,09 juta orang jatuh sakit dan ~125 ribu meninggal per tahun (~14 kematian/jam), sementara pada 2024 hanya ~885 ribu kasus ditemukan (Kemenkes) → ~200 ribu orang menularkan tanpa terdeteksi.

Program nasional TOSS-TB memperluas skrining CXR + distribusi X-ray portabel, tetapi tiga hambatan bertemu:
1. Volume bacaan CXR melampaui kapasitas pembaca ahli (puskesmas tanpa dokter/radiolog).
2. Alat CAD yang ada bersifat komersial (qXR, Cad4TB) atau masih tahap awal tanpa kalibrasi & penanganan ketidakpastian (TBScreen.AI UGM, pilot terbatas 2025).
3. Skrining berhenti di skor – tidak menyambung ke rujukan, kepatuhan, dan tindak lanjut.

Frasa inti: loop perawatan TB bergerak lebih lambat daripada penularan TB.

## 2. Target user

| Prioritas | Siapa | Kebutuhan |
| ---|---|---|
| Utama (MVP) | Kader/petugas TB puskesmas di daerah terpencil & fasilitas tanpa dokter | Skrining CXR → keputusan triase + surat rujukan dalam < 3 menit |
| Sekunder | Pasien TB & keluarga | Kepatuhan obat & pemantauan harian, bahasa manusia |
| Tersier | Klinisi/pengelola program | Ringkasan trajektori risiko untuk tindak lanjut |

## 3. Win conditions (dipetakan dari rubrik 25 poin)

| Kriteria | Ambisi kita |
| ---|---|
| Idea & Innovation | Bukan skor CAD ke-1000: loop triase terkalibrasi + honest evaluation + last-mile design |
| Implementation | Model nyata terlatih (bukan wrapper) + evaluasi eksternal + aplikasi live ter-deploy |
| Health Impact & Rigor | Angka WHO/Kemenkes + guideline resmi + protokol evaluasi ala TPP WHO + limitations jujur |
| Design & Usability | UI ID-first, mobile-first untuk kader; alur skrining < 3 menit |
| Presentation | Demo nyata + video 3 menit + slide jujur "selesai vs roadmap" |

## 4. Constraints & non-goals

- Kapasitas: solo, 6-8 jam/hari efektif. Tanpa GPU lokal → training di Colab/Kaggle (ADR-006).
- Deadline policy: platform = 13 Okt 23:45 EDT; halaman Rules (basi) menulis 7 Okt. Target internal: submit ≤ 7 Okt WIB (buffer). Verifikasi via email ke biocataanalysis@gmail.com (draft di `03-eksekusi.md`).
- Data: publik saja (daftar di `02-arsitektur.md`). Tanpa validator klinis → mitigasi: guideline resmi + framing "alat triase, bukan diagnosis" + limitations jujur.
- NON-GOALS (dilarang masuk MVP): alat diagnosis; menggantikan GeneXpert; CAD untuk <15 tahun (di luar rekomendasi WHO); analisis suara batuk; multi-penyakit; auth multi-user/RBAC; integrasi SITB otomatis; fitur apapun tanpa baris di coverage matrix (`02-arsitektur.md`).

## 5. Log keputusan (ADR)

| ID | Keputusan | Status |
| ---|---|---|
| ADR-001 | Domain = TB paru (beban #2 dunia; loop lengkap; dataset publik; wedge latensi jelas) | accepted – 2 Okt 2026 |
| ADR-002 | Invarian inti = state risiko terkalibrasi R(x,t) sebagai satu-satunya mesin; semua fitur = proyeksinya | accepted – 2 Okt 2026 |
| ADR-003 | Scope freeze MVP: Recognition=DEEP, Access=MEDIUM, Interpretation=MEDIUM, Monitoring=LIGHT, Continuity=LIGHT (detail di `02`) | accepted – 2 Okt 2026 |
| ADR-004 | Nama produk | ditunda sampai UI freeze |
| ADR-005 | Deadline policy: target internal 7 Okt, submit awal, buffer 8-13 Okt hanya untuk polish/stretch tercatat | accepted – 2 Okt 2026 |
| ADR-006 | Jalur training: Colab/Kaggle GPU (T4) – tidak ada GPU lokal; model kecil + checkpoint tiap epoch wajib | accepted – 2 Okt 2026 |
| ADR-007 | Studi representasi (dalam scope Recognition-DEEP): bandingkan supervised fine-tune (EB0/ConvNeXt) vs probe SSL (DINOv2 umum + RAD-DINO khusus X-ray dada) di AUROC eksternal + kalibrasi; pilih pemenang untuk API. Bukan scope baru – ini pemilihan inti mesin. | accepted – 2 Okt 2026 |

Kebijakan perubahan (anti-berubah-ubah):
1. Setelah freeze, setiap penambahan scope wajib menghapus item setara (zero-sum) dan dicatat sebagai ADR superseding.
2. Tidak ada fitur yang tidak punya baris di coverage matrix.
3. Semua perubahan besar harus selesai dalam < 30 menit diskusi; kalau tidak, ditolak default.
```

```
# Pohon Masalah TB → Pemilihan Wedge

## 1. Akar (root cause)

**Loop perawatan TB (deteksi → diagnosis → pengobatan → pemantauan) bereaksi lebih lambat daripada laju penularan dan progresi penyakit.**

Bukan "kurang dokter", bukan "kurang alat" – itu gejala. Mekanismenya: latensi setiap tahap loop > waktu yang dibutuhkan penyakit untuk menular ke orang lain / memburuk pada pasien.

**Bukti delay (terverifikasi):**
- Median diagnostic delay di primary care Indonesia 4 hari (IQR 1-11), **tetapi 19,3% episode >14 hari dan 7,6% >30 hari** (Annals of Global Health, 2025, aogh.5369).
- Patient pathway analysis: median 20 hari (IQR 7-72), **57,3% mengalami long diagnostic delay** (Clinical Microbiology & Infection, 2021).
- TBM (TB meningitis): median **66 hari** menuju diagnosis (PMC, 2025).
- Kualitas fasilitas DOTS berkorelasi dengan total delay (Jogjakarta, Trop Med Int Health, 2011).

## 2. Cabang masalah (MECE, mengikuti tahap loop)

| # | Cabang | Isi | Bukti |
|---|---|---|---|
| A | **Detection gap** | ±200k kasus/tahun tak pernah masuk sistem; case finding masih pasif; kasus ringan/asimtomatik; stigma; kapasitas skrining | 1,07-1,09 jt estimasi vs ~885k ditemukan (GTR 2025; Kemenkes 2024) |
| B | **Reading/access gap** | X-ray portabel didistribusikan masif, tapi pembaca ahli langka; GeneXpert tidak merata; keputusan rujukan lambat | WHO 2021/2025: CAD direkomendasikan *justru karena* kekurangan pembaca; Kemenkes distribusi X-ray portabel |
| C | **Interpretation gap** | Skor/angka tanpa penjelasan; pasien & kader tidak paham arti; stigma | TBScreen.AI = "probability 0-100%" tanpa mekanisme penjelasan |
| D | **Treatment continuity** | Regimen 6 bulan; LTFU; beban DOTS; TPT & investigasi kontak belum optimal | Wamenkes (2026): penemuan kasus, keberhasilan pengobatan, investigasi kontak & TPT masih jadi tantangan |
| E | **System continuity/learning** | Data antar-kunjungan terfragmentasi; tak ada trajektori risiko; program tak belajar dari kasus yang lolos | SITB/monitoring manual; tidak ada mekanisme loop |

## 3. Analisis bottleneck (Theory of Constraints)

- **Inflow constraint: deteksi.** Tidak bisa mengobati yang tidak ditemukan. Setiap kasus tertunda = tambahan transmisi + risiko kematian.
- Di dalam deteksi, constraint operasional pada push nasional = **throughput pembacaan CXR di ujung rantai** (puskesmas tanpa dokter; volume naik setelah roll-out alat portabel).
- **Wedge = kompresi latensi di titik skrining & triase**, lalu sambungkan loop ke hilir (rujukan → kepatuhan → tindak lanjut).
- Cabang D & E adalah bottleneck sekunder: intervensi di sana hanya menyentuh pasien yang *sudah* tertangkap sistem.

## 4. Mengapa solusi yang ada belum cukup (terverifikasi – ini diferensiasi kita)

| Solusi | Status | Celah |
|---|---|---|
| qXR (Qure.ai), Cad4TB | Komersial, mapan, dipakai program komunitas Indonesia (dilaporkan 190 kabupaten) | Tertutup; berhenti pada prioritisasi/skor; tidak menutup loop; threshold tetap isu implementasi |
| TBScreen.AI (UGM × KONEKSI, Agu 2025) | CAD lokal pertama; ~936 citra; pilot Klaten & Mimika | Akurasi dilaporkan ~64% (swa-lapor); tanpa kalibrasi/ketidapastian; belum terbuka |
| Model riset dari dataset publik | "AUROC 0,99" | **Audit medRxiv Agu 2026**: class-conditional acquisition confounding di Montgomery/Shenzhen/Rahman/TBX11K – 0,97-1,00 pada split acak → **0,57-0,88** pada transfer LOCO → 0,569 pada kohort asing. 88,4% "normal" Rahman berasal dari satu arsip RS; semua TB dari koleksi khusus. Mode gagal klinis: scar paru → FP percaya diri (0,84) |

**Wedge statement (satu kalimat untuk juri):**
> *Bukan skor lagi – sebuah loop triase terkalibrasi yang tahu kapan dirinya ragu, tahan uji pindah-domain, berjalan di ujung rantai layanan, dan menyambung skrining → rujukan → kepatuhan → tindak lanjut.*

## 5. Cabang → intervensi (preview coverage matrix)

| Cabang | Intervensi (proyeksi dari R(x,t)) | Depth |
|---|---|---|
| A Detection | Model CXR ensemble terkalibrasi + band triase | DEEP |
| B Access | Kebijakan aksi t (target TPP WHO) + paket rujukan cetak | MEDIUM |
| C Interpretation | Grad-CAM + penjelasan bahasa manusia + sitasi guideline | MEDIUM |
| D Continuity | Kartu pasien: check-in harian → R(t) → alert eskalasi | LIGHT (demo) |
| E Learning | Ringkasan trajektori untuk klinisi/program | LIGHT (demo) |

## 6. Referensi kunci

- WHO Global TB Report 2025, §1.1 TB incidence (Indonesia 10% beban global; #2).
- WHO consolidated guidelines on TB, Module 2: Screening (2021), rekomendasi #10 (CAD menggantikan pembaca manusia, ≥15 tahun, kondisional).
- WHO policy statement: Use of CAD software for TB screening (Mei 2025) – proses TAG 2025; isu pemilihan threshold masih terbuka; WHO calibration toolkit.
- WHO Target Product Profile: sensitivitas >0,90; spesifisitas >0,70.
- Bilal A. "Auditing Class-Conditional Acquisition Confounding Across Five Open TB CXR Corpora", medRxiv 2026 – repo audit: github.com/AhmadBilal227/tb-cxr-triage-pakistan.
- Kemenkes: TOSS-TB; ~885k kasus ditemukan 2024; proyeksi ~1,08-1,09 jt kasus 2024-2026; TB ditetapkan sebagai emergency.
```

```
# Arsitektur & Scope Freeze (ADR-003)

## 1. Invarian inti: state risiko `R(x, t)`

Satu objek komputasi per pasien yang berevolusi waktu:

...
R(x, t) = {
  p_tb : probabilitas TB terkalibrasi (temperature-scaled ensemble),
  u : ketidakpastian (ensemble disagreement),
  z : peta bukti spasial (Grad-CAM),
  band : aksi { rujuk-prioritas | ragu-periksa-manusia | negatif-skinning },
  t : riwayat pembaruan (trajektori)
}
...

**Metode penyatu:** satu mesin risiko + satu kebijakan aksi. Setiap cabang pohon masalah = **proyeksi** dari `R` yang sama – bukan fitur yang hidup sendiri-sendiri.

...


```
graph LR
 CXR --> R["MESIN R(x,t)
model ensemble + kalibrasi"]
 check_in[check-in] --> R
 R --> A["proyeksi A
triase rujukan"]
 R --> BC["B/C
penjelasan
Grad-CAM"]
 R --> DE["D/E
loop kepatuhan
& ringkasan"]
```


...

## 2. Coverage matrix – SCOPE MVP (FROZEN setelah approval)

| Cabang | Proyeksi dari R | Fitur MVP | Depth | Bukti sukses |
|---|---|---|---|---|
| A Detection | p_tb + u | Upload CXR → skor + CI + band | **DEEP** | Sens@Spec, AUROC+CI, ECE, tabel transfer |
| B Access | band → aksi | Triage band + surat rujukan (print/PDF) | MEDIUM | Kesepakatan band vs guideline; 1 alur <3 menit |
| C Interpretation | z → penjelasan | Overlay Grad-CAM + teks ID + "apa yang TIDAK ditunjukkan" | MEDIUM | Paham oleh 3+ orang non-medis (user test kecil) |
| D Monitoring | R(t) update | Kartu pasien: check-in minum obat/gejala → alert naik | LIGHT (demo) | Skenario demo memburuk → eskalasi tampil |
| E Continuity | R(t) → ringkasan | Export ringkasan klinisi (print) | LIGHT (demo) | Artefak printable muncul |

**Di luar scope (bukan roadmap yang dijanjikan, hanya ide yang sengaja TIDAK dikerjakan):** audio batuk, pediatric, multi-penyakit, integrasi SITB, auth multi-user, mobile native.

## 3. Data plan

| Dataset | Isi | Peran | Catatan |
|---|---|---|---|
| TBX11K (vbookshelf/tbx11k-simplified) | 8.399 berlabel 512²: **660 active TB**, 7.600 negatif (3.800 healthy + 3.800 sick-but-no-TB); 139 latent dikeluarkan; +3.302 `unknown_*` tanpa label (tidak dipakai) | **Train/val utama** | Label dari `data.csv` (otoritatif); task sulit by design: sick-but-no-TB = negatif |
| Rahman (tawsifurrahman) – folder `TB_Chest_Radiography_Database/` | **700 TB / 3.500 normal** | **Train** | WAJIB dedup – 88,4% "normal" dari satu arsip RS; semua TB dari koleksi khusus (Bilal 2026) |
| NLM Shenzhen | **336 TB / 326 normal** (662 citra) | **External test** | Sumber resmi NLM; tidak boleh tersentuh training |
| NLM Montgomery | **58 TB / 80 normal** (138 citra) + ClinicalReadings (usia/sex) | **External test** + subgroup | idem |
| Mendeley Pakistani TB CXR V2 | kohort RS Pakistan | External tambahan (stretch) | doi:10.17632/8j2g3csprk.2 |

Imbalance train ≈ 1.360 TB : 11.100 negatif (~1:8) → pakai `--balanced` + metrik yang tidak sensitif kelas (AUROC, sens@spec) sebagai metrik utama.

**Aturan integritas (tidak bisa dinegosiasi):**
1. Dedup perseptual (pHash/embedding) + provenance audit mengikuti metode Bilal 2026 (tools + source-matched hashes tersedia publik).
2. Laporkan jumlah duplikat yang dibuang, di README dan evaluasi.
3. Split manifest dikunci sebelum training (file JSON versioned di repo).
4. Tidak ada citra dari korpus eksternal yang masuk training – dicek ulang via hash sebelum final submission.

## 4. Protokol evaluasi (senjata rigor)

| Aspek | Metrik/metode |
|---|---|
| Akurasi utama | **Sensitivitas @ spesifisitas 0,90 & 0,70** (target TPP WHO: sens >0,90, spec >0,70) + 95% CI bootstrap |
| Diskriminasi | AUROC + CI |
| Kalibrasi | ECE, reliability curve, Brier score (sebelum vs sesudah temperature scaling) |
| Ketidakpastian | Proporsi "ragu" (deferral rate) + akurasi pada subset ragu |
| Transfer/honesty | Tabel: within-corpus vs provenance-dedup LOCO vs unseen-corpus (meniru desain audit) |
| Baseline pembandingan | (1) linear probe fitur beku, (2) 1 arsitektur fine-tune, (3) ensemble + kalibrasi |
| Subgrup | Usia/jenis kelamin (tersedia di NLM); pembahasan kualitatif scar-FP |
| Limitations (wajib di PDF & video) | Retrospektif; confounding akuisisi dataset publik; tanpa data Indonesia; bukan alat diagnosis |

## 5. Stack teknis

| Layer | Pilihan | Alasan |
|---|---|---|
| ML | PyTorch + timm (EfficientNet-B0, ConvNeXt-T), scikit-learn | Cepat dilatih, terbukti, export ONNX mulus |
| Training | Colab/Kaggle GPU (T4) atau lokal bila tersedia | B0 pada ~10k citra: ~1-2 jam/arsitektur |
```

Serving	FastAPI + onnxruntime (CPU), Docker → deploy HF Spaces	Gratis, publik, stabil untuk demo
FE	Next.js + Tailwind, mobile-first, ID-first	Kecepatan build + kualitas UI (kekuatan kita)
Offline (stretch)	onnxruntime-web (WASM) + PWA	Momen demo "airplane mode tetap jalan"
Repo	mono-repo: `ml/` `api/` `web/` `docs/` `submission/`	Bersih untuk code PDF & juri

6. Narasi demo video (3 menit)

1. **Masalah** (20 dtk): angka BPS/WHO + satu kalimat wedge.
2. **Skrining** (60 dtk): kader upload CXR → band + heatmap + penjelasan + surat rujukan.
3. **Loop** (40 dtk): kartu pasien → check-in → R(t) naik → alert eskalasi → ringkasan klinisi.
4. **Rigor** (40 dtk): tabel evaluasi – termasuk **jatuhnya performa di unseen corpus** (kejujuran ini yang membedakan).
5. **Roadmap jujur** (20 dtk): selesai vs direncanakan vs tidak dikerjakan.

```
# Eksekusi & Risk Register

## 1. Timeline (asumsi: deadline aman 7 Okt; jika 13 Okt terkonfirmasi → 8-13 Okt hanya polish/stretch)

| Hari | Tanggal | Kerja | Output |
| --- | --- | --- | --- |
| D0 | 2 Okt (malam) | Freeze docs; kirim email panitia; download data (Kaggle API); init repo; setup Colab | docs frozen; data lokal; repo jalan |
| D1 | 3 Okt | Prep data: dedup pHash, split manifest, EDA; baseline linear probe; mulai fine-tune arch-1 | Manifest split + baseline terukur |
| D2 | 4 Okt | Fine-tune arch-2 (+arch-3 bila lancar); temperature scaling; ensemble; eval harness | Tabel & grafik evaluasi v1 |
| D3 | 5 Okt | API + web end-to-end; Grad-CAM; triage band; surat rujukan; layar loop; deploy HF + Vercel | Demo hidup |
| D4 | 6 Okt | Polish UI; final eval run; 1-pager PDF; code PDF; writeup Devpost; video (script+rekam); submit draft | Submission draft |
| D5 | 7 Okt | Buffer + QA + **SUBMIT** | Submitted ☒ |
| D6+ | 8-13 Okt | HANYA stretch tercatat ADR: WASM offline, korpus eksternal tambahan, efisiensi | Opsional |

## 2. Deliverables & Definition of Done

| Deliverable | DoD |
| --- | --- |
| Repo GitHub publik | README (masalah, cara run, hasil, limitation), LICENSE, `submission/` lengkap; `make`/script reproducible |
| 1-Pager PDF | Tepat 1 halaman: masalah + angka bersitasi, solusi, bukti eval, roadmap |
| Code PDF | Auto-generate dari repo (script `ml/tools/export_code_pdf.py`) |
| Demo video (~3 mnt) | Narasi EN (juri internasional), UI ID; demo fitur benar-benar jalan; seksi rigor & roadmap jujur |
| Devpost writeup | Same story: problem → wedge → eval → honesty → roadmap |

## 3. Risk register

| # | Risiko | P | Dampak | Mitigasi |
| --- | --- | --- | --- | --- |
| R1 | Deadline ganda (7 vs 13 Okt) | M | Tinggi | Email hari ini ($4); submit ≤ D5; buffer 8-13 Okt tidak dipercaya |
| R2 | GPU tidak tersedia | M | Tinggi | Colab/Kaggle T4; model kecil (B0); checkpoint tiap epoch; arch-3 opsional |
| R3 | Leakage/confounding dataset | H | Tinggi | Dedup + provenance audit + split terkunci; jadikan *bagian cerita*, bukan kejutan |
| R4 | Tanpa validator klinis | H | Sedang | Framing "triase, bukan diagnosis"; sitasi guideline resmi; limitations eksplisit |
| R5 | Scope creep | H | Tinggi | ADR gate; zero-sum; fitur tanpa baris coverage matrix = ditolak |
| R6 | Download Kaggle gagal (auth) | L | Sedang | kaggle CLI + fallback unduh manual; mirrors |
| R7 | Deployment HF Spaces mati saat judging | L | Sedang | Video demo = rekaman layar lokal; link demo sebagai bonus, bukan satu-satunya bukti |

## 4. Draft email verifikasi deadline

**To:** biocataanalysis@gmail.com
**Subject:** UnivaBio – quick check on submission deadline

> Hi UnivaBio team,
> I noticed the Rules page lists "Hackathon End: October 7" while the Devpost schedule shows submissions until October 13 (11:45pm EDT). Could you confirm which deadline applies for submissions?
> Thank you!

## 5. Kebijakan change control

1. Semua keputusan tercatat sebagai ADR di `00-charter.md` $5; perubahan = ADR baru yang menggantikan (superseding).
2. Zero-sum: tambah 1 fitur = hapus 1 fitur setara.
3. Fitur wajib punya baris di coverage matrix (`02-arsitektur.md` $2).
4. Keputusan besar (>30 menit debat) ditolak default demi momentum.

## 6. Log progres

- **2 Okt – D0 selesai:** rencana frozen (ADR-003/005/006); data terunduh (TBX11K 8.399 berlabel + Rahman 4.200 + NLM 800);
**manifest v1** (train 10.288 / val 1.815 / external 800; 338 duplikat internal + **19 kebocoran cross-corpus** dibuang); repo publik berisi kode + artefak manifest; metadata klinis NLM 789 baris (21 scar-flagged); **smoke test ML end-to-end di CPU** – 2 bug nyata ditemukan & diperbaiki (kondisi spesifikitas terbalik; exporter ONNX butuh `dynamo=False` + toleransi relatif); UI web lengkap (Skrining, Pendampingan, Ringkasan klinisi, Tentang) build hijau.
```

```
# Evidensi Data – Manifest v1 (2 Okt 2026)

## 1. Pipeline

`ml/data/prepare_data.py`: discovery berlabel → pHash 64-bit → klaster Hamming ≤3 (union-find transitif) → proteksi integritas
external → split val per-klaster (seed 42). Output: `manifest.json`, `dedup_report.json`, `train.csv`, `val.csv`, `external.csv`
(path relatif – portable lokal ≠ Colab).

## 2. Hasil dedup (angka konkret untuk submission)

| Metrik | Nilai |
| --- | --- |
| Citra berlabel diproses | 13.260 |
| Tak-terklasifikasi | 3.442 (3.302 `unknown` tanpa label + 139 latent + 1 tak tercatat) |
| **Duplikat internal dibuang** | **338** |
| **Kebocoran cross-corpus dibuang** | **19** citra train terbukti near-dup citra external NLM → dihapus dari training |
| Korpus external | utuh 800 (Shenzhen 662 + Montgomery 138) |

> 19 itu penting: contoh konkret bahwa dataset publik memang saling terkontaminasi – persis mekanisme yang didokumentasikan audit
Bilal (medRxiv 2026). Kita tidak sekadar mengklaim, kita mengukur dan Membuktikan lalu membuangnya.

## 3. Split v1 (FROZEN – jangan berubah)

| Split | n | TB | NEG | Komposisi |
| --- | --- | --- | --- | --- |
| train | 10.288 | 1.123 | 9.165 | Rahman 3.552 (584 TB) + TBX11K 6.736 (539 TB) |
| val | 1.815 | 211 | 1.604 | Rahman 637 + TBX11K 1.178 |
| external | 800 | 394 | 406 | Montgomery 138 + Shenzhen 662 |

- Imbalance train ≈ **1:8,2** → training wajib `--balanced`; metrik utama AUROC & sens@spec.
- `sick_but_no_tb` (3.800 di TBX11K) dipertahankan sebagai negatif → task sengaja lebih sulit & lebih realistis untuk triase ("bukan
TB" termasuk penyakit paru lain).

## 4. Keterbatasan (jujur – akan masuk PDF/video)

1. pHash menangkap near-duplikat langsung; **reprocessing agresif multi-langkah bisa lolos** (provenance recovery sulit bahkan di
paper audit).
2. Label dari dataset publik bersifat radiografis, bukan konfirmasi bakteriologis.
3. Tidak ada data Indonesia di training/evaluasi → domain shift wajib dilaporkan apa adanya.

## 5. Cara verifikasi ulang

```bash
.venv/bin/python ml/data/prepare_data.py --raw data/raw --out ml/data/artifacts # deterministik (seed 42)
python -c "import json; m=json.load(open('ml/data/artifacts/manifest.json')); print(m['meta']['counts'])"
```

## 6. Hasil training v1 & "momen kejujuran" (2 Okt)

Model	Val AUROC (within-corpus)	Ext AUROC [95% CI]	Ext sens@spec90	Ext ECE
EfficientNet-B0	0,9999	0,638	–	–
ConvNeXt-Tiny	0,9861	0,675	–	–
**Ensemble (EB0+ConvNeXt, temp-scaled)**	–	**0,675 [0,636-0,712]**	0,266	0,212
Linear probe (fitur beku)	0,9802	**0,637 [0,597-0,671]**	0,254	0,322

**Interpretasi (jadi tulang punggung narasi submission):**

1. **Ilusi within-corpus terbukti pada data kita** – bahkan probe linear dengan fitur beku ImageNet mencapai 0,98 di val, tapi jatuh
ke ~0,64 saat pindah korpus. Persis pola yang didokumentasikan audit Bilal (2026): evaluasi split-acak membesarkan angka, deployment
membongkarnya.
2. **Over-triage eksternal:** 79,5% citra masuk band "rujuk prioritas" (ambang warisan val tidak transfer); band "negatif skrining"
masih memuat 24,7% TB → sistem **tidak boleh** dipakai rule-out tanpa konfirmasi manusia. Ini justifikasi empiris untuk pita "ragu" +
human-in-the-loop – dan mengukur secara konkret *gap threshold* yang diakui WHO masih terbuka.
3. **Kalibrasi tetap tantangan lintas-domain:** ECE eksternal 0,212-0,322 setelah temperature scaling (T=1,2-2,4). Model overkonfiden
ekstrem (logit val ~786).
4. **Subgrup adil:** AUROC eksternal F 0,669 / M 0,682; usia 0-30: 0,631 · 30-50: 0,707 · 50+: 0,737 (tidak ada disparitas besar;
celah di kelompok muda).
5. **Aksi perbaikan (ADR-007):** studi representasi SSL (DINOv2 + RAD-DINO khusus X-ray dada) – audit melaporkan probe SSL mencapai
~0,88 pada transfer terkontrol; hasil menyusul di bagian 7.
6. **Deferral berbasis ketidakpastian (ensemble disagreement):** pada data eksternal, kelompok 20% dengan disagreement tertinggi
punya TB-rate 31% vs 54% pada sisanya →  $\tau_u=0,40$  dipakai di API `model_config.json` sebagai pemicu band "ragu".

## 7. Studi representasi SSL (ADR-007) – HASIL FINAL (2 Okt)

Representasi (probe linear, fitur beku)	Val AUROC	**Eksternal AUROC [95% CI]**	sens@spec90	ECE
**RAD-DINO** (X-ray dada, SSL) – **DIPILIH**	0,9982	**0,8874 [0,862-0,909]**	**0,713**	0,247
Kombinasi DINOv2+RAD-DINO	–	0,8579 [0,831-0,883]	0,670	–
Stack RAD-DINO + supervised	–	0,8092	–	–
DINOv2 (SSL umum)	0,9987	0,7357 [0,698-0,768]	0,383	0,329
Supervised ensemble (EB0+ConvNeXt)	–	0,6751 [0,636-0,712]	0,266	0,212
Linear probe (fitur ImageNet beku)	0,9802	0,6366 [0,597-0,671]	0,254	0,322

**Keputusan final (ADR-007):**

- **Skor = probe RAD-DINO tunggal** (eksternal AUROC 0,887; sens@spec90 0,713). Catatan jujur: TPP WHO mensyaratkan sensitivitas
>0,90 – kita belum sampai, dan itu dilaporkan.
- **Ketidakpastian = std 4 anggota lintas-keluarga** (RAD-DINO, DINOv2, EB0, ConvNeXt): di eksternal, defer 25% → TB-rate grup ragu
```

32% vs 55% sisanya; $\tau_u=0,25$.

- **Ensemble/stacking** dengan supervised menurunkan performa** di domain asing (0,887-0,809) → dibuang.
- Pelajaran utama: **representasi pra-latih** pada domain (X-ray dada) mendominasi fine-tuning** untuk transfer lintas-rumah-sakit (+0,21 AUROC vs supervised; sens@spec90 2,7× lebih baik). Sejalan audit Bilal (2026): SSL 0,883 pada LOCO.
- Demo API final: kasus TB asli → p 0,98/u 0,006 → "rujuk"; kasus normal domain-shift → p 0,32/u 0,37 → "ragu".

```
# Model Card – SIGAP TB Triage v1

> Format mengikuti praktik umum model card (Mitchell et al.). Status: **prototipe riset UnivaBio 2026 – bukan alat medis.**

## 1. Ringkasan

Item	Detail
Model final	**RAD-DINO** (ViT-B/14, self-supervised pada X-ray dada) + **linear probe** terkalibrasi
Tugas	Klasifikasi biner pada citra X-ray dada: TB (radiografis) vs bukan-TB (termasuk `sick-but-no-TB`)
Peran dalam sistem	Menghasilkan **skor risiko** p(TB); bukan diagnosis. Dipakai bersama pita deferral.
Keluaran	`p_tb` (probabilitas terkalibrasi), `u` (ketidakpastian lintas-keluarga), `band` ∈ {rujuk_prioritas, ragu, negatif_skrining}
Versi	`models-v1` (GitHub Release) · probe_rad_dino.npz · T=0,749

## 2. Penggunaan yang dimaksudkan

- **Untuk:** triase skrining oleh kader/petugas puskesmas – memprioritaskan siapa yang perlu GeneXpert hari ini, dan **menahan keputusan ketika model tidak yakin**.
- **Pantang digunakan untuk:** diagnosis, meniadakan pemeriksaan konfirmasi, populasi <15 tahun (di luar rekomendasi CAD WHO), kehamilan sebagai satu-satunya dasar keputusan, atau setting tanpa jalur rujukan.
- **Pengguna:** petugas terlatih; output selalu disertai disclaimer dan instruksi konfirmasi.

## 3. Data

Korpus	Peran	n	Label
TBX11K (Kaggle)	train/val	7.914 setelah dedup	radiografis; `sick-but-no-TB` = negatif
Rahman et al. (Kaggle)	train/val	4.189 setelah dedup	radiografis
NLM Shenzhen + Montgomery	**external test**	800	radiografis + readings klinis

- Dedup pHash (Hamming ≤3): **338 duplikat internal** + **19 kebocoran train-external** dibuang sebelum training. Manifest frozen (seed 42).
- Tidak ada data Indonesia dalam training/evaluasi (dilaporkan sebagai limitation).
- Model SSL (RAD-DINO) pra-latih pada data publik pihak ketiga (MIMIC-CXR) – lisensi mengikuti sumber.

## 4. Evaluasi

Evaluasi	AUROC [95% CI]	Sens@Spec90	ECE
Val internal (split acak, per-kelompok)	0,9982	0,96	–
**Eksternal (Shenzhen+Montgomery)**	**0,887 [0,862-0,909]**	**0,713**	0,247
Subgrup eksternal – perempuan / laki-laki	0,87 / 0,90 (perkiraan dari run terkait)	–	–
Pembandingan: supervised EB0+ConvNeXt	0,675 [0,636-0,712]	0,266	0,212
Pembandingan: DINOv2 umum	0,736 [0,698-0,768]	0,383	0,329

- **Uji perilaku deployment** (`tools/model_checks.py`, 60 citra eksternal via HTTP API): API vs evaluasi offline → **Pearson r=0,986**, mean|Δp|=0,034, AUROC sampel **0,864 vs 0,867** offline – jalur deployment mereproduksi evaluasi setelah preprocessing disamakan (pra-resize 512px).
- Robustness kompresi **JPEG q85**: mean|Δp|=0,065, maks 0,28 → rekomendasi operasional: kirim citra kualitas penuh; hindari kompresi berat.
- Angka internal (0,98-0,9999) dilaporkan untuk transparansi tetapi **tidak boleh dianggap** ukuran keselamatan: dataset publik mengandung confounding akuisisi (Bilal 2026).

## 5. Kalibrasi & ketidakpastian

- Temperature scaling di-fit pada val (T=0,749 untuk probe final).
- `u` = std probabilitas 4 anggota lintas-keluarga (RAD-DINO, DINOv2, EB0, ConvNeXt); τ_u=0,25.
- Pada eksternal: defer 25% teratas → TB-rate 32% vs 55% pada sisanya (stratifikasi jujur, bukan ajaib).
- ECE eksternal 0,247 – **kalibrasi lintas-domain masih lemah**; threshold dari val tidak sepenuhnya transfer (over-triage terdokumentasi).

## 6. Keterbatasan & risiko

1. Label radiografis, bukan konfirmasi bakteriologis → nilai AUROC bergantung kualitas bacaan sumber.
2. Domain shift: performa dan threshold turun di rumah sakit asing; τ harus dikalibrasi per-site (roadmap).
3. `reprocessing` agresif dapat lolos dedup perseptual.
4. Tidak ada data Indonesia; bias populasi (MIMIC/China) belum diukur penuh.
5. Skar/fibrosis dapat tampak "TB-like" (mode gagal klinis yang dikenal) – alasan band ragu + pembacaan manusia.
6. Sensitivitas pada Spec90 = 0,713 < target TPP WHO (>0,90) – **belum memenuhi**.

## 7. Pertimbangan etis & operasional

- Data publik hanya; tidak ada PII; tidak ada redistribusi citra MIMIC.
- Output demo tersimpan lokal (browser) bila pengguna menekan "simpan ke kartu".
- Rekomendasi: setiap implementasi lapangan harus melalui validasi lokal, persetujuan etik, dan pengawasan klinis.

## 8. Reproduksi

```bash
bash scripts/fetch_models.sh # artefak models-v1
bash ml/data/download_kaggle.sh && bash ml/data/download_nlm.sh
.venv/bin/python ml/data/prepare_data.py --raw data/raw --out ml/data/artifacts
.venv/bin/python ml/ssl_probe.py --emb-dir <embeddings> --out ml/runs/ssl_eval # perlu kernel SSL
.venv/bin/python ml/evaluate.py --runs ... --split external --temp-scale
```

Empat bug nyata ditemukan lewat pengujian berlapis (metrik spesifisitas terbalik, export ONNX, grid temperature, preprocessing API vs evaluasi) – semuanya terdokumentasi di riwayat commit.
```